UNIVERSITÉ PIERRE ET MARIE CURIE - PARIS 6

U.F.R INFORMATIQUE

THESE

présentée pour obtenir le grade de

DOCTEUR DE l'UNIVERSITÉ PIERRE ET MARIE CURIE

Spécialité : Réseaux

Abdesselem KORTEBI

Titre :

Evaluation d'une architecture IP orientée flot

Directeur de thèse :

M. Guy PUJOLLE

Jury

Rapporteur	Roch GUÉRIN	Professeur à l'université de Pennsylvania
Rapporteur	Gérard HÉBUTERNE	Professeur à l'INT Evry
Examinateur	Serge FDIDA	Professeur à l'université Pierre et Marie Curie
Examinateur	Daniel KOFMAN	Professeur à l'ENST
Examinateur	Guy PUJOLLE	Professeur à l'université Pierre et Marie Curie
Examinateur	Sara OUESLATI	Ingénieur de recherche à France Télécom R&D

A mes parents

Remerciements

Je tiens à exprimer ma gratitude envers Sara Oueslati et Jim Roberts pour avoir encadré cette thèse et guidé mes recherches. Leurs conseils m'ont été d'un grand apport et leur disponibilité et gentillesse très appréciées.

Je remercie Guy Pujolle, mon directeur de thèse pédagogique, qui m'a proposé d'effectuer cette thèse au sein de l'équipe TRM à France Télécom R&D.

J'exprime ma reconnaissance envers Gérard Hébuterne et Roch Guérin pour avoir accepté d'être les rapporteurs de ce mémoire, ainsi que Serge Fdida et Daniel Kofman, membres du jury.

Je remercie les autres membres de mon équipe : Jordan Augé, Nabil Benameur, Thomas Bonald, Nidhi Hegde, Matthieu Jonckheere, Philippe Olivier et Alexandre Proutière pour les moments sympathiques qu'on a passé, mais aussi pour les discussions techniques intéressantes qu'on a eu. Je remercie également Luca Muscariello, avec qui j'ai été amené à collaborer dans la bonne humeur.

Mes remerciements à Jean-François Legros et Christian Guillemot, directeurs des laboratoires, respectivement, OAT et CPN où cette thèse s'est déroulée successivement ; ainsi qu'à tous les membres de ces deux laboratoires.

Un grand merci à mes proches, notamment Yacine, Riadh et Samir, ainsi qu'à mes amis pour leur soutien.

Résumé

Cette thèse traite du problème de la qualité de service dans le réseau IP. Le sujet en question a évidemment déjà fait l'objet de plusieurs travaux de recherches qui ont donné lieu à diverses propositions au succès mitigé. L'approche que nous présentons est quelque peu différente des mécanismes de QoS existants, dans la mesure où elle tient davantage compte de la nature stochastique du trafic IP. Il s'avère que l'échelle du flot est la plus appropriée pour la modélisation du trafic et pour l'évaluation de critères de performance.

Ainsi, nous décrivons une architecture orientée flot appelée *cross-protect* qui associe le contrôle d'admission implicite (sans signalisation ni réservation de ressources) et une discipline de service de type *fair queueing* par flot au sein des files d'attente des routeurs. L'objectif étant de garantir un partage équitable de la bande passante et un débit satisfaisant pour les flots élastiques tout en assurant des délais et pertes négligeables pour les paquets des flots *streaming*.

Une évaluation de *cross-protect* est conduite via des simulations qui illustrent un des atouts majeurs de cette architecture, à savoir la différenciation de service implicite, évitant le besoin de marquage des paquets.

Nous analysons également l'application d'un contrôle d'admission basé sur les mesures pour garantir une performance adéquate aux flots *streaming*. Un algorithme simple, adapté à *cross-protect*, est proposé et testé par le biais de simulations.

Dans le souci de montrer la faisabilité d'une telle architecture orientée flot, nous traitons le problème d'extensibilité des algorithmes de *fair queueing*. En effet, l'idée répandue est que le partage équitable de la bande passante est souhaitable eu égard aux avantages indéniables qu'il offre, mais qu'il n'est pas réalisable sur des liens à grande capacité, où le nombre de flots en cours est très élevé. Nous verrons qu'en réalité, seuls les flots actifs ont besoin d'être maintenus ; leur nombre se limite à quelques centaines seulement indépendamment de la capacité du lien. Pour ce faire, nous avons basé notre étude sur des traces de trafic réelles, relativement récentes, afin de mettre l'accent sur l'importance des caractéristiques des flots IP et leur impact sur l'extensibilité du *fair queueing* ainsi que sur la QoS perçue par les utilisateurs.

Mots-clés : QoS dans les réseaux IP, *flow-aware networking*, contrôle d'admission implicite, *fair queueing*, différenciation de service implicite, contrôle d'admission basé sur les mesures.

Abstract

This thesis deals with quality of service in IP networks. Obviously, this problem has been the subject of intense research that lead to various standard proposals with mitigated successes. We present an approach that is somewhat different from existing QoS mechanisms because it takes more into account the stochastic nature of IP traffic. It turns out that flow scale is more appropriate for traffic modelling and performance evaluation.

We describe a flow-aware architecture called cross-protect that combines implicit admission control (without signalling nor resource reservation) and per-flow fair queueing in router queues. The objective is to provide fair bandwidth sharing and a satisfactory throughput for elastic flows, while ensuring negligible delay and loss for streaming flow packets.

An evaluation of cross-protect is conducted by means of simulations, and illustrates one of the major assets of this architecture, namely implicit service differentiation that dispenses with specific packet marking.

Measurement-based admission control algorithms constitute an attractive way to provide streaming flows with statistical QoS guarantees. The benefits of this kind of approach are highlighted and a simple algorithm, adapted to cross-protect, is proposed and tested using simulations.

In order to demonstrate the feasibility of such flow-aware architecture, we consider the scalability problem of fair queueing algorithms. Despite the benefits of fair scheduling for bandwidth allocation, the deployment of per-flow fair queueing in the Internet is limited mainly because of scalability concerns. The common belief is that the number of flows in progress on high speed links is very large. Maintaining per-flow state has been considered infeasible. We show that only active flows need to be scheduled ; their number is limited to a few hundreds whatever link capacity. Our study is based on real traces that underline the impact of IP traffic characteristics on the scalability of fair queueing and on the perceived QoS by the users.

Key words : Quality of service in IP networks, flow aware-networking, implicit admission control, fair queueing, implicit service differentiation, measurment-based admission control algorithms.

Table des matières

Introduction **1**
1 Mécanismes de QoS existants . . . 2
1.1 Intserv . . . 2
1.2 Diffserv . . . 3
2 Caractéristiques du trafic IP . . . 5
2.1 Variation du trafic . . . 5
2.2 Flots à débit variable . . . 6
2.3 Le modèle Poisson de sessions . . . 7
2.4 Flots élastiques et flots *streaming* . . . 8
2.5 Relation entre demande, capacité et performance . . . 9
3 Contenu de la thèse . . . 9

1 Une architecture FAN première génération **13**
1.1 Description de l'architecture FAN 1^{re}G . . . 13
1.1.1 Identification des flots . . . 14
1.1.2 Contrôle d'admission . . . 15
1.1.3 Estimation de la bande passante disponible . . . 15
1.2 Mécanismes de protection du trafic en surcharge . . . 18
1.2.1 Protection du trafic avec Diffserv . . . 19
1.2.2 Protection du trafic avec le contrôle d'admission sélectif . . . 21
1.3 Conclusion . . . 23

2 *Cross-protect* : architecture FAN deuxième génération **27**
2.1 Architecture d'un routeur *cross-protect* . . . 28
2.1.1 Identification des flots . . . 28
2.1.2 Acheminement des paquets . . . 29
2.1.3 Ordonnancement des paquets . . . 30
2.2 Algorithme PFQ . . . 32
2.2.1 Structures de données . . . 32
2.2.2 Pseudo-code . . . 33
2.2.3 Indicateurs de congestion . . . 37
2.2.4 Complexité . . . 38
2.3 Garanties de QoS . . . 39

2.3.1 Contrôle d'admission . . . 39
2.3.2 Différenciation de service . . . 40
2.3.3 Vulnérabilité . . . 40
2.4 Evaluation de *cross-protect* par simulation . . . 41
2.4.1 Scénarios de simulation . . . 41
2.4.2 Résultats . . . 43
2.4.3 Simulation d'un lien à 100 Mb/s . . . 45
2.4.4 Comparaison entre PFQ et PQ . . . 46
2.5 Réalisation alternative avec l'algorithme PDRR . . . 47
2.5.1 Algorithme PDRR . . . 48
2.5.2 Résultats de simulations . . . 52
2.5.3 Algorithmes existants . . . 58

3 Contrôle d'admission basé sur les mesures pour les flots *streaming* 59
3.1 Classification des approches de contrôle d'admission . . . 59
3.1.1 Garanties déterministes vs. garanties statistiques . . . 60
3.1.2 Approche paramétrique vs. MBAC . . . 60
3.1.3 Multiplexages avec et sans buffer . . . 62
3.2 Etat de l'art sur les algorithmes MBAC pour les flots *streaming* . . . 62
3.3 Algorithme MBAC basé sur l'approximation Poisson du nombre de rafales . . . 69
3.3.1 Description . . . 70
3.3.2 Evaluation préliminaire par simulation . . . 71
3.3.3 Algorithme adaptatif . . . 75
3.4 Algorithme MBAC basé sur l'approximation Gaussienne du débit agrégé 76
3.5 Résultats comparatifs de simulation . . . 77
3.5.1 Trafic synthétique . . . 77
3.5.2 Trafic vidéo . . . 78
3.5.3 Trafic réel . . . 79
3.5.4 Discussion et perspectives . . . 81

4 Etat de l'art sur les mécanismes de partage équitable de la bande passante 83
4.1 Introduction . . . 83
4.2 Algorithmes de fair queueing . . . 84
4.2.1 Degré d'équité . . . 84
4.2.2 Algorithmes à base d'estampille temporelle . . . 85
4.2.3 Algorithmes à base de *round robin* . . . 87
4.2.4 Algorithmes hybrides . . . 89
4.3 Mécanismes d'équité approximative . . . 89

5 Analyse de l'extensibilité de l'ordonnancement *fair queueing* 95
5.1 Le modèle PS . . . 96
5.2 Analyse préliminaire . . . 96
5.2.1 Résultats de simulation . . . 98
5.2.2 Modèle analytique mono débit . . . 99
5.2.3 Analyse post simulation . . . 101
5.3 Traces de trafic . . . 102
5.3.1 Statistiques des traces . . . 103
5.3.2 Nombre de flots en cours . . . 104
5.3.3 Débits des flots . . . 107
5.3.4 Distribution de la taille des paquets . . . 108
5.4 Evaluation par simulation de l'extensibilité du *fair queueing* . . . 109
5.4.1 Charges originales . . . 110
5.4.2 Charges modifiées : 0.6 et 0.9 . . . 110
5.5 Modèle analytique multi-débit . . . 113
5.5.1 Modèle de trafic . . . 113
5.5.2 Flots actifs et flots *bottlenecked* . . . 115
5.5.3 La notion de *busy cycle* . . . 115
5.5.4 Distribution conditionnelle de la taille de l'AFL . . . 117
5.5.5 Distribution du nombre de flots en cours . . . 119
5.5.6 Distribution de la taille de l'AFL . . . 123
5.6 Analyse des résultats . . . 125
5.7 Extension des résultats à PDRR . . . 127
5.8 Surcharge . . . 128

6 Réduction du coût d'implémentation de l'architecture FAN 2ndG 131
6.1 Réduction du coût de gestion des flots actifs . . . 131
6.1.1 Identification des flots *non bottlenecked* . . . 132
6.1.2 pseudo-code modifié de PFQ . . . 133
6.1.3 pseudo-code modifié de PDRR . . . 133
6.1.4 Précision du mécanisme . . . 134
6.1.5 Evaluation par simulation . . . 137
6.2 Réduction du coût de gestion des flots en cours . . . 140
6.2.1 PFL à adressage direct . . . 141
6.2.2 Insertion probabiliste dans la PFL . . . 144

Conclusion 147

Annexe 1 : Expérimentations 151
1 Expérimentation de Diffserv . . . 151
2 Expérimentation du contrôle d'admission sélectif . . . 152
3 Expérimentation de *cross-protect* . . . 153
3.1 Génération de trafic . . . 154

3.2 Résultats 155

Annexe 2 : Exemples de fonctionnement de PFQ et PDRR **159**

Annexe 3 : Traces vidéos et traces réelles **163**
1 Traces vidéos 163
2 Traces réelles 165
2.1 Variations du débit cumulé 165
2.2 Variations du nombre de flots en cours 165

Annexe 4 : Formules de récurrence **169**
1 Preuve de la proposition 3 169
2 Preuve de la proposition 4 170
3 La formule de récurrence Kaufman-Roberts 170

Annexe 5 : Expressions des distributions manipulées **173**
1 Loi exponentielle 173
2 Loi Pareto 173
3 Loi Pareto tronquée 174
4 Loi Weibull 174
5 Distribution complémentaire du temps de séjour dans une file M/M/1 174

Table des figures **175**

Liste des tableaux **177**

Glossaire **178**

Bibliographie **181**

Introduction

Cette thèse s'inscrit dans le cadre général du problème de la qualité de service (QoS) dans les réseaux IP. Une architecture basée sur le concept de flot y est proposée et évaluée. Cette architecture tire profit des enseignements fournis par les études précédentes effectuées au sein de l'unité de recherche et développement TRM (Traffic and Resource Management) à France Télécom R&D. La thèse s'est déroulée au sein de cette équipe de recherche composée, entre autres, de N. Benameur, S. Oueslati et J. Roberts qui avaient déjà jeté les bases de l'architecture orientée flot tel que nous le verrons au cours du paragraphe 2. Au préalable, nous décrivons rapidement deux architectures de QoS existantes (Intserv et Diffserv) en indiquant certains de leurs défauts ; nous terminons cette introduction en spécifiant le contenu de la thèse.

La convergence des différents services voix, vidéo et données sur un réseau IP commun capable de répondre aux exigences de QoS des diverses applications envisagées constitue un des objectifs d'actualité dans le monde des réseaux de télécommunications. Des travaux précédents de Roberts et. al. [95, 16] ont déjà mis l'accent sur les conséquences potentiellement néfastes qu'entraînerait une mise en œuvre de ce réseau intégrant tout type de service en se basant sur les mécanismes de QoS normalisés connus tels que Intserv, Diffserv et MPLS. Ce constat a amené à préconiser une approche originale au sens où elle tient davantage compte de la nature statistique du trafic IP et de la façon dont elle affecte les performances. Il s'agit donc de mettre en évidence l'inadéquation des approches existantes pour garantir une QoS et de proposer une architecture alternative orientée flot.

Une des solutions possibles (et d'ailleurs largement pratiquée) pour résoudre le problème de QoS dans le réseau IP est le surdimensionnement. Un réseau *best effort* surdimensionné permettrait de satisfaire les exigences des utilisateurs, la majorité du temps. Cependant, plusieurs inconvénients font que le surdimensionnement ne constitue pas une solution viable pour le réseau multiservices, parmi lesquels on peut citer : la sous-utilisation des liens, l'absence de contrôle de la disponibilité du réseau, le risque de délais inacceptables pour les paquets des applications temps-réel, le contrôle de congestion est tributaire de la collaboration des utilisateurs, l'établissement de la nature des risques à prendre en compte est loin d'être triviale, l'absence d'un modèle économique valide, etc.

Certains routeurs actuels implémentent des mécanismes de différenciation de service destinés à remplir les critères de QoS des applications appartenant à une classe dite *premium*. En réalité, il a été montré que la différenciation de service n'est effective que pour des charges de trafic importantes [9]. Par contre, en cas de surcharge la performance du trafic *best effort* est très mauvaise entraînant un certain gaspillage de ressources à cause des risques d'interruption prématurée des transferts comme on le verra dans le chapitre 1.

C'est pourquoi, de notre point de vue, il est plus judicieux de protéger tous les flots en cas de surcharge et d'offrir un accès sélectif au réseau en appliquant un contrôle d'admission implicite. Ainsi, l'architecture orientée flot, qu'on appellera FAN (pour *Flow-Aware Networking*) première génération, proposée par Roberts et. al. [10, 5, 98] constitue le point de départ de nos travaux de recherche. La contribution de cette thèse concerne l'évaluation d'une architecture FAN deuxième génération qui introduit des améliorations significatives par rapport à la précédente.

1 Mécanismes de QoS existants

Plusieurs mécanismes de QoS ont été proposés dans la littérature. Dans ce paragraphe, nous nous restreignons aux modèles de service Intserv et Diffserv.

1.1 Intserv

Le modèle de service Intserv a été introduit en 1997 [107]. Il s'agit d'apporter des garanties de QoS pour tout flot en fonction de la classe à laquelle il appartient. Pour ce faire, une signalisation est effectuée grâce au protocole RSVP (*Resource ReSerVation Protocol*) [22] dans le but de procéder à une réservation de ressource. Cette signalisation véhicule des paramètres descriptifs de chaque flot. Une réservation est maintenue selon le principe *soft state* via des réactualisations périodiques.

Bien que ce concept ne fût pas un succès, nous décrivons de façon très succincte ses classes de service car il est emblématique d'une approche basée sur la réservation de ressources par flot en fonction de descripteurs de trafic individuels :

- *Guaranteed* (GS) : elle offre des garanties strictes et déterministes sur les délais de bout en bout et sur les pertes de paquets. Ainsi, les applications de cette classe bénéficient d'un délai de traversée du réseau garanti. Notons que des garanties déterministes peuvent être apportées en appliquant le modèle *network calculus*, par exemple, au prix d'une sous-utilisation du lien.
- *Controlled load* (CL) : elle offre une qualité de service équivalente à celle d'un service *best effort* dans une situation où la charge offerte est relativement faible. Un contrôle d'admission pourrait être utilisé à cet effet.

Deux autres classes non normalisées sont également citées dans la littérature :

- *Predective* : elle concerne les applications ayant une borne supérieure sur leur délai de bout en bout ainsi qu'un faible taux de perte.
- *Controlled delay* : elle assure une perte de paquets négligeable et permet aux applications de changer dynamiquement leurs exigences de délais quand elles observent des délais non satisfaisants.

La principale raison pour laquelle Intserv n'a jamais vraiment été déployé est liée au problème de passage à l'échelle, ou extensibilité, puisque cette approche nécessite la gestion des flots individuellement. En effet, le nombre de réservations effectuées en parallèle sur un lien à grande capacité peut être extrêmement élevé entraînant un coût mémoire important pour le maintien d'un état par flot outre un nombre d'opérations de signalisation et de réservation conséquent.
De plus, les descripteurs de trafic utilisés pour les classes CL et GS sont les paramètres du *leaky bucket*. Or, la caractérisation des flots a priori n'est pas évidente pour des applications à débit variable, comme on va le voir dans le paragraphe 2.1 ainsi qu'au cours du chapitre 3.

1.2 Diffserv

Suite à l'échec de Intserv, Diffserv [14] a été proposé, en 1998, par la communauté IETF (*Internet Engineering Task Force*) en se basant sur le concept d'agrégation de flots en classes afin d'éviter le problème d'extensibilité.

Chaque classe est identifiée par une valeur codée dans le champ DSCP (*DiffServ Code Point*) de l'entête IP. Cette classification s'effectue au niveau des routeurs de bord (*edge routers*) à l'entrée du réseau.

Les routeurs du cœur ne sont responsables que de l'acheminement des paquets. Quand un paquet, marqué de son champs DSCP, est reçu au niveau d'un nœud, il est transmis au prochain nœud selon le *Per Hop Behaviour* (PHB) associé à la classe du paquet. Le PHB détermine la façon dont les buffers du routeur et le lien sont partagés par les différentes classes de trafic.

Outre le PHB associé au trafic *best effort* et qui n'assure aucune garantie, deux autres groupes sont employés.

Le PHB EF (*Expedited Forwarding*), représentant la classe de service *premium* [57], est adapté aux applications temps-réel dans la mesure où il permet de garantir un faible délai et aucune perte. Une file de type prioritaire, WRR (*Weighted Round Robin*) ou CBQ (*Class Based Queueing*) peut être utilisée dans cette optique.
Toutefois, un contrôle d'admission est nécessaire afin de s'assurer que le trafic EF ne dépasse pas la capacité qui lui est allouée. Ainsi, les flots du groupe EF obtiennent des garanties statistiques grâce au traitement prioritaire réservé à leurs paquets et au maintien d'une capacité disponible suffisante pour appliquer un multiplexage sans buffer. Néanmoins, ils doivent être écrêtés au débit qu'ils déclarent an niveau des routeurs *edge*.

Le PHB AF (*Assured Forwarding*) [55] recouvre 4 différentes classes. L'agrégat

de flots appartenant à une des classes AF doit être conforme aux paramètres de *leaky bucket* associés dans le cadre d'un contrat de trafic. La tâche de vérification de la conformité (*policing*) est effectuée par les routeurs de bord. L'objectif est d'assurer l'acheminement (sans perte) des paquets IP de ce PHB avec une très grande probabilité tant que les flots respectent leurs profils, les contraintes de délai ne sont pas prises en compte.
Au sein d'une même classe AF, trois niveaux de priorité peuvent être spécifiés dans l'éventualité de rejeter des paquets. En cas de congestion dans une des classes, les paquets de basse priorité, i.e. ceux marqués avec un niveau de préférence de perte (*drop precedence*) élevé, sont rejetés en premier.
Un ordonnancement de type WFQ (*Weighted Fair Queueing*), WRR ou CBQ peut être utilisé pour le partage de la bande passante entre les différentes classes AF. Un mécanisme de gestion active de la file de type WRED (*Weighted Random Early Detection*), par exemple, permet de gérer les niveaux de préférence de perte.

Comme on peut le constater, plusieurs niveaux de différenciation peuvent être spécifiés avec le PHB AF. En pratique, l'efficacité de ce type de différenciation relative est contestable, comme décrit plus loin, si on veut obtenir une différenciation réellement tangible.

L'inconvénient de l'architecture Diffserv vient du fait qu'elle n'est pas basée sur des modèles mathématiques qui prennent en compte la nature dynamique du trafic. Il en résulte que les garanties apportées ne sont pas très bien définies. Cette difficulté est accentuée par le manque de contrôle des chemins empruntés par les flots et, donc, de leur performance. L'absence d'une réelle différenciation pose également des problèmes de tarification.
Par ailleurs, la mise en œuvre de Diffserv n'est pas triviale due à la difficulté de la gestion de la QoS de bout en bout à cause du remarquage inter-domaine, le besoin d'appliquer un contrôle d'admission (via les *bandwidth brokers* ou autres), la gestion des conditionneurs de trafic (qui s'assurent que les flots sont conformes à leurs paramètres déclarés et vérifient la disponibilité des ressources, entre autres), etc.

Une comparaison entre Diffserv et le contrôle d'admission implicite dans le cadre d'une architecture FAN est présentée à la fin du premier chapitre. Elle met en lumière la nature de la différenciation réalisée dans chaque cas lorsqu'un modèle de trafic dynamique est considéré.

Il est donc primordial d'essayer de mieux comprendre le trafic IP afin de se doter d'outils appropriés pour le gérer. La nature statistique du trafic Internet est discutée dans la prochaine section où nous rappelons les principaux résultats obtenus au sein de l'équipe, concernant les caractéristiques du trafic IP.

2 Caractéristiques du trafic IP

Les caractéristiques statistiques du trafic ont un impact direct sur les garanties de QoS réalisables. Ainsi, il faut avoir à l'esprit que tout mécanisme de QoS doit prendre en compte le caractère dynamique du trafic, c'est-à-dire que les flots arrivent et partent à des instants aléatoires. Dans ce qui suit, nous donnons un bref aperçu sur ces principales caractéristiques.

2.1 Variation du trafic

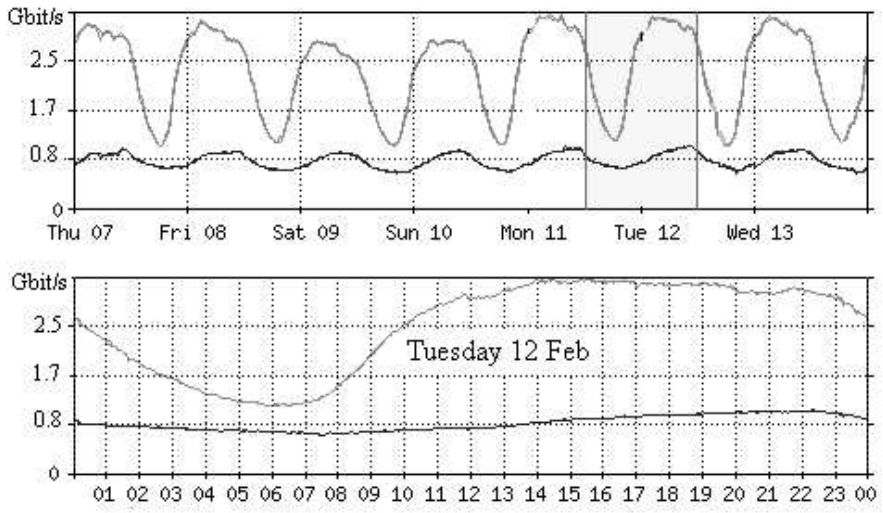


FIG. 1 – Demande de trafic journalière et hebdomadaire sur un lien OC192

La figure 1 représente des variations à long terme, à caractère périodique, qui ont été observées sur un lien à 10 Gb/s. On distingue clairement une période d'activité pendant laquelle la demande de trafic atteint approximativement la même valeur sur plusieurs jours successifs. Le réseau doit être dimensionné de façon à répondre aux exigences des utilisateurs sous cette charge maximale.

Pour établir la relation entre la demande, la capacité et les performances, on suppose que le trafic dans la période de pointe peut être modélisé par un processus stochastique stationnaire. Ce processus est extrêmement complexe au niveau paquet [76] à cause, notamment, du phénomène connu d'auto-similarité qui se traduit par de larges variations sur différentes échelles de temps. Une modélisation à l'échelle flot s'avère plus appropriée dans la mesure où les caractéristiques du trafic s'en trouvent simplifiées. Il est également possible d'en déduire des performances au niveau paquet.

2.2 Flots à débit variable

Un autre aspect à prendre en considération est la nature même de certaines applications qui se distinguent par un débit variable. Il est donc impossible de les caractériser par une seule valeur canonique adaptée plutôt à des applications à débit constant (CBR).

La figure 2 retrace le débit d'une séquence vidéo codée en MPEG extraite d'une course de Formule 1 [115]. On s'aperçoit que le débit varie sur des échelles temporelles multiples exhibant ainsi un comportement dit auto-similaire. Une des conséquences de cette variabilité est la difficulté de caractériser un flot vidéo de manière exploitable par un mécanisme de contrôle de trafic donné. En particulier, un mécanisme à base de *leaky bucket*, qui est inclus dans plusieurs architectures de QoS [92], n'est pas adapté pour décrire ce type de variation [96]. En effet, si les paramètres d'un descripteur de type *leaky bucket* sont facilement vérifiables (pour déterminer si un flot est conforme ou pas), ils ne sont pas faciles à comprendre et à assigner par les utilisateurs, ni aisément exploitables pour une allocation de ressource.

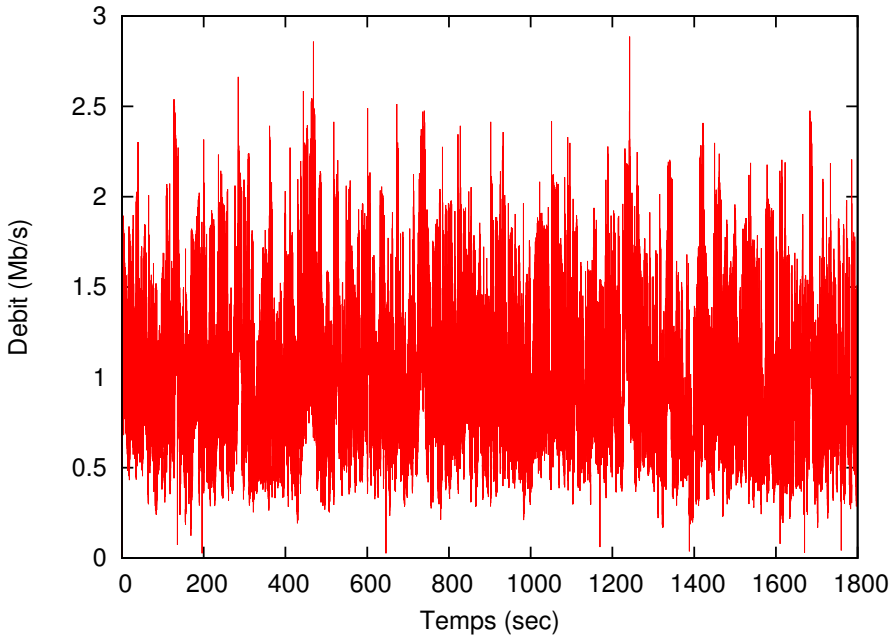


FIG. 2 – Variation du débit d'une vidéo MPEG

Cette variabilité n'est pas propre au trafic vidéo, elle se retrouve également dans du trafic de données agrégées, sur un LAN (*Local Area Network)*, par exemple [76].

Pour contourner ces difficultés, un modèle au niveau flot avait été proposé [9]. Nous le décrivont succinctement dans la prochaine section.

2.3 Le modèle Poisson de sessions

Le trafic IP au niveau d'un lien réseau peut être vu comme étant une superposition de sessions générées par un grand nombre d'utilisateurs indépendants. Une session est relative à une période d'activité d'un utilisateur ou d'une application donnés.
En pratique, il est difficile de reconnaître et de délimiter les sessions. Cependant, une session est constituée d'une succession de flots séparés par des temps de réflexion (*think times*). Les flots sont à leur tour composés d'une suite de paquets comme illustré par la figure 3.
Chaque utilisateur contribuant, de manière indépendante, avec une faible proportion à la charge totale, il en résulte naturellement un processus d'arrivée Poisson de ces sessions. Ceci constitue l'un des rares invariants du trafic IP et a été confirmé empiriquement, notamment, par les travaux de Floyd et Paxson [46].
On appellera ce modèle le *modèle Poisson de session* [9]. Il est à la base de notre modélisation analytique ; de surcroît il est valable pour différents types de trafic tels que la navigation Web, e-commerce, FTP, P2P ...

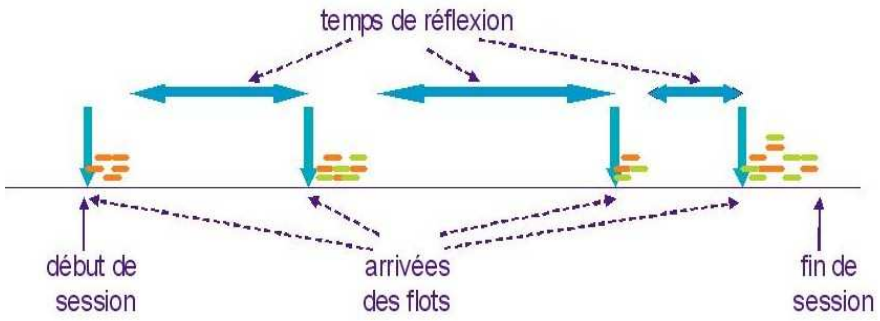


FIG. 3 – Structure d'une session en flots et paquets

La succession de flots et de temps de réflexion au sein des sessions peut être modélisée par un réseau de Kelly [62] où les clients visitent deux files. La première correspond au lien et la seconde modélise le temps de réflexion. Les arrivées extérieures (sessions) suivent un processus de Poisson et chaque client peut quitter le réseau suite à une visite de la file représentant le lien. Bien entendu, les visites successives du lien sont séparées par des visites de la file modélisant le temps de réflexion comme indiqué dans la figure 4.

Un des résultats importants de ce type de réseau est que la distribution du nombre de clients dans chaque file est celle où les arrivées des clients à la première file suivraient un processus de Poisson. Par conséquent, on peut supposer que le processus d'arrivée des flots est Poisson (même si ce n'est pas le cas dans la réalité)

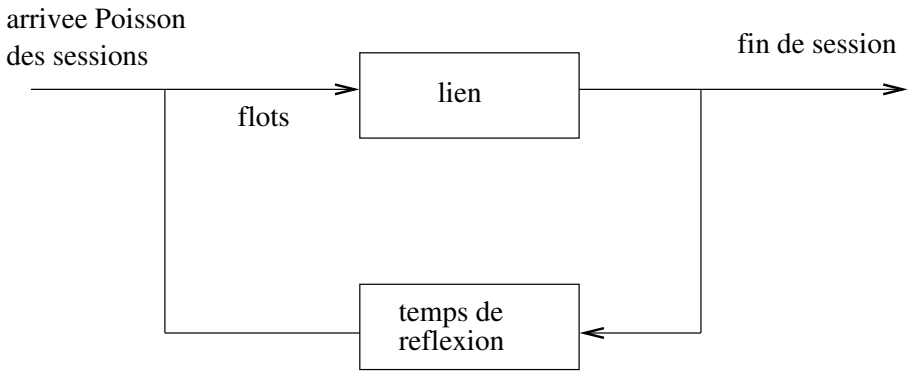


FIG. 4 – Modélisation des arrivées des flots

tant que les arrivées des sessions le sont. Ceci est vrai même si les tailles des flots successifs d'une même session sont corrélées et qu'il y a dépendance entre les flots et les temps de réflexion.

L'identification des flots constitue un facteur primordial au sein d'une architecture FAN, elle sera discutée dans le chapitre suivant.

2.4 Flots élastiques et flots *streaming*

Dans un contexte Internet multiservices, le trafic peut être décomposé en deux principales catégories à savoir les flots élastiques et les flots *streaming*.

Les flots élastiques sont générés par les applications utilisant généralement le protocole TCP pour le transfert de documents tels que les transferts de fichiers via FTP, le Web, les MP3, etc. Ils sont caractérisés par la taille du document à transférer, le principal critère de QoS de ce type de trafic est le débit réalisé. En effet, ces applications ont la faculté de s'adapter au débit disponible le long du chemin reliant la souce à la destination, et régulent leur débit grâce à l'algorithme de contrôle de congestion de TCP.

Les flots *streaming* sont, quant à eux, représentatifs des applications audio et vidéo. Ils sont caractérisés par un débit (qui peut être variable) et une durée intrinsèques. En raison de leur caractère temps-réel, leurs exigences de QoS se ramènent à de faibles délais et un taux de perte négligeable. Ils utilisent, d'ailleurs, le protocole UDP qui, contrairement à TCP, ne met en place aucun mécanisme de retransmission des paquets perdus.

Le trafic Internet est, actuellement, largement dominé en volume et en nombre de flots par le trafic élastique.

Cette classification peut être considérée comme robuste par rapport aux différentes applications et leur éventuelle évolution.

2.5 Relation entre demande, capacité et performance

Il est primordial de comprendre l'impact de la demande de trafic offert et de la capacité disponible sur les performances observées.

La demande de trafic est obtenue en multipliant le taux d'arrivée des flots par leur taille moyenne pour du trafic élastique. Dans le cadre d'un trafic *streaming*, elle correspond au produit du taux d'arrivée des flots par leur débit par leur durée moyenne.

Pour étudier la performance du trafic élastique, on suppose que les flots TCP se partagent équitablement la bande passante, même si en réalité cette équité dépend d'autres facteurs qui la rendent partielle. L'hypothèse d'équité permet de simplifier les modèles utilisés tout en fournissant des indications sur les facteurs déterminants pour la performance des flots élastiques et donc, par la même occasion, sur la façon d'apporter des garanties de QoS.
Les études effectuées au niveau d'un lien montrent qu'en l'absence de contraintes externes limitant le débit des flots, le débit moyen réalisé par flot est égal à la capacité résiduelle moyenne, $C - A$, où C est la capacité du lien et A la demande totale [9]. Ainsi, le débit réalisé sur un chemin du réseau est déterminé principalement par le lien ayant la plus faible capacité résiduelle [17]. Lorsque ce débit résiduel est supérieur au débit de la ligne d'accès de l'utilisateur, le réseau est dit transparent. Dans le cas contraire, ce lien constitue le goulot d'étranglement (on parle alors de lien *bottleneck*).
Si les liens du réseau sont typiquement transparents en charge normale, l'impact de la congestion nuit à la performance des flots en cas de surcharge, qui se produit lorsque la demande de trafic dépasse la capacité [21]. Le débit réalisé décroît alors progressivement jusqu'à ce que certaines applications soient abandonnées.

La performance du trafic *streaming* peut être établie grâce à d'autres types de modèles. Nous y reviendrons au chapitre 3.

Ainsi, l'analyse de cette dépendance entre demande et performance conduit à la remise en question de l'adéquation des mécanismes de QoS standards. En résumé, les performances du réseau sont excellentes tant que la demande de trafic reste inférieure à la capacité. Par contre, dès que la demande dépasse la capacité, les performances se dégradent nettement. Théoriquement, le nombre de flots dans le système croît indéfiniment, dans la mesure où ils en arrivent plus qu'ils en partent. En pratique, cela conduit le plus souvent à l'abandon de certains transferts à cause de l'impatience des utilisateurs ou des applications.

3 Contenu de la thèse

Après avoir introduit le contexte général de la thèse et rappeler les principaux résultats qui ont servi de base pour les travaux entrepris, nous résumons son contenu dans cette section.

Ce document s'organise en six chapitres. Le premier chapitre revient sur l'architecture FAN première génération qui avait déjà été proposée suite aux travaux de recherches de Roberts et. al. Nous y établissons également une comparaison entre le mécanisme Diffserv et le contrôle d'admission afin d'évaluer la capacité de chacune des solutions à protéger le trafic en cas de surcharge. Cette comparaison est effectuée par le biais de simulations, elle met en évidence le gaspillage de ressource dû à la gestion du trafic *best effort* avec Diffserv, d'une part et l'efficacité du contrôle d'admission pour garantir un accès sélectif au réseau en cas de surcharge, d'autre part.

Les chapitres suivants comportent les contributions principales de la thèse. Dans le second chapitre, nous présentons l'architecture orientée flot deuxième génération appelée *cross-protect* au tour de laquelle cette thèse s'articule. Il s'agit d'un concept innovant qui permet d'effectuer une différenciation de service implicite en associant le contrôle d'admission et une discipline de service de type *fair queueing* au sein des files d'attente des routeurs. Ainsi, il est possible d'accorder un traitement prioritaire aux paquets des flots *streaming* sans nécessiter un marquage explicite, les flots élastiques se partagent, quant à eux, équitablement la bande passante résiduelle. Pour ce faire, nous proposons deux algorithmes de *fair queueing* qui mettent en œuvre cette différenciation implicite. Une évaluation de performance par des simulations *ns* est présentée, elle confirme la faculté des algorithmes proposés à garantir de faibles délais ainsi que des pertes négligeables aux paquets des flots *streaming* tout en offrant un débit satisfaisant aux flots élastiques. Les simulations en question ont été conduites en situations de charge normale (inférieure à 1) et de surcharge.

Le chapitre 3 traite de la performance des flots *streaming* lorsqu'un contrôle d'admission est utilisé. Nous pensons qu'un contrôle d'admission basé sur les mesures (MBAC) est souhaitable pour garantir de faibles délais aux paquets des applications temps-réel. Dans un premier temps, nous procédons à un tour d'horizon des différents algorithmes de MBAC préalablement proposés dans la littérature ; nous expliquons les raisons pour lesquelles ils ne sont pas exploitables dans le cadre de *cross-protect*. Ensuite, nous proposons un algorithme simple adapté à l'architecture *cross-protect*, basé sur une approximation Poisson du nombre de rafales (*bursts*), afin d'apporter des garanties de QoS statistiques pour les flots *streaming*. Cet algorithme est testé, et comparé avec un autre algorithme MBAC basé sur une approximation Gaussienne du débit agrégé, par des simulations *ns* en considérant divers scénarios de trafic, y compris du trafic vidéo et des traces de trafic réelles.

Nous présentons un état de l'art sur les mécanismes de partage équitable de la bande passante au cours du chapitre 4. Nous décrivons succinctement certains algorithmes de *fair queueing* proposés dans la littérature ainsi que des méthodes d'équité approximative qui mettent l'accent sur l'importance d'un partage équitable de la bande passante.

Le chapitre 5 aborde la question d'extensibilité (*scalability*) des algorithmes de *fair queueing*, qui revient à évaluer le nombre de flots actifs à traiter par le scheduler. Nous montrerons que le nombre de flots à gérer est limité à quelques centaines

seulement avec une très grande probabilité et ce, indépendamment, de la capacité du lien considéré. Par conséquent, on en déduit que l'ordonnancement de type *fair queueing* est à la fois extensible et faisable. Cette étude a été conduite en considérant des traces de trafic réelles dont certaines statistiques sont mises en avant (en particulier, le débit des flots constitue un facteur important dans l'évaluation du nombre de flots actifs). Des simulations ont été effectuées sur les traces en question, leurs résultats sont expliqués grâce à un modèle analytique.

Dans le dernier chapitre, nous traiterons des aspects liés à la mise en œuvre d'une architecture FAN, en général, et de *cross-protect*, en particulier, en introduisant notamment des améliorations concernant la gestion des structures de données associées. En particulier, nous montrerons comment la taille de la liste des flots actifs peut être réduite. L'objectif de cette analyse étant de fournir des arguments supplémentaires incitatifs en vue d'une mise en œuvre de la solution *cross-protect* sur des routeurs commerciaux.

Chapitre 1

Une architecture FAN première génération

Ce chapitre est scindé en deux parties. Dans un premier temps, nous rappelons brièvement l'architecture orientée flot proposée par Roberts et. al., on s'y référera en tant que première génération de FAN [10, 98].
Par la suite, nous effectuons une comparaison entre le mécanisme Diffsev et le contrôle d'admission implicite par des simulations *ns*. L'objectif est de mettre en évidence la meilleure approche pour la protection des services en cas de surcharge [6].

1.1 Description de l'architecture FAN 1^{re}G

L'analyse du trafic IP nous enseigne que la modélisation à l'échelle flot est la plus appropriée et que la performance en charge normale est bonne ; par contre, il faut éviter qu'un phénomène de surcharge se produise.
Au lieu de compter sur l'impatience des utilisateurs ou des applications pour stabiliser les performances d'un lien congestionné, il serait plus judicieux de limiter proactivement la demande en appliquant un contrôle d'admission. Le critère d'admission d'un nouveau flot devrait être tel qu'en charge normale ($A < 0.9C$, par exemple), la probabilité de blocage soit négligeable, tandis qu'en surcharge ($A > C$), l'excédant de trafic soit rejeté, garantissant du coup un débit satisfaisant pour les flots admis et permettant de préserver l'efficacité globale du réseau.

La cohabitation des flots *streaming* et élastiques au niveau des mêmes liens est plus rentable pour le réseau car elle permet d'exploiter la synergie qui existe entre ces deux types de flots. Cette intégration repose sur les aspects suivants :

- Un marquage de paquets explicite pour distinguer les deux classes de service est effectué.
- Un traitement prioritaire est accordé aux paquets des flots *streaming*, ce qui permet de leur garantir de faibles délais et pertes. Ces flots *streaming* doivent alors être policés à un débit crête maximum.

– Les flots élastiques se partagent la capacité résiduelle (non utilisée par les flots *streaming*).

La synergie entre les deux classes se reflète dans leur performance, dans la mesure où les flots *streaming* voient un lien peu chargé et les flots élastiques utilisent toute la capacité disponible. Elle est due au fait que la qualité de service associée à chaque classe s'exprime à des échelles de temps différentes : à l'échelle du paquet pour les flots *streaming*, à l'échelle du flot pour les flots élastiques.

Une architecture orientée flot (FAN) repose sur les trois éléments présentés dans ce qui suit.

1.1.1 Identification des flots

Le contrôle d'admission préconisé est implicite, i.e. sans signalisation ni réservation de ressource. En effet, comme la majorité des flots IP sont très courts, il fallait éviter de retomber dans les travers de Intserv. Ainsi, les nouveaux flots sont détectés et identifiés "au vol".

Les flots sont identifiés à partir de certains champs des entêtes des paquets. Afin de délimiter les flots dans le temps, on peut associer aux champs des entêtes une durée d'inter-paquet maximum appelée *time out*. Une identification simple des flots reposerait sur les adresses IP source et destination. Un choix plus judicieux, comporterait les numéros de ports.

Idéalement, un flot pourrait être spécifié par les applications voire les utilisateurs en fonction de leurs besoins. Dans cette perspective, le champ *flow label* du protocole IPv6 sur 20 bits représente une solution bien adaptée à une architecture FAN. Il faut signaler, toutefois, que la sémantique originelle de ce champ est légèrement différente de celle qu'on envisage dans le cadre d'une architecture FAN [35]. En effet, le *flow label* avait été pensé pour des architectures telles que Intserv pour maintenir un état par flot. Dans ce contexte, il est nécessaire de s'assurer qu'un état donné est bien effacé de la mémoire avant de réutiliser le même *flow label*. Dans notre cas, l'état du flot se réduit à l'instant d'observation de son dernier paquet (outre son identifiant bien entendu) ; ce qui implique qu'il est tout à fait possible pour un utilisateur d'employer le même label pour des connexions successives ou parallèles pour qu'elles puissent être identifiées comme un seul flot par les routeurs.

On pourrait également envisager d'utiliser 2 bits (parmi les 20 bits) du *flow label* pour spécifier si l'identifiant du flot s'obtient en combinant le label avec l'adresse source, avec l'adresse destination, avec les deux adresses ou aucune d'elles. Ceci permettrait d'associer plusieurs connexions au niveau transport à un même flot. Par exemple, tous les éléments d'une page Web pourraient appartenir à un seul flot en fonction de l'adresse de destination.
Cette approche permettrait plus de souplesse dans l'identification des flots compte tenu de la diversité et de l'évolution des applications.

Typiquement, les flots seraient identifiés en appliquant une fonction de hachage sur les attributs d'entête sélectionnés. Il s'agit de trouver le bon compromis entre la

complexité du hachage et la probabilité de collision entre des flots distincts. Signalons, tout de même, qu'une collision n'a pas d'effets graves puisqu'elle n'induirait éventuellement qu'à l'acceptation de quelques flots par erreur, comme on le verra plus tard.

Une liste des flots en cours PFL (*Protected Flow List*) est maintenue, elle contient deux types d'informations : l'identifiant de chaque flot et le temps d'observation de son dernier paquet.

1.1.2 Contrôle d'admission

A chaque arrivée d'un paquet, le flot auquel il appartient est identifié. Si le nouveau paquet appartient à un flot déjà en cours (et donc présent dans la liste PFL), le paquet est transmis normalement. Quand un nouveau flot arrive (paquet avec un identifiant non présent dans la table), une décision d'admission pour le flot doit être prise en fonction de l'indicateur de congestion utilisé. Le paquet est donc soit transmis et le flot rajouté dans la liste, ou bien rejeté si le critère d'admission n'est pas rempli.

Comme les paquets des flots *streaming* bénéficient d'un traitement prioritaire, et sous réserve qu'ils constituent une minorité du trafic (ce qui est largement le cas actuellement), seule la QoS des flots élastiques est compromise en cas de surcharge. Ainsi, le critère d'admission est uniquement lié au débit des flots élastiques, les flots *streaming* sont rejetés dans les mêmes conditions.

Il s'avère qu'un tel critère doit rejeter un nouveau flot dont l'admission éventuelle conduirait à un débit instantané des flots en cours sur le lien inférieur à approximativement 1% de la capacité du lien [5]. En effet, 1 % de la capacité du lien constitue en général une valeur bien plus grande qu'un utilisateur considérerait comme valeur acceptable (pour la plus part des flots le débit de la ligne d'accès est inférieur à $0.01C$). Cependant, il est important de noter que la relaxation du seuil d'admission n'apporterait aucun gain ; en ce sens que la capacité du lien est utilisée (presque) entièrement en surcharge et la probabilité de blocage est approximativement $(A - C)/A$, et ce quelque soit le seuil d'admission. Il est donc préférable de choisir un seuil relativement élevé pour permettre aux flots qui n'ont pas de limitations de débits d'accès de compléter leur transfert rapidement.
Notons que si la charge due aux flots *streaming* est trop importante, leurs paquets risquent de subir une augmentation de délais ; un critère d'admission supplémentaire serait alors nécessaire.

La figure 1.1 illustre un scénario d'utilisation d'un tel mécanisme de contrôle d'admission dans un réseau au niveau d'un goulot d'étranglement.

1.1.3 Estimation de la bande passante disponible

Le critère d'admission est basé sur l'estimation de la bande passante disponible, elle correspond au débit qu'aurait un nouveau flot admis. Deux méthodes ont été

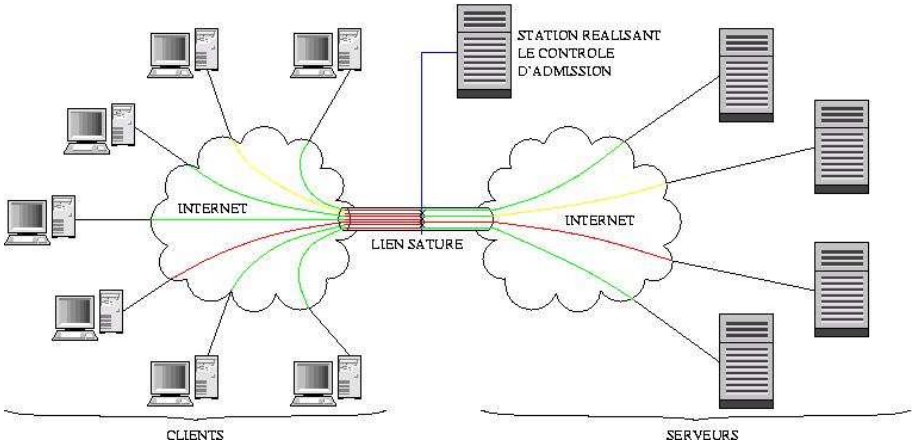


FIG. 1.1 – Réseau muni d'un contrôle d'admission au niveau d'un goulot d'étranglement

proposées pour effectuer cette mesure.

Méthode du TCP fantôme

La mesure de la bande passante se fait grâce à une connexion TCP fantôme permanente. Cette méthode est inspirée des travaux de Afek et. al. [2]. Le but du contrôle d'admission n'étant pas de garantir un débit strict par flot, l'estimation n'a pas besoin d'être extrêmement précise. La connexion fantôme envoie continuellement des paquets, elle est donc sujette au contrôle de congestion de TCP en fonction des pertes qu'elle subit au même titre que les autres connexions.
La bande passante est calculée en moyennant les débits en fonction des paquets acquittés sur des intervalles de temps avec un lissage exponentiel, i.e.

$$\text{Estimation}_n = \alpha \times \text{Estimation}_{n-1} + (1-\alpha)\frac{\tau_n}{\delta} \text{ où } 0 < \alpha < 1$$

δ est la longueur de l'intervalle de mesures (0.1 seconde), α est un coefficient qui permet de lisser les fluctuations instantanées du débit et τ_n est le nombre de bits acquittés sur cet intervalle. Le seuil d'admission peut être pris entre 0.5% et 2% de la capacité du lien (par exemple, 1%). Ainsi un nouveau flot est rejeté si le débit courant du TCP fantôme est inférieur à $0.01C$.

Même si cette solution a fait ses preuves en donnant lieu à des résultats satisfaisants sur une plate-forme expérimentale, elle présente l'inconvénient d'être active en injectant du trafic inutilement. De plus, elle est difficilement intégrable dans un réseau opérationnel puisqu'elle nécessiterait l'établissement d'une connexion entre deux routeurs, l'un agissant comme puits de trafic. Ceci imposerait au routeur d'envoyer des paquets ACK alors qu'il ne fonctionne habituellement pas au niveau TCP.

Méthode passive

Cette solution dite *Top N* consiste à mesurer les débits réalisés par les connexions en cours et en sélectionner les meilleurs comme indication de la bande passante disponible. En effet, la phase de *slow start* de TCP et les contraintes externes (débit d'accès, congestion en amont ...) limitent le débit de certaines connexions qui n'est donc pas représentatif de la bande passante disponible sur le lien, d'où le choix de N plus forts débits.

Cette technique tient compte d'un deuxième critère pour le classement des flots, outre leur débit, afin qu'ils puissent intégrer la liste *Top N*. Il s'agit de la notion d'activité d'un flot, c'est-à-dire le temps écoulé depuis l'émission de son dernier paquet. Ainsi, un flot ayant eu un bon débit sera progressivement déclassé dans la liste au fur et à mesure que son temps d'inactivité s'accroît, son débit n'étant plus représentatif de l'état courant du lien. Nous détaillons dans ce qui suit le fonctionnement de cet algorithme.

L'estimation s'effectue en trois étapes :

- Dans un premier temps, le débit instantané des flots est calculé tous les p paquets émis (ou bien sur des périodes temporelles fixes) en divisant le volume d'octets envoyés par la durée du cycle. Dans la liste des flots en cours, un compteur est utilisé à cet effet mémorisant le nombre d'octets émis par chaque flot. Soit r_i le débit instantané du flot i.
- A partir du débit instantané du flot i, une estimation à long terme R_i est faite via un lissage exponentiel :

$$R_i = \alpha R_i + (1 - \alpha) r_i \text{ où } 0 \leq \alpha \leq 1$$

  L'indice de classement du flot τ_i est déduit comme suit :

$$\tau_i = log(R_i) + t_i$$

  où t_i est l'instant d'observation du dernier paquet du flot i (extrait de la liste des flots).
  Le paramètre τ_i constitue le critère permettant aux flots d'intégrer la liste *Top N*. Les flots ayant les N plus élevés τ_i sont ceux présents dans la *Top N*. Ainsi, un flot i est rajouté à la liste à l'issue d'une mesure de son τ_i si $\tau_i > min$, où min est la plus petite valeur de τ au sein de la liste (dans le cas où le flot i est déjà dans la liste, une simple mise à jour est effectuée).
  On remarque que l'impact des débits instantanés mesurés diminue exponentiellement avec le temps, ce qui permet de déclasser les flots au fur et à mesure que leur degré d'activité baisse. L'estimation fournie par la $TopN$ reflète alors la bande passante disponible à l'instant courant.
- Pour calculer l'estimateur de la bande passante disponible E à partir de la *Top N*, deux méthodes sont possibles :
  Calcul de la moyenne : Il s'agit tout simplement de calculer périodiquement

la moyenne des débits des flots composant la *Top N* :

$$E = \frac{1}{N} \sum_{i \in Top\ N} R_i$$

Lissage exponentiel : A chaque insertion d'un flot dans la liste avec un débit R_i, l'estimateur est recalculé :

$$E = \beta E + (1 - \beta) R_i \text{ où } 0 \leq \beta \leq 1$$

C'est cet estimateur E qui représente le critère d'admission des nouveaux flots, il est comparé avec le seuil. Notons que E est bien calculé à partir des débits R_i mais c'est le paramètre τ_i qui détermine les flots qui sont éligibles dans la *Top N*.

Cette méthode a également été testée par simulations *ns* et expérimentations, elle s'est avérée efficace. Le choix d'une valeur de N égale à 5 ou 10 est raisonnable. Signalons que cette approche est vulnérable en cas de comportement agressif des flots, notamment ceux qui ne réagissent pas à la congestion comme le font les connexions TCP standards (les flots UDP à fort débit pourraient biaiser la mesure, par exemple).

1.2 Mécanismes de protection du trafic en surcharge

Dans cette section, nous nous intéressons à la performance des flots élastiques ; ces travaux font suite à une étude effectuée pour le compte de la branche opérationnelle de France Télécom dans l'optique d'augmenter le taux d'utilisation du réseau RBCI (Réseau *Backbone* et de Collecte Internet) qui est actuellement inférieur à 50% (ainsi, en cas de panne, le reroutage du trafic ne cause pas de surcharge).

Plus précisément, nous considèrons deux classes de trafic en l'occurrence *premium* (PR) et *best effort* (BE) et nous comparons deux solutions de protection de la classe *premium* en situation de surcharge (qui pourrait être provoquée par une panne) via des simulations *ns*. Dans un cas, nous appliquons le modèle de service Diffserv et dans l'autre cas, nous appliquons un contrôle d'admission sélectif, dont le principe est décrit dans la section 1.2.2.
Nous verrons plus loin que les conséquences de la protection du trafic *premium* différent avec les deux solutions.
On distingue les deux critères de différenciation suivants :

- La transparence : ce critère indique dans quelle mesure le débit réalisé n'est contraint que par des limitations externes telles que le débit d'accès. Une transparence élevée est synonyme du fait que le réseau ne contraint le débit des flots qu'en faible mesure.
- L'accessibilité : elle représente la proportion des flots qui vont au bout de leur transfert (i.e., ceux qui ne sont pas bloqués ni interrompus).

1.2.1 Protection du trafic avec Diffserv

Simulations

La topologie simulée est celle représentée par la figure 1.2. Des flots TCP sont générés par 10 sources de trafic suivant un processus de Poisson dont l'intensité produit une charge cible au niveau du lien *bottleneck* de capacité 10 Mb/s, équipé d'un buffer d'une taille de 100 paquets. La distribution de la taille des flots TCP est de type Pareto tronquée avec un paramètre de forme égal à 1.5, une moyenne de 25 paquets et une taille maximum de 1 Mo (sachant que la taille des paquets est de 1 Ko). Ce choix permet de reproduire une distribution de taille de flot à queue lente telle qu'observée dans la réalité [34]. Une loi Pareto tronquée a une variance finie, elle converge plus vite vers sa moyenne.
Les charges (ρ) offertes sont 0.9 et 1.2 avec une proportion de 30% due au trafic PR. La durée des simulations est de 300 secondes.

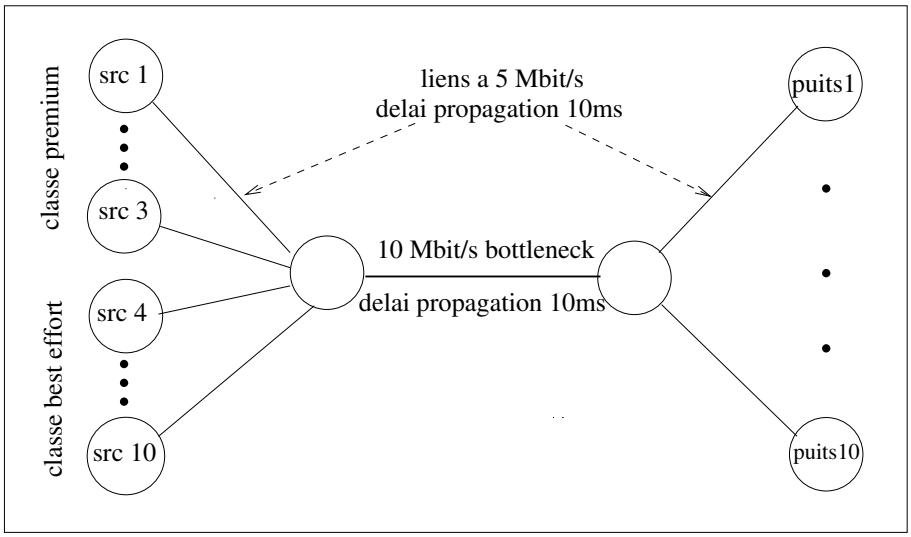


FIG. 1.2 – Topologie simulée

Afin d'accorder un traitement prioritaire aux flots PR dans le cadre de Diffserv, trois mécanismes ont été utilisés :

- *Priority Queueing* (PQ) : les paquets des flots PR sont traités avant ceux des flots BE avec une priorité stricte non préemptive.
- *Weighted Fair Queueing* (WFQ) : la file d'attente du lien *bottleneck* est gérée avec une discipline de service WFQ, où les poids affectés aux classes PR et BE sont respectivement 70% et 30%.
- *Weighted Random Early Detection* (WRED) : ce mécanisme de type AQM (*Active Queue Management*) est utilisé avec une file FIFO. Il permet de rejeter les paquets quand la taille de la file se trouve entre deux seuils $s1$ et $s2$ avec une certaine probabilité. Lorsque la taille dépasse $s2$, tout nouveau paquet est rejeté. Notons que la taille de la file est calculée selon un lissage exponentiel.

Les valeurs des seuils utilisés sont 20 et 40 paquets pour les flots BE, 40 et 80 paquets pour les flots PR. Le paramètre de probabilité de perte est 0.1 pour les flots BE et 0.02 pour les flots PR, la probabilité de perte effective croît linéairement avec la taille de la file.

Ces mécanismes correspondent à des schémas standards mis en œuvre actuellement dans les routeurs afin de réaliser une différenciation de service.

La figure 1.3 illustre les débits réalisés par les flots sous des charge de 0.9 et 1.2 avec une simple file FIFO et sans appliquer Diffserv. La représentation associée est en forme de diagrammes en rectangle. Un rectangle représente un flot, sa hauteur est proportionnelle au débit du flot et la projection de ses extrémités sur l'axe des abscisses les instants de début et fin du flot en question. L'emplacement des rectangles sur l'axe des ordonnées est arbitraire. Le code couleur utilisé indique un débit d'autant plus élevé que la couleur est claire (jaune pour un débit supérieur à 400 Kb/s et rouge pour un débit inférieur à 20 Kb/s).
La figure confirme la relation entre la demande de trafic et la performance exprimée dans la section 2.5 de l'introduction. En effet, pour une charge inférieure à 1, le débit obtenu par les flots est satisfaisant (rectangles jaunes). En surcharge, la performance est très mauvaise, puisque les rectangles sont colorés en rouge, étirés (indiquant un rallongement de la durée des flots) et d'une hauteur très petite (faible débit).

La figure 1.4 est relative au scénario Diffserv avec PQ, en surcharge. Elle indique que le débit obtenu par les flots PR est excellent (500 Kb/s en moyenne) au détriment des flots BE qui sont très mal lotis. Un comportement similaire est observé avec WFQ et WRED.

La figure 1.5 trace le nombre de flots PR et BE pour les trois scénarios de Diffserv. On remarque que, indépendamment de la discipline de service appliquée, le nombre de flots PR est faible et stable, il vaut 6 en moyenne. En revanche, le nombre de flots BE croît régulièrement au fil de la simulation, puisque le taux d'arrivée de ces flots est supérieur à leur taux de départ.
Une minorité du trafic est donc protégée en cas de surcharge, les flots BE sont pénalisés outre mesure. En pratique, le nombre de flots se stabilise suite au phénomène d'impatience qui sera abordé dans la section 1.2.2.

Des résultats d'expérimentations de Diffserv en surcharge sont présentés dans l'annexe 1. Ils confirment les observations obtenues par simulation.

Nous avons également simulé le même scénario de trafic en charge normale (inférieure à 1). Le tableau 1.1 indique le débit moyen réalisé par les flots élastiques dont la taille est supérieure à 50 paquets lorsque la charge offerte ρ est égale à 0.7. On constate que le débit obtenu par les deux classes est satisfaisant, sachant que le débit maximum atteignable par les flots est 2.6 Mb/s (en divisant la taille maximum de la fenêtre TCP par le RTT). La différence de débit n'est donc pas importante quelle que soit la discipline de service utilisée (même si le débit le plus élevé pour la classe PR est à mettre à l'actif de la discipline PQ, comme prévu).

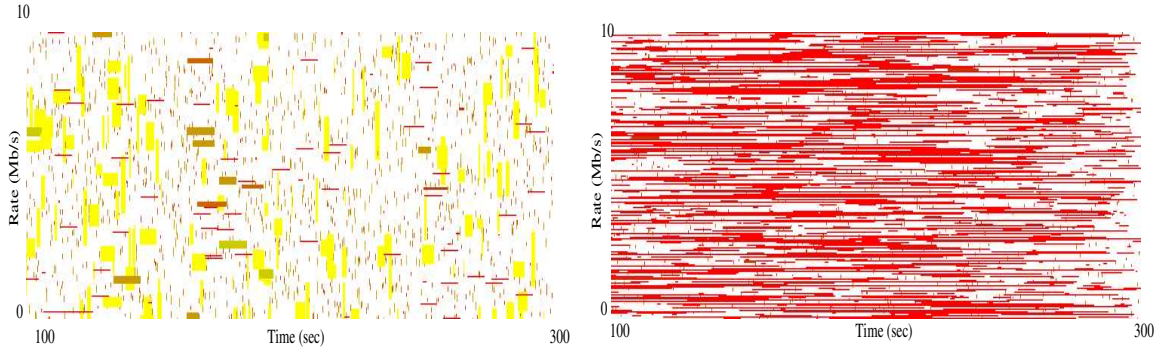


FIG. 1.3 – Diagrammes en rectangle sans Diffserv, ρ=0.9 (gauche) et ρ=1.2 (droite)

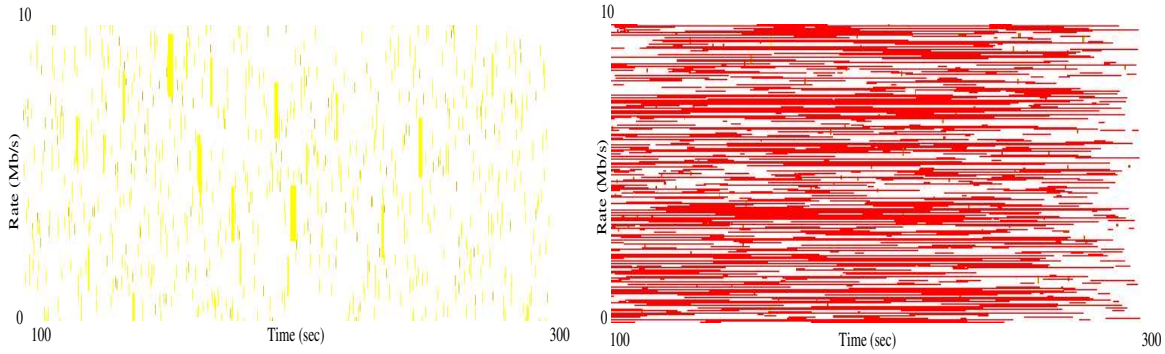


FIG. 1.4 – Diagrammes en rectangle, scénario PQ, ρ=1.2, flots PR (gauche) et flots BE (droite)

1.2.2 Protection du trafic avec le contrôle d'admission sélectif

Simulations

Nous avons simulé le même profil de trafic décrit dans la section 1.2.1 en appliquant un contrôle d'admission sélectif avec une charge de 1.2. Les seuils d'admission choisis étaient de 1% et de 4% de la capacité du lien pour les flots PR et BE respectivement. Ainsi, si la bande passante disponible est comprise entre 100 Kb/s et 400 Kb/s, seuls les flots PR sont admis, en deçà, tous les flots sont bloqués et au

Mécanisme	Débit PR (Mb/s)	Débit BE (Mb/s)
PQ	1.26	1.07
WFQ	1.23	1.07
WRED	1.18	1.08

TAB. 1.1 – Débit des flots élastiques, simulations Diffserv, $\rho = 0.7$

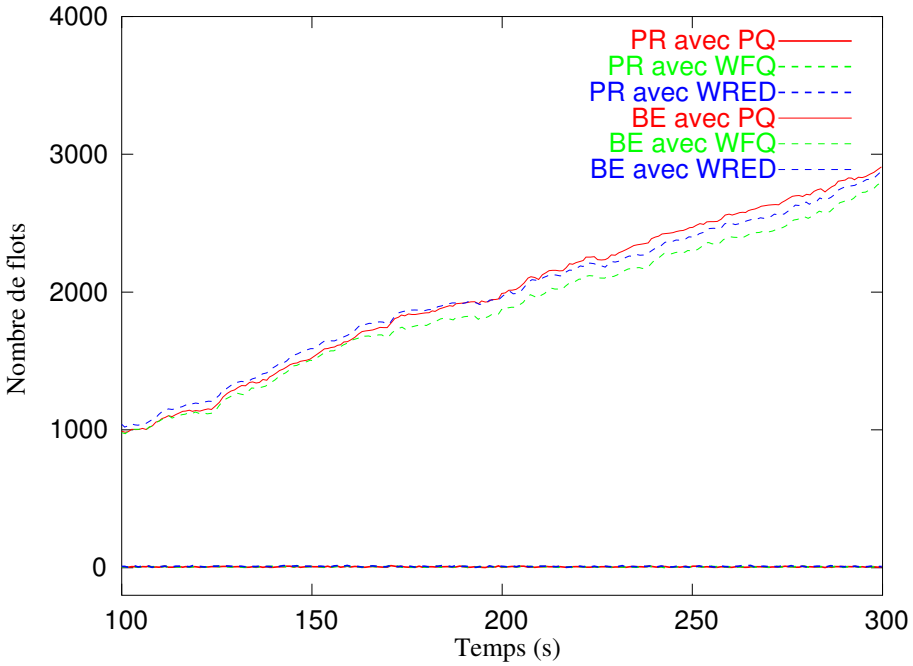


FIG. 1.5 – Nombre de flots des classes PR et BE vs. temps, simulations Diffserv

delà, tous les flots sont admis. La bande passante disponible étant mesurée avec la méthode passive *Top N* présentée dans la section 1.1.3 que nous avons implémenté dans *ns*.

En outre, dans le souci de reproduire un scénario réaliste, où le nombre de flots ne croît pas indéfiniment (exagérant la dégradation de performance des flots BE), le phénomène d'impatience est pris en compte. Nous avons repris le modèle d'impatience analysé dans [21] qui considère qu'un flot est abandonné dès que son temps de transfert dépasse un seuil prédéterminé en fonction de sa taille. Ce seuil dépend de deux paramètres : un temps de tolérance τ et un débit minimum espéré θ.
Ainsi, un flot de taille σ est interrompu après t secondes si $t > \tau + \sigma/\theta$. Les valeurs choisies pour ces paramètres sont $\theta = 20$ Kb/s et $\tau = 10$ secondes.

Les résultats comparatifs obtenus avec l'option PQ de la solution Diffserv et le contrôle d'admission ainsi défini sont consignés dans le tableau 1.2. On se limite au scénario PQ de Diffserv car on a déjà vu dans la section 1.2.1 que les autres mécanismes (WFQ et WRED) offrent des performances similaires.
Cette comparaison concerne les débits réalisés ainsi que le taux de blocage ou d'abandons. On constate que 35% des flots BE sont interrompus avec Diffserv. En considérant que ces flots induisent un gaspillage de ressource du fait qu'ils ne vont pas à leur terme, 23% de la capacité du lien est ainsi perdue. D'un autre côté, le contrôle d'admission, en bloquant 28% des flots, évite que ceux qui sont admis ne soient interrompus. Notons que seuls les flots BE sont bloqués, la différenciation en faveur des flots PR dans ce cas est faite au niveau de l'accès au réseau. En effet, tous les

flots admis obtiennent de bons débits (qu'ils relèvent de la classe PR ou de la classe BE). Le débit moyen de la totalité des flots est limité par la phase *slow start* de TCP ; en se focalisant sur les flots dont la taille est plus grande que 50 paquets (les "éléphants"), le débit moyen est proche du seuil d'admission (400 Kb/s).

On notera que les flots PR obtiennent un meilleur débit avec Diffserv. Cependant, cet avantage serait vite atténué sur des liens à grande capacité où les seuils de 1% et 4% sont typiquement plus importants que les débits d'accès des flots, ce qui impliquerait que les flots en question se contenteraient de réaliser leurs débits d'accès.

	PQ		AC	
Débit :	PR	BE	PR	BE
- tous les flots (Kb/s)	502	61	201	183
- flots éléphants (Kb/s)	1226	45	350	306
Abandons/Blocage	0%	35%	0%	28%

TAB. 1.2 – Comparaison entre le scénario PQ de Diffserv et le contrôle d'admission (AC), simulation avec impatience, ρ=1.2

La figure 1.6 trace l'évolution du nombre de flots PR et BE avec Diffserv (scénario PQ) et le contrôle d'admission. Par rapport à la figure 1.5, on constate que le nombre de flots BE se stabilise avec Diffserv grâce à l'abandon de certains d'entre eux. Néanmoins, ce nombre reste plus élevé que celui obtenu avec le contrôle d'admission indiquant que le débit de la classe BE est faible (61 Kb/s en moyenne). Le nombre de flots BE se stabilise au tour de 500 avec Diffserv, une valeur compatible avec le seuil d'impatience choisi (20 Kb/s). Un seuil plus bas aurait donné un nombre de flots plus élevé, un taux d'abandon plus faible et donc un débit plus faible. Naturellement, avec le contrôle d'admission le nombre de flots BE est plus grand que le nombre de flots PR, car ils représentent 70% de la charge offerte et non pas parce qu'ils sont moins bien traités.

Enfin, signalons que les deux mécanismes, à savoir le contrôle d'admission et Diffserv avec PQ, induisent une utilisation totale du lien à 99%. Cependant, la différence est que seule 77% de cette utilisation est due au trafic utile (non interrompu) avec Diffserv alors qu'elle est de 100% dans le cas du contrôle d'admission d'où l'intérêt d'un blocage proactif des flots.

Des résultats d'expérimentations du contrôle d'admission sélectif, qui corroborent ceux obtenus avec les simulations, sont présentés dans l'annexe 1.

1.3 Conclusion

Nous avons rappelé dans ce chapitre les principaux résultats d'analyse du trafic IP obtenus grâce aux travaux de recherche précédemment entrepris au sein de

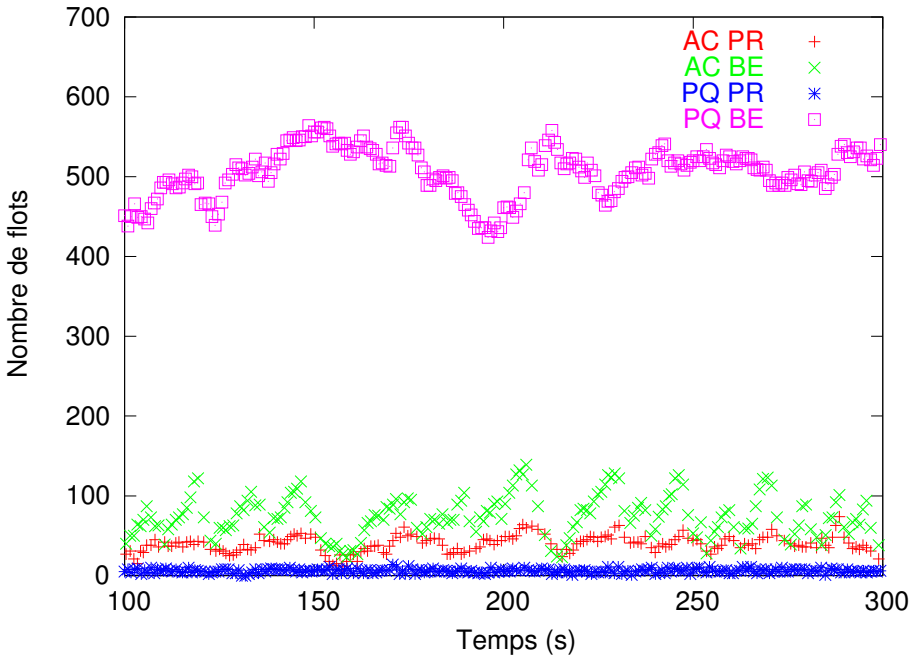


FIG. 1.6 – Evolution du nombre de flots PR et BE avec le scénario PQ de Diffserv et le contrôle d'admission, simulation avec impatience, ρ=1.2

l'équipe et qui ont débouché sur la première génération de FAN. Il en ressort qu'une modélisation à l'échelle flot est appropriée pour l'analyse de la performance des flots élastiques. Elle permet de prendre en compte la nature stochastique du trafic. Les flots peuvent être classés en deux grandes catégories : élastique et *streaming*. Le contrôle d'admission implicite est indispensable pour préserver la qualité de service et pour assurer une utilisation efficace en cas de surcharge occasionnelle.

Dans un deuxième temps, on a pu voir que la différenciation apportée par Diffserv est quasiment imperceptible en régime nominal. En surcharge, le trafic *premium* est protégé au détriment d'un certain gaspillage de ressources dû à la gestion du trafic *best effort*. En effet, les flots *best effort* obtiennent un débit très faible, ce qui conduit à l'interruption de certains d'entre eux. Ainsi, un degré de transparence de quasiment 100% est assuré pour les flots de la classe *premium* (notamment dans un contexte opérationnel où les capacités des liens sont nettement plus importantes que les débits d'accès des flots) alors que les flots *best effort* ont une accessibilité limitée. En revanche, les flots *premium* ne sont pas interrompus. Le taux d'abandon dépend de plusieurs facteurs que l'on ne maîtrise pas (le degré d'impatience des utilisateurs, par exemple), ce qui fait qu'il est impossible de prédire la performance.

Les comparaisons effectuées avec le contrôle d'admission par le biais de simulations ont confirmé cette tendance. Le contrôle d'admission sélectif permet de privilégier les flots prioritaires en terme d'accessibilité (100% des flots PR étaient admis) au réseau en cas de congestion. En choisissant les seuils d'admission convenablement,

il est également possible de garantir une transparence proche de 100% pour tous les flots admis.

En dépit de ses avantages, la première génération de FAN présente quelques insuffisances :

- Dans un contexte de réseau multiservices, où les flots *streaming* font l'objet d'un contrôle d'admission, les conditions d'admission supposent un débit crête maximum. Ce débit crête doit être contrôlé par les routeurs de bord, typiquement. Cela constitue donc une contrainte rigide ; d'autant plus que l'écrêtage s'appliquerait indépendamment des conditions de charge, empêchant ainsi certains flots *streaming* au débit crête important de bénéficier des plages horaires à faible charge (par exemple, un transfert à haut débit entre serveurs durant la nuit).
- L'équité dans le partage de la bande passante est tributaire de la collaboration des utilisateurs via le protocole TCP. De plus, ceci peut constituer un frein au développement d'autres protocoles de transport mieux adaptés aux transferts à haut débit.
- Un marquage explicit des paquets est nécessaire afin d'identifier les paquets des flots *streaming* puisqu'il faut les traiter en priorité.

Dans le prochain chapitre, nous présentons une évolution de l'architecture FAN (FAN deuxième génération) basée sur un concept original de différenciation implicite qui permet de pallier à ces défauts.

Chapitre 2

Cross-protect : architecture FAN deuxième génération

Les insuffisances de la première génération de *flow-aware networking* (FAN) présentées à la fin du chapitre précédent nous ont conduit à proposer une seconde génération de FAN qui permet de parer à ces problèmes. Cette deuxième génération a été baptisée *cross-protect*.
Cette architecture s'inscrit dans la continuité de la première génération de FAN en reprenant ses principes fondamentaux que sont la notion de flot et le contrôle d'admission implicite, i.e. sans signalisation ni réservation de ressources.
L'idée est d'exploiter la nature même des deux types de flots à savoir *streaming* et élastique et leurs propriétés intrinsèques afin d'effectuer une différenciation implicite. En effet, les paquets des flots *streaming* sont espacés en fonction de leur débit d'émission, les flots élastiques de leur côté envoient des paquets en rafales.

L'architecture *cross-protect* associe le contrôle d'admission implicite et une discipline de service de type *fair queueing* au sein des files d'attente des routeurs cœurs, ce qui lui permet de tirer profit des avantages qu'ils procurent.

L'appellation *cross-protect* provient du fait qu'on ait une protection mutuelle entre, d'une part, le contrôle d'admission implicite et, d'autre part, le *fair queueing* par flot. En effet, le contrôle d'admission permet d'assurer l'extensibilité de l'ordonnancement *fair queueing*, en limitant le nombre de flots à traiter, alors que ce dernier lui fournit les mesures de congestion nécessaires. De plus, les deux types de flots, à savoir les flots élastiques et *streaming*, obtiennent des performances satisfaisantes sans se gêner mutuellement pour autant.

Le principal avantage de cette architecture réside dans la différenciation implicite qu'elle apporte entre les flots élastiques et *streaming* sans avoir à exercer un marquage explicit de leurs paquets. Ceci se traduit par des délais et des pertes négligeables pour les applications temps-réel et par un débit satisfaisant pour les transferts TCP. Une conséquence directe de cette différenciation implicite est le maintien de la même interface utilisateur/réseau qu'on a actuellement dans un réseau *best effort* vu qu'aucun marquage n'est exigé.

Un autre apport de *cross-protect* est l'équité du partage de la bande passante, qui est garantie indépendamment de la coopération des utilisateurs, ce qui décharge les applications de la contrainte d'être TCP *friendly*.

2.1 Architecture d'un routeur *cross-protect*

Un routeur *cross-protect* tel que représenté par la figure 2.1 combine l'utilisation du contrôle d'admission implicite et du *fair queueing* par flot. Le contrôle d'admission est effectué au niveau des interfaces d'entrée tandis que le scheduling de type *fair queueing* est appliqué sur les interfaces de sortie suivant un algorithme particulier nommé PFQ (*Priority Fair Queueing*) auquel une section de ce chapitre est consacrée.
Les opérations d'identification des flots et les décisions d'acheminement des paquets qui arrivent sont exécutées au niveau des interfaces d'entrée. En sortie, les paquets sont envoyés sur le lien suivant l'ordre dicté par l'algorithme de *fair queueing* utilisé. Dans les prochains paragraphes, nous décrirons de façon plus détaillée ces différentes étapes.

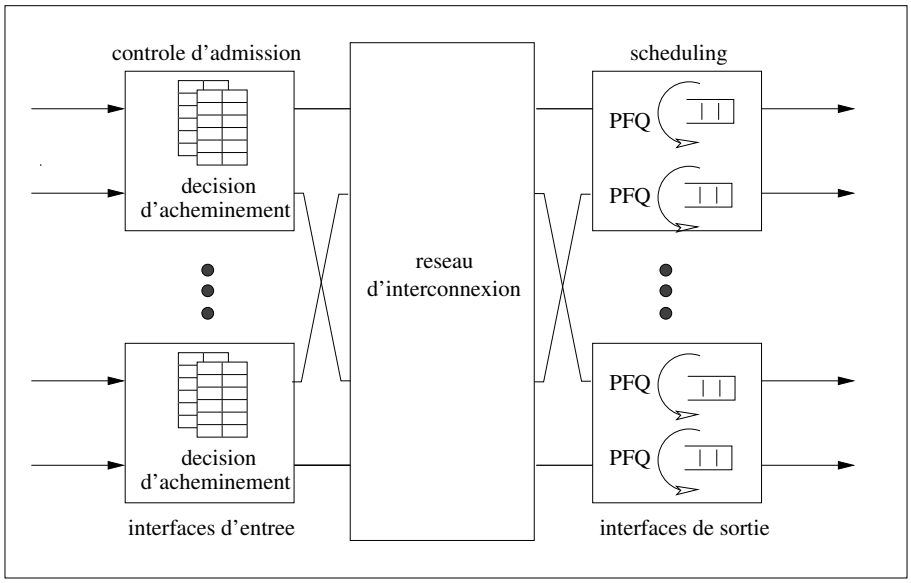


FIG. 2.1 – Un routeur *cross-protect*

2.1.1 Identification des flots

L'opération d'identification des flots se fait de la même manière que dans l'architecture FAN première génération telle que décrite dans le paragraphe 1.1.1. Les flots sont donc reconnus au vol en fonction de certains attributs de leurs entêtes IP et TCP (ou UDP). De plus, un flot est limité dans le temps, au sens où les arrivées

de ses paquets sont espacées d'un temps maximum appelé *time out*.
Un flot est identifié au niveau d'une interface réseau donnée et admet autant d'instances dans un réseau qu'il traverse d'interfaces le long de son chemin.

2.1.2 Acheminement des paquets

La fonction première d'un routeur IP classique est d'aiguillier les paquets vers les liens de sortie qui leur permettent d'atteindre leurs destinations. Actuellement, les décisions d'acheminement ne tiennent pas compte de l'état de congestion des liens de sortie.

Un routeur *cross-protect* réalise quelques tâches supplémentaires lors de la prise de décisions d'acheminement des paquets. Ainsi, la transmission des paquets arrivant n'est plus systématique. En cas de congestion sur un lien de sortie, les paquets qui doivent y transiter sont rejetés. Pour ce faire, une structure de donnée appelée PFL (*Protected Flow List*) qui contient les flots en cours est maintenue.

Comme dans le cadre de l'arcitecture FAN 1^{re}G, une entrée dans la PFL consiste en deux champs seulement. Le premier est l'identifiant du flot et le second représente le temps d'observation du dernier paquet du flot. Les flots sont rajoutés à la PFL selon les modalités qu'on décrira plus loin. Ils sont effacés s'ils n'envoient aucun paquet pendant la durée du *time out*, ceci est détecté grâce au temps d'observation de leur dernier paquet.

L'organigramme de la figure 2.2 résume les différentes actions qui font suite à l'arrivée d'un paquet au niveau d'une des interfaces d'entrée d'un routeur *cross-protect*. A chaque arrivée d'un nouveau paquet, le flot auquel il appartient est identifié. S'il s'agit d'un flot déjà en cours, i.e. présent dans la PFL, le temps de passage du dernier paquet du flot en question est mis à jour et le paquet est transmis vers l'interface de sortie correspondante. En revanche, si le paquet appartient à un nouveau flot i.e. non présent dans la PFL, il est soumis à la décision du contrôle d'admission qui dépend de l'état de congestion du lien de sortie associée. Si l'indicateur de congestion utilisé (la nature de cet indicateur est discutée plus tard) signale que le lien est saturé, alors le paquet est rejeté faisant office de notification implicite de blocage pour le nouveau flot. Autrement (lien non congestionné), le paquet est acheminé vers la bonne interface de sortie et le flot est rajouté à la liste des flots protégés. Comme on peut le constater, le blocage ne concerne que les nouveaux flots, tous ceux qui ont déjà été entamés devraient aller au terme de leur transfert bénéficiant du statut de protection accordé par la PFL. Notons que le critère d'admission peut dépendre de certains attributs de l'entête IP des paquets déterminant une classe de service si un contrôle d'admission sélecttif est appliqué.

Comme on le verra au cours du chapitre 5, le nombre de flots en cours sur des liens à grande capacité peut atteindre quelque centaines de milliers. Pour des raisons de capacité mémoire, la liste des flots protégés est dimensionnée pour contenir un nombre de flots limité. La taille de la PFL devrait garantir une faible probabilité de saturation. Néanmoins, la saturation de la liste ne constitue pas un événement

dramatique. En effet, dans ce cas, les nouveaux flots ne sont pas rajoutés à la PFL mais leurs paquets sont transmis, sauf en cas de congestion. Le seul désagrément causé est qu'ils n'acquièrent pas le statut de flots protégés dès l'envoi de leur premier paquet. Ceci n'est préjudiciable qu'en situation de congestion où ils seraient interrompus.
Il serait également possible d'insérer les nouveaux flots dans la PFL avec une probabilité p qui pourrait dépendre de la classe de service . Si la valeur de p est relativement faible (0.1 par exemple) alors la majorité des flots courts ne sont pas rajoutés à la liste, tandis que les flots longs le sont avec une très forte probabilité après l'émission de leurs premiers paquets. Cette possibilité sera analysée dans le chapitre 6.
Afin d'optimiser la gestion de la PFL et pour des raisons d'extensibilité, il serait préférable d'avoir plusieurs listes par interface d'entrée (par exemple, une liste par port de sortie).

Notons enfin que des constructeurs comme Caspian [112] ont déjà développé des produits qui maintiennent une liste des flots accommodant jusqu'à 6 millions d'entrées et capable de gérer un taux d'arrivée de 2 millions de flots par seconde ; ce qui prouve la faisabilité d'une architecture orientée flot.

2.1.3 Ordonnancement des paquets

Une fois acheminés, les paquets sont stockés dans la file d'attente de sortie correspondante en attendant d'être émis sur le lien. L'ordre de traitement de ces paquets est déterminé par le scheduler.
Au sein d'un routeur *cross-protect*, c'est l'algorithme PFQ (*Priority Fair Queueing*) qui est en charge de l'ordonnancement. PFQ est inspiré d'un algorithme existant appelé SFQ (*Start-time Fair Queueing*) [51].

En réalité, l'objectif est d'utiliser un algorithme de type *fair queueing* afin de garantir un partage équitable de la bande passante qui se passe de la coopération des utilisateurs. Plusieurs algorithmes de *fair queueing* ont été proposés dans la littérature. Nous passons en revue certains d'entre eux dans le chapitre 4.
Le choix s'est porté, initialement, sur SFQ en raison du bon degré d'équité qu'il assure, ainsi que pour sa faible complexité par rapport à WFQ (*Weighted Fair Queueing*) ou SCFQ (*Self Clocked Fair Queueing*) (voir section 4.2.2). Dans la suite de ce chapitre, nous présenterons PDRR (*Priority Deficit Round Robin*) qui lui est basé sur DRR (*Deficit Round Robin*), un autre algorithme encore plus simple réalisant un partage équitable entre les flots.

Cependant, SFQ n'est pas adapté pour la gestion des flots *streaming*, car il peut causer des délais de paquets et une gigue relativement importants, puisqu'il ne met en place aucun mécanisme de priorité. C'est pourquoi, nous l'avons adapté en proposant PFQ qui permet de garantir un faible délai pour les paquets des flots dont le débit est inférieur au débit équitable en leur accordant un traitement prioritaire au sein de la file d'attente. Ainsi, en maintenant le débit équitable suffisamment élevé,

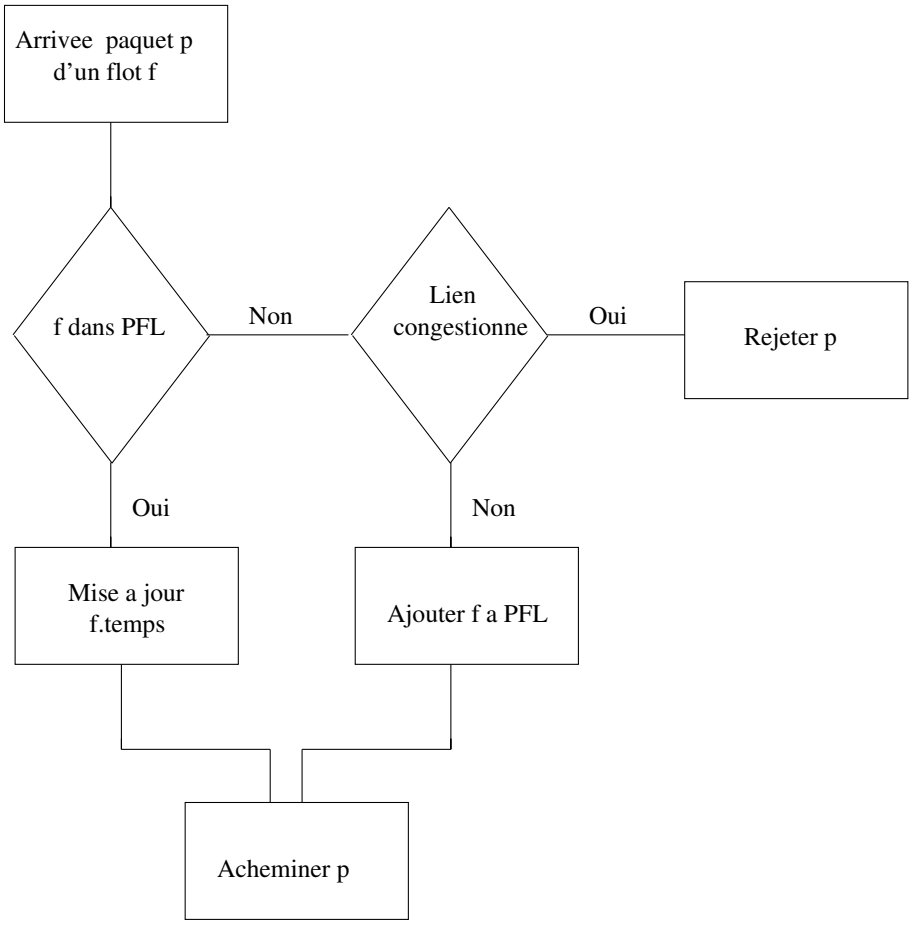


FIG. 2.2 – Opérations exécutées à l'arrivée d'un paquet

par le contrôle d'admission, la plupart des flots *streaming* verraient leurs paquets traités en priorité.
Au contraire, les paquets des flots élastiques, au sens où ils peuvent atteindre le débit équitable, sont *backlogged*. Ils ne sont donc pas insérés dans la file prioritaire. Le maintien d'un débit équitable au dessus d'un seuil satisfaisant leur assure une bonne qualité de service. Bien entendu, comme aucun marquage n'est effectué, certains paquets appartenant aux flots élastiques (TCP) peuvent également bénéficier d'un traitement prioritaire, alors que cela n'est pas nécessaire. Néanmoins, dans la mesure où ces paquets proviendraient de flots dont le débit est inférieur au débit équitable courant, ils ne gênent pas les autres flots élastiques.
Dans ce qui suit, nous utiliserons la terminologie flots *bottlenecked* pour désigner des flots dont le débit à l'entrée de l'ordonnanceur est supérieur au débit équitable. Ils réalisent alors le débit équitable en sortie du scheduler. Nous appellerons aussi les flots dont le débit est inférieur au débit équitable les flots *non bottlenecked*. Nous reviendrons sur cette distinction au cours du chapitre 5. Comme le débit équitable varie dans le temps, certains flots peuvent osciller entre les catégories *bottlenecked*

et *non bottlenecked*.

On considère que PFQ réalise un partage max-min avec un débit équitable θ. On rappelle qu'un partage est max-min, comme schématisé dans la figure 2.3, avec un débit équitable (*fair rate*) θ, si le débit qu'obtient un flot dont le débit crête est p_i est le minimum entre θ et p_i et que la somme des débits réalisés (r_i) par tous les flots est le minimum entre la capacité du lien (C) et la somme des débits crêtes, i.e.

$$r_i = min(\theta, p_i), \quad \forall i \text{ et } \sum_i r_i = min(C, \sum_i p_i)$$

Un partage de type max-min au niveau de chaque lien conduit à un partage max-min au niveau du réseau [13], ce qui est un objectif de partage approprié pour les flots élastiques [54].

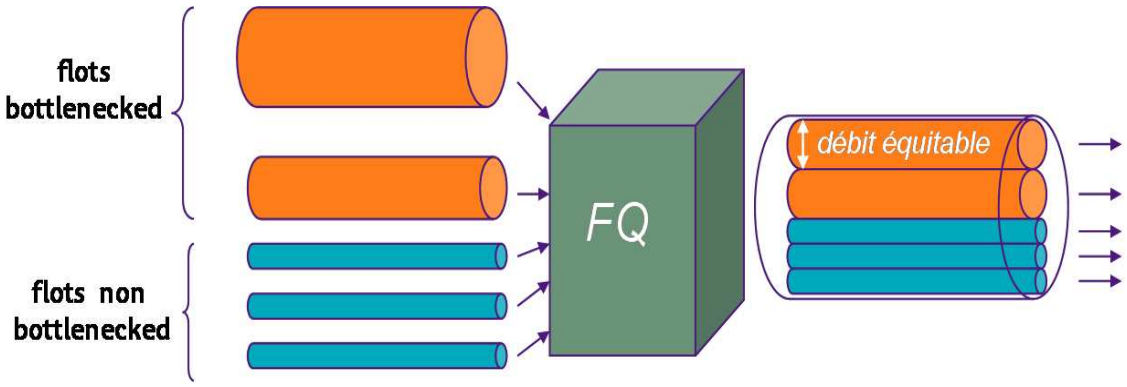


FIG. 2.3 – Partage max-min via un scheduler *fair queueing* (FQ)

2.2 Algorithme PFQ

Dans cette section, nous décrivons de manière plus détaillée les fonctions de l'algorithme PFQ.

2.2.1 Structures de données

L'algorithme PFQ fait partie de la catégorie des algorithmes de *fair queueing* basés sur la notion d'estampille temporelle. Son fonctionnement repose principalement sur l'utilisation de deux structures de données : la file des paquets et la liste des flots actifs.

File des paquets

Cette structure est de type PIFO (*Push In First Out*), où les paquets sont insérés suivant l'ordre croissant de leurs estampilles temporelles. Le prochain paquet à extraire est donc celui dont l'estampille est la plus faible (en cas d'égalité, c'est

l'ordre d'arrivée qui détermine l'ordre d'insertion). Un pointeur de priorité P est également maintenu, il permet d'insérer les paquets dans la partie prioritaire de la file. La partie prioritaire est gérée en FIFO (*First In First Out*).
D'un point de vue implémentation, chaque flot aurait sa propore file FIFO de paquets. Le tri en fonction des estampilles temporelles ne concerne que les premiers paquets de chaque flot qui constituent donc l'échéancier. Au fur et à mesure que les paquets, appartenant à un flot donné, sont envoyés à partir de l'échéancier, le paquet qui se trouve en tête de la file du flot en question est rajouté à l'échéancier en fonction de son estampille temporelle en vue d'une prochaine transmission.

Liste des flots actifs

La seconde structure est appelée liste des flots actifs (AFL). Elle maintient un état par flot qui a (ou a récemment eu dans un sens qu'on précisera dans ce qui suit) un paquet dans la file PIFO. Une réalisation possible de la liste des flots actifs est montrée dans la figure 2.4. Chaque entrée contient les champs suivants :

- identificateur du flot : représente l'identifiant du flot actif.
- *backlog* : indique la taille de la file du flot en octets. Ce champ permet de rejeter les paquets des files les plus longues.
- *finish tag* : estampille temporelle qui sera associée au prochain paquet du flot en question.
- *bytes* : compteur du nombre d'octets insérés en priorité par le flot en question.
- liste chaînée des paquets : ce champ consiste en deux pointeurs, l'un sur la tête de la file (premier paquet du flot), il est utilisé lors de l'extraction du prochain paquet à traiter, et l'autre sur la fin de la file (dernier paquet du flot) qui sert pour l'insertion de nouveaux paquets.

identificateur de flot	backlog	finish tag	bytes	liste chainee de paquets
F	F.Queue	F.finishTag	F.bytes	F.FIFO

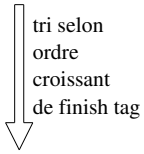


FIG. 2.4 – Structure de la liste des flots actifs avec l'algorithme PFQ

2.2.2 Pseudo-code

Les traitements effectués par l'algorithme PFQ sont relatifs à la gestion de deux types d'événements : l'arrivée d'un nouveau paquet et la fin de service d'un paquet. Dans les deux pseudo-codes associés que l'on va présenter, L désigne la taille du

paquet et F le flot auquel il appartient. Pour les besoins de l'algorithme, une variable globale appelée temps virtuel est maintenue, elle est égale à l'estampille temporelle du paquet en cours de service.

Pseudo-code à l'arrivée des paquets

```
1. si (PIFO saturée) alors rejecter le paquet en tête de la file de flot la plus longue
2. si (F ∈ AFL)
3.     F.backlog += L
4.     si (F.bytes ≥ MTU)
5.         insérer {paquet,F.finishTag} dans PIFO
6.     sinon
7.         insérer {paquet, temps virtuel} dans PIFO derrière P et mettre à jour P
8.         priority bytes += L
9.         F.bytes += L
10.    F.finishTag += L
11.sinon
12.    insérer {paquet,temps virtuel} dans PIFO derrière P et mettre à jour P
13.    priority bytes += L
14.    si (AFL n'est pas saturée)
15.        rajouter flot F
16.        F.finishTag = temps virtuel +L
17.        F.backlog= L
18.        F.bytes= L
```

Lors de l'arrivée d'un nouveau paquet, avant de l'insérer, il faut d'abord tester si la file n'est pas saturée (ligne 1). Dans le cas où la file est déjà pleine, un paquet doit être sélectionné pour être détruit. Plusieurs stratégies pour le choix du paquet à rejeter existent. La plus connue est, sans doute, la stratégie *drop tail* qui consiste à rejeter le dernier paquet dont l'arrivée cause le débordement de la file. En dépit de sa simplicité, cette technique n'est ni performante ni équitable. Nous préconisons de rejeter le premier paquet du flot dont le *backlog* est le plus important, i.e. dont la taille de la file est la plus grande en terme d'octets, comme suggéré par Suter et al [102]. Cette approche permet de pénaliser les flots les plus gourmands ; la perte de leur premier paquet accélère leur réaction. De plus, cette technique garantit que les flots *streaming* qui ont généralement un *backlog* faible soient épargnés par les pertes.

Ensuite, la première vérification à faire est de savoir si le flot est actif ou pas. Si le flot est déjà présent dans la liste des flots actifs (AFL), son *backlog* est incrémenté par la taille du nouveau paquet (ligne 3). Tous les flots ayant le droit d'insérer jusqu'à MTU octets dans la file prioritaire, on teste si le compteur *bytes* du flot en question dépasse MTU (ligne 4). Si c'est le cas, le flot est considéré comme *bottlenecked* et son paquet est inséré dans la file du flot avec comme estampille temporelle le *finish tag* (ligne 5).
Par contre, si le flot n'a pas encore atteint MTU octets dans la file prioritaire, le paquet est inséré dans cette file juste derrière le pointeur P (ligne 7) avec comme estampille le temps virtuel. Dans ce cas de figure, un compteur global appelé *priority*

bytes est incrémenté par la taille du paquet, il sert à calculer la charge de la partie prioritaire de la file comme il sera décri dans la prochaine section. De la même façon, le compteur *bytes* du flot est incrémenté (ligne 9). Notons que le pointeur P permet de distinguer les paquets des flots *bottlenecked* et *non bottlenecked* ayant des estampilles temporelles identiques. Les flots sont autorisés à envoyer MTU octets dans la file prioritaire afin d'éviter qu'un flot qui émet deux paquets de faibles tailles pendant le même cycle ne devienne *backlogged*.
Une fois le paquet du flot inséré (que ce soit dans la partie prioritaire ou bien dans la file du flot), le *finish tag du flot* est mis à jour (ligne 10).
Si le flot n'est pas actif, le paquet est automatiquement inséré dans la file prioritaire à l'endroit indiqué par le pointeur P (ligne 12) et acquiert l'estampille temps virtuel. Par la même occasion, le compteur *priority bytes* est mis à jour (ligne 13).
Si la liste des flots actifs n'est pas saturée, le nouveau flot y est rajouté (ligne 15) et ses différents champs sont initialisés, comme indiqué par les instructions 16-18.
La saturation de l'AFL a pour impact de traiter en priorité, éventuellement à tort, le(s) prochain(s) paquet(s) du flot qui n'a pu être inséré. Cependant, si un tel flot a un débit plus grand que le débit équitable, il y a de fortes chances pour qu'il soit capturé rapidement (inséré dans l'AFL quand elle ne sera plus saturée). Le dimensionnement de la taille de la liste des flots actifs sera traité dans le chapitre 5. En cas de saturation de l'AFL, il serait également possible d'envisager de rajouter les nouveaux flots à la place d'autres flots actifs pour éviter qu'un flot à fort débit ne soit pas détecté comme actif pendant longtemps. En particulier, si les flots déjà présents dans l'AFL n'émettent plus de paquets, tous les nouveaux paquets sont traités en priorité. Par conséquent le temps virtuel n'évolue pas et aucun flot n'est alors supprimé de l'AFL ; cette situation ne perdurerait pas en limitant la charge due aux paquets prioritaires comme on le verra plus loin.

Ainsi, le fonctionnement de cet algorithme fait que tous les flots dont le débit est inférieur au débit équitable (i.e. ceux qui ne sont pas actifs ou ceux qui n'ont pas envoyé plus de MTU octets dans la file prioritaire) bénéficient d'un traitement prioritaire.

Dans le cœur du réseau, nous préconisons un partage parfaitement équitable entre les flots ce qui se traduit par des poids identiques pour tous les flots dans le pseudo-code de PFQ. Ceci est motivé par le fait qu'une différenciation via des poids n'est pas très perceptible sur des liens à grande capacité, où, de toute manière, le débit équitable est important surtout que l'on sait que la majorité des flots sont contraints par leurs débits d'accès [95].
Par contre, à l'accès du réseau il serait intéressant d'effectuer un partage pondéré entre les flots. Le pseudo-code présenté ci-dessus reste valable à l'exception près du calcul du *finish tag* qui doit tenir compte du poids de chaque flot. Les instructions 10 et 16 seraient alors respectivement modifiées comme suit :

$$F.\text{finishTag} \mathrel{+}= L/r \text{ et } F.\text{finishTag} = \text{temps virtuel} + L/r$$

où r représente le poids du flot F. Les flots dont le poids est important seraient

privilégiés, puisque le *finsish tag* est inversement proportionnel à r. Typiquement les valeurs de r seraient associées à des classes de service déductibles à partir des entêtes IP.

Pseudo-code au départ des paquets

Les opérations exécutées par PFQ à la suite du départ d'un paquet sont données par le pseudo-code ci-dessous :

```
1. si (PIFO vide)
2.     supprimer tous les flots de AFL
3. sinon
4.     servir paquet en tête de file
5.     TS = estampille temporelle de ce paquet
6.     F.backlog −= L
7.     si (TS > temps virtuel)
8.         temps virtuel = TS
9.         pour tout flot f ∈ AFL
10.            si (f.finishTag ≤ temps virtuel)
11.                supprimer f de AFL
```

Lorsqu'un paquet quitte la file après la fin de sa transmission, les opérations à effectuer dépendent du fait qu'il reste ou pas d'autres paquets à traiter.
Si la file est vide, il n'y a aucun paquet à servir. Dans ce cas, tous les flots deviennent non actifs puisqu'ils n'ont pas de paquets en attente de traitement et la valeur du temps virtuel est arbitraire en période d'inactivité. La liste des flots actifs est alors purgée (ligne 2).

En revanche, si la file n'est pas vide, le prochain paquet à servir est traité. Le *backlog* du flot d'appartenance du paquet est décrémenté (ligne 6) [1]. Comme le temps virtuel est égal à l'estampille du paquet en cours de service, il n'y a pas d'opérations supplémentaires à effectuer quand le temps virtuel ne change pas. Par contre, si le temps virtuel évolue (ligne 8), des flots sont susceptibles de devenir non actifs. Ainsi, tous les flots dont la valeur du *finish tag* est inférieure ou égale au nouveau temps virtuel sont supprimés de la liste des flots actifs (lignes 10-11). En effet, si un flot ayant un *finish tag* plus petit que le temps virtuel n'est pas effacé de l'AFL, son prochain paquet contracterait une estampille égale au *finish tag* (instruction 5 du pseudo-code relatif à l'arrivée d'un nouveau paquet) inférieure au temps virtuel courant, ce qui serait aberrant. Un exemple de fonctionnement de l'algorithme PFQ est donné dans l'annexe 2.

Ainsi, tous les flots qui ont des paquets en attente dans la file sont présents dans la liste des flots actifs. L'AFL contient aussi les flots dont le *finish tag* est supérieur au temps virtuel courant même s'ils n'ont pas de paquets dans la file ; puisque ce sont des flots potentiellement encore *bottlenecked*.

[1] un traitement particulier est nécessaire si le flot n'avait pas été rajouté à l'AFL au préalable due à sa saturation

2.2.3 Indicateurs de congestion

Afin d'effectuer le contrôle d'admission, deux indicateurs de congestion sont mesurés continuellement de façon périodique par l'algorithme PFQ.

Fair rate

Le premier indicateur est le *fair rate* (*FR*) ou le débit équitable. Il indique le débit qu'obtiennent les flots *bottlenecked* ou, en d'autres termes, la bande passante disponible sur le lien.
Pour le mesurer, on calcule le débit que réalise un flot fictif continuellement *backlogged* (i.e. qui a toujours des paquets en attente de traitement). On suppose que les paquets de ce flot sont de taille 1 octet et qu'ils sont insérés avec les paquets réels selon l'ordre dicté par PFQ. Ainsi, au cours d'une *busy* période de la file, le nombre d'octets que le flot fictif aurait pu émettre est directement donné par l'évolution du temps virtuel (rappelons que le temps virtuel est incrémenté en fonction des tailles des différents paquets qui sont servis). Pendant que la file est vide, la bande passante disponible est la totalité de la capacité du lien.
Sur un intervalle de mesure $[t_1, t_2]$, si l'on note par $v(t)$ la valeur du temps virtuel à l'instant t, S la durée totale pendant laquelle la file était vide durant cet intervalle et C la capacité du lien on a :

$$fair\ rate = \frac{\max\{S \times C, (vt(t_2) - vt(t_1)) \times 8\}}{(t_2 - t_1)}. \tag{2.1}$$

Le premier terme est significatif quand le lien est peu chargé donnant lieu à des périodes de silence plus longues que les *busy* périodes. Dans ce cas, le flot fictif obtient toute la capacité. Le second terme correspond au cas contraire, il permet d'avoir une estimation approximative du débit réalisé par un flot *backlogged* durant l'intervalle de mesure.
Les simulations conduites ont montré qu'il n'était pas nécessaire d'affiner cet estimateur pour des charges intermédiaires. Rappelons qu'il n'est pas primordial d'avoir une estimation exacte de la bande passante disponible, une approximation est amplement suffisante pour l'application du contrôle d'admission.

Priority load

Le deuxième indicateur de congestion est le *priority load* (*PL*), il n'est rien d'autre que la charge due à la partie prioritaire de la file. L'estimation du *PL* repose sur le compteur *priority bytes* qui indique le nombre d'octets insérés dans la partie prioritaire de la file pendant un intervalle de temps $[t_1, t_2]$.
Si l'on note par $pb(t)$ la valeur du compteur *priority bytes* à l'instant t et par C la capacité du lien, on a :

$$priority\ load = \frac{(pb(t_2) - pb(t_1)) \times 8}{C(t_2 - t_1)}. \tag{2.2}$$

2.2.4 Complexité

De façon générale, les algorithmes de *fair queueing* basés sur la notion d'estampille temporelle ont une complexité qui est en $O(LogA)$ où A représente le nombre de flots actifs.

Plus particulièrement, si l'on note par B le nombre de flots *bottlenecked*, l'opération de tri en fonction des estampilles temporelles a une complexité qui est en $O(LogB)$. En effet, le tri ne concerne que les flots qui ont plus d'un seul paquet dans la file. Une estampille temporelle étant affectée à chaque paquet selon les spécificités de l'algorithme PFQ. Typiquement chaque flot aurait sa propre file FIFO de paquets, l'échéancier (scheduler) n'a besoin de maintenir que les premiers paquets de chaque flot triés tel qu'indiqué par la figure 2.5. Les paquets appartenant aux flots *non bottlenecked* sont, quant à eux, traités dans la file prioritaire et servis en FIFO.

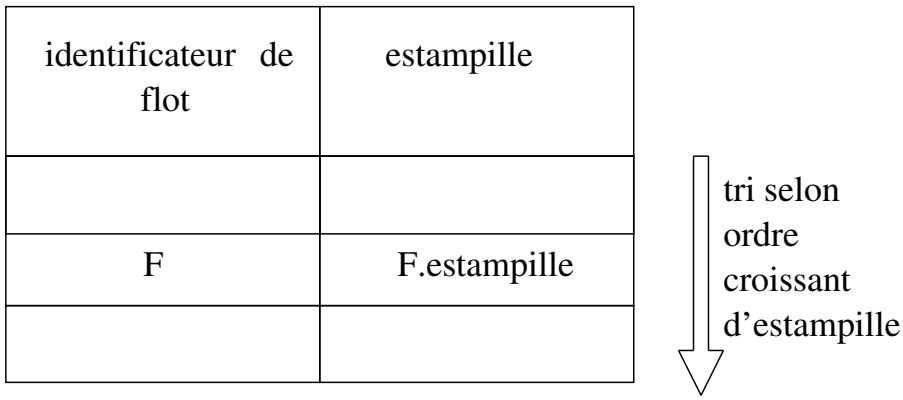


FIG. 2.5 – Structure de l'échéancier avec l'algorithme PFQ

Pour les besoins de suppression des flots qui deviennent non actifs, un tri en fonction du *finish tag* est souhaitable pour tous les flots actifs. Sa complexité est donc en $O(LogA)$.

Le test de présence d'un flot au sein de la liste des flots actifs peut, quant à lui, être effectué en un temps constant (O(1)) grâce à l'emploi d'une mémoire de type CAM (*Content Adressable Memory*) qui permet de réaliser cette tâche en *hardware*. Une mémoire CAM est constituée de semi-conducteurs (SRAM, habituellement) munis de circuits logiques de comparaison, ce qui permet d'effectuer des recherches en parallèle sur un seul cycle horloge.

Contrairement à ce qu'on pourrait croire, l'architecture *cross-protect* est parfaitement extensible en dépit du fait qu'elle repose sur l'application d'un algorithme de *fair queueing*. En effet, dans le chapitre 5 nous montrerons que le nombre de flots actifs reste relativement faible, se limitant à quelques centaines de flots seulement avec une très grande probabilité, et ce indépendamment de la capacité du lien.

2.3 Garanties de QoS

L'architecture *cross-protect* a pour but d'apporter une QoS pour tous les flots même en situation de surcharge. En l'occurrence, les garanties de QoS se matérialisent par :

- Des délais de paquets et des pertes négligeables pour les flots *streaming* dont le débit crête est inférieur à un certain seuil qui est plus faible que le débit équitable minimum imposé par le contrôle d'admission.
- Un débit équitable satisfaisant pour les flots élastiques.

Contrairement à la première génération de FAN, la limitation relative au débit crête des flots *streaming* n'est pas rigide. En effet, en l'absence de congestion, les critères de QoS sont remplis même pour les flots dont le débit crête est plus important que le seuil garanti, du moment que le *fair rate* soit supérieur à leur débit crête.

2.3.1 Contrôle d'admission

Comme on a pu le voir au cours du chapitre précédent, tant que la demande de trafic est inférieure à la capacité du lien, la performance est satisfaisante. Par contre, dès que la charge totale dépasse 1, on observe une dégradation des performances. La solution déjà proposée est d'appliquer un contrôle d'admission implicite pour rejeter proactivement l'excès du trafic.

cross-protect reprend cette idée en appliquant le contrôle d'admission en fonction des indicateurs de congestion fournis par l'algorithme PFQ. Afin de garantir un débit équitable minimum pour les flots élastiques, une des conditions d'admission est de maintenir la valeur mesurée du *fair rate* au delà d'un certain seuil, noté $seuil1$. D'autre part, dans le but d'assurer de faibles délais pour les flots *streaming*, la charge de la partie prioritaire de la file ne doit pas excéder un deuxième seuil, noté $seuil2$. Ainsi, un nouveau flot est admis si les deux conditions suivantes sont remplies :

$$FR \geq seuil1 \text{ et } PL \leq seuil2$$

Des travaux antérieurs [5] ont montré qu'un choix pour le seuil du FR d'une valeur de l'ordre de 1% de la capacité du lien était adéquat. Pour ce qui est du seuil du PL, son calibrage sera discuté dans le prochain chapitre.

Les valeurs instantanées du FR et PL sont mesurées telles que indiquées respectivement par les équations 2.1 et 2.2, puis un lissage exponentiel est appliqué, c'est-à-dire que pour le FR on a :

$$FR = \alpha \times FR + (1 - \alpha) \times mesure \text{ où } 0 \leq \alpha \leq 1$$

Il en est de même pour le PL.
Les durées des intervalles de mesures du FR et PL sont relatives aux échelles de temps qui sont pertinentes pour leurs paramètres de QoS respectifs. Elle est de l'ordre de quelques centaines de milliers de secondes pour le FR et légèrement inférieure concernant le PL.

2.3.2 Différenciation de service

Le contrôle d'admission couplé à l'ordonnancement *fair queueing* permet de garantir la transparence du réseau pour tous les flots admis au sens où il préserve le signal des flots *streaming* et ne cause pas de dégradation de débit pour les flots élastiques. Une différenciation entre les flots peut toutefois être effectuée en terme d'accessibilité au réseau et non pas en terme de QoS perçue. En effet, les utilisateurs appartenant à une classe qui requière une haute disponibilité du réseau seraient quasiment toujours épargnés par le blocage.Ceci peut être réalisé en jouant sur les seuils d'admission des différentes classes. Par exemple, le seuil relatif au FR appliqué pour les flots d'une classe i plus prioritaire que la classe j serait plus petit, donc moins contraignant.
La priorité d'un flot pourrait être déterminée à partir d'un champ classe de service (le champ DSCP, par exemple) ou bien en fonction des adresses IP.

De manière plus formelle, si on suppose qu'on a m classes de service et un ensemble de fonctions d'admissibilité $admit_i(fr, pl)$ pour $i = 1, \ldots, m$. Une classe i est plus prioritaire que j si $i < j$, on a pour les mêmes couples (fr, pl) :
$admit_1 \geq admit_2 \geq \ldots \geq admit_m$. Un flot d'une classe i est accepté si $admit_i = 1$ et refusé si $admit_i = 0$.
Ce type de différenciation a déjà été considéré dans [5] dans le cas particulier où $admit_i$ ne dépend que de l'argument fr.

2.3.3 Vulnérabilité

On pense que *cross-protect* n'induit pas de failles de sécurité au delà de celles qui peuvent être exploitées dans le cadre des architectures existantes.

En effet, la présence des listes PFL et AFL ne constitue pas une opportunité supplémentaire pour des attaques de type DoS (*Denial of Service*). Deux possibilités sont envisageables :

- Un utilisateur pourrait changer l'identifiant de flot pour chaque paquet qu'il envoie. Ceci conduirait à la saturation de la PFL. Cependant, les méfaits de cette saturation ne sont visibles que si le lien est congestionné à cet instant, auquel cas les paquets de cet utilisateur risquent également d'être rejetés. De plus, l'insertion des nouveaux flots dans la PFL avec une probabilité p réduirait l'impact d'une telle attaque.
- Un utilisateur pourrait garder le même identifiant pour plusieurs flots. Ceci éviterait aux autres flots de passer par le contrôle d'admission du moment que la durée inter-paquet reste inférieure au *time out*. Néanmoins, ces flots seraient considérés comme étant un seul et unique flot par le scheduler ; ils devront alors se partager un même débit qui ne sera pas plus grand que le débit équitable.

L'impact sur les autres utilisateurs apparaît donc minime voire inexistant dans les deux cas.

2.4 Evaluation de *cross-protect* par simulation

L'évaluation de performance de l'architecture *cross-protect* a été conduite via des simulations *ns* principalement. Un prototype logiciel a également été développé, par un autre membre de l'équipe, afin de conduire des expérimentations dont les résultats sont exposés dans l'annexe 1.
L'objectif de cette évaluation est de vérifier que *cross-protect* opère bel et bien une différenciation de service implicite tout en remplissant les critères de QoS des divers flots.

Après avoir implémenté l'algorithme PFQ dans *ns* (*Network Simulator* [2]), nous avons conduit diverses simulations en utilisant du trafic synthétique (artificiel) mais aussi du trafic issu de traces réelles.

2.4.1 Scénarios de simulation

Afin de tester *cross-protect* dans un contexte de trafic dynamique où les flots arrivent et quittent le système comme c'est le cas dans la réalité, nous générons des flots suivant un processus d'arrivée Poisson.
On définit la charge des flots élastiques, notée ρ_e, comme étant le rapport entre le taux d'arrivée des flots (λ_e) fois leur taille moyenne (σ_e) et la capacité (C) du lien i.e.

$$\rho_e = (\lambda_e \times \sigma_e)/C$$

De même pour les flots streaming, la charge, notée ρ_s, est définie comme suit :

$$\rho_s = (\lambda_s \times \text{débit moyen} \times \text{durée moyenne})/C$$

Topologie : La topologie simulée est celle représentée dans la figure 2.6. Elle consiste en un lien *bottleneck* constituant le goulot d'étranglement, où une file de type PFQ est mise en place. Le contrôle d'admission est effectué, le cas échéant, au niveau de ce lien. Le lien *bottleneck* est alimenté par 8 sources de trafic, dont la moitié génère du trafic TCP Reno et l'autre moitié du trafic UDP à destination de 4 nœuds puits.
Sauf mention contraire, la capacité du lien *bottleneck* est de 10 Mb/s, les autres liens ayant des capacités de 5 Mb/s. Les délais de propagation étant de 10ms sur chaque lien, le RTT (*Round Trip Time*) vaut 60ms.

Modèles de trafic : Les flots élastiques (TCP) émettent des paquets de taille 1000 octets avec une fenêtre TCP maximum de 20 paquets, ce qui implique que leur débit d'accès est 2.66 Mb/s.
La distribution de la taille des flots élastiques est de type Pareto tronquée avec un paramètre de forme égal à 1.5. Ce choix est motivé par le fait que la loi Pareto

[2] http ://www.isi.edu/nsnam/ns/

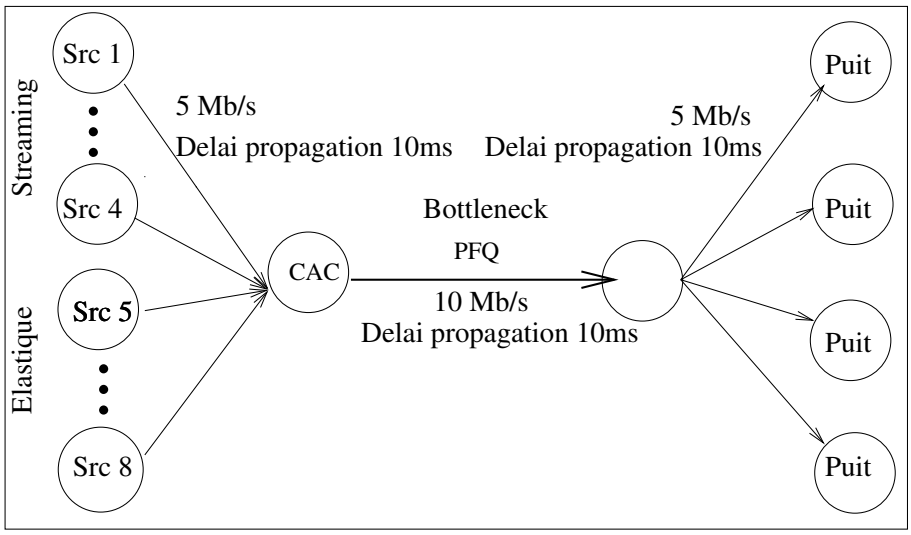


FIG. 2.6 – Topologie simulée

est à queue lente (*heavy tail*), ce qui correspond aux distributions observées dans l'Internet. De plus, comme déjà mentionné, une loi Pareto tronquée converge plus vite vers sa moyenne. Le nombre de paquets moyen par flot est 25, le taille maximum est 1000 paquets et celle minimum vaut 8 paquets.

Les flots *streaming* (UDP) sont de type *on/off*, les durées des périodes *on* et *off* sont exponentiellement distribuées avec une moyenne de 500ms. Le débit d'émission à l'état *on* est 64 Kb/s, il en résulte un débit moyen de 32 Kb/s. Ces paramètres ont été choisis pour se rapprocher d'un profil de trafic de type voix. La taille des paquets générés est 190 octets. Outre le débit, les flots UDP sont caractérisés par leur durée. La durées moyenne des flots suit une loi exponentielle de moyenne 60 secondes.

La taille du buffer a été fixée à 100 paquets ce qui correspond approximativement au produit bande passante fois le délai.
La durée des simulations est de 500 secondes, les statistiques étant récoltées sur les 400 dernières secondes pour éliminer les effets de l'état transitoire.

Scénarios : Nous avons simulé trois scénarios où le trafic élastique représente 80% de la charge totale et les flots *streaming* représentent les 20% restant :

a) une charge offerte de 0.9, sans contrôle d'admission, avec l'objectif de montrer qu'en charge normale (pas très proche de 1) *cross-protect* effectue une différenciation implicite de service et garantit des performances adéquates.

b) une charge offerte de 1.1, sans contrôle d'admission, dans l'intention de montrer que les performances s'écroulent en surcharge.

c) une charge offerte de 1.1, avec contrôle d'admission, afin de confirmer que le contrôle d'admission restaure des performances décentes.

2.4.2 Résultats

Les courbes tracées dans la figure 2.7 représentent, de haut en bas :

1) les variations du *priority load* (*PL*).
2) les variations du *fair rate* (*FR*) normalisé (divisé par C).
3) les variations de la taille de la PFL (liste des flots en cours sans tenir compte du *time out*) et de la taille de l'AFL (liste des flots actifs).
4) les variations du nombre total de paquets et de la taille de la file prioritaire.

Pour des raisons de clarté, on se focalise sur les résultats relatifs à la période allant de 150 à 200 secondes dans les différentes simulations. Les valeurs des différents paramètres tracés dans les courbes de la figure 2.7 sont obtenues aux instants d'arrivées et départs de flots au cours des simulations.

Analyse en régime de charge normale : Lorsque la charge du lien est 0.9, tous les paquets des flots *streaming* sont traités en priorité, ce qui leur garantit un faible délai moyen égal à 0.48ms. La proportion des paquets dont le délai dépasse 1.6ms se réduit à 1% seulement. Ces résultats s'expliquent par le fait que la valeur moyenne du *PL* est relativement faible : 0.37 et que le *FR* moyen est de l'ordre de 2 Mb/s ce qui fait que le débit des flots *streaming* est plus faible que le débit équitable. Comme on peut le voir, le *FR* exhibe de larges variations au cours du temps ; quand la file est vide il est proche de 1 (ce qui est d'ailleurs confirmé par la courbe indiquant le nombre total de paquets dans la file). En période de saturation, il est plus faible. Néanmoins il reste suffisamment élevé pour garantir un bon débit aux flots élastiques. En effet, le débit moyen des flots élastiques est 395 Kb/s sachant que la majorité d'entre eux sont trop courts pour atteindre le débit équitable (à cause du mécanisme de *slow start* de TCP). C'est pourquoi, 30% des paquets appartenant aux flots TCP sont insérés dans la file prioritaire conduisant à une valeur du *PL* (0.37) plus importante que les 0.18 due aux flots *streaming* (rappelons que la charge des flots *streaming* est égale à 0.2×0.9). Le débit moyen des flots éléphants (ceux qui ont une taille d'au moins 100 paquets) est de 800 Kb/s.
Le nombre de flots en cours reste inférieur à 115, avec une moyenne de 74, alors que le nombre de flots actifs est encore plus faible (15 flots en moyenne) confirmant ainsi que tous les flots en cours ne sont pas actifs au même moment.
Enfin, on peut remarquer que l'occupation de la file des paquets varie beaucoup dans le temps et que le nombre de paquets dans la file prioritaire est très faible. Le nombre moyen de paquets dans la file est 38, alors que la taille moyenne de la file prioritaire est 0.68 seulement. Quand la taille maximum de 100 est atteinte, des paquets sont rejetés. Le taux de perte observé est de 0.6% sachant qu'aucun paquet de flots *streaming* n'a été perdu.

Analyse en régime de surcharge, sans contrôle d'admission : En situation de surcharge, les performances se dégradent car le système devient instable. Le *fair*

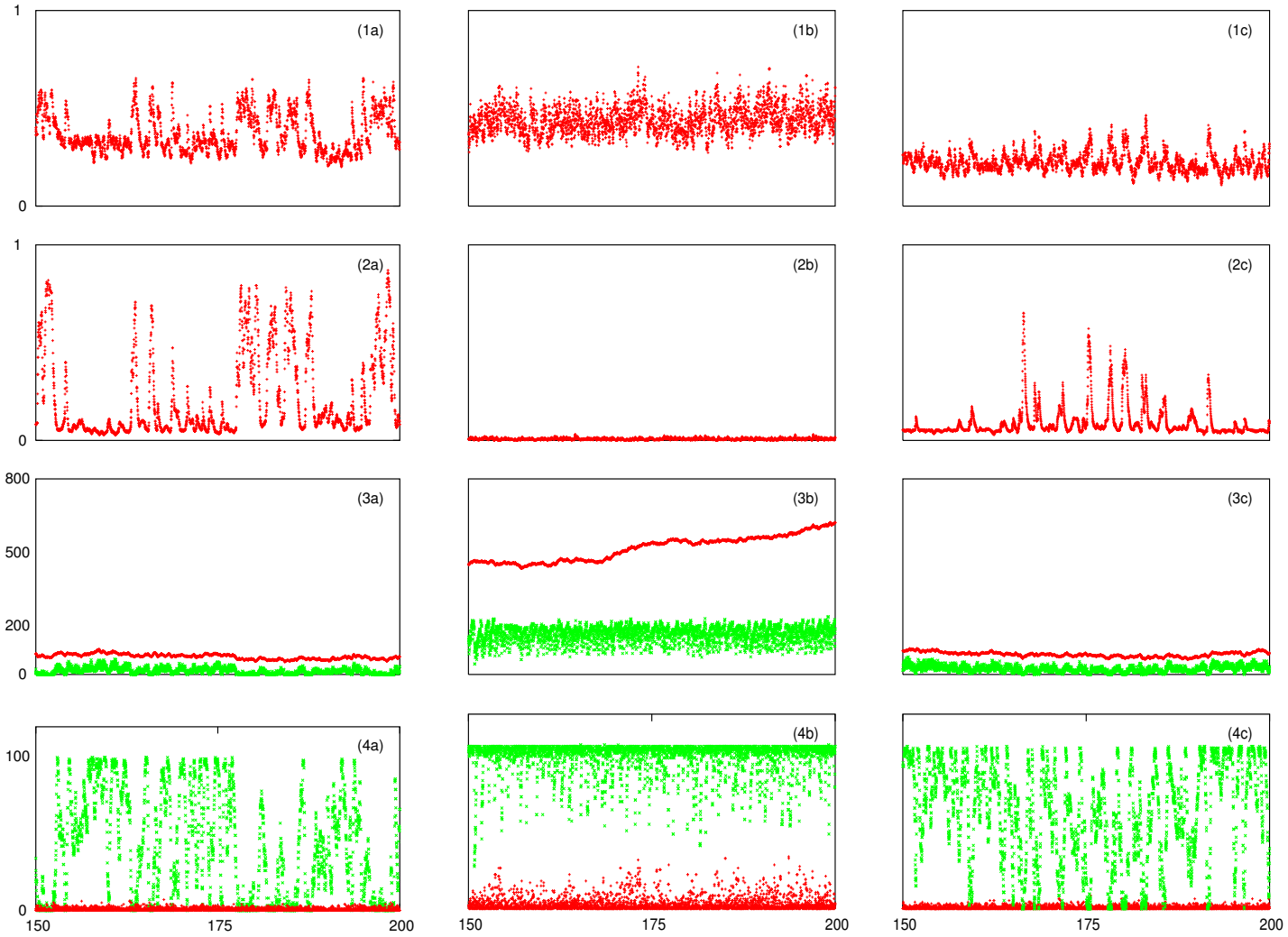


FIG. 2.7 – Résultats des simulations avec un lien à 10 Mb/s sous a) charge normale , b) surcharge sans contrôle d'admission, c) surcharge avec contrôle d'admission. Courbes relatives à 1) *PL*, 2) *FR* normalisé, 3) tailles PFL (rouge/foncé) et AFL (vert/clair), 4) tailles file prioritaire (rouge/foncé) et file totale (vert/clair).

rate est très faible indiquant que les flots obtiennent un mauvais débit. Les paquets des flots *streaming* subissent des délais importants (4.5ms en moyenne et 122ms au maximum), et ce en dépit de la contribution minoritaire du *priority load* à la charge totale (0.43 en moyenne). L'attente excessive pour ces paquets provient du fait qu'ils sont devenus *backlogged* car le débit de 64 Kb/s est plus grand que le débit équitable. Le nombre de flots en cours croît continuellement puisque le taux d'arrivée des flots dépasse le taux de départ (figure (3b)). Ce phénomène n'a pas été observé pour le nombre de flots actifs, car la file de paquet étant fixée à 100 paquets limite, indirectement, la taille de l'AFL. Autrement, le nombre de flots actifs aurait suivi la même tendance.
Comme prévu, la file de paquets est souvent saturée, ce qui entraîne un taux de perte de 11%. Notons qu'il s'agit d'un comportement pathologique de PFQ, qui n'est pas censé fonctionner en surcharge sans contrôle d'admission.

Analyse en régime de surcharge avec contrôle d'admission : En appliquant le contrôle d'admission, on revient à une situation de stabilité et les résultats sont proches de ceux obtenus sous la charge de 0.9. En effet, le délai moyen des paquets *streaming* est 0.49ms et le nombre de flots dans le système reste borné.
Selon les condition d'admission appliquées, un nouveau flot est accepté si la valeur estimée du *FR* (réevaluée toutes les 15ms) est plus grande que 100 Kb/s (représentant 1% de la capacité) et si le *PL* (mesuré toutes les 10ms) est plus petit que 0.7. Notons que seul le critère reltif au *FR* était déterminant au cours de cette simulation, conduisant à un taux de blocage de 11%. Cette même condition d'admission donne lieu à un taux de blocage de moins de 1% lorsque la charge offerte est 0.9.
Le *fair rate* est moins variable (figure (2c)) que dans le cas de la charge 0.9 (figure (2a)), indiquant une utilisation importante du lien (le taux d'utilisation moyen est 98%). Il se maintient au dessus du seuil de 100 Kb/s grâce au contrôle d'admission. Le débit moyen des flots élastiques est 310 Kb/s, et vaut 347 Kb/s pour les flots éléphants.

Des résultats similaires à ceux qu'on vient de commenter ont été obtenus, en utilisant des flots *streaming* dont les durées des périodes *on* et *off* suivent une loi Pareto (au lieu d'une loi exponentielle). Pour des raisons de concision, ils ne sont pas présentés.

2.4.3 Simulation d'un lien à 100 Mb/s

Afin de donner quelques premières indications quant à l'extensibilité de *cross-protect*, une simulation *ns* a été menée sur un lien *bottleneck* d'une capacité de 100 Mb/s avec une topologie identique à celle représentée par la figure 2.6 en opérant les ajustements adéquats (capacité des liens intermédiaires à 50 Mb/s et un buffer d'une taille maximum de 1000 paquets). Le même type de trafic que celui décrit dans la section 2.4.1 a été généré.

La figure 2.8 illustre l'évolution du nombre de flots en cours (taille de la PFL) et du nombre de flots actifs (taille de l'AFL) entre les instants 150 et 200 secondes de la simulation lorsque la charge offerte est 0.9.
Une comparaison rapide avec la troisième ligne de la figure 2.7 (cas 3a) nous indique que le nombre de flots en cours augmente sur le lien à 100 Mb/s, alors que le nombre de flots actifs reste inférieur à 150. La légère différence existante entre les tailles de l'AFL sur les liens à 10 Mb/s et 100 Mb/s est due à l'augmentation de la charge prioritaire dont la moyenne atteint 0.41 lorsque C vaut 100 Mb/s. Cette augmentation est à imputer aux flots TCP qui ne sont pas en mesure d'atteindre le débit équitable (qui devient plus important dans ce cas) à cause de leur débit d'accès limité à 2.66 Mb/s. Leurs paquets sont alors insérés dans la partie prioritaire de la file. Ceci n'est pas préjudiciable aux flots *streaming* dont les paquets subissent un très faible délai moyen égal à 0.06ms. De surcroît, le délai maximum observé au cours de la simulation est de 0.63ms.

L'impact de la proportion des flots *bottlenecked* et *non bottlenecked* sur la taille de la liste des flots actif sera examiné de plus près au cours du chapitre 5 qui sera l'occasion de revenir sur l'extensibilité du *fair queueing*.

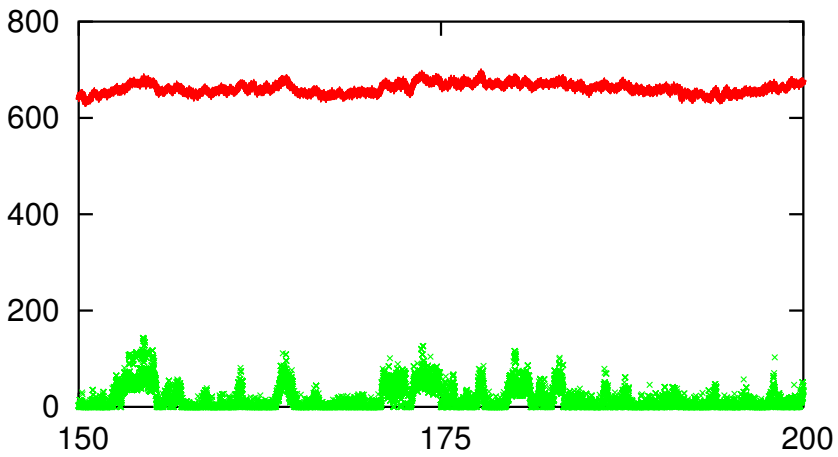


FIG. 2.8 – Nombre de flots en cours (rouge/foncé) et actifs (vert/clair) sur un lien à 100 Mb/s en fonction du temps, charge 0.9

Des résultats similaires (non présentés) sont obtenus avec une charge offerte de 1.1 sur un lien de 100 Mb/s en appliquant le contrôle d'admission.

2.4.4 Comparaison entre PFQ et PQ

Nous nous sommes posé la question de savoir qu'elle est la perte de performance en terme de délai pour les paquets des flots *streaming* à cause de leur non

identification explicite. En d'autres mots, quel est le prix à payer pour assurer une différenciation implicite ?
Afin d'y répondre, nous avons effectué une comparaison entre les délais des paquets appartenant aux flots *streaming* avec l'algorithme PFQ et ceux obtenus avec une discipline de service PQ (*Priority Queueing*) qui accorde une priorité stricte à ce type de paquets suite à un marquage explicite. Pour cela, nous avons reproduit la simulation du scénario a) de la section 2.4.1 (charge offerte 0.9) en utilisant une file de type PQ au lieu de PFQ. Evidemment, les résultats de la file PQ constituent une borne inférieure pour les délais subits par les paquets des flots *streaming*.

La figure 2.9 indique la distribution complémentaire des délais des paquets provenant des flots *streaming* (CCDF), i.e. la probabilité que le délai soit supérieur à une valeur donnée. Comme on peut le constater, l'algorithme PFQ offre des délais très proches de ceux garantis par PQ sauf pour ce qui est de la queue de la distribution. En effet, le délai moyen est de 0.38ms avec PQ contre 0.48ms sous PFQ.
Concernant la queue de la distribution, l'impact des paquets TCP qui sont insérés en priorité avec PFQ se fait sentir. Néanmoins, il y a 1% de chance seulement que le délai dépasse 1.6ms qui est, du reste, un délai relativement faible. Globalement, la différence entre les deux algorithmes demeure légère.
Cette différence n'étant pas importante sur un lien à 10 Mb/s, elle sera encore moins significative sur des liens à grande capacité, où la vitesse de transmission des paquets est plus élevée. On s'aperçoit alors de tout l'intérêt d'une différenciation implicite telle que celle apportée par PFQ.

De plus, l'application du *fair queueing* entraîne une petite amélioration du débit des flots élastiques qui obtiennent en moyenne 395 Kb/s avec PFQ contre 365 Kb/s avec PQ. Notons que les valeurs moyennes sont calculées sur l'ensemble des débits réalisés par les connexions TCP une fois terminées.

Dans la prochaine section, nous décrivons une façon alternative de réaliser l'ordonnacement de type *fair queueing* dans *cross-protect*.

2.5 Réalisation alternative avec l'algorithme PDRR

Bien que le nombre de flots actifs soit relativement faible (comme on va pouvoir le confirmer dans le chapitre 5), les algorithmes de *fair queueing* basés sur la notion de *round robin* présentent l'avantage par rapport à PFQ d'avoir une complexité en O(1), c'est-à-dire indépendante du nombre de flots actifs et non pas logarithmique. Ce genre d'algorithme traite les files des flots de façon cyclique.
En théorie, ces algorithmes basés sur le *round robin* présentent un degré d'équité moins bon que ceux basés sur les estampilles temporelles. Cependant, en pratique cette différence n'est pas aussi importante, car le trafic IP est dynamique (les flots arrivent et quittent le système perpétuellement alors que les analyses d'équité considèrent un nombre de flots *backlogged* fixe).

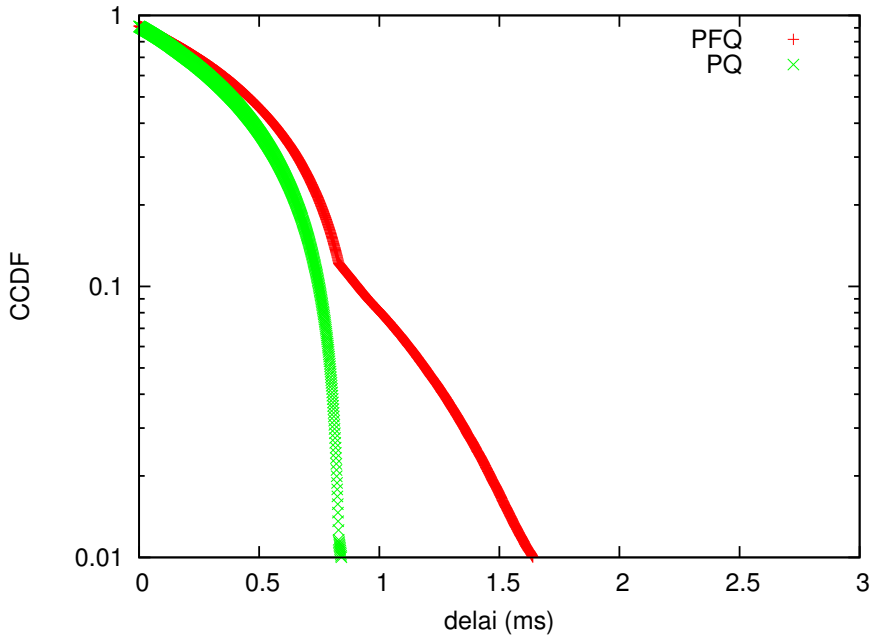


FIG. 2.9 – Distribution complémentaire des délais des paquets des flots *streaming*

Le but de *cross-protect* étant de garantir un partage équitable de la bande passante, l'ordonnancement PFQ peut être remplacé par un autre algorithme de *fair queueing* du moment que ce dernier permet d'effectuer une différenciation implicite en faveur des flots dont le débit est plus faible que le débit équitable.

De plus, en prenant en compte des aspects liés à la mise en œuvre de *cross-protect*, il s'avère que le constructeur Cisco propose des routeurs qui implémentent des variantes de l'algorithme *Deficit Round Robin (DRR)* [100]. Afin de maximiser les chances d'industrialisation de *cross-protect*, nous avons proposé une réalisation alternative, où le scheduler PFQ est remplacé par un nouvel algorithme PDRR (*Priority Deficit Round Robin*), ayant fait l'objet d'un dépôt de brevet [71].

2.5.1 Algorithme PDRR

L'algorithme PDRR est inspiré de DRR, il en hérite les propriétés d'équité et de complexité en O(1). Chaque flot dispose de sa propre file FIFO ; de plus par souci d'améliorer les délais des paquets appartenant aux flots *streaming*, une file prioritaire PQ est utilisée. Les paquets des flots dont le débit est inférieur au débit équitable y sont insérés, la différenciation est donc toujours implicite en fonction des débits des flots.

D'autres algorithmes basés sur DRR et qui gèrent une file prioritaire pour les applications temps-réel avaient été proposés dans la littérature. Ils seront discutés à la fin de cette section.

Pseudo-code

Tout comme pour l'algorithme PFQ, nous décrivons les opérations effectuées par PDRR lors de l'insertion d'un nouveau paquet et lors de l'extraction du prochain paquet à émettre.
Le pseudo-code de PDRR est présenté avec quelques instructions en italique. Ces instructions sont celles qui différent par rapport à DRR, les autres étant identiques. Dans le code nous considèrons que chaque flot dispose de son propre quantum Q_i, bien qu'on envisage plutôt un partage équitable (quanta égaux), en particulier au sein d'un réseau dorsale (*backbone*).

PDRR maintient à son tour une liste des flots actifs (AFL) qui contient les flots qui ont des paquets dans la file. Cette liste permet également de déterminer l'ordre de service des différents flots qui sont visités cycliquement. Elle comporte les champs classiques, à savoir l'identifiant du flot, son compteur de déficit DC_i, son quantum Q_i et des pointeurs pour réaliser la file FIFO du flot contenant ses paquets. Un champ supplémentaire $BytesCount_i$ mémorise le nombre d'octets insérés dans la file prioritaire (PQ) par le flot en question. La taille totale de la file du flot pourrait également être indiquée par un compteur d'octets *backlog*. Un pointeur global indique le prochain flot à traiter au sein de l'AFL.

Insertion d'un paquet : L'opération d'insertion d'un nouveau paquet p permet de déterminer la file dans laquelle il sera placé.
Dans un premier temps, il est nécessaire de vérifier si le buffer de paquets est saturé (en supposant que l'on dispose d'un espace mémoire commun pour stocker les paquets de tous les flots). Si tel est le cas, un paquet à rejeter doit être choisi. On adopte la stratégie décrite dans le cadre de PFQ et qui consiste à rejeter le premier paquet du flot dont le *backlog* est le plus important (lignes 2-3). Notons que les mises à jour de la variable *backlog* (incrémentation lors de l'arrivée d'un paquet et décrémentation lors d'un départ) associée à chaque flot ne sont pas spécifiées dans le pseudo-code.
Si le paquet nouvellement arrivé n'appartient pas à un flot actif (i.e. présent dans l'AFL), le flot est rajouté à la liste des flots actifs en fin de cycle et le paquet est inséré à la fin de la file PQ (lignes 5-9). Signalons que si l'AFL est saturée, le flot n'y est pas rajouté mais son paquet est inséré dans PQ. Le compteur *BytesCount* est incrémenté par la taille de p.
En revanche, si le paquet provient d'un flot déjà actif, c'est le compteur *BytesCount* qui détermine la file où il sera inséré. Tant que *BytesCount* est inférieur au quantum du flot (Q_i), le paquet est rajouté à PQ (lignes 11-13), autrement il est placé en fin de la file associée à son flot i (ligne 15).
Ainsi, la priorité est accordée aux paquets des flots envoyant moins d'un quantum par cycle (les flots non actifs étant supprimés conformément à l'instruction 32) ce qui leur garantit de faibles délais.

```
1. à l'arrivée d'un paquet p
2. si (buffer saturé) alors
3.       Rejeter un paquet
4. i = Exraire_flot(p)
5. si (i ∉ AFL)
6.       Rajouter (i) à AFL
7.       DC_i = 0
8.       ByteCount_i = taille(p)
9.       Insérer(PQ,p)
10. sinon
11.      ByteCount_i += taille(p)
12.      si (ByteCount_i ≤ Q_i)
13.        Insérer(PQ,p)
14.      sinon
15.        Insérer(Queue_i,p)
```

Extraction d'un paquet : L'opération d'extraction du prochain paquet à transmettre récupère le paquet qui se trouve en tête de la file prioritaire tant que celle-ci n'est pas vide (lignes 17-20). Lorsqu'un paquet est envoyé à partir de PQ, le compteur de déficit de son flot est décrémenté par la taille du paquet en question (ligne 21). Ceci permet d'empêcher tout flot d'envoyer plus d'un seul quantum par cycle (cas où son premier paquet est inséré dans PQ et le second dans sa propre file).
Lorsque PQ est vide et qu'il y a au moins un flot actif (ligne 22), le flot se trouvant en tête de cycle est sélectionné pour être servi (ligne 23). Son compteur de déficit est alors incrémenté par son quantum (ligne 24). Les instructions 25 à 34 consistent à envoyer DC_i octets pour le compte du flot i. En effet, un flot peut envoyer plusieurs paquets successivement au cours d'un même cycle, son déficit étant à chaque fois décrémenté par la taille du paquet correspondante. Le déficit permet de mémoriser la partie du quantum non utilisée pendant les cycles (*rounds*) précédents ce qui améliore l'équité à long terme.
Si la file d'un flot devient vide à l'issue de son service, le flot est supprimé de l'AFL puisqu'il n'est plus actif. Ceci est le cas de tous les flots qui émettent moins d'un quantum par cycle, ce qui se traduit par un débit inférieur au débit équitable. Autrement (file du flot non vide), le flot est replacé en fin de cycle. Le prochain flot à servir est mis à jour en fonction de l'ordre établi par la succession des flots actifs dans l'AFL. En réalité, les instructions 32 et 34 consistent juste à modifier les pointeurs appropriés (tête de l'AFL et queue de l'AFL).

Notons que selon le pseudo-code qu'on vient de décrire, la file PQ n'interrompt pas le service d'un flot au milieu de son quantum. On pourrait également envisager de servir la file PQ, une fois qu'elle n'est plus vide, dès que le paquet du flot en cours de service est émis.

Un exemple comparatif du fonctionnement de PDRR et DRR est donné dans l'annexe 2.

```
16. Tant que (VRAI) faire
17.       Tant que (PQ non vide) faire
18.            p = Extraire(PQ)
19.            i = Extraire_flot(p)
20.            Envoyer(p)
21.            DC_i -= taille(p)
22.       si (AFL non vide) alors
23.            récuperer tête AFL, flot i
24.            DC_i += Q_i
25.            Tant que ((DC_i ≥ 0) et (Queue_i non vide)) faire
26.                 L = taille (Tête(Queue_i))
27.                 si (L ≤ DC_i) alors
28.                    Envoyer(Extraire(Queue_i))
29.                    DC_i -= L
30.                 sinon
31.                    break /*quitter boucle tant que*/
32.            effacer (i) de AFL
33.            si (Queue_i non vide) alors
34.                 rajouter (i) à AFL
```

Indicateurs de congestion

Les mêmes indicateurs de congestion que pour PFQ sont mesurés dans PDRR. En considérant un flot constamment *backlogged* avec un quantum de taille MTU, le *fair rate* s'obtient en divisant le nombre d'octets que ce flot aurait pu émettre par la durée de l'intervalle de mesure. De la même manière que pour PFQ, la durée des silences (périodes d'inactivité de la file) est également prise en compte lors du calcul du *FR*. Si on note par *fairBytes* le nombre d'octets que le flot fictif aurait pu envoyer pendant l'intervalle $[t_1, t_2]$ (il est incrémenté par MTU à chaque visite du flot fictif), par S la durée du silence et C la capacité du lien on a :

$$FR = \frac{\max\{S \times C, fairBytes \times 8\}}{(t_2 - t_1)}$$

Le *priority load* représente la charge de la file prioritaire, il est mesuré sur des intervalles périodiques en se basant sur le compteur *priority bytes* qui est incrémenté à chaque fois qu'un paquet est inséré dans PQ (l'instruction correspondante n'est pas spécifiée dans le pseudo-code).

Complexité

Contrairement à DRR, la liste des flots actifs peut contenir des flots dont la file est vide. Il s'agit des flots dont les paquets sont insérés dans PQ. Ainsi, l'opération d'extraction de paquets telle que décrite peut rencontrer des files vides, qui devraient être supprimées jusqu'à atteindre une file non vide. De ce fait, sa complexité n'est plus en O(1).

Une modificaton de PDRR, où seuls les flots dont les files ne sont pas vides sont maintenus dans l'AFL, est proposée dans le chapitre 6. Cette solution induit une complexité en O(1) pour l'opération d'extraction.

Par ailleurs, une valeur du quantum supérieure ou égale à MTU est nécessaire afin de maintenir une complexité en O(1) pour l'opération d'extraction de paquets. Autrement, plusieurs files pourraient être visitées avant qu'un paquet ne puisse être réellement émis.

L'opération d'insertion de nouveaux paquets a une complexité en O(1), tout comme la détection de présence d'un flot dans l'AFL, grâce à l'emploi de mémoires de type CAM.

2.5.2 Résultats de simulations

Nous avons conduit deux séries de simulations *ns*, la première avec du trafic synthétique, pour effectuer une comparaison avec les résultats obtenus en utilisant PFQ, et la seconde avec des traces réelles afin d'étudier l'amélioration des délais des paquets des flots *streaming* par rapport à DRR.
Dans toutes ces simulations, les quanta des flots sont égaux à MTU.

Trafic synthétique

Nous avons simulé le même scénario a) décrit dans 2.4.1 en utilisant l'algorithme PDRR au lieu de PFQ. La charge offerte était donc de 0.9 dont 80% provenant des flots TCP. La figure 2.10 montre l'évolution du *PL*, *FR*, taille de l'AFL et taille des files de paquets avec PDRR et PFQ.
Comme on peut le remarquer, les deux algorithmes offrent des performances quasi identiques. En effet, on n'enregistre que des différences minimes dans le *priority load* dont la valeur moyenne est légèrement plus faible avec PDRR (0.37 contre 0.39). Ceci est dû au fait que les flots restent un peu plus longtemps dans l'AFL avec PDRR, dans la mesure où ils ne sont supprimés que lorsqu'ils sont visités et que leur file est vide ; alors qu'avec PFQ ils sont effacés dès que leur *finish tag* dépasse le temps virtuel. Le nombre moyen de flots actifs avec PDRR est 20 contre 15 pour PFQ.
Les variations du *fair rate* sont similaires pour les deux algorithmes, les deux courbes sont pratiquement confondues, ce qui se reflète dans les débits des flots élastiques : 392 Kb/s avec PDRR contre 395 Kb/s avec PFQ.
On observe également les mêmes tendances quant à l'occupation des buffers, le nombre de paquets moyen étant respectivement de 36.5 et 36 pour PDRR et PFQ . Enfin, les délais des paquets des flots *streaming* sont aussi faibles avec PDRR (0.47ms en moyenne).

On en déduit que les deux algorithmes se comportent presque de la même façon offrant des performances satisfaisantes pour des charges inférieures à 1. En particulier, le degré d'équité moindre de PDRR semble être largement compensé par sa plus

faible complexité, puisque les performances ne s'en ressentent guère dans le contexte du trafic dynamique simulé.

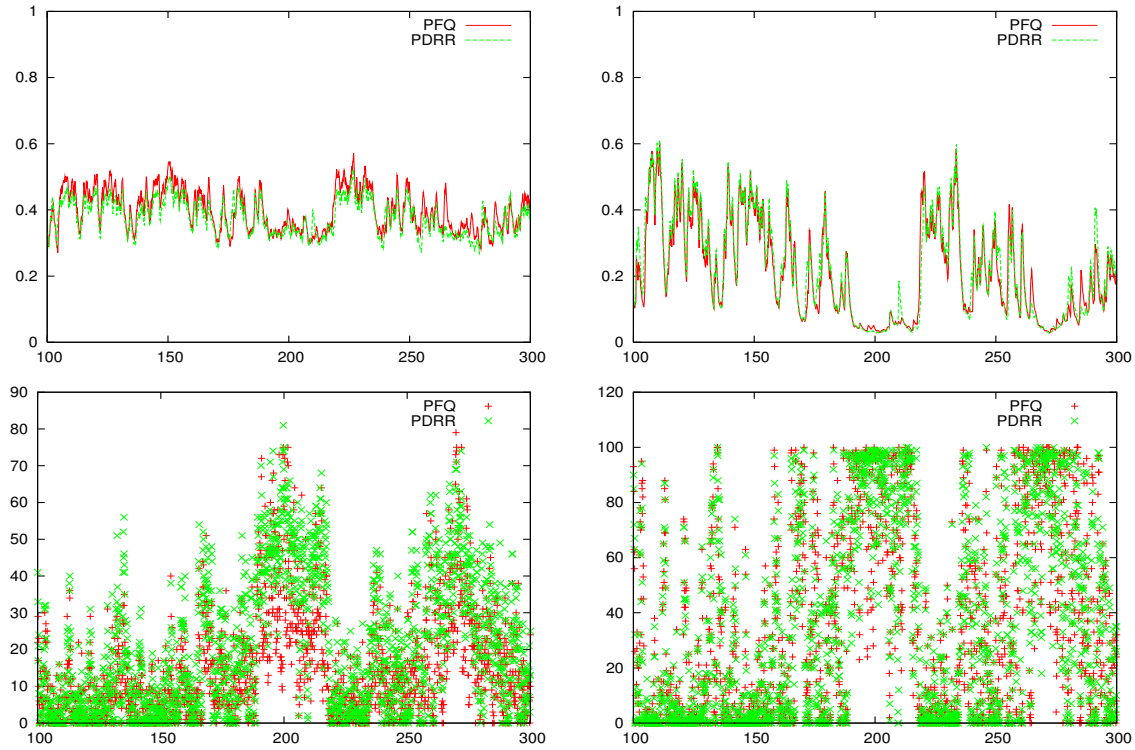


FIG. 2.10 – Comparaison entre PDRR et PFQ. Courbes relatives à *PL* (en haut à gauche), *FR* normalisé (en haut à droite), tailles AFL (en bas à gauche), tailles file totale (en bas à droite), trafic synthétique.

Trace réelle

Pour illustrer le concept de différenciation implicite, nous allons montrer que PDRR assure des délais négligeables pour les paquets des flots *streaming* dont le débit est inférieur au débit équitable via une simulation avec du trafic issu d'une trace réelle auquel on rajoute des connexions UDP.
La trace en question a été collectée le 14/08/2002 de 10h30 à 10h35 sur un lien OC48 (2.5Gb/s) entre Indianapolis et Kansas sur le réseau de recherche Américain Abilene ; elle est accessible à partir du site NLANR [3]. Elle comporte plus de 19 millions de paquets appartenant à quelques 2.3 millions de flots identifiés en utilisant le quintuple adresses IP source et destination, numéros de port, protocole outre un *time out* de 20 secondes. Le nombre de flots moyen en cours avoisinait les 162000 en tenant compte du *time out*. Le taux d'utilisation du lien était de 13%. Une analyse

[3] http ://pma.nlanr.net/Traces/Traces/long/ipls/1

plus poussée de cette trace sera présentée dans le chapitre 5.

La simulation consiste à rejouer les paquets de la trace Abilene sur un lien utilisant PDRR avec également 5 connexions CBR (*Constant Bit Rate*) permanentes dont les débits sont 64 Kb/s, 10 Mb/s, 20 Mb/s, 30 Mb/s et 40 Mb/s émettant des paquets de taille MTU (1500 octets). La capacité du lien a été fixée à 483 Mb/s de façon à produire une charge totale de 0.9. Cette configuration de simulation allie à la fois la présence de flots issus d'une trace réelle et de simples flots CBR dont on peut observer les performances en terme de délai.

La courbe de gauche dans la figure 2.11 indique la distribution complémentaire des délais des flots CBR en fonction de leurs débits. Il apparaît que les flots de faible débit obtiennent des délais négligeables, car leurs paquets son toujours traités en priorité. C'est le cas pour le flot de 64 Kb/s qui obtient un délai moyen de 0.038ms mais aussi pour le flot de 10 Mb/s qui observe un délai moyen de 0.048ms. Les délais augmentent lorsque les débits des flots sont plus importants. Ceci s'explique par le fait, que plus le débit du flot est grand, plus ses paquets risquent de ne pas être insérés dans PQ (vu que le débit équitable pourrait devenir plus petit) et donc subir des attentes plus longues. Les paquets du flot de débit 40 Mb/s attendent en moyenne 1.67ms.
En fait, les paquets des flots au débit élevé deviennent occasionnellement *backlogged*, à cause des variations aléatoires du *fair rate* comme le montre la courbe de droite de la figure 2.11. Elle retrace un échantillon des variations instantanées du *FR* et la valeur lissée mesurée avec un coefficient de filtre égal à 0.9 sur des intervalles de 100ms.
Bien que le *FR* moyen soit de 85 Mb/s, certains paquets du flot à 40 Mb/s subissent des délais importants car les variations instantanées du *FR* sur de courtes échelles de temps atteignent des valeurs beaucoup plus faibles que 40 Mb/s. Les valeurs instantanées sont calculées en fonction de la durée d'un cycle donné en divisant le quantum (1500 octets) par la durée du cycle (le temps nécessaire pour revenir au flot fictif). Un flot bascule dans l'état *backlogged* dès lors que la durée d'un cycle est plus grande que son intervalle d'inter-paquet.

Une deuxième série de simulations a été conduite afin de comparer les délais des paquets des flots *streaming* obtenus avec PDRR avec ceux de DRR.
La figure 2.12, à gauche, représente la distribution complémentaire des délais des paquets des flots UDP obtenue avec la simulation du scénario a) de la section 2.4.1. Conformément aux prévisions, elle indique que le délai moyen est nettement plus faible grâce à PDRR (0.47ms contre 8.4ms avec DRR). Cette différence résulte suite au traitement prioritaire dont ont pu bénéficier les paquets UDP, car leur débit de 64 Kb/s est inférieur au débit équitable.

La figure 2.12, à droite, est relative à la distribution complémentaire des délais des paquets appartenant à une seule connexion CBR (de débit 64 Kb/s) qui est rajoutée à la trace Abilene. La trace a été rejouée sur un lien à 361 Mb/s induisant une charge de 0.9. La différence relative entre les délais avec PDRR et DRR est

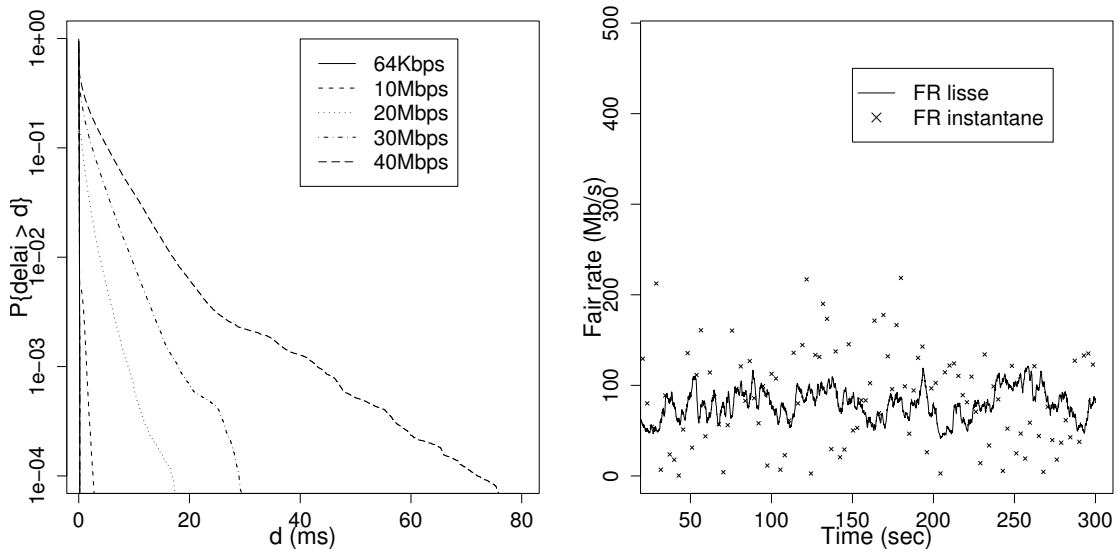


FIG. 2.11 – Distribution complémentaire des délais des paquets et évolution du *fair rate*, trace Abilene plus flots CBR.

pratiquement la même que celle obtenue avec le trafic synthétique de la figure de gauche. Néanmoins, la capacité du lien étant plus importante, les délais moyens avec DRR sont relativement faibles : 0.46ms. Evidemment, ils sont encore plus petits avec PDRR : 0.03ms.
Sachant que la durée de transmission d'un paquet de taille MTU sur un lien de 361 Mb/s est de 33.2μs et que le nombre de flots actifs est limité à quelques centaines seulement. On aboutit du coup à des délais qui ne sont pas très importants avec DRR pour des capacités de lien de cet ordre ou plus.
L'apport de PDRR est donc réduit sur des liens à très grande capacité. Toutefois, la mise en place d'un mécanisme de priorité dans le cadre de PDRR n'étant pas coûteuse, il est préférable de se doter d'un tel scheduler capable d'assurer des délais négligeables pour les paquets des applications temps-réel. De plus, le concept de *channelization* induit le partage des liens à capacité importante en cannaux de débits plus faibles [65].

La troisième courbe tracée correspond aux délais dans une file M/M/1. Elle indique que les délais dans la file prioritaire PQ peuvent être prédits en connaissant la charge moyenne (PL) et la taille moyenne des paquets (déduites à partir des simulations). Il est clair qu'une file M/M/1 ne représente pas un modèle exact, elle permet uniquement de fournir une approximation de la performance de PDRR en terme de délais des paquets prioritaires.

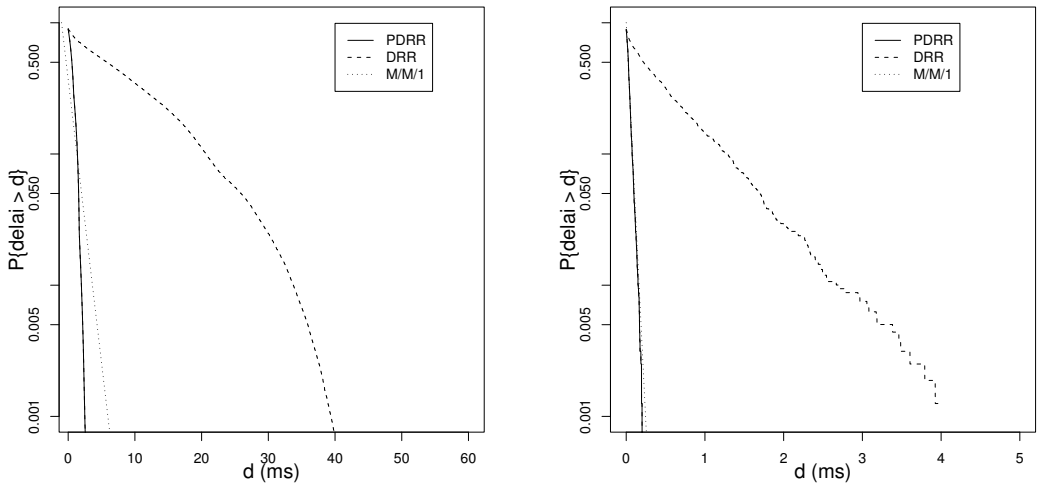


FIG. 2.12 – Comparaison des distributions des délais des packets du flot CBR avec PDRR et DRR, trafic synthétique (gauche) et trace Abilene (droite)

Performance en surhcarge

Comme dans le cas de l'algorithme PFQ, nous avons pu vérifier que PDRR assure une bonne performance en termes de débits pour les flots élastiques et délais pour les paquets des flots *streaming* en charge normale (90%).

En surcharge, il est nécessaire d'appliquer un contrôle d'admission. Dans le but de tester PDRR avec le contrôle d'admission, nous avons simulé les scénarios b) et c) de la section 2.4.1 i.e. du trafic synthétique avec une charge offerte de 1.1 avec et sans contrôle d'admission. Le trafic réel n'étant pas adapté à des simulations en surcharge vue que les connexions TCP auraient réagit aux pertes de paquets qui se seraient produites (en charge normale, la taille du buffer utilisée évitait la perte de paquets).
La taille du buffer a été fixée à 1000 paquets (afin qu'elle ne soit pas un facteur limitant pour le nombre de flots actifs) tandis que la taille maximum de l'AFL était 500 flots.
Le seuil d'admission relatif au *fair rate* est fixé à 100 Kb/s impliquant le blocage des flots dès lors que la valeur mesurée du *FR* est plus petite que 100 Kb/s. Une condition sur le *priority load* n'est pas nécessaire puisque sa valeur moyenne est relativement faible (0.25) dans cette configuration de simulation.
Le calcul du *fair rate* se fait sur des intervalles de durée 100ms avec un lissage exponentielle de paramètre 0.9. Il en est de même pour le *priority load*.

Les courbes de droite et de gauche de la figure 2.13 tracent les variations respectivement du *PL* et du nombre de flots actifs, avec et sans contrôle d'admission. On observe que le comportement des deux scénarios est identique durant les 60 premières secondes de simulation, où le système monte en charge. Dès que la surcharge survient, la valeur du *FR* descend en dessous des 100 Kb/s et le contrôle d'admission

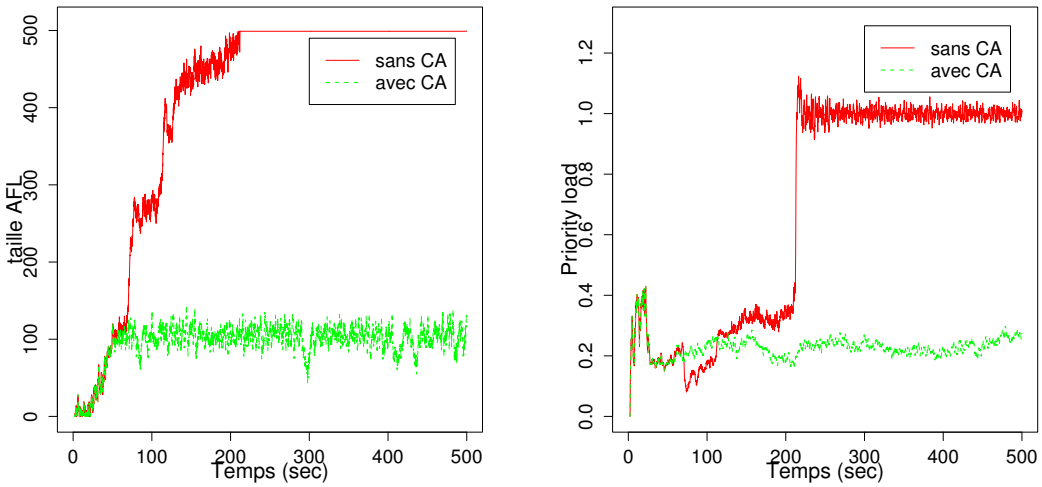


FIG. 2.13 – Variations de la taille de l'AFL (gauche) et *priority load* (droite) vs. temps, trafic synthétique en surcharge

se met en route lorsqu'il est activé.
Quand le contrôle d'admission n'est pas opérationnel, PDRR, comme PFQ, exhibe un comportement pathologique. Le nombre de flots en cours croît continuellement, la liste des flots actifs sature en atteignant sa limite de 500 flots au bout de 210 s. A partir de cet instant, tous les paquets des nouveaux flots qui arrivent sont insérés dans la file prioritaire, puisque leurs flots n'ont pu être rajoutés à l'AFL. Le scheduler ne traite que des paquets de la PQ, puisque celle-ci ne se vide jamais. Les flots déjà actifs, n'étant pas servis, ne sont donc pas supprimés de l'AFL. Ce foncionnement anormal perdure jusqu'à la fin de la simulation. L'ordonnancement PDRR se transforme alors en un simple FIFO.
L'évolution du *PL* confirme cela, puisqu'il atteint la valeur de 1 après 210 s causant des délais extrêmement élevés (400ms) et des débits désastreux.

L'application du contrôle d'admission permet encore une fois de préserver la performance. L'évolution du nombre de flots actifs suggère que la valeur du *FR* est proche du seuil de 1% (puisque une centaine de flots sont présents dans l'AFL).
La valeur moyenne du *PL* est 0.25, ce qui laisse présager de faibles délais pour les paquets prioritaires. En effet, le délai moyen des paquets appartenant aux flots UDP est 0.69ms. De leur côté, les flots TCP réalisent un débit moyen de 129 Kb/s.
La probabilité de blocage des flots est 0.1 donnant lieu à une utilisation moyenne du lien de l'ordre de 99%.

Des simulations de la trace Abilene en surcharge avec un contrôle d'admission, qu'on omettra de présenter, ont confirmé la capacité de PDRR à satisfaire les critères de QoS des différents types de flots.

Dans la prochaine section, nous décrirons succinctement d'autres algorithmes, proposés dans la littérature, basés sur DRR et qui utilisent une file prioritaire pour gérer le trafic temps-réel.

2.5.3 Algorithmes existants

DRR peut occasionner des délais relativement importants pour les paquets des flots temps-réel. En effet, le délai maximum que pourrait subir un paquet d'un flot donné est proportionnel aux nombres de flots actifs.

Conscients de cet handicap, Shreedhar et Varghese, les auteurs de DRR, ont proposé une version modifiée appelée DRR+ [100] pour traiter des flots appartenant à des classes de trafic exigeant de faibles délais. Le traitement prioritaire associé à ces flots consiste à les placer en tête de cycle quand leurs paquets arrivent. En outre, ces flots doivent respecter un contrat de trafic qui limite le nombre d'octets transmis par cycle. Cette solution permet de réduire le délai des paquets des flots prioritaires ; par contre, elle entraîne un traitement en LIFO (*Last In First Out*) entre les flots prioritaires. De plus, elle nécessite de conditionner les flots prioritaires afin de s'assurer qu'ils ne violent pas leur contrat de trafic.

L'algorithme DRR++ [78] utilise une file prioritaire pour traiter les paquets des flots à faible délai. La file prioritaire est servie en alternance avec les autres files en fonction des compteurs de déficit et des quanta.

L'algorithme MDRR [113], mis en œuvre dans certains routeurs Cisco, maintient une file prioritaire (PQ) consacrée aux paquets des flots temps-réel. Deux modes d'extraction de paquets sont préconisés. Dans le mode alterné, PQ est servie en alternance avec les autres files (PQ est visitée à chaque fois qu'une file DRR termine son service). Dans le mode priorité stricte, PQ est servie dès qu'elle contient au moins un paquet. Tous les flots appartenant à la file PQ se partagent le même compteur de déficit et utilisent le même quantum, ceci pourrait induire des iniquités entre les flots en question.

Il importe de souligner que toutes ces méthodes reposent sur un marquage explicite des paquets des flots prioritaires. Elles sont donc différentes de PDRR.

Chapitre 3

Contrôle d'admission basé sur les mesures pour les flots *streaming*

Après s'être penché sur la performance des flots élastiques au cours du premier chapitre, nous nous intéressons à présent à la performance des flots *streaming* et au type de contrôle d'admission appliqué dans le cadre de *cross-protect* qui permet de remplir leurs exigences de QoS.

Dans un premier temps, nous parcourons rapidement certaines approches utilisées pour apporter des garanties de QoS aux flots *streaming*. Ensuite, avant de proposer un algorithme de contrôle d'admission basé sur les mesures adapté au contexte de *cross-protect* [72], certains algorithmes existants sont passés en revue en expliquant les raisons pour lesquelles ils ne sont pas exploitables dans le cadre de *cross-protect*.

3.1 Classification des approches de contrôle d'admission

Le succès grandissant de l'Internet a permis l'émergence de nouvelles applications dites temps-réel telles que la téléphonie sur IP, vidéoconférences, visioconférences, etc. A la différence du trafic *best effort*, les applications temps-réel nécessitent une qualité de service (QoS) qui se traduit par une faible gigue et des délais ainsi que des pertes négligeables.
Les recherches effectuées indiquent que la gigue subie par les paquets des flots *streaming* reste contrôlable simplement en limitant la charge maximum du lien au tour de 90% [19]. Ainsi, le contrôle d'admission (CA) s'impose pour garantir cette QoS. Il permet d'offrir une qualité de service à tout flot nouvellement admis, sans pour autant dégrader celle des autres flots en cours. En effet, une solution basée sur le contrôle des sources grâce à un *feedback* indiquant l'état du réseau (dans la lignée des mécanismes de congestion de TCP) n'est pas adaptée aux applications temps-réel.

Classiquement, la performance du trafic *streaming* repose sur le concept de

contrat de trafic qui assure des garanties de QoS en se basant sur des descripteurs de trafic utilisés pour caractériser les flots a priori. Des mécanismes de contrôle assurent que tant que le trafic est conforme à ses descripteurs, ses garanties de QoS sont respectées (grâce au contrôle d'admission).

On distingue deux types de contrôle d'admission en fonction de la nature des garanties qu'ils apportent : déterministes ou statistiques.

3.1.1 Garanties déterministes vs. garanties statistiques

Les premiers algorithmes de contrôle d'admission étaient conçus pour apporter des garanties déterministes, où les conditions d'admission sont dérivées à partir de l'étude de pire cas. Ces algorithmes assurent le respect des critères de QoS de tous les flots admis.

Leur principal inconvénient réside dans le faible taux d'utilisation qu'ils engendrent, ce qui n'est pas économiquement rentable pour les opérateurs de télécommunications.

Par exemple, l'allocation de ressources basée sur le débit crête conduit à une sous-utilisation du lien, puisque les flots n'émettent pas constamment à leur débit crête. Il en est de même avec une approche basée sur le modèle *network calculus*.

En considérant des garanties statistiques plutôt que des garanties déterministes, c'est-à-dire que les critères de QoS des flots admis sont remplis avec une très grande probabilité et non pas de façon certaine (à 100%), il est possible d'améliorer nettement l'utilisation des liens. Ce type de contrôle d'admission suffit pour la plupart des applications *streaming* qui peuvent tolérer, dans une certaine limite, des dépassements de délais ou des pertes de paquets (résorption de la gigue grâce à des *play-out* buffers, par exemple).

Ainsi, l'application du contrôle d'admission nécessite la connaissance de certains paramètres de trafic afin de gérer les ressources convenablement. Pour ce faire, deux méthodes sont applicables. La première consiste à spécifier les paramètres descriptifs des flots à l'avance par les utilisateurs. La seconde recourt à des mesures qui permettent de déduire ces paramètres. Nous revenons sur ces deux méthodes dans le prochain paragraphe.

3.1.2 Approche paramétrique vs. MBAC

Comme mentionné précédemment, la notion de contrat de trafic se base sur la caractérisation des flots a priori avec certains paramètres descriptifs. Néanmoins, cette méthode présente plusieurs inconvénients. En effet, cette approche paramétrique nécessite l'utilisation de descripteurs de trafic adaptés. Or, ce genre de mécanisme est basé sur la déclaration par les applications de leurs paramètres de trafic (débit et *burstiness*, par exemple) qui sont généralement difficiles à connaître avec exactitude. De plus, la variation du trafic sur plusieurs échelles temporelles fait qu'il ne peut

être caractérisé de façon appropriée par des descripteurs du type *leaky bucket* (cf. introduction, §2.1). Par ailleurs, les paramètres des applications dont le contenu est généré de façon dynamique (avec un algorithme de compression, par exemple) ne peuvent être connus d'avance ; ce qui rend délicat le choix des paramètres de *leaky bucket*.

Enfin, lorsque les spécifications des sources déclarées par les applications sont trop conservatrices, le lien est sous-utilisé. Ces difficultés ont été constatées dans le cas des réseaux ATM (*Asynchronous Transfer Mode*) et relais de trame (*frame relay*), ce qui a amené les opérateurs à appliquer, de manière empirique, un facteur de surréservation (*overbooking*) qui peut compromettre les garanties de QoS annoncées.

Dans l'optique de s'affranchir de descripteurs de trafic peu utiles conduisant à des performances incertaines et de pallier aux insuffisances des méthodes déterministes, une autre approche de contrôle d'admission a vu le jour. Il s'agit du contrôle d'admission basé sur les mesures ou, en anglais, *Measurement Based Admission Control* (MBAC). Cette méthode est orientée applications temps-réel dans la plupart des cas. Elle exploite le gain engendré par le multiplexage statistique des flots, afin de maximiser l'utilisation du lien, tout en assurant une bonne QoS aux flots admis. L'approche MBAC est destinée aux applications qui n'exigent pas de garanties strictes (au sens déterministes) dans la mesure où elle repose sur une estimation des ressources disponibles.

Tout algorithme de MBAC est constitué de deux parties : la condition d'admission et l'estimateur appliqué aux mesures de congestion effectuées. Un nouveau flot est accepté si les mesures indiquent que la condition d'admission est satisfaite.
L'application d'un algorithme de MBAC décharge les utilisateurs de la spécification des paramètres de trafic. En effet, les mesures effectuées continuellement permettent de déduire ces paramètres.

Une méthode MBAC doit, idéalement, remplir les critères suivants :

- Robustesse : assure la QoS en se basant sur une description minimale du trafic (débit crête uniquement, par exemple).
- Efficacité : elle doit maximiser le taux d'utilisation du lien, tout en remplissant les critères de QoS des flots admis.
- Faisabilité : elle doit être implémentable (en termes de passage à l'échelle et de complexité, de l'algorithme) et suffisamment simple afin qu'elle puisse être exploitée.

Quand le débit des flots est variable, les algorithmes MBAC se révelent efficaces pour atteindre un taux d'utilisation important du lien.

Notons que les algorithmes MBAC qui reposent sur des mesures globales sont plus performants que ceux qui nécessitent la mesure de statistiques par flots, car ils sont plus simples et plus robustes. De plus, la QoS perçue par les utilisateurs lorsqu'ils sont multiplexés dépend de l'agrégat de leur trafic.

3.1.3 Multiplexages avec et sans buffer

Afin d'étudier la performance des flots *streaming*, il est commode de supposer qu'ils sont fluides avec un débit instantané connu. Dans un contexte de modélisation fluide, on distingue deux types de multiplexage : avec et sans buffer. Le multiplexage avec buffer permet d'absorber l'excédant de trafic qui arrive par rapport à la capacité du lien C. Le multiplexage sans buffer suppose des délais nuls et repose sur la condition que le débit cumulé global reste inférieur à la capacité C. L'efficacité d'un multiplexage sans buffer dépend donc de l'application d'un contrôle d'admission basé sur les mesures pour limiter la probabilité de débordement.

Une étude comparative effectuée dans [95] met en évidence que le multiplexage sans buffer surclasse d'autres approches, à savoir l'allocation au débit crête, le *network calculus* déterministe et la relaxation stochastique du *network calculus* (en supposant que les sources sont indépendantes) en terme d'utilisation du lien ; en particulier lorsque le rapport C/p est important, où p est le débit crête maximum considéré.

Nombreux sont les algorithmes de MBAC qui utilisent ce modèle fluide de *bufferless multiplexing* (multiplexage sans buffer). Il s'agit de s'assurer que le débit cumulé mesuré des flots ne dépasse la capacité du lien qu'avec une faible probabilité :

$$Pr[\sum_{i \in \mathcal{F}} r_i(t) > C] < \epsilon$$

où $r_i(t)$ est le débit du flot i à l'instant $t, \mathcal{F}$ représente l'ensemble des flots partageant le lien et ϵ est très petit. Ce type de multiplexage permet d'obtenir des délais négligeables. Le multiplexage sans buffer est efficace si tous les débits crêtes sont inférieurs à 1% de la capacité du lien, approximativement, car il permet d'atteindre un taux d'utilisation élevé.
Il est à noter qu'à l'échelle paquet un buffer d'une petite taille est utilisé pour absorber une éventuelle congestion à ce niveau. Les performances du *bufferless multiplexing* sont discutées dans [97], chapitre 16.

3.2 Etat de l'art sur les algorithmes MBAC pour les flots *streaming*

Le contrôle d'admission pour les applications temps-réel a suscité un engouement important qui a débouché sur la proposition de plusieurs algorithmes MBAC, parmi lesquels on peut citer [47, 53, 58, 93, 56, 33, 38]. Dans ce qui suit, nous décrivons certains de ces algorithmes en s'attardant sur les plus intéressants, de notre point de vue, en l'occurence [53] et [47].

Jamin et al [58] ont proposé un des premiers algorithmes de MBAC qui est souvent cité en référence dans la littérature. Il s'agit de l'algorithme *Measured Sum* (MS), il a été développé dans le cadre de la classe *predective* de Intserv. Ils ont effectué une analyse des délais des flots en fonction de leur classe. Les flots appartenant

à la classe *guaranteed* ont une proportion de la capacité qui leur est réservée, ils sont traités en WFQ (*Weighted Fair Queuing*). Les différentes classes de *predictive* sont traitées en priorité entre elles et se partagent ce qui reste de la bande passante.

Quand un nouveau flot *predictive* de classe k arrive les délais des classes $k, k+1 \ldots$ augmentent, où la classe i est plus prioritaire que la classe $i+1$. L'arrivée d'un flot *predictive* a plus d'impact sur les délais des classes *predictive* moins prioritaires que l'arrivée d'un flot *guaranteed*, car WFQ isole ses flots.

Les conditions d'admission sont basées sur un taux d'utilisation maximum, ainsi que sur des délais par classe. Un nouveau flot déclare ses paramètres de *leaky bucket*, tous les flots appartenant à une même classe ont les mêmes paramètres. Les mesures se font sur des fenêtres temporelles et les délais sont estimés en fonction du temps de séjour dans la file de chaque paquet. Cet algorithme s'appuie sur deux paramètres T et S qui décrivent les fenêtres de mesures relatifs aux délais et débits, respectivement.
Les simulations effectuées ont indiqué que lorsque T est grand l'algorithme est plus stable mais offre un taux d'utilisation moindre, tandis que pour des valeurs de T petites il y a une meilleure réactivité, mais également plus de dépassements de délais dus à une utilisation plus élevée. Le choix d'une faible valeur de S conduit à un contrôle d'admission conservateur, car la mesure du débit cumulé est moins stable.

Comme on peut le constater, cet algorithme distingue plusieurs classes de flots et nécessite donc de les reconnaître explicitement. Une classification aussi fine n'est pas indispensable pour garantir une QoS aux applications temps-réel à partir du moment où l'on regroupe les flots au sein de deux grandes classes, en l'occurrence *streaming et élastique*, comme mentionné précédemment. De plus, il faut maintenir des statistiques pour toutes ces classes afin de pouvoir évaluer les conditions d'admission. Un autre problème réside dans le fait que les performances de l'algorithme soient très liées aux paramètres précédemment décrits (T et S).

La version simplifiée de l'algorithme MS (pas de notion de classes et aucune contrainte de délais) consiste à accepter un nouveau flot si

$$\widehat{v} + r \leq \mu C$$

où $\widehat{v}$ est la mesure du débit cumulé, r le débit déclaré du nouveau flot, μ un taux d'utilisation cible (0.9 par exemple) et C la capacité du lien.

Casetti et Kurose [28] ont proposé une amélioration de l'algorithme MS en utilisant une fenêtre de mesure adaptative (le paramètre T décrit ci-dessus). Ils utilisent un modèle plus simple que celui de [58] avec une seule classe *predictive* et aucun flot de classe *guaranteed*. Les conditions d'admission sont réduites à la condition sur le délai, i.e. $\widehat{D} + b/C \leq D_{max}$ et la condition sur l'utilisation qui est $\widehat{V} + r \leq \nu C$ où r et b sont les paramètres du *leaky bucket* déclarés par le nouveau flot, C la capacité du lien, $\widehat{D}$ le délai mesuré et $\widehat{V}$ la bande passante mesurée.
Pour obtenir une fenêtre adaptative, l'idée est de diminuer la taille de la fenêtre afin d'augmenter l'utilisation jusqu'à ce qu'une charge maximum soit atteinte où la fenêtre est de nouveau augmentée et ainsi de suite. La charge maximum est ajustée

en fonction du pourcentage des paquets qui dépassent leur borne de délai. D'après les simulations effectuées les sources homogènes permettent d'avoir une meilleure utilisation que celles hétérogènes. L'apport de cet algorithme est qu'il décharge l'utilisateur du choix crucial de la valeur de T ; néanmoins il nécessite à son tour un bon paramétrage des facteurs d'incrémentations et décrémentations pour le réajustement des différentes valeurs. De plus, il ne tient pas compte de la nature du trafic.

Gibbens et al [47] ont élaboré une méthode de MBAC simple basée sur un modèle fluide *bufferless*. Ils proposent de policer les applications afin qu'elles respectent leur débit crête. L'admission d'un nouveau flot (ou appel) se fait en fonction de la charge courante ; si elle est inférieure à un seuil pré-calculé le flot est accepté. L'algorithme concerne uniquement la partie admission, ils ne proposent pas de méthodes de mesures de la charge du lien.

Le calcul du seuil d'admission est réalisé suivant une procédure d'estimation qui tient compte du compromis entre l'acceptation et le rejet d'un nouveau flot. Les sources sont décrites par un modèle *on/off* et la charge induite par les flots est représentée par une distribution binomiale. Cette dernière hypothèse s'est avérée conservatrice. La charge instantanée produite par la superpositions des appels $S_n(t)$ est donnée par : $S_n(t) = X_1(t) + X_2(t) + \ldots + X_n(t)$, où les $X_i(t)$ sont des variables aléatoires indépendantes et identiquement distribuées représentant la charge induite par l'appel i à l'instant t. Les sources ont un débit crête unitaire et un débit moyen p avec $Pr[X_i(t) = 1] = p, \quad Pr[X_i(t) = 0] = 1 - p$.
Les appels arrivent suivant un processus de Poisson d'intensité ν et $\lambda = \nu p$.
Le modèle utilisé suppose que le débit moyen (p) des flots est relativement stable et tente de maximiser le taux d'utilisation en se fixant un taux de perte limite (10^{-9}). Le modèle tient compte de la séparation entre les échelles appel, rafales (*bursts*) et cellules (en supposant que les variables aléatoires $X_i(t)$ sont indépendantes).

Afin de minimiser le nombre de flots admis par erreur quand la charge est élevée, une stratégie de *back off* est employée. Elle consiste à refuser tout nouveau flot arrivant après un premier blocage, jusqu'à ce qu'il y ait un départ de flot du système.

L'inconvénient majeur de cette méthode est que le calcul du seuil d'admission nécessite la connaissance au préalable de la valeur de λ, ce qu'on est pas censé savoir. De plus, la technique de *back off* peut être préjudiciable au taux d'utilisation.

Gibbens et Kelly [48] ont également étudié différents schémas de contrôle d'admission basés sur des conditions d'admissions issues des *Chernoff bounds* et évaluées grâce à diverses tangentes de la fonction de *effective bandwidth*, telles que la tangente à l'origine, au *peak*, etc. Ils ont considéré plusieurs classes de connexions, chaque classe étant caractérisée par ses débits crête et moyen. Ces algorithmes nécessitent, au moins, la connaissance de la charge offerte, du nombre de flots dans chaque classe et leur débit crête.

L'approche développée par Grossglauser et Tse [53] constitue une technique intéressante. Son principal apport consiste en l'introduction d'une échelle de temps

critique $\widetilde{T_h}$, qui constitue le temps nécessaire pour se remettre d'une admission erronée. Elle permet de parer aux erreurs d'estimation de mesures, en les prenant en compte explicitement.
Ils distinguent deux types de fluctuation du débit. Les fluctuations plus rapides que $\widetilde{T_h}$ imposent la réservation d'une partie de la bande passante pour pouvoir gérer ces variations, tandis que les fluctuations plus lentes sont compensées par les départs des flots en cours.

Le modèle utilisé est un lien *bufferless* de capacité C, le nombre maximum de flots que le lien peut contenir est donné par $n = C/\mu$ où μ est le débit moyen par flot et la durée des flots est exponentielle de moyenne T_h. Le contrôle d'admission œuvre pour minimiser la probabilité de perte i.e. $Pr[S_t > C] < p_q$, où S_t représente la quantité de bande passante consommée par l'agrégat des flots à tout instant t et p_q est très faible.
L'algorithme mesure la moyenne ainsi que la variance du débit des flots sur des fenêtres de taille $K\widetilde{T_h}, \quad K >> 1$. A partir de ces mesures, le nombre de flots maximum à accepter M_t est déduit grâce à une approximation Gaussienne concernant le débit cumulé des flots. Il est connu que les approximations Gaussiennes conviennent pour un nombre de sources relativement grand. Ce nombre de flots M_t est lié à la probabilité de perte fixée p_q via l'équation :

$$Q\left(\frac{C - M_t\widehat{\mu}_t}{\sqrt{M_t}\widehat{\sigma}_t^H}\right) = p_q$$

où Q(.) est la fonction de répartition de la loi Normale $N(0,1)$, $\widehat{\mu}_t$ est le débit moyen mesuré avec un filtre passe bas et $\widehat{\sigma}_t^H$est l'écart type, mesuré avec un filtre passe haut, du débit par flot.

Ainsi, un nouveau flot est accepté si :

$$C - (N_t + 1)\widehat{\mu}_t > \alpha_q\widehat{\sigma}_t^H\sqrt{N_t + 1}, \quad \alpha_q = Q^{-1}(p_q)$$

où N_t est le nombre de flots dans le système à l'instant t.

Quand le lien est à grande capacité et donc $n = C/\mu$ est important, la charge mesurée se rapproche de la charge moyenne ; par contre la variance reste aléatoire ce qui fait que le nombre de flots à admettre l'est aussi. Comme spécifié ci-dessus, les mesures se font sur des fenêtres de longueur proportionnelle à K. Quand la valeur de K est faible il y a plus de risques de dépasser la probabilité de perte voulue car dans ce cas la variance est sous estimée conduisant à l'acceptation de beaucoup trop de flots. De ce fait, il faudrait viser une probabilité de perte encore plus petite que celle que l'on voudrait.

Pour déterminer la valeur du temps critique, les auteurs ont considéré les différentes échelles de fluctuation du débit par rapport à $\widetilde{T_h}$.
Quand les fluctuations de la bande passante sont à une échelle plus lente que $\widetilde{T_h}$, un nouveau flot peut être accepté dés qu'il y a un départ (ils supposent qu'il y a toujours des flots en attente d'admission).

Dans le cas où les fluctuations du débit cumulé sont plus rapides, le nombre maximum de flots à accepter n^* vérifie : $n^* = n - \frac{\sigma\alpha_q}{\mu}\sqrt{n} + o(\sqrt{n})$ en considérant que la moyenne et l'écart type, respectivement μ et σ, sont connues. Il y a donc une déviation de l'ordre de $\sqrt{n}$ flots. Le taux de départ des flots étant n/T_h, il faudrait $\sqrt{n}/(\frac{n}{T_h})$ comme temps pour annuler les erreurs de mesures. Ainsi, ils déduisent que

$$\widetilde{T_h} = \frac{T_h}{\sqrt{n}}$$

La quantité $\frac{\sigma\alpha_q}{\mu}\sqrt{n}$ peut être interprétée comme une marge de sécurité (bande passante à laisser de côté). Les analyses menées ont montré que les pertes fluides sont dues à la fluctuation de la bande passante et non pas à la fluctuation du nombre de flots.

Afin d'étudier l'impact des erreurs de mesures dans [52], les auteurs ont utilisé *impulsive model*, où il n'y a pas de départs de flots, i.e. les flots acceptés ont une durée infinie. L'étude a montré que les erreurs dans l'estimation du nombre de flots à admettre sont dues plus aux erreurs dans la mesure de la moyenne qu'à celles de la variance.

Les simulations conduites ont montré que cette méthode réalise un bon taux d'utilisation, et que la probabilité de perte diminue avec $\widetilde{T_h}$. Cependant l'algorithme nécessite la connaissance du nombre de flots dans le système à tout moment outre les instants d'arrivées et de départs des flots, ce qui est loin d'être évident.
Cette méthode complète leur travail précédent [52] qui reposait sur des mesures par flot individuel, car il est plus facile de prédire le comportement de l'agrégat que celui d'un flot individuel (loi des grands nombres).

Dans le cas où le nombre de flots N_t est inconnu, une autre condition d'admission a été proposée :

$$C - A_t^\lambda - r > \alpha_q \widehat{\sigma}_t^{AH}$$

où A_t^λ et $\widehat{\sigma}_t^{AH}$ sont évalués à partir du débit cumulé S_t sans se ramener au débit moyen par flot (puisque le nombre de flots est inconnu), r représente le débit crête du nouveau flot.
Comme l'admission d'un nouveau flot n'est pas répercutée immédiatement dans la mesure de la bande passante agrégée et afin d'éviter qu'il n'y ait trop de flots acceptés, ils proposent de rajouter r au débit total à chaque admission ; l'impact de cette addition diminue exponentiellement avec le temps.

A leur tour Qui et Knightly [90] proposent une méthode où ils tiennent compte de l'incertitude des mesures. Leur modèle utilise un buffer, sa taille est choisie de sorte à ce que les paquets non perdus respectent leur critère de délais. Les auteurs calculent le débit maximum de l'agrégat sur un intervalle donné et lui associent un niveau de confiance afin de décider de l'admissibilité des nouveaux flots en fonction de la probabilité de perte. Le niveau de confiance représente la probabilité qu'il n'y ait pas de perte si un nouveau flot est admis en fonction des mesures de débit courantes, il est introduit car il n'y a pas de garantie que le débit restera toujours borné

par l'enveloppe calculée. Il est déterminé à partir de la distribution de l'enveloppe de débit, la moyenne et la variance.
L'enveloppe de débit ainsi calculée n'utilise pas l'approximation Gaussienne mais une distribution de Gumbel, ce qui permet de l'appliquer même dans le cas où le nombre de sources multiplexées est faible ou si elles sont réparties sur plusieurs classes. La mesure du débit maximum de l'agrégat sur un intervalle permet de capturer les variations sur de petites échelles de temps, tandis que la mesure de la variance prend en compte les fluctuations sur des échelles plus importantes. Un nouveau flot déclare son débit crête ainsi que son délai maximum par le biais d'une signalisation RSVP, il est accepté si sa contrainte de délai est respectée avec une probabilité liée au niveau de confiance. Le modèle ne fait aucune hypothèse particulière sur le type des sources.

Floyd [44] a proposé un algorithme de MBAC destiné à la classe *controlled load* (CL) de Intserv, il est basé sur la mesure de la capacité équivalente (CE) pour l'agrégat de trafic. Le contrôle d'admission s'applique au trafic CL policé par un *leaky bucket*. La CE est évaluée grâce à *Hoeffding bounds* en tenant compte d'une faible probabilité de perte ϵ. Une nouvelle connexion est acceptée si la somme de la CE de sa classe associée et le débit crête est inférieure à la bande passante allouée à cette classe. La condition d'admission peut être formulée avec : $CE + p_n \leq C$ où CE est la capacité équivalente estimée de la classe, p_n est le débit crête du flot soumis au contrôle d'admission et C est la capacité allouée à la classe.
L'algorithme applique la stratégie de *back off* déjà utilisée par Gibbens et al [47], de ce fait il nécessite la détection des instants de fin des connexions. La méthode suppose également qu'il n'y pas de corrélations dans le temps entre les débits calculés, ce qui n'est pas le cas lors d'une audio conférence par exemple. En outre, la valeur optimale de ϵ devrait être trouvée via des expérimentations.

Notons également que les études basées sur la capacité équivalente, également appelée *effective bandwidth*, qui représente la bande passante dont aurait besoin le nouveau flot, considèrent souvent un nombre de flots constant pour la calculer ; ce qui n'est pas le cas dans la réalité.

Les algorithmes cités ci-dessus mesurent la moyenne voire la variance du trafic. Reisslein [93] a proposé une technique de MBAC basée sur la théorie des grandes déviations qui utilise la fonction génératrice des moments du processus d'arrivée des flots. Les mesures sont effectuées sur l'agrégat du trafic. Les débits crêtes des flots sont limités à l'entrée du réseau. Les mesures se font en moyennant les arrivées sur des slots, en accordant plus de poids aux mesures les plus récentes ; et en tenant compte des débits crêtes des flots nouvellement admis car ils ne sont pas reflétés immédiatement dans les mesures.
La condition d'admission se base sur un taux de perte maximum fixé pour un multiplexage *bufferless*. Une nouvelle connexion est acceptée si la probabilité de perte estimée est inférieure à cette valeur fixée (10^{-6}, par exemple).
L'auteur a comparé sa méthode avec celles de Jamin et. al. [58] ainsi que Grossglau-

ser et Tse [53] via des simulations sur des traces de vidéos MPEG 1. L'algorithme de Reisselein permet d'admettre plus de flots que les deux autres tout en maintenant approximativement la même probabilité de perte.
L'utilisation de la fonction génératrice des moments n'étant pas envisageable en temps-réel pour appliquer un contrôle d'admission, l'intérêt de cette technique s'en trouve réduit.

Etudes comparatives

Quelques travaux de comparaisons entre différents algorithmes de MBAC ont été menés [24], [105]. Les algorithmes ont été testés en termes de taux d'utilisation et de pertes, dans la mesure où chacun d'entre eux tente de maximiser l'utilisation tout en minimisant les pertes qui constituent le critère de QoS pour les flots. En effet, les algorithmes testés disposent de paramètres qui leur permettent de contrôler l'agressivité des admissions et donc de maîtriser les pertes.
Jamin et. al. [24] ont comparé les six (6) algorithmes suivants :
- *Measured Sum* (MS) : décrit précédemment [58], son estimateur utilise une fenêtre temporelle.
- *Hoeffding Bounds* (HB) : développé par Floyd [44], son estimateur utilise un lissage exponentiel.
- *Tangent at Peak* (TP) : présenté dans [48], il utilise la tangente au *peak* de la courbe de *equivalent bandwidth* obtenue grâce aux *Chernoff bound*s. Un nouveau flot est admis si : $np(1-e^{-sp})+e^{-sp}\widehat{\nu} \leq C$ où n est le nombre de flots admis, p est le débit crête des flots, s est un paramètre de *Chernoff bounds*, $\widehat{\nu}$ est la charge mesurée et C est la capacité du lien. Cet algorithme utilise un estimateur du type *point sample*, i.e. il considère uniquement la valeur calculée sans tenir compte de l'historique.
- *Tangent at Origin* (TO) : décrit également dans [48], il utilise une tangente à l'origine de la courbe de *equivalent bandwidth*. Un nouveau flot est admis si l'équation suivante est satisfaite : $e^{sp}\widehat{\nu} \leq C$
L'estimateur employé est la valeur instantanée (*point sample*).
- *Measure CAC* (MC) : cet algorithme [33] basé sur la théorie des grandes déviations admet un nouveau flot si la somme du débit crête du flot en question et la mesure de la bande passante occupée par les flots présents est inférieure à la capacité du lien. L'estimation de la bande passante cumulée est évaluée en tenant compte d'une probabilité de perte cible.
- *Aggregate Traffic Envelopes* (TE) : déjà décrit dans la section précédente [90].
Ces algorithmes diffèrent dans leur théorie de base, dans leurs conditions d'admissions ainsi que dans leurs processus de mesures.

Les comparaisons ont été effectuées via des simulations *ns*. Les résultats de simulations ont indiqué que ces différents algorithmes ont pratiquement les mêmes performances (courbes utilisation vs. pertes presque identiques), les légères différences qui sont apparues sont dues aux estimateurs et non aux conditions d'admission.

Tous les algorithmes réalisent une bonne utilisation, mais ne respectent pas le taux de perte annoncé en fonction de leur paramétrage. Par conséquent, une opération de calibrage est nécessaire pour déduire les paramètres adéquats. Les simulations ont également montré qu'un algorithme basé sur un nombre maximum de flots à accepter fixe est moins performant quand on a affaire à la LRD (*Long Range Dependance* ou dépendance à long terme), ceci s'explique par le fait que cette dernière induit des variations dans le trafic sur plusieurs échelles de temps et l'algorithme n'adapte pas son nombre de flots maximum à ces variations. De ce fait la LRD peut constituer une raison supplémentaire pour utiliser les MBAC, d'autant plus qu'elle fait qu'une petite augmentation du nombre de flots peut engendrer une nette augmentation de la perte. Les MBAC sont connus pour leur faculté à s'adapter à ce genre de changement.

D'après Mandjes et van der Berg [105], les MBAC ont de meilleurs performances avec un trafic Pareto qu'avec exponentiel *on/off* concernant le taux d'utilisation réalisé. Ils ont testé deux algorithmes, à savoir MS [58] et une variante de l'approche présentée par Brichet et Simonian [25] par la biais de simulations. Les conclusions étaient identiques à celles trouvées par Jamin et. al. [24] à savoir que les deux algorithmes ont des performances très proches.

Il est à noter que d'après les travaux de Reisslein [93], les performances des différents algorithmes de MBAC avec *bufferless multiplexing* sont légèrement différentes entre elles. Chose qui n'apparaissait pas avec l'utilisation d'un buffer dans [24] et [105]. Les simulations ont indiqué que l'algorithme de Grossglauser et Tse [53] permet d'admettre plus de flots que celui de [58] (MS) pour un taux de perte equivalent ; la mesure de la variance conduit ainsi à une meilleure utilisation.

Les différents algorithmes de MBAC présentés utilisent deux types de multiplexage : avec buffer et *bufferless*. Le dimensionnement de la taille du buffer peut nécessiter la connaissance de certaines caractéristiques des sources, mais il permet d'augmenter le taux d'utilisation. Tandis qu'avec *bufferless multiplexing*, la probabilité de perte ne dépend que de la distribution du débit et non pas d'éventuelles corrélations (y compris la propriété d'auto-similarité) au niveau paquet. C'est pourquoi, le multiplexage sans buffer constitue, à notre avis, une méthode plus robuste.

Dans la suite de ce chapitre, nous présentons deux algorithmes MBAC adaptés pour *cross protect*, l'un repose sur une approximation Poisson du nombre de rafales, et l'autre sur une approximation Gaussienne du débit global.

3.3 Algorithme MBAC basé sur l'approximation Poisson du nombre de rafales

On a pu voir au cours du chapitre 2 que la garantie d'un faible délai pour les paquets des flots *streaming* passe par une limitation de la valeur du *priority load*.

Dans cette section, nous décrivons la façon dont ce seuil est calculé.

Pour le traitement des flots dont les paquets sont insérés dans la partie prioritaire de la file avec le mécanisme *cross-protect* nous préconisons un algorithme de MBAC basé sur une approche de multiplexage sans buffer. L'algorithme en question doit être simple et adapté aux connaissances minimes du trafic dont on dispose. En effet, les instants de départ des flots sont inconnus, les instants d'arrivées ne sont que partiellement connus, puisque le contrôle d'admission est effectué de manière distribuée au sein du routeur (une instance de décision par port d'entrée) et le nombre de flots dans le système n'est pas disponible (même si une estimation peut être obtenue). De plus, les caractéristiques détaillées des flots, autres qu'un débit crête maximum, ne peuvent être prises en compte. Ainsi, les algorithmes présentés dans le paragraphe 3.2 ne sont pas adaptés pour *cross-protect* dans la mesure où ils exigent la connaissance, d'une façon ou d'une autre, d'informations que l'on ignore.

Par conséquent, nous avons été amenés à proposer un algorithme de MBAC, simple à appliquer, orienté *cross-protect*.

3.3.1 Description

On suppose que le trafic est modélisé par un processus stationnaire qui est conforme au modèle Poisson de session introduit dans le paragraphe 2.3 de l'introduction, i.e. que les sessions arrivent selon un processus de Poisson et qu'elles sont constituées d'une succession de flots et de temps de réflexion. La seule information sur le trafic dont on dispose est que le débit crête maximum de tous les flots est p. On suppose également que ces flots émettent des paquets de taille MTU, ce qui constitue un pire cas dans le cadre de notre étude [19], selon un processus *on/off* de rafales (*bursts*) [72].

Sous ces hypothèses, et en l'absence d'un contrôle d'admission, le nombre de rafales suit une loi de Poisson. En effet, une session peut être modélisée comme étant un client visitant un réseau stochastique composé de deux files infinies, le premier serveur représente l'émission des rafales et le second les périodes de silence (inter-rafales). Les visites successives à ces deux files s'effectuent de manière alternée. Il est connu que la distribution du nombre de client dans ce genre de réseau est la même que si les arrivées internes (rafales) étaient Poisson [62]. On sait alors que le nombre de clients dans une file $M/G/\infty$ suit une distribution de Poisson i.e. :

$$Pr[\text{nombre de rafales} = n] = \frac{\lambda^n}{n!} e^{-\lambda}$$

où λ est le taux d'arrivée des rafales égal aussi au nombre de rafales moyen.

Le rôle du contrôle d'admission est d'assurer une probabilité de débordement (dépassement de la capacité C) ϵ faible. Soit Λ la mesure du débit agrégé des flots sur le lien (demande de trafic en bits/s), $Pr[\Lambda > C]$ représente la probabilité de débordement. Nous évaluons la charge maximum tolérée Λ_ϵ correspondante à cette

probabilité de débordement comme suit :

$$\sum_{np>C} \frac{(\Lambda_\epsilon/p)^n}{n!} e^{-(\Lambda_\epsilon/p)} = \epsilon$$

Où n représente le nombre de rafales et p le débit crête considéré.
Une probabilité de débordement de 10^{-2} peut être visée vu que l'utilisation d'un buffer au niveau paquet permet d'absorber momentanément l'excédant de trafic.

Soit s le seuil d'admission, on a alors :

$$s = \Lambda_\epsilon / C$$

La valeur du seuil dépend essentiellement du rapport C/p. Certaines valeurs typiques sont consignées dans le tableau 3.1. Comme on peut le constater, lorsque C/p est élevé (1000 par exemple), le seuil d'admission est important (supérieur à 0.9). Ceci pourrait être le cas typiquement sur les liens *backbone*.
Ainsi, un nouveau flot est admis si la mesure courante du *priority load* (PL) est inférieure au seuil s.

C/p	100	100	1000	1000
ϵ	0.01	0.001	0.01	0.001
Seuil	0.79	0.73	0.93	0.91

TAB. 3.1 – Seuils d'admission en fonction de C/p et ϵ

La mesure de la charge due au PL s'effectue sur des intervalles de durée fixe égale à MTU$\times 8/p$ (période pendant laquelle tout flot envoie au plus MTU octets). Un lissage exponentiel de paramètre α est effectué afin de tenir compte des mesures passées et d'assurer une meilleure stabilité.

3.3.2 Evaluation préliminaire par simulation

Dans un premier temps, nous effectuons une évaluation préliminaire de l'algorithme MBAC basé sur la la loi de Poisson afin de déduire l'impact du débit des flots sur la performance. Pour ce faire, nous avons conduit des simulations *ns* en utilisant une topologie composée de 4 sources de trafic reliées à 4 destinations via un lien *bottleneck* d'une capacité de 10 Mb/s, où le contrôle d'admission est appliqué. La taille du buffer est fixée à 100 paquets.
Les sources génèrent du trafic UDP de type *on/off*. La durée des flots suit également une loi exponentielle de moyenne 1 minute. La taille des paquets est 1000 octets. Le processus d'arrivées des flots est Poisson et la durée des simulations est 500 secondes, les statistiques étant récoltées sur les 350 dernières secondes.

Deux scénarios de simulations ont été considérés, l'un où les flots ont un débit crête de 64 Kb/s et un débit moyen de 32 Kb/s (la durées des périodes *on* et *off* est exponentielle de moyenne 500ms) et l'autre où les flots ont un débit crête de 32

Kb/s et un débit moyen de 28.8 Kb/s (les périodes *on* et *off* sont exponentiellement distribuées de moyennes respectives de 900ms et 100ms). Dans chaque scénario, 5 charges offertes ont été testées : 0.8, 0.9, 1, 1.1 et 1.2. Le but de ces simulations est d'observer la performance de l'algorithme basé sur la loi de Poisson décrit dans le paragraphe 3.3.1 en termes d'utilisation, délais, probabilité de débordement et blocage.

Pour calculer le seuil d'admission, on considère que le débit crête maximum est 64 Kb/s. Ainsi, on peut évaluer l'impact du choix d'un débit crête plus grand que le véritable débit des flots dans le scénario où il vaut 32 Kb/s. On applique le contrôle d'admission, dans les deux scénarios, avec un seuil égal à 0.88 correspondant à une probabilité de débordement de $5\ 10^{-2}$. La durée de l'intervalle de mesure est 125ms (8K/64K), la charge est mesurée avec un lissage exponentiel de paramètre $\alpha = 0.9$. Dès qu'une mesure dépasse le seuil, tous les flots arrivant pendant le prochain intervalle sont bloqués.

La figure 3.1 de gauche indique, pour les valeurs de charges offertes considérées lorsque le débit crête est 64 Kb/s, le taux d'utilisation du lien et la probabilité de débordement. Cette dernière est égale à la probabilité d'avoir plus de 156 paquets durant un même intervalle de mesure. En effet, l'émission de 157 paquets de taille 1K octets dans un intervalle de 125ms conduit à un débit agrégé supérieur à la capacité du lien *bottleneck*. On constate que la probabilité de débordement augmente, comme prévu, avec l'utilisation. Néanmoins, elle reste inférieure à $5\ 10^{-2}$. Le taux d'utilisation tend progressivement vers le seuil d'admission au fur et à mesure que la charge offerte croît.
Dans le scénario où le débit crête est 32 Kb/s, la probabilité de débordement est 10^{-3} lorsque la charge vaut 1.2 ; elle est nulle pour les charges inférieures à 1.2. Ceci s'explique par le fait que le choix du seuil d'admission est basé sur un débit crête plus important : 64 Kb/s. On considère que la probabilité de débordement est la probabilité que le nombre de paquets soit supérieur à 312 sur un intervalle d'une durée de 250ms.

La figure 3.1 de droite trace la probabilité que le délai subi par les paquets soit plus grand que 3ms (cette valeur est choisie uniquement à titre indicatif) pour les deux scénarios. Les délais sont légèrement plus importants lorsque le débit crête est 32 Kb/s du fait que l'utilisation associée l'est aussi. Néanmoins, cette différence est minime.

La figure 3.2 représente le taux de blocage des flots en fonction des charges offertes. Comme on peut le constater, le blocage ne dépend pas des caractéristiques du trafic (débit crête dans ce cas) mais uniquement de la charge totale. En effet, le taux de blocage théorique est donné par :

$$(ch - u)/ch$$

Où ch représente la charge offerte et u l'utilisation du lien. La courbe du blocage théorique de la figure 3.2 est obtenue à partir des taux d'utilisations atteints dans les simulations du scénario de débit crête 32 Kb/s : 0.8 (où la charge offerte après simulation était 0.81), 0.84, 0.86, 0.864 et 0.87. Cette courbe est très proche de celle

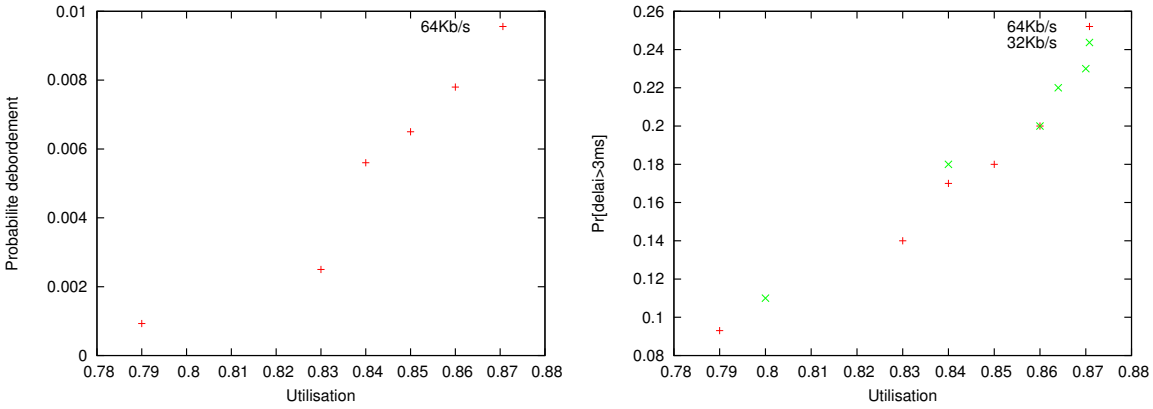


FIG. 3.1 – Probabilité de débordement (gauche) et délais (droite) vs. utilisation, trafic synthétique

obtenue par simulation.

La figure 3.3 est obtenue lorsque le seuil d'admission est 0.91 dans le scénario où le débit crête vaut 32 Kb/s correspondant à une probabilité de débordement cible de 5 10^{-2}, l'intervalle de mesure est de durée 250ms. Elle trace la probabilité que le délai des paquets dépasse 3ms en fonction de l'utilisation réalisée suite aux charges offertes précédentes. Cette courbe confirme que le taux d'utilisation peut être amélioré au prix d'une augmentation des délais.

Vérification de l'hypothèse Poisson pour le nombre de rafales

La figure 3.4 de gauche représente la distribution du nombre de paquets observés par intervalle (de durée 125ms) dérivée à partir de la simulation de la charge offerte 0.8 du trafic *on/off* avec un débit crête de 64 Kb/s décrite dans le paragraphe précédent. On constate que cette distribution est bien approximée par la loi de Poisson de même moyenne (126.5) correspondante à Λ/p; en tenant compte des imprécisions de la simulation dues à la discrétisation de l'intervalle de mesure.

La figure 3.4 de droite trace la distribution du nombre de paquets arrivés pendant chaque intervalle de mesure de durée 12ms (1500 $\times 8/1$ M) issue de la trace de trafic ADSL qui sera introduite dans la section 5.3. Dans ce cas, on a considéré un débit crête maximum égal à 1 Mb/s correspondant au débit d'accès des lignes ADSL.
Notons que même si la majorité des flots de la trace en question sont élastiques (trafic TCP), ils ont un débit qui représente une faible fraction de la capacité du lien (155 Mb/s), et par conséquent, ils peuvent être assimilés à du trafic *streaming* pour les besoins de cette étude.
On remarque, que la distribution du nombre de paquets par intervalle est bien approximée par une loi de Poisson de même moyenne (96) en dépit de quelques imprécisions inhérentes (les flots n'émettent pas forcément au débit crête et la taille des paquets n'est pas toujours MTU).

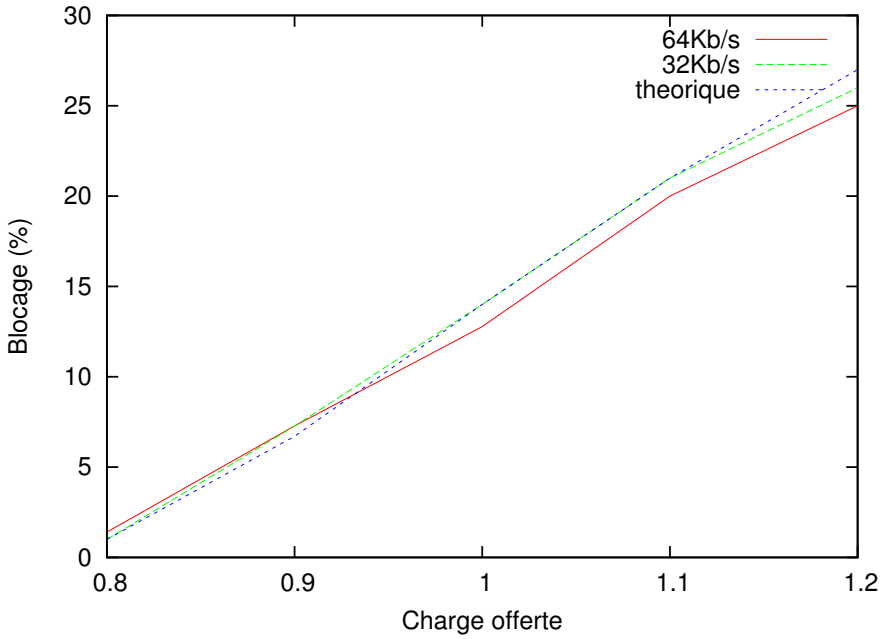


FIG. 3.2 – Taux de blocage en fonction de la charge offerte

Impact du paramètre de lissage α

Afin d'étudier l'impact du coefficient de lissage exponentiel α sur les performances de l'algorithme de MBAC basé sur la loi de Poisson, nous avons conduit des simulations avec une valeur de α adaptative. D'après [64, 63], la valeur adaptative est ajustée en fonction des "erreurs" e (bruits) de mesure où :

$$e = \frac{|ch_l - ch_i|}{ch_l}$$

avec ch_l représentant la charge lissée et ch_i la mesure de charge instantanée. La valeur de e est calculée avec un lissage exponentiel également. Le paramètre α est inversement proportionnel à e, lorsque e est élevée, α doit être petit afin que la mesure soit réactive. Quand e est faible, α doit être grand car la mesure est stable.

Les simulations indiquent que la valeur de α n'influe pas significativement sur les performances en termes d'utilisation du lien et de délais des paquets. Les différences qui en découlent, par rapport à $\alpha = 0.9$, sont minimes.

D'autres simulations conduites avec des valeurs de α fixes différentes de 0.9 confirment que ce paramètre n'a pas d'impact prépondérant sur les résultats, la valeur 0.9 assure néanmoins une meilleure stabilité.

Avant de procéder à une évaluation plus complète de l'algorithme MBAC décrit dans la section 3.3.1, notamment avec du trafic hétérogène et vidéo, nous présentons dans ce qui suit un algorithme à seuil adaptatif.

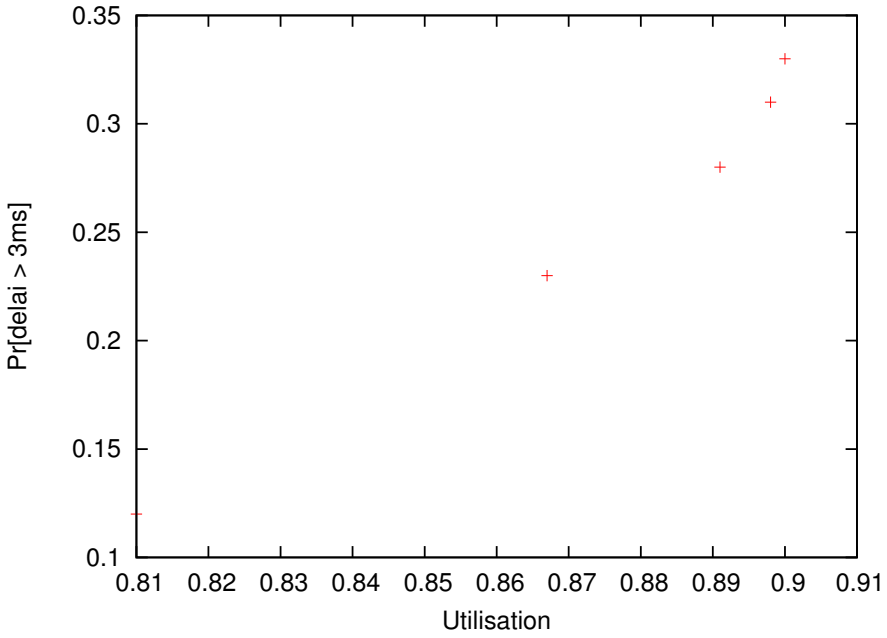


FIG. 3.3 – Délais vs. utilisation, seuil d'admission = 0.91, débit crête = 32 Kb/s

3.3.3 Algorithme adaptatif

L'algorithme adaptatif tente d'améliorer les performances du MBAC basé sur la loi de Poisson pour le nombre de rafales en réajustant le seuil d'admission dynamiquement en fonction de la probabilité de débordement observée à long terme. Si la probabilité de débordement observée est inférieure à la probabilité cible, le seuil d'admission est incrémenté pas à pas. En revanche, si la probabilité de débordement effective est plus importante que celle voulue, le seuil est décrémenté. Le seuil est maintenu dans un intervalle compris entre une valeur minimum et une valeur maximum (ces deux valeurs peuvent être déterminées en choisissant des probabilités de débordement cible très faibles ou relativement importantes).

On considère qu'un débordement survient à chaque fois que le débit cumulé des flots dépasse la capacité du lien. La mesure du débit cumulé se fait via un lissage exponentiel de paramètre β sur des intervalles de longueur fixe qui doit garantir à la fois la stabilité et la réactivité des mesures (si la durée est trop grande, le débordement n'est pas observé, et si elle est trop petite, l'arrivée de quelques paquets simultanément risque de conduire à un débordement non significatif). Les résultats de simulations indiquent que le choix d'une durée de 10ms et $\beta = 0.9$ est adéquat. Notons que cet intervalle est différent de celui où la charge moyenne est mesurée pour les besoins du contrôle d'admission, puisque il s'agit de mesures indépendantes.
Le taux de débordement à long terme est mesuré sur une fenêtre glissante qui tient compte des x dernières mesures de débit cumulé. Lorsque la probabilité de débordement cible ϵ est 10^{-2}, une valeur de $x = 1000$ semble être suffisante. L'évaluation

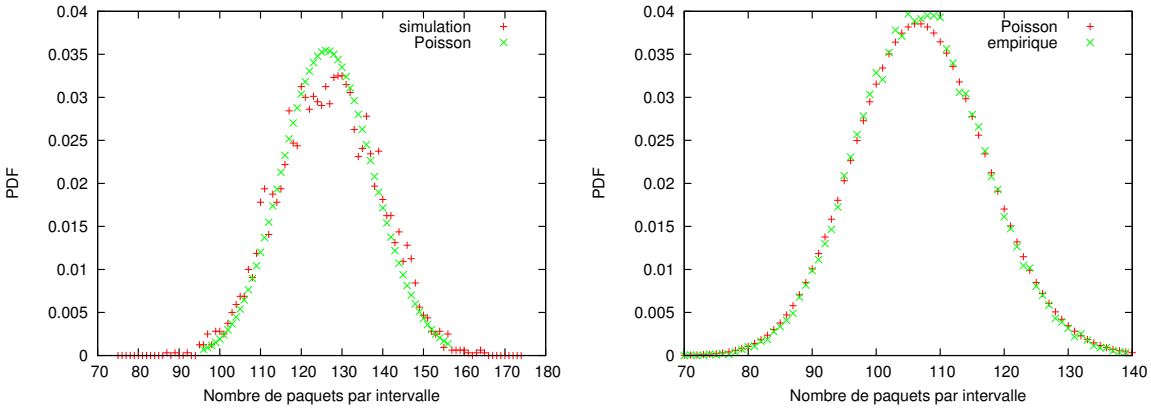


FIG. 3.4 – Distribution du nombre de paquets par intervalle avec du trafic *on/off* (gauche) et la trace ADSL (droite)

de cet algorithme sera présentée dans la section 3.5, où le seuil maximum a été fixé à 0.95, le minimum à 0.7, le pas d'incrémentation et décrémentation est 0.01 et la durée de l'intervalle de mesure des débordements est 10ms.

Comme on peut le voir, les algorihtmes basés sur la loi de Poisson (à seuil fixe et adaptatif) ne mesurent que la moyenne de la charge. Afin de savoir si la mesure de la variance de la charge permettrait d'améliorer les perforamnces, nous avons testé un algorithme basé sur l'approximation du débit total par la loi Normale en gardant la même approche de multiplexage sans buffer. Cet algorithme est décrit dans le prochain paragraphe.

3.4 Algorithme MBAC basé sur l'approximation Gaussienne du débit agrégé

En s'inspirant des travaux de Grossglauser et Tse (décrits dans le paragraphe 3.2), nous avons mis au point un algorithme qui se base sur une approximation Gaussienne concernant l'agrégat de débit des flots afin de déterminer le seuil d'admission. Ainsi, un nouveau flot est admis si la mesure de la charge est inférieure à s, où :

$$s = 1 - \alpha_q \frac{\sigma}{C},\ \alpha_q = Q^{-1}(\epsilon)$$

avec $Q(.)$ est la distribution complémentaire de la loi Normale centrée réduite, C la capacité du lien, ϵ la probabilité de débordement cible et σ la mesure de la variance du débit agrégé des flots. La mesure de la variance se fait sur une fenêtre comportant K valeurs de charge instantanées, alors que la charge est mesurée par un lissage exponentiel sur des intervalles de durée fixe. Lors des simulations que l'on présentera dans la prochaine section, la durée de l'intervalle de mesure est 100ms, $K = 100$ et $\alpha_q = 2.3$ correspondant à $\epsilon = 10^{-2}$.

3.5 Résultats comparatifs de simulation

Après avoir décrit les différents algorithmes proposés, nous procédons à une comparaison des méthodes basées sur la loi de Poisson avec seuil fixe et adaptatif ainsi que de l'algorithme basé sur la loi Normale par le biais de simulations *ns* en utilisant divers modèles de trafic. Dans ce qui suit on se réferera à ces algorithmes, respectivement, par Poisson, adaptatif et Gaussien.

3.5.1 Trafic synthétique

Deux scénarios de trafic de type *on/off* ont été simulés.

Flots *on/off* homogènes

Nous avons simulé une charge offerte $\rho = 0.9$ avec des flots *on/off* à débit crête 64 Kb/s et débit moyen 32 Kb/s selon les mêmes modalités spécifiées dans la section 3.3.2. Le seuil fixe utilisé par l'algorithme de Poisson est 0.83 correspondant à $\epsilon = 10^{-2}$. La durée de l'intervalle de mesure de la charge est 125ms (8K/64K) pour les algorithmes Poisson et adaptatif.

Le tableau 3.2 dresse une comparaison entre les trois algorithmes, et fournit les résultats correspondants à un scénario sans contrôle d'admission. On constate que même en charge normale (<1), le délai moyen des paquets est élevé (8.8ms) quand le contrôle d'admission n'est pas appliqué. L'algorithme adaptatif permet d'avoir une utilisation légèrement plus importante que les autres, tout en maintenant des délais satisfaisants (1.3ms en moyenne). Aucun débordement (nombre de rafales supérieur à 156) n'a été observé avec les trois algorithmes.

Indicateurs de performance	Poisson	adaptatif	Gaussien	sans CA
Utilisation	0.8	0.83	0.81	0.9
Délai moyen (ms)	1	1.3	1.1	8.8
Pr[délai > 10ms]	$5\ 10^{-4}$	$2\ 10^{-3}$	$6\ 10^{-4}$	0.17
Blocage (%)	10	8	9	-

TAB. 3.2 – Simulation flots *on/off* homogènes, $\rho = 0.9$

Flots *on/off* hétérogènes

Dans ce scénario les flots appartiennent à deux classes en fonction de leur débit crête. La moitié de la charge est due aux flots avec un débit crête de 200 Kb/s et l'autre moitié est composée de flots à débit crête 40 Kb/s. Le débit moyen des deux classes est 20 Kb/s.

Le seuil d'admission de l'algorithme Poisson est 0.73 correspondant à $\epsilon = 10^{-2}$ obtenu en considérant le débit crête maximum à savoir 200 Kb/s. La durée de l'intervalle de mesures est de 40ms (8K/200K) pour les algorithmes Poisson et adaptatif.

Le tableau 3.3 indique les performances obtenues. Encore une fois, sans contrôle d'admission les délais des paquets sont importants. Le contrôle d'admission préserve la performance. L'amélioration apportée par l'algorithme adaptatif au taux d'utilisation par rapport à Poisson est plus importante dans ce cas. En effet, la présence de flots dont le débit est inférieur au débit crête considéré rend l'algorithme à seuil fixe conservateur. On peut s'attendre à ce que l'écart entre les deux algorithmes augmente lorsque la proportion des flots avec le débit crête maximum diminue. L'algorithme Gaussien arrive lui aussi à mieux s'adapter aux variations de débits crêtes, il offre une utilisation moyenne de 0.76.
Notons que la probabilité de débordement (somme des débits des flots dans l'état *on* supérieure à la capacité) est pratiquement la même avec les trois algorithmes, elle vaut 8 10^{-3}.

Indicateurs de performance	Poisson	adaptatif	Gaussien	sans CA
Utilisation	0.71	0.78	0.76	0.9
Délai moyen (ms)	0.69	1	0.88	8.9
Pr[délai $>$ 10ms]	9 10^{-5}	3 10^{-3}	10^{-3}	0.19
Blocage (%)	20	14	16	-

TAB. 3.3 – Simulation flots *on/off* hétérogènes, $\rho = 0.9$

3.5.2 Trafic vidéo

Afin de tester les algorithmes avec un autre type de flots *streaming*, nous avons effectué des simulations avec du trafic vidéo. Quatre vidéos codées en H263 VBR (*Variable Bit Rate*) ont été utilisées [114] pour obtenir des débits hétérogènes, les vidéos en question ainsi que leurs principales caractéristiques sont spécifiées dans le trableau 3.4. Leurs variations de débit sont indiquées dans le paragraphe 1 de l'annexe 3. Les traces vidéos consistent en une suite de trames chacune avec une taille donnée et un temps associé.

Vidéos	Débit crête (Mb/s)	Débit moyen (Mb/s)	Durée (minute)
Formule 1 (F1)	2.8	0.533	30
Star Wars (SW)	1.3	0.115	45
Star Trek (ST)	2.2	0.170	60
South Park (SP)	2.5	0.310	20

TAB. 3.4 – Vidéos simulées

La topologie simulée est celle spécifiée dans 3.3.2, les flots arrivent selon un processus de Poisson sur le lien *bottleneck* de capacité 280 Mb/s, la taille du buffer

est 2800 paquets. La charge offerte est $\rho = 1$. La durée des simulations est 500 secondes.
Chaque nouveau flot commence à un point aléatoire de la trace vidéo correspondante, il a une durée exponentielle de moyenne 1 minute. Les paquets sont émis aux instants indiqués par la trace associée ; quand la fin de la vidéo est atteinte l'émission des paquets reprend depuis le début.

Lorsque la taille de la trame est supérieure à 1000 octets, elle est fragmentée en plusieurs paquets d'une taille maximum 1000 octets. Ces paquets sont envoyés de deux façons possibles :

1. type1 : les paquets sont émis avec un débit constant égal à 2.8 Mb/s (débit crête maximum).
2. type2 : les paquets sont émis avec un débit variable égal, à l'échelle trame, à la taille de la trame divisée par la durée de l'inter-trame, sachant que les trames ont des tailles variables. Dans ce cas, le débit crête maximum considéré est 2.8 Mb/s correspondant à la vidéo de F1.

Le seuil d'admission pour l'algorithme Poisson est fixé à 0.83 et la durée de l'intervalle de mesure est 2.8ms (8K/2.8M).

Le tableau 3.5 indique que l'algorithme Poisson est conservateur dans le sens où il réalise une utilisation nettement inférieure à celle obtenue avec les algorithmes adaptatif et Gaussien qui aboutissent à une utilisation supérieure à 0.9 avec des délais moyens de l'ordre de 0.55ms. En effet, sur des liens à grande capacité, la vitesse de transmission d'un paquet de taille MTU étant faible, les délais qui en résultent ne sont pas importants même sous une charge aussi élevée. A nouveau, la présence de flots dont le débit est inférieur au débit crête maximum pénalise l'approche Poisson. Notons que l'algorithme adaptatif appliquait le seuil maximum (0.95) pendant la majeur partie du temps (une fois le régime stationnaire atteint), vu qu'il n'y avait pas de débordement observé.
Sans contrôle d'admission, le délai moyen est catastrophique, il est égal à 37ms.

Des résultats similaires, rapportés dans le tableau 3.6 sont obtenus avec le trafic vidéo de type2, où le taux d'utilisation atteint avec l'algorithme Gaussien est le plus important (0.94 contre respectivement 0.93 et 0.82 pour adaptatif et Poisson). Signalons que les délais moyens dans ce cas sont inférieurs à ceux induits par le trafic de type1 (0.34ms avec l'algorithme Gaussien), car les flots atteingnent le débit crête moins souvent (puisque les paquets des trames dont la taille est supérieure à 1000 octets sont émis avec un débit variable inférieur au débit crête). D'autres simulations où les paquets des trames de tailles supérieures à 1000 octets sont envoyés successivement les uns à la suite des autres (avec un débit égal donc au débit du lien) ont été conduites. Qualitativement, elles fournissent le même type de résultat.

3.5.3 Trafic réel

Dans le but de confronter les algorithmes de MBAC à un contexte plus réaliste, nous avons simulé des traces de trafic réelles collectées sur un lien ADSL de 155Mb/s

Indicateurs de performance	Poisson	adaptatif	Gaussien	sans CA
Utilisation	0.81	0.92	0.93	1
Délai moyen (ms)	0.38	0.54	0.56	37
Pr[délai $>$ 10ms]	0	0	0	0.6
Blocage (%)	17	9	8	-

TAB. 3.5 – Simulation trafic vidéo type1, $\rho = 1$

Indicateurs de performance	Poisson	adaptatif	Gaussien	sans AC
Utilisation	0.82	0.93	0.94	1
Délai moyen (ms)	0.18	0.3	0.34	36
Pr[délai $>$ 10ms]	0	0	0	0.58
Blocage (%)	17	9	7	-

TAB. 3.6 – Simulation trafic vidéo type2, $\rho = 1$

du réseau de France Télécom. Une de ces traces sera décrite de façon plus détaillée dans la section 5.3.

Les flots de la trace étant limités par un débit d'accès de 1 Mb/s, on les assimile à du trafic *streaming* même s'ils correspondent à du trafic TCP dans leur grande majorité. Nous avons produit une charge offerte de $\rho = 1$ en multiplexant 4 traces de trafic sur un lien à 155 Mb/s. La durée des simulations est 100 secondes.

Le seuil d'admission de l'algorithme Poisson est fixé à 0.83 (correspondant à une probabilité de débordement 10^{-2} en considérant un débit crête de 1 Mb/s). La durée de l'intervalle de mesure est 12ms (12K/1M).

Le tableau 3.7 confirme les résultats obtenus avec du trafic vidéo, au sens où les algorithmes adaptatif et Gaussien réalisent un meilleur taux d'utilisation que Poisson tout en garantissant de faibles délais moyens (0.52ms). Le taux d'utilisation atteint par l'algorithme Poisson est proche du seuil, la méthode adaptative conduit à l'utilisation la plus élevée et par conséquent à un taux de blocage moindre.
Naturellement, sans contrôle d'admission, les délais des paquets sont extrêmement importants (47ms en moyenne).

Indicateurs de performance	Poisson	adaptatif	Gaussien	sans CA
Utilisation	0.82	0.91	0.87	1
Délai moyen (ms)	0.27	0.52	0.51	47
Pr[délai $>$ 10ms]	10^{-4}	10^{-3}	10^{-3}	0.95
Blocage (%)	29	13	23	-

TAB. 3.7 – Simulation trafic réel, $\rho = 1$

3.5.4 Discussion et perspectives

En résumé on a pu voir que dans le cadre de flots *on/off* homogènes, les performances des trois algorithmes sont très proches.
Lorsqu'on a affaire a du trafic hétérogène, où les débits d'une partie des flots sont inférieurs au débit crête considéré, les algorithmes adaptatif et Gaussien offrent de meilleurs performances que Poisson à seuil fixe. Il en est de même pour du trafic vidéo et réel où le taux d'utilisation est amélioré sans dégrader les délais des paquets de manière inacceptable.
Néanmoins, l'algorithme adaptatif nécessite un bon paramétrage (intervalle de mesure du débordement, incrémentation et décrémentation du seuil) ce qui soulève la question de sa de robustesse vis à vis du choix de ces paramètres.
L'algorithme Poisson présente l'avantage d'être simple indépendamment du type de trafic au prix d'une légère baisse du taux d'utilisation par rapport à celle qu'on pourrait atteindre si on disposait de plus d'informations. Cependant, sur des liens à grande capacité où le rapport C/p est important, un taux d'utilisation aussi élevé que 90% peut être atteint. De plus, il nécessite une mémoire de mesure plus faible.
Ainsi, l'algorithme basé sur la loi Normale semble offrir le meilleur compromis entre performance et robustesse même si le choix de la mémoire des mesures à prendre en compte (la valeur de K ainsi que la durée de l'intervalle) est important. Une faible mémoire de mesure peut conduire à sous estimer la variance, et par conséquent à admettre trop de flots.

Les liens du réseau fonctionnent dans la plupart du temps sous des charges normales qui ne sont pas très proches de 1. Le contrôle d'admission doit pouvoir protéger les flots en cours lorsqu'une panne surgit causant une augmentation subite importante de la charge. Une panne implique un taux d'arrivée, de flots vus comme nouveaux, élevé dans un intervalle de mesure donné. Comme l'acceptation des nouveaux flots ne se répercute pas immédiatement dans la mesure de la charge moyenne, il faut mettre au point une parade pour éviter d'admettre trop de flots en un seul coup. L'addition du débit crête à la mesure de la charge suite à chaque admission d'un flot permettrait de régler ce problème. Toutefois, cette approche peut conduire à une sous-utilisation du lien, notamment lorsque la majorité des flots sont courts et n'atteignent pas, par conséquent, le débit crête considéré. Ce phénomène a pu être vérifié avec les simulations effectuées sur la trace ADSL. La gestion de ce type d'événement pourrait faire l'objet de travaux de recherche futurs.

Chapitre 4

Etat de l'art sur les mécanismes de partage équitable de la bande passante

Au cours de ce chapitre, nous donnons un aperçu sur les mécanismes de partage équitable de la bande passante existants. Dans un premier temps, nous décrivons, de manière non exhaustive, certains algorithmes de *fair queueing* connus. Par la suite, nous nous intéressons aux mécanismes d'équité approximative qui tentent de réduire la complexité due aux algorithmes de *fair queueing*.

4.1 Introduction

Le partage de la bande passante par les flots élastiques se fait actuellement grâce au protocole TCP et son algorithme de contrôle de congestion qui permet aux sources de réguler leurs débits en fonction des pertes qu'elles subissent. Ainsi, l'équité du partage entre les flots dépend de divers facteurs tels que le RTT (*Round Trip Time*) (le débit est inversement proportionnel au RTT), la taille maximum de la fenêtre de congestion, la version de TCP, etc. L'équité de l'allocation de la bande passante est également soumise à la coopération des utilisateurs ; elle pourrait être mise à mal par des applications qui n'utilisent pas TCP ou qui utilisent des versions agressives de TCP. Ce problème n'existerait pas si l'équité était imposée par le réseau. En effet, le contrôle de congestion appliqué par le protocole TCP s'effectue au niveau des nœuds finaux, i.e. chez les utilisateurs. Ainsi, la complexité est repoussée aux extrémités, et le réseau reste le plus simple possible en n'effectuant que l'opération de transport des données.

D'un autre côté, les problèmes de partage de la bande passante et de la QoS sont liés. Un des facteurs importants intervenant dans les architectures de QoS est représenté par la discipline de service utilisée au sein des files d'attente des routeurs. Le scheduler détermine l'ordre de traitement des paquets appartenant aux différents

flots et conditionne, par conséquence, le partage de la bande passante. La discipline la plus connue et répandue, due à sa simplicité, est FIFO. Cette discipline n'assure ni le partage équitable ni la QoS, vu que la bande passante pourrait être accaparée par une minorité de flots et que les paquets des applications temps-réel pourraient subir des délais importants.
L'émergence de nouvelles applications temps-réel utilisant UDP et la nécessité d'apporter de la QoS à certains types de flots font que le réseau devrait, quand même, être en mesure de participer au contrôle de congestion.

En vue d'assurer un partage équitable de la bande passante, une solution basée sur l'ordonnancement *fair queueing* (FQ) est plus attractive. Cela a pour effet de procurer une isolation entre les flots en garantissant un débit équitable commun indépendamment des caractéristiques des flots individuels.

Dans la prochaine section, nous donnerons un bref aperçu des algorithmes de *fair queueing* existants ainsi que de leur complexité.

4.2 Algorithmes de fair queueing

En raison des avantages que procure le partage équitable de la bande passante, plusieurs algorithmes de *fair queuing* ont été proposés depuis les premiers travaux entrepris par Nagle [84] en 1985. Les principaux axes de recherche concernaient le développement d'algorithmes de faible complexité tout en offrant un bon degré d'équité et des bornes de performance satisfaisantes notamment en terme de délais par paquets.

Avant de présenter certains de ces algorithmes, on décrit la notion de degré d'équité.

4.2.1 Degré d'équité

Intuitivement, on dit qu'une allocation de la bande passante est équitable si elle fournit, à tout moment, le même débit à tous les flots qui sont en mesure d'atteindre ce débit équitable. Ce concept d'équité peut être généralisé en tenant compte d'un partage pondéré de la bande passante.
De manière plus formelle, si on note par r_i le poids du flot i et par $S_i(t_1, t_2)$ le service total obtenu par le flot i (en bits) pendant l'intervalle de temps $[t_1,t_2]$, l'indice d'équité de Golestani [49] est donné par : $\frac{S_i(t_1,t_2)}{r_i} - \frac{S_j(t_1,t_2)}{r_j} = 0$ où les flots i et j sont considérés comme *backlogged*, i.e. ont toujours des paquets à envoyer au cours de l'intervalle $[t_1, t_2]$. L'indice de Golestani mesure la différence de service entre deux flots *backlogged* quelconque.

En réalité, ceci constitue une définition idéale de l'équité. Le service reçu par les flots à tout instant dépend de l'entité de service utilisée, en d'autres termes des

paquets des flots et de leur taille. L'objectif des algorithmes de *fair queueing* est de s'assurer que la quantité $|\frac{S_i(t_1,t_2)}{r_i} - \frac{S_j(t_1,t_2)}{r_j}|$ soit le plus proche possible de 0.

Golestani [49] a montré que si un scheduler garantit que :

$$|\frac{S_i(t_1,t_2)}{r_i} - \frac{S_j(t_1,t_2)}{r_j}| \leq H(i,j)$$

pour tout intervalle $[t_1, t_2]$, alors

$$H(i,j) \geq \frac{1}{2}(\frac{L_i^{max}}{r_i} + \frac{L_j^{max}}{r_j})$$

où L_i^{max} représente la taille de paquet maximum du flot i et $H(i,j)$ une fonction qui détermine la borne inférieure du degré d'équité d'un algorithme donné.

Une autre mesure d'équité est donnée par l'indice de Bennet et Zhang [11] WFI (*Worst-case Fairness Index*), elle indique une borne supérieure sur la différence de service qu'obtient un flot par rapport à celui qu'il aurait dû obtenir dans le cas idéal.

En théorie, cette notion de mesure d'équité permet de comparer l'efficacité des différents algorithmes de ce point de vue. En pratique, l'importance et la signification des ces mesures sont fortement réduites à cause du fait qu'elles concernent un certain nombre de flots continuellement *backlogged* ; or en réalité la population des flots actifs est très changeante et elle est dominée par les flots non *bottlenecked* comme on va pouvoir le vérifier dans le prochain chapitre.

Les algorithmes de fair queuing peuvent être classés en deux catégories : ceux qui sont basés sur la notion d'estampille temporelle et ceux qui sont basés sur la notion de *round robin*.

4.2.2 Algorithmes à base d'estampille temporelle

Ce type d'algorithmes tente d'émuler l'algorithme GPS (*Generalized Processor Sharing*) [89], qui constitue une allocation idéale mais qui ne peut être implémentée, en calculant une estampille temporelle pour chaque paquet. Les paquets sont ensuite servis suivant un ordre croissant de leurs estampilles.

Comme son nom l'indique, GPS est une généralisation du modèle PS (*Processor Sharing*) [74] où les flots se partagent de manière équitable la bande passante (ils sont servis en même temps). La généralisation consiste à utiliser des poids par flot, ainsi la quantité de bande passante obtenue par chaque flot est proportionnelle à son poids. L'algorithme GPS considère un trafic de type fluide où les flots sont servis bit par bit assurant ainsi une équité parfaite. Il est clair que ce type d'approche ne peut être mis en œuvre dans la mesure où les paquets constituent une entité atomique non partageable en bits.
Même si GPS ne peut être implémenté, il a servi comme modèle de référence et comparaison pour les autres algorithmes de *fair queueing*.

WFQ (*Weighted Fair Queuing*), proposé par Demers et. al. [36] et étudié par Parekh et Gallager [89] sous le nom de PGPS (*Packet* GPS), est une approximation de GPS au niveau paquet. Les paquets sont triés et insérés en fonction de leur

estampille qui est égale au *finish tag* du flot auquel ils appartiennent. Le *finish tag* représente l'instant où le paquet aurait quitté le système s'il avait été traité avec l'algorithme GPS. Le calcul du *finish tag* (F_i^k) associé à un flot i, suite à l'arrivée de son $k^{\text{ième}}$ paquet, tient compte de la taille du paquet (L_i^k), du poids du flot (r_i) et nécessite le maintien d'une variable appelée temps virtuel (VT), comme indiqué ci-dessous :

$$F_i^k = S_i^k + L_i^k/r_i, \text{ avec } S_i^k = Max\{F_i^{k-1}, V(a_i^k)\}$$

où S_i^k est appelé *start tag* du flot i et $V(a_i^k)$ représente la valeur du temps virtuel à l'instant de l'arrivée du paquet k. Le temps virtuel représente l'évolution du temps au sein du système GPS équivalent et doit être mis à jour continuellement après les arrivées et départs de paquets. Entre deux instants t_1 et t_2, on a :

$$V(t_2) = V(t_1) + \frac{C(t_2 - t_1)}{\sum_{i \in \mathcal{B}(t_1,t_2)} r_i}$$

où C est la capacité du lien et $\mathcal{B}(t_1, t_2)$ l'ensemble des flots *backlogged* (qui ont des paquets dans la file) pendant (t_1, t_2). Quand la file se vide, VT peut être remis à 0. L'algorithme WFQ se distingue par le très bon degré d'équité qu'il offre ($H(i,j)$ est proche de la borne inférieure donnée par $\frac{1}{2}(\frac{L_i^{max}}{r_i} + \frac{L_j^{max}}{r_j})$) ainsi que la faible borne de délai qu'il assure. Néanmoins, il souffre d'une complexité qui dépend du nombre de flots à gérer. En effet, la complexité de WFQ est en O(N) où N représente le nombre de flots actifs concernant l'opération de mise à jour du temps virtuel. Elle est en O($log\ N$) pour ce qui est du tri en fonction des estampilles temporelles.

L'algorithme *Worst-Case Fair Weighted Fair Queueing* (WF^{2}Q) [11] introduit des améliorations par rapport à WFQ en terme d'équité sur le court terme, mais il reconduit la même complexité. Alors que WFQ trie tous les paquets en fonction de leurs estampilles (le prochain paquet à servir est celui dont le *finish tag* est le plus petit), WF^{2}Q ne considère que les paquets qui auraient dû commencer leur service dans le système GPS.
Plus tard, Bennet et Zhang ont montré que (WF^{2}Q) pouvait être modifié dans l'optique de réduire sa complexité à O($log\ N$) [12].

Afin d'éviter de calculer le temps virtuel perpétuellement, des algorithmes dits *self clocked* ont été proposés. Ils considèrent que le temps virtuel est égal à l'estampille du paquet en cours de service. Ainsi, VT n'est plus lié à l'évolution du système idéal GPS. La complexité de ces algorithmes s'en trouve alors réduite à O($log\ N$), où N représente le nombre de flots actifs. Parmi ces algorithmes, on peut citer *Self-Clocked Fair Queueing (SCFQ)* [49] et *Start-time Fair Queueing* (SFQ) [51].

L'algorithme SCFQ trie les paquets en fonction de leur *finish tag* calculé à partir du temps virtuel (VT). Cependant, SCFQ induit un délai maximum par paquet plus important que WFQ.
Il a été montré [50] que SCFQ cause un délai maximum égal à $\frac{\sum_{i=1}^{N} L_i^{max}}{C}$ supplémentaire par rapport à WFQ où C est la capacité du lien.

L'index d'équité de Golestani de SCFQ est donné par $H(i,j) = (\frac{L_i^{max}}{r_i} + \frac{L_j^{max}}{r_j})$, il n'est qu'à un facteur 2 de la borne inférieure.

L'algorithme SFQ trie les paquets en fonction de leur *start tag* calculé à partir de VT. Le *start tag* n'est rien d'autre que le *finish tag* auquel est soustrait la taille du paquet en question divisée par le poids de son flot d'appartenance. Tout comme SCFQ, SFQ a une complexité en O($log\ N$) et offre le même degré d'équité, mais il assure une borne de délai maximum plus faible. Par conséquent, SFQ est plus performant que SCFQ.

Il à noter qu'il existe d'autres algorithmes à base d'estampilles, qu'on ne présentera pas ici, tels que [109, 43].

4.2.3 Algorithmes à base de *round robin*

Ce genre d'algorithme ne nécessite pas le calcul d'estampilles temporelles. Les flots sont traités cycliquement en *round robin*. A chaque visite, un flot peut envoyer un certain nombre d'octets correspondant à un ou plusieurs paquets. L'avantage de cette approche réside dans la faible complexité qu'elle procure : O(1). Autrement dit, les opérations d'insertion et d'extraction de paquets se font en un temps constant indépendamment du nombre de flots à gérer. La réduction de complexité se traduit par une légère perte d'équité par rapport aux algorithmes à base d'estampilles.

Plusieurs algorithmes à base de *round robin* ont vu le jour. *Deficit Round Robin* (DRR) [100] est l'un des plus connus, il a été développé par Shreedhar et Varghese en 1996. Sa popularité est due au fait que contrairement à un simple algorithme *round robin*, il tient compte de la variabilité de la taille des paquets en utilisant un compteur de déficit par flot. Ce compteur permet, en effet, de garder en mémoire la proportion de bande passante non utilisée par un flot lors des cycles précédents (à cause du problème d'alignement de la taille des paquets et du quantum). Cette particularité lui garantit un bon degré d'équité.
Ainsi, chaque flot i a le droit d'envoyer jusqu'à $quantum_i + DC_i$ octets par cycle, où DC_i représente le compteur de déficit du flot et vérifie :

$$0 \leq DC_i < MTU, \quad \forall i$$

L'indice d'équité de Golestani pour DRR donne $H(i,j) = 1 + \frac{L_i^{max}}{r_i} + \frac{L_j^{max}}{r_j}$ lorsque $Min_i\{r_i\} = 1$. On constate, que pour certaines valeurs de poids, cette mesure est nettement plus grande que celle relative aux algorithmes SFQ et SCFQ.
La simplicité de cet algorithme et sa complexité en O(1), ont en fait un sérieux prétendant pour le choix de schedulers de type *fair queueing* à implémenter au sein des routeurs sur des liens à grande capacité. D'ailleurs, certaines variantes de DRR telle que MDRR [113] sont déjà déployées dans les routeurs Cisco.
Notons que la complexité en O(1) est conditionnée par l'utilisation de quanta d'au moins MTU. En effet, si un quantum est inférieur à MTU, il se pourrait que l'ex-

traction d'un paquet nécessite le parcours de plusieurs files avant de trouver un flot en mesure d'envoyer un paquet.

D'autres algorithmes qui améliorent les performances de DRR en termes d'équité et de délais ont ensuite été proposés. Ces algorithmes tentent également de résoudre certains problèmes liés au fonctionnement de DRR. Les plus connus étant la *burstiness* qu'il engendre (un flot qui émet des paquets espacés à l'origine peut les voir envoyer groupés à la sortie de DRR par le jeu des quanta et déficits) et le temps de latence avant qu'un nouveau flot ne commence à être traité (puisque les nouveaux flots sont insérés en fin de cycle).

L'algorithme *Nested*-DRR [59] partage chaque cycle DRR en plusieurs sous cycles internes. Si Q_{min} constitue le plus petit quantum alloué aux flots, alors pendant les sous cycles internes, chaque flot est servi avec un quantum égal à Q_{min}. Les flots ayant des quanta plus élevés sont servis plusieurs fois au cours d'un même cycle externe. Il est clair que cet algorithme n'apporte une amélioration par rapport à DRR que lorsqu'il y a une différence significative entre les divers quanta alloués aux flots.

L'algorithme *Pre-order* DRR [104] emploie un nombre fixe (p) de files prioritaires. Les paquets extraits au cours d'un même cycle DRR sont insérés dans ces files prioritaires suivant un certain ordre avant d'être transmis. Ainsi, l'ordre de transmission de ces paquets peut être réarrangé afin d'améliorer l'équité entre les flots. Cependant, cette opération de tri résulte en une complexité qui est en $O(log\ p)$.

Le scheduler *Elastic Round Robin* (ERR) [60] détermine à chaque cycle le nombre d'octets que tout flot est autorisé à émettre. Ceci est réalisé par le biais du maintien d'un compteur de surplus SC par flot, qui indique la différence entre le nombre d'octets émis et le nombre d'octets autorisés. La valeur du nombre d'octets autorisés est élastique (i.e. change dynamiquement), elle est évaluée à partir du SC maximum observé durant le cycle précédent.

L'algorithme *Prioritized Elastic Round Robin* (PERR) [61] combine l'idée d'utiliser des files prioritaires issue de Pre-order DRR et la notion de quantum élastique développée dans le cadre de ERR. PERR et ERR sont adaptés aux types de réseaux où l'on ne connaît pas la taille des paquets avant de les avoir entièrement transmis. C'est pourquoi le compteur SC mémorise la quantité d'octets envoyés en trop par rapport au quantum autorisé sur chaque cycle.

stochastic fairness queueing [79] procure une équité approximative. En effet, cet algorithme utilise un nombre limité de files d'attente. Les flots sont associés aux files par le biais d'une fonction de hachage. Ainsi, plusieurs flots peuvent se partager une même file suite à des collisions possibles lors de la répartition des flots due à la fonction de hachage. Les files sont servies cycliquement et ont la possibilité d'envoyer leurs quanta d'octets par cycle.

4.2.4 Algorithmes hybrides

D'autres algorithmes qui associent la notion d'estampille temporelle à des files par flot visitées en *round robin* ont été développés.

A l'instar de *Bin Sort Fair Queueing* [29], *Stratified Round Robin* [91] (SRR) ; à ne pas confondre avec *Smoothed Round Robin* [31] ; combine l'utilisation des notions de *round robin* et estampille temporelle afin de tirer profit de leurs avantages respectifs. De plus, il offre une borne de délai maximum indépendante du nombre de flots actifs.
SRR répartit les flots en classes en fonction de leurs poids. Une classe contient tous les flots dont les poids sont dans le même intervalle entre deux puissances de 2 successives. Plus spécifiquement, une classe F_k est définie comme suit :
$F_k = \{f_i : \frac{1}{2^k} \leq w_i < \frac{1}{2^{k-1}}\}$, pour $k \geq 1$ où w_i représente le poids du flot f_i.
En considérant que le débit minimum alloué à un flot est r, le nombre de classes à maintenir est : $n = \lceil log_2(C/r) \rceil$. L'algorithme SRR met en place deux niveaux de scheduling, le premier est en intra classe et le second en inter-classe. Les flots d'une même classe sont traités en *round robin* en se basant sur les quanta et les compteurs de déficit. Une *deadline* est associée à chaque classe, le scheduling inter-classe consiste à sélectionner la classe dont la *deadline* est la plus faible.
La complexité de SRR est en O($log\ n$), sachant que le nombre de classes est nettement plus petit que le nombre de flots actifs. Une implémentation *hardware* qui conduit à une complexité en O(1) a été proposée.

Après avoir passé en revue divers algorithmes de *fair queueing*, nous décrivons certains mécanismes d'équité approximative qui ont été proposés afin de contourner la difficulté, supposée, de mise en place de ces algorithmes.

4.3 Mécanismes d'équité approximative

Comme mentionné dans [23], il existe deux approches pour effectuer un contrôle de congestion au sein des routeurs. La première consiste à utiliser des disciplines d'ordonnancement par flot tel que le *fair queueing*. Nos travaux rentrent dans ce cadre.
La seconde approche repose sur le maintien d'une file FIFO conjointement à la mise en place de mécanismes de gestion active de files d'attente, connus sous le nom d'*active queue management* (AQM) en anglais.

La motivation derrière cette deuxième approche est qu'en théorie, les algorithmes de *fair queueing* offrent des avantages indéniables ; mais qu'en pratique, le maintien d'un état par flot est infaisable sur des liens à grande capacité. Ainsi, des mécanismes d'équité approximative qui ne présentent pas de problèmes liés à l'extensibilité ont été proposés dans le but de garantir une certaine équité dans le partage de la bande passante. Ces mécanismes sont censés être plus simples à mettre en œuvre que les schedulers à base de *fair queueing*, nous verrons que cela n'est pas forcément justifié.

Comme mentionné précédemment, la discipline de service la plus simple à implémenter est FIFO, où les paquets sont traités en fonction de leur ordre d'arrivée. En cas de saturation de la file, le mécanisme *drop-tail* consiste à rejeter le paquet qui cause le débordement du buffer. Ce mécanisme cause des problèmes bien connus tels que la monopolisation de la file par un faible nombre de connexions et la possibilité que l'état de saturation perdure longuement (suite à la réaction tardive des connexions TCP à la congestion).

Plusieurs techniques d'équité approximative sont décrites dans la littérature. Globalement, elles proposent de rejeter des paquets (en cas de saturation) de manière probabiliste en fonction d'une estimation des débits des flots. Dans ce qui suit, nous allons brièvement présenter les plus connues d'entre elles. Au préalable, nous décrivons RED (*Random Early Detection*) [45] qui fut l'un des premiers mécanismes d'AQM proposés (1993).

RED : RED préconise de rejeter des paquets avant que le buffer ne devienne saturé afin de mieux appréhender les situations de congestion. Il repose sur l'utilisation de seuils de congestion $s1$ et $s2$ (où $s1 < s2$). Lorsque la taille du buffer est inférieure à $s1$, aucun paquet n'est rejeté. Quand la taille du buffer est comprise entre $s1$ et $s2$, les nouveaux paquets sont rejetés avec une probabilité qui croît avec la taille du buffer. Si jamais, le nombre de paquets dans la file dépasse $s2$, tout nouveau paquet qui arrive est rejeté.
Notons que la taille du buffer est mesurée avec un lissage exponentiel, elle ne reflète donc pas la taille instantanée de la file. Le choix d'un bon paramètre de lissage exponentiel et des seuils de congestion sont un des casses têtes liés au fonctionnement de RED. En réalité, il est impossible de trouver un choix universel de ces paramètres. Pire encore, dans certaines configurations, il a été démontré [30] que RED n'introduit pas d'amélioration par rapport à une simple file FIFO pour du trafic du type HTTP.
May et al ont montré que GRED (*Gentle* RED) [82] qui utilise la taille de la file instantanée et non pas lissée est mieux adaptée à un environnement réel où la majorité des flots sont courts ; alors que RED a été évaluée en considérant plutôt des flots permanents. Naturellement, un lissage exponentiel avec un paramètre qui accorde un poids trop important aux mesures passées nuit à la réactivité du mécanisme RED. Cependant, GRED est sensible aux variations instantanées importantes de la charge et aux arrivées de paquets en rafales, elle ne constitue donc pas une solution robuste et viable pour tout type de trafic.
Si cette technique de RED est meilleure que le simple *drop-tail*, elle ne pénalise pas plus les flots dont le débit est important que les flots à faible débit. Par conséquent, RED n'introduit pas d'équité entre les flots. RED a, néanmoins, servi de modèle de base pour d'autres AQM plus sophistiqués.

Variantes de RED : Différentes variantes de RED ont été proposées dans le but de parer à certains de ses défauts. Parmi elles on peut citer, ARED (*Adaptive*

RED) [42] qui ajuste dynamiquement le paramètre probabilité de perte maximum utilisée lors du calcul de la probabilité effective de rejet d'un paquet lorsque la taille de la file est comprise entre les deux seuils. La probabilité de perte maximum croît avec la taille moyenne de la file.
Les mêmes auteurs de ARED ont développé BLUE [41] qui se base sur les événements de perte de paquets et silence du lien (aucun paquet dans la file) pour ajuster la probabilité de perte maximum.
DRED (*Dynamic RED*) [4] tente de maintenir une taille de file (la valeur lissée) proche d'une taille cible afin d'atteindre un taux d'utilisation voulu. La probabilité de perte maximum est ajustée en conséquence.

Mécanismes à base de liste de flots : Les variantes évoquées ci-dessus (ARED, BLUE, DRED) agissent uniquement sur le choix de la probabilité de perte de RED. Autrement dit, elles ne mettent en place aucun mécanisme d'équité. Dans ce qui suit, nous présentons certaines approchent qui remédient à cet inconvénient.

FRED (*Flow Random Early Drop*) [77], AFD (*Approximate Fair Droping*) [87] et RED-PD (RED *with Preferential Dropping*) [80] se basent sur une liste des flots pour déterminer leur débit. Cette liste ne contient que les flots qui ont des paquets dans la file (*bottlenecked*). De ce fait, leur complexité dépend de la taille de cette liste. Les flots qui ont le plus de paquets dans le buffer subissent des pertes plus importantes.
FRED nécessite à sont tour un bon paramétrage des seuils de congestion par flot supplémentaires (seuils d'occupation minimum et maximum par flot) qu'il introduit outre ceux relatifs à RED. FRED maintient également une variable *strike* spécifique à chaque flot de la liste ; elle indique le nombre de fois où le flot en question n'a pas réagit à une notification de congestion. Les flots dont la valeur de *strike* est élevée sont pénalisés.

AFD exploite le fait que la distribution des débits des flots a tendance à être à queue lente (*heavy tail*), c'est-à-dire que la majorité des flots ont de faibles débits mais que le gros du volume transféré est réalisé par des flots à débits importants. Ainsi, AFD maintient un second buffer de taille fixe contenant des paquets échantillonnés afin d'estimer les débits des flots. Le renouvellement des paquets de ce buffer se fait de manière probabiliste en remplaçant des paquets aléatoirement choisis.
La probabilité de rejeter un paquet appartenant à un flot donné dépend du débit du flot en question et du débit équitable. Le débit équitable étant déduit à partir d'un lissage exponentiel effectué sur la taille courante de la file et la taille cible souhaitée.

RED-PD s'appuie sur les pertes de paquets récentes pour identifier les flots dont le débit dépasse le débit de référence et rejeter leurs paquets. Le débit de référence est obtenu en considérant le débit réalisé par une connexion TCP étant donné un certain RTT et taux de perte (qui sont des paramètres à fixer).

L'évaluation de ces techniques (FRED, AFD et RED-PD) est souvent biaisée du fait qu'un nombre de flots constant est considéré, plutôt qu'une population dyna-

mique.

Dans le même cadre de recherche, SRED (*Stabilized* RED) [86] est également un mécanisme qui tente d'améliorer les performances de RED en identifiant les flots les plus longs. Ceci est fait en maintenant une liste de flots de taille fixe M appelée liste des flots zombis. Les éléments de cette liste contiennent l'identificateur du flot, un compteur du nombre de paquets du flot et le temps d'observation du dernier paquet du flot.
Tant que la liste des flots zombis n'a pas atteint sa taille limite, tout nouveau paquet entraîne l'insertion de son flot dans la liste avec une valeur initiale du compteur de paquets égale à 0 et comme temps d'observation du dernier paquet l'instant courant. Une fois la liste remplie, à l'arrivée d'un paquet l'identificateurs de son flot est comparé avec celui d'un flot choisi aléatoirement parmi ceux qui sont présents dans la liste des flots zombis. S'il s'agit du même flot, le compteur de paquets est incrémenté et le temps d'observation du dernier paquet est mis à jour. En revanche, si le test de comparaison est négatif, le flot du nouveau paquet est rajouté à la liste avec une probabilité p fixée à la place du flot choisi aléatoirement.
Ainsi, la liste des flots zombis contient les flots les plus récents, ceux dont le débit est important (plusieurs paquets envoyés) sont les plus exposés aux pertes. La probabilité de rejet des nouveaux paquets tient compte de la taille instantanée du buffer.
Comme on peut le constater, cette méthode nécessite quand même la gestion d'une liste de flots. De plus, l'éternel problème du choix des paramètres adéquats
(M, p, ...) pour le bon fonctionnement de cette approche se pose.

CHOKe : CHOKe [88] est un mécanisme basé sur RED. En cas de congestion, à l'arrivée d'un nouveau paquet, le flot auquel il appartient est comparé avec le flot d'un paquet choisi au hasard. Si les deux flots coïncident, les deux paquets sont rejetés, sinon le nouveau paquet est rejeté avec une certaine probabilité de la même manière qu'avec RED.
En pratique, le rejet du second paquet, déjà présent dans le buffer suite à un résultat de comparaison positif avec un nouveau paquet, n'est pas une mince affaire. D'un point de vue implémentation, les auteurs proposent de rajouter un bit dans l'entête des paquets qui sert d'indicateur. Ce bit est mis à 1 (par *hardware*) lorsque le paquet doit être rejeté, ce dernier n'est pas supprimé de la file tout de suite. Quand un paquet arrive en tête de file pour être servi, il est rejeté si jamais le bit en question est à 1. CHOKe ne maintient donc aucune information sur les flots actifs. Cependant, l'équité apportée n'est qu'approximative. Elle peut être améliorée en augmentant le nombre de comparaison effectuée entre un nouveau paquet et les paquets existant. Le nombre de comparaison optimal reste un facteur à déterminer.

CSFQ : Les auteurs de *Core-Stateless Fair Queueing* (CSFQ) [101] proposent de déplacer la complexité vers les routeurs de bords (*edge*) qui maintiennent un état par flot et rajoutent un label par paquet indiquant le débit des flots. Les routeurs

cœur, qui se trouvent déchargés de la gestion des flots actifs, utilisent de simples files FIFO et appliquent un rejet de paquets probabiliste (en cas de saturation) basé sur le débit des flots. Les flots ayant les débits les plus élevés, notamment ceux qui dépassent le débit équitable estimé par le routeur cœur, sont les plus concernés par les pertes de paquets. Ceci a pour effet d'améliorer l'équité du partage de la bande passante.
En dépit de son intérêt, cette technique présente deux inconvénients majeurs. Le premier est que les routeurs de bords doivent tenir compte de tous les flots en cours, alors qu'avec les algorithmes de *fair queueing*, comme on pourra le constater, il suffit de gérer uniquement les flots qui ont (ou ont récemment eu) des paquets dans les files d'attente (flots actifs). Le second inconvénient réside dans la nécessité d'apporter des changements aux entêtes des paquets IP et, par le même ocacasion, à l'interface actuelle du trafic *best effort* afin de prendre en compte les labels indiquant les débits des flots.

Tout comme CSFQ, *Rainbow Fair Queueing* [27] préconise de rejeter les paquets au niveau des routeurs cœur en fonction d'informations sur le débit des flots rajoutées par les routeurs de bords. Les routeurs cœur ne maintiennent aucun état par flot. Les flots sont classifiés en couches (couleurs) d'après leurs débits estimés et leurs paquets sont marqués en conséquence. L'information véhiculée par les paquets est la couleur et non pas le débit explicite du flot comme dans CSFQ. Les routeurs cœur maintiennent un seuil de couleur dynamique, ajusté en fonction de l'état de congestion. Ils rejettent les paquets dont la couleur dépasse le seuil.
La principale différence avec CSFQ réside dans le fait que les applications ont la possibilité de marquer certains paquets qu'elles ne souhaiteraient pas perdre avec une faible couleur. Toutefois, l'intérêt de cette particularité est limité dans la mesure où il est difficile d'accorder plus d'importance à un paquet plutôt qu'à un autre mise à part pour des flots vidéo. En outre, cette technique souffre des mêmes insuffisances, sus-mentionnées, que CSFQ.

Résumé : En résumé, on a pu voir que ces différentes techniques n'apportent qu'une équité approximative. De plus, dans certains cas, il faut maintenir une liste des flots dont la taille est, certes, nettement inférieure au nombre de flots en cours ; mais qui est de toute façon de l'ordre du nombre de flots actifs que les algorithmes de *fair queueing* doivent gérer. Nous montrerons dans le prochain chapitre que ce nombre reste faible indépendamment de la capacité des liens.

Chapitre 5

Analyse de l'extensibilité de l'ordonnancement *fair queueing*

Lors du chapitre précédent, nous avons décrit quelques algorithmes de *fair queueing* (FQ) en évoquant leur capacité à assurer un partage équitable de la bande passante. Pourtant, en dépit de ses atouts, le FQ n'a jamais été déployé à grande échelle dans le réseau Internet à cause de considérations liées à son extensibilité.
En effet, l'idée répandue est que le nombre de flots en cours croît avec la capacité des liens ; par conséquent sur des liens à très grande vitesse, où le nombre de flots en cours est extrêmement élevé, les opérations nécessitées par les algorithmes de FQ deviennent infaisables. Il est alors inconcevable de maintenir un état par flot tel qu'exigé par les différentes approches de *fair queueing*.

Au cours de ce chapitre, on va voir que ces considérations sont erronées et qu'en réalité la complexité du FQ dépend plutôt du nombre de flots actifs (ceux qui ont ou ont récemment eu des paquets dans la file) qui lui est indépendant du débit du lien. Nous montrerons donc que le *fair queueing* est parfaitement extensible. Pour ce faire, nous évaluons le nombre de flots actifs qui doivent être gérés par l'ordonnanceur.
Nous commençons par rappeler quelques résultats importants du modèle PS (*Processor Sharing*), particulièrement pertinents dans le cadre de cette étude. Par la suite, nous présentons une première analyse basée sur un modèle simplifié qui fournit des indications sur l'extensibilité du *fair queueing*. Enfin, nous effectuons une analyse plus complète à partir de traces de trafic réelles qui met l'accent sur l'impact des caractéristiques des flots sur l'extensibilité du *fair queueing*.
Ce travail a été réalisé en collaboration avec Luca Muscariello de l'école polytechnique de Turin lors de sa visite à France Télécom R&D.

5.1 Le modèle PS

Le partage équitable de la bande passante entre plusieurs flots sur un lien donné peut être modélisé par le biais du modèle PS (*Processor Sharing*).
Ainsi, en supposant que les sessions arrivent selon un processus de Poisson tel que décrit dans la section 2.3 de l'introduction, on peut dériver des résultats d'évaluation de performance en se basant sur la file M/G/1/PS.
Un des avantages de cette file réside dans son insensibilité aux caractéristiques détaillées du trafic tel que la distribution de la loi de service (dans notre cas, il s'agit en l'occurrence de la distribution de la taille des flots). On a alors les mêmes résultats que pour une simple file M/M/1.
En effet, la distribution du nombre de clients dans le système suit une loi géométrique de paramètre ρ où ρ représente la charge de la file (La figure 5.1 représente la loi géométrique de paramètre 0.9), i.e.
$Pr[\text{nombre flots en cours} = n] = \rho^n(1-\rho)$. Par conséquent, le nombre moyen de flots en cours sur un lien est donné par :
$E[\text{nombre de flots en cours}] = \rho/(1-\rho)$. Comme l'indique la figure 5.2, le nombre moyen de flots reste très faible même sous une charge aussi importante que 0.9, par exemple, où il vaut 9 seulement.
Or, on sait qu'en pratique le nombre de flots en cours sur des liens à grande capacité est de l'ordre de quelques dizaines, voir centaines, de milliers (cette observation est confirmée dans la section 5.3, où nous présentons un certain nombre de traces de trafic réelles). On en déduit que la majorité des flots en cours ne peuvent être *bottlenecked* sur le lien dont ils se partagent la capacité. En effet, un partage équitable tel que modélisé par le système PS implique que tous les flots réalisent le même débit équitable qu'ils sont en mesure d'atteindre. On dit alors que les flots en question sont *bottlenecked* au niveau de ce lien dans le sens où leur débit est contraint par la capacité disponible dictée par le partage effectué sur le lien en question.

Nous définissons le débit intrinsèque comme étant le débit que réaliserait un flot si le lien qu'il traverse était de capacité infinie. Ainsi, les flots *non bottlenecked* se distinguent par un débit intrinsèque inférieur au débit équitable. Cette répartition des flots en catégorie *bottlenecked* et *non bottlenecked* a un impact direct sur l'évaluation du nombre de flots à prendre en considération par un scheduler de type *fair queueing*, comme on va pouvoir le voir au cours des prochaines sections de ce chapitre.

5.2 Analyse préliminaire

Dans un premier temps, nous avons conduit une analyse préliminaire [69] sur l'extensibilité des algorithmes de FQ. Cette analyse a été basée sur la simulation d'une seule trace de trafic et le développement d'un modèle analytique relativement simple permettant de donner une explication aux résultats observés. Dans ce qui suit, nous décrivons rapidement cette analyse afin de donner quelques premières

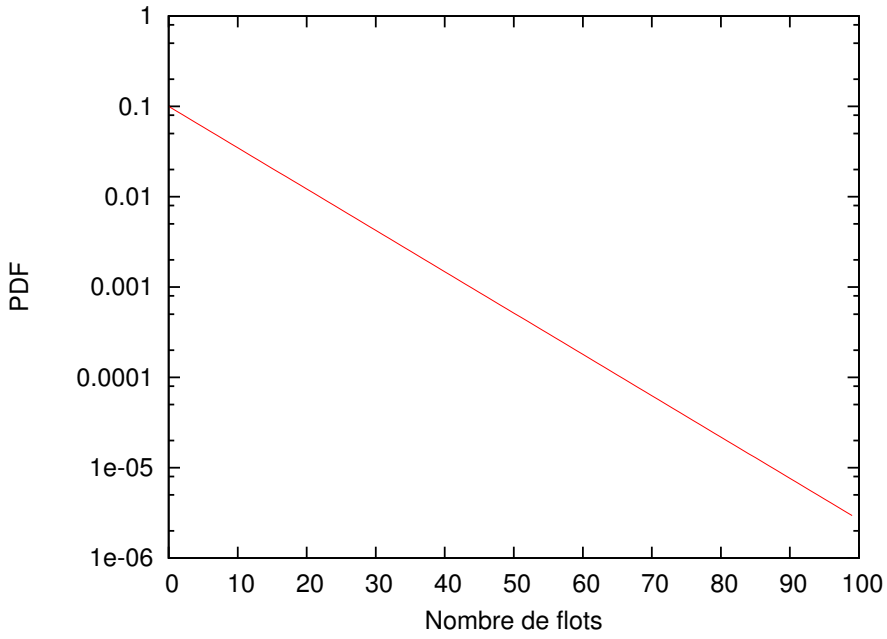


FIG. 5.1 – Distribution géométrique du nombre de flots, $\rho = 0.9$

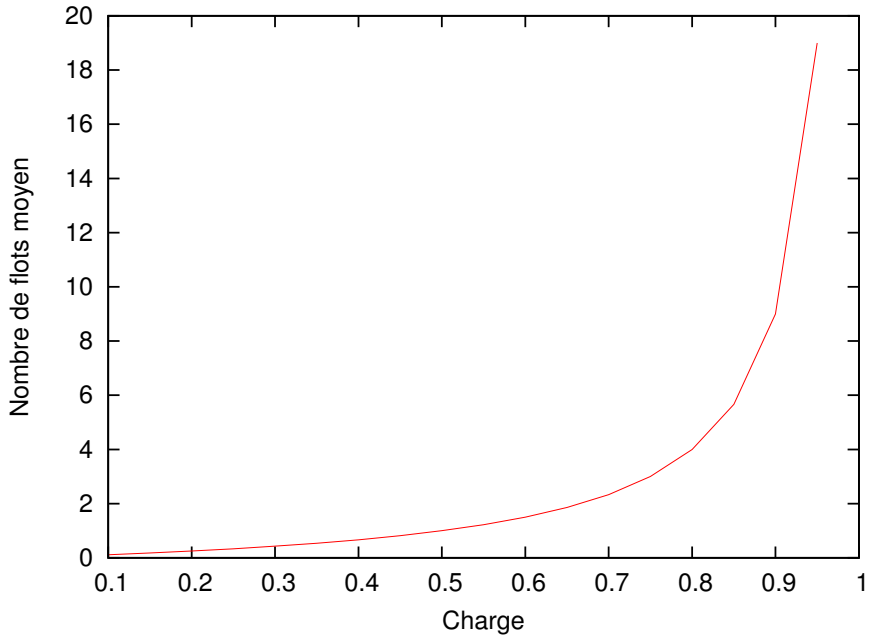


FIG. 5.2 – Nombre de flots moyen avec une loi géométrique en fonction de la charge

indications concernant le nombre de flots à gérer par les algorithmes de FQ ; avant de présenter une étude plus approfondie dans la section 5.5.

5.2.1 Résultats de simulation

En utilisant la trace décrite ci-dessous, nous avons effectué des simulations *ns* avec l'algorithme PFQ. En variant la capacité du lien simulé, on produit automatiquement différentes charges offertes pour chaque scénario de simulation. Ceci permet de voir l'impact de la charge totale sur la distribution de la taille de la liste des flots actifs (AFL).

Modèle de trafic : Nous avons utilisé une trace de trafic issue du réseau de recherche Abilene obtenue à partir du site de NLANR [1]. Elle a été collectée le 14/08/2002 sur un lien OC48 reliant Indianapolis à Cleveland (dans le sens *west-bound*). Les données extraites sont relatives à la période entre 9 :20 :40 et 9 :21 :40 (i.e. d'une durée de 1 minute).
La trace en question contient plus de 4 millions de paquets appartenant à 111000 flots (identifiés via le quintuple habituel et un *time out* de 20 secondes). La distribution de la taille des paquets est approximativement de type tri modale avec 46% de paquets dont la taille est autour de 40 octets, 6% autour de 576 octets et 48% autour de 1500 octets. La majorité des flots ont de faibles débits, le débit moyen maximum observé était de 10 Mb/s. Le nombre de flots en cours moyen est dans les 7000 flots (un flot n'est plus considéré en cours dès l'observation de son dernier paquet). Notons que la majorité écrasante (97%) des octets de la trace est due aux flots TCP. Enfin le taux d'utilisation du lien est assez faible, il est égal à 16%.

En simulant un lien avec une capacité de 2.5 Gb/s correspondant à la charge originale de 0.16, la taille de l'AFL ne dépasse pas les 7 flots. La file de paquets se comporte comme une simple FIFO (car tous les flots ont un débit inférieur au débit équitable qui est très élevé), ainsi les flots s'accumulent dans l'AFL jusqu'à la fin des *busy* périodes de la file.
En diminuant la capacité des liens pour augmenter les charges, la durée des *busy* périodes est rallongée causant ainsi une augmentation du nombre de flots actifs. De plus, le débit équitable étant plus faible, certains flots deviennent *bottlenecked*. Les capacités simulées étaient de 2.5 Gb/s, 1 Gb/s, 500 Mb/s et 450 Mb/s pour atteindre, respectivement, des charges de 0.16, 0.41, 0.84 et 0.93.
La figure 5.3 représente la distribution complémentaire de la taille de la liste des flots actifs pour chaque scénario de simulation. Comme on peut le constater, même sous une charge aussi importante que 0.93, le nombre de flots actifs reste inférieur à 215, ce qui est beaucoup plus faible que le nombre de flots en cours (7000). Le modèle analytique présenté dans la prochaine section expliquera ses résultats.

[1] http ://pma.nlanr.net/Traces/long/ipls1.html

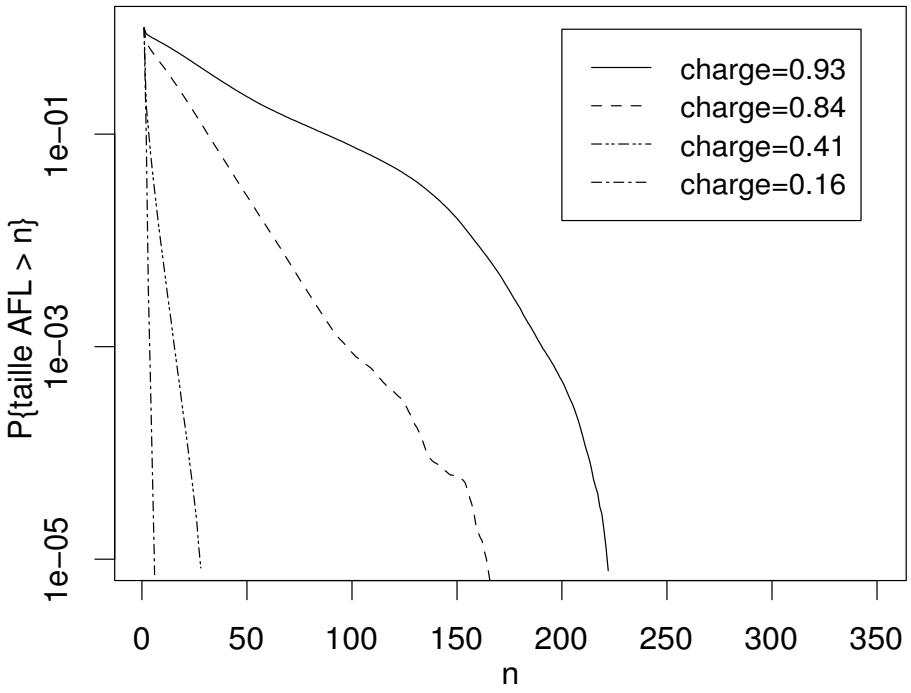


FIG. 5.3 – Distribution complémentaire de la taille de la l'AFL, charges : 0.16, 0.41, 0.84, 0.93.

5.2.2 Modèle analytique mono débit

Dans le but d'expliquer les différences entre le nombre de flots en cours, actifs et *bottlenecked*, nous présentons un modèle simple. On suppose que les flots sont générés selon le modèle Poisson de session introduit dans la section 2.3, ils se partagent un lien de capacité C.
De plus, on suppose que tous les flots ont un même débit constant C/m où m est un entier. On note par N_m le nombre de flots en cours. Lorsque $N_m \geq m$, tous les flots réalisent le même débit équitable.
L'hypothèse ci-dessus correspond au modèle fluide de [9]. L'expression de la distribution du nombre de flots en cours est donnée par :

$$\pi_m(n) = (1-\rho)f(\rho) \times \begin{cases} \frac{m!}{n!}(\rho m)^{n-m}, & \text{pour } n < m, \\ \rho^{n-m}, & \text{pour } n \geq m, \end{cases} \tag{5.1}$$

où

$$f(\rho) = \frac{(m\rho)^m/m!}{(1-\rho)\sum_{k=0}^{m-1}(m\rho)^k/k! + (m\rho)^m/m!}.$$

avec ρ représentant la charge du lien égale à (taux d'arrivée des flots $\times$ taille moyenne des flots / C).

Notons par A_m le nombre de flots actifs lorsqu'une nouvelle entrée doit être rajoutée à l'AFL. Nous nous intéressons à la distribution de A_m. Pour cela, on

suppose que les paquets sont de taille constante. Deux cas peuvent alors se présenter : On sait que tant que $N_m < m$, le scheduler agit comme une file FIFO. Le nombre de flots actifs est alors différent de 0 uniquement pendant les *busy* périodes de la file. Durant une *busy* période, chaque nouvelle arrivée de paquet donne lien à une nouvelle entrée dans l'AFL incrémentant ainsi A_m. Comme spécifié dans le fonctionnement de PFQ, à l'issue d'une *busy* période la liste des flots actifs est vidée.
Dans le cas où $N_m \geq m$, le nombre de flots actifs n'est rien d'autre que le nombre de flots en cours (puisque tous les flots en cours sont *bottlenecked*).

La distribution $b_m(i,n)$ du nombre de paquets servis au cours d'une *busy* période est obtenue à partir des résultats de la file nD/D/1 [106]. On a, pour $1 \leq n < m$ et $1 \leq i < n$:

$$b_m(i,n) = \binom{n-1}{i-1} \frac{m-n}{m-n+1} \frac{i^{i-2}(m-i)^{n-i-1}}{m^{n-2}} \tag{5.2}$$

En notant par $d_m(j,n)$ la probabilité que $A_m = j$ sachant que $N_m = n$, on a :

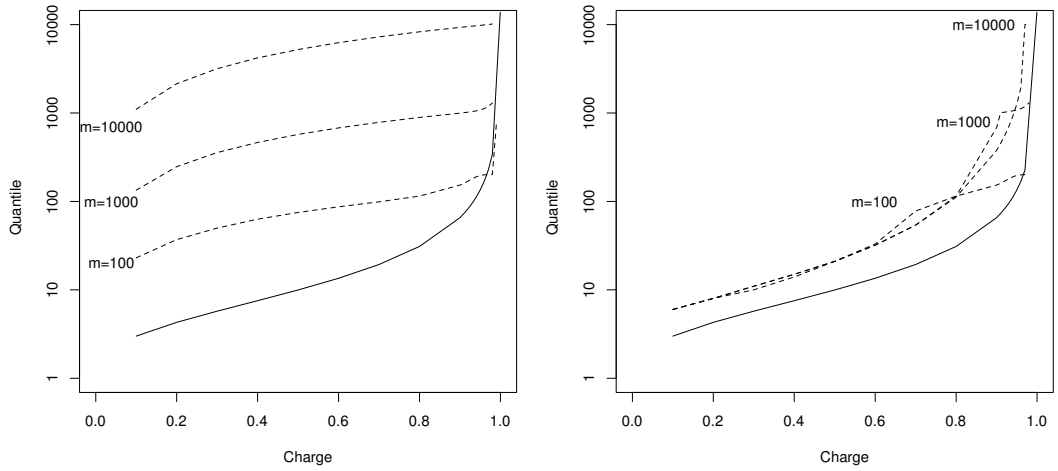
$$\begin{aligned} d_m(j,n) &= (1 - \frac{n-1}{m}) \sum_{i=j+1}^{n} b_m(i,n), \text{ pour } 0 \leq j < n < m \\ & \quad 1, \text{ pour } n \geq m \text{ et } j = n, \text{ ou } j = n = 0 \\ & \quad 0, \text{ sinon.} \end{aligned}$$

La première ligne de cette dernière équation correspond au fait qu'un nouveau paquet trouve, à son arrivée, une AFL de taille égale au nombre de paquets ayant contribués à la *busy* période de la file nD/D/1. La probabilité que ce nombre de paquets soit égal à j est donnée par : Pr[*busy* période $> j$] / E[durée *busy* période], puisque le nouveau paquet va également contribuer à la *busy* période en cours.
Enfin, la distribution du nombre de flots actifs $a_m(j)$ est déduite en déconditionnant par rapport à la distribution 5.1 :

$$a_m(j) = \sum_{n \geq 0} d_m(j,n) \pi_m(n). \tag{5.3}$$

Le nombre de flots *bottlenecked* B_m vaut soit n avec une probabilité $\pi_m(n)$, pour $n \geq m+1$, soit 0 avec une probabilité $\sum_{0 \leq i \leq m} \pi_m(i)$. La figure 5.4 trace le quantile 99.9% des distributions de N_m et A_m en fonction de la charge du lien ρ et du paramètre de débit m. Lorsque m vaut 1, tous les flots en cours sont actifs et *bottlenecked* entraînant la même distribution géométrique pour N_1, A_1 et B_1. Le quantile de cette distribution est représenté par la ligne continue de la figure 5.4 (a) et (b). Les quantiles de B_m ne sont pas tracés car ils sont très faibles sauf lorsque ρ est proche de 1. Les courbes de A_m de la figure 5.4 (b) exhibent un comportement non uniforme quand ρ augmente, ceci est dû au changement dans l'expression de la distribution 5.3 lorsque N_m dépasse m.

Les résultats obtenus indiquent que le FQ est à la fois extensible, en ce sens que le nombre de flots actifs ne dépend pas de la capacité du lien, et faisable, vu que la taille de l'AFL est relativement faible.



(a) Flots en cours N_m (b) Flots actifs A_m

FIG. 5.4 – Quantiles 99.9% des distributions du nombre de flots vs. charge : $m = 1, 100, 1000, 10000$.

5.2.3 Analyse post simulation

Il est clair que les flots issus du trafic réel ont un débit variable. Selon que leurs débits soient plus grands ou plus petits que le débit équitable, ces flots sont soit dans la catégorie *bottlenecked* soit *non bottlenecked*. Donc pour simplifier, nous considèrons que les flots appartiennent à deux classes, l'une est constamment *bottlenecked* et l'autre contient des flots qui sont toujours *non bottlenecked*. En outre, on suppose que les paquets appartenant aux flots *non bottlenecked* arrivent suivant un processus de Poisson et peuvent avoir une taille quelconque. Ainsi, la distribution de la taille de l'AFL peut être déduite à partir des résultats sur la *busy* période d'une file M/G/1 [85], sachant que le comportement de l'AFL est régi par la notion de *busy cycle* qui sera définie dans la section 5.5.3. Ainsi, on considère une convolution de n *busy* périodes d'une M/G/1, où n représente le nombre de flots *bottlenecked*. Les flots *bottlenecked* se partageant la capacité résiduelle équitablement, on a :

$$\pi(n) = (1 - \rho_b)\rho_b^n, \text{ avec } \rho_b = A_1/(C - A_2)$$

où A_1 et A_2 représentent la demande de trafic, respectivement, des classes *bottlenecked* et *non bottlenecked*.

En se basant sur les charges moyennes induites par les classes *bottlenecked* et *non bottlenecked* obtenues après les simulations décrites dans la section 5.2.1, où le *priority load* représente la charge due aux flots *non bottlenecked*, et sur la distribution de la taille des paquets des flots *non bottlenecked* de la trace évoquée dans la section 5.2.1, on obtient la figure 5.5. La figure en question établit une comparaison entre les simulations *ns* (lignes en pointillés) et le modèle analytique (lignes continues) qui fournit, malgré tout, une bonne approximation comme on peut le voir.

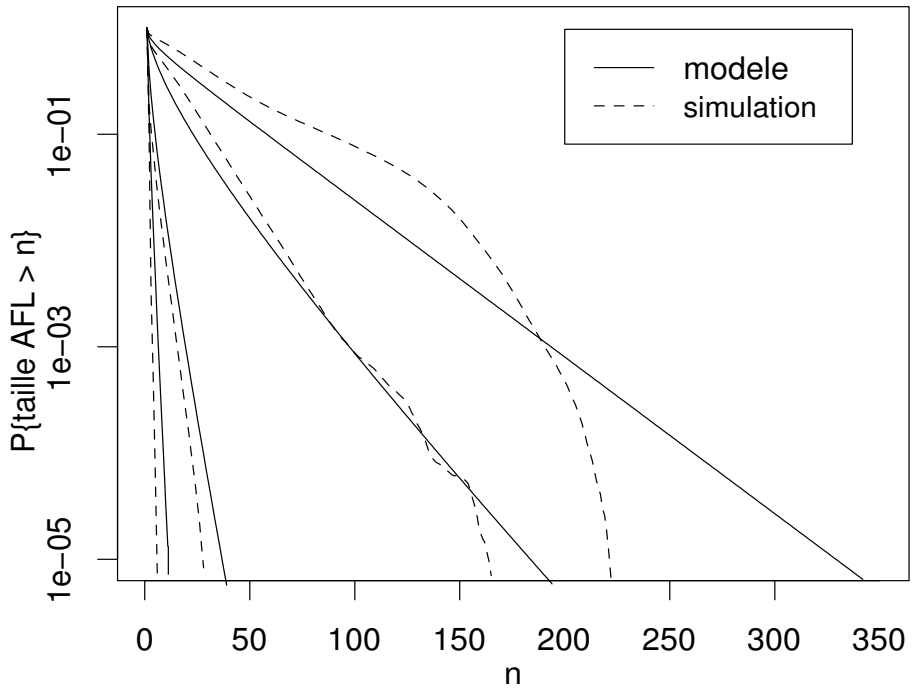


FIG. 5.5 – Distribution complémentaire de la taille de l'AFL, simulation et analyse.

Même si cette analyse préliminaire n'est basée que sur une seule trace d'une durée relativement courte et que les modèles de trafic utilisés (5.2.2) sont largement perfectibles (pour obtenir un système multi-débits et ne pas utiliser des charges moyennes a posteriori), elle permet néanmoins d'apporter quelques éléments de réponse quant aux paramètres qui déterminent le nombre de flots à prendre en compte par les algorithmes de FQ.

Suite à l'obtention de ces premiers résultats encourageants, nous avons entrepris une étude plus poussée en se basant sur plusieurs traces de trafic de durées plus longues, ainsi qu'un modèle analytique plus complet, qui prend en compte la diversité de débit des flots pour aboutir à la distribution de l'AFL.

5.3 Traces de trafic

Notre étude sur l'extensibilité des algorithmes de *fair queuing* a été menée en utilisant trois traces de trafic Internet réelles :

1. *ADSL* : cette trace a été collectée sur un lien OC3 du réseau de France Télécom le 25/08/2003. Elle contient 5 minutes de données représentant le trafic descendant vers plusieurs utilisateurs ADSL.
2. *Ab-I* : cette trace a été collectée sur un lien OC48 du réseau de recherche Abilene entre Indianapolis et Kansas City. Elle représente 5 minutes de trafic

Statistiques	*ADSL*	*Ab-I*	*Ab-III*
Capacité	155Mb/s	2.5Gb/s	10Gb/s
Nombre de paquets total	2.6M	19M	156M
Nombre de flots total	850K	2.3M	683K
Utilisation	28%	13%	19%
Nombre moyen de flots en cours	65500	162000	127000

TAB. 5.1 – Statistiques des traces de trafic

récolté le 14/08/2002.

3. *Ab-III* : cette trace a été collectée sur un lien OC192 du réseau de recherche Abilene III entre Indianapolis et Chicago le 01/06/2004. Elle contient 2 minutes de données.

Les traces Abilene sont disponibles sur le site web NLANR[2]. Le réseau Abilene est un réseau dorsal (*backbone*) de Internet2 reliant principalement des universités Américaines.

5.3.1 Statistiques des traces

Le tableau 5.1 présente les principales statistiques des traces décrites ci-dessus. Les flots ont été identifiés à partir du quintuple classique (adresses IP source et destination, numéros de ports et protocole) issu des traces, outre l'utilisation d'un *time out* de 20 secondes. Bien entendu, d'autres valeurs possibles pour le *time out* auraient pu être choisies (10s, 2s, ...), sa durée n'ayant aucun impact sur les résultats d'extensibilité du *fair queueing*.

On constate que les trois traces présentent des statistiques différentes. En particulier, la taille des flots de la trace *Ab-III* est beaucoup plus importante que celles relatives aux deux autres traces. Ceci est dû au fait que la trace *Ab-III* contient un plus faible nombre total de flots ayant des débits élevés. Comme on pouvait s'en douter, les flots TCP constituent 95% du nombre total d'octets au sein des trois traces.
L'utilisation des trois liens est relativement faible avec un maximum de 28% pour la trace ADSL comme indiqué par les Figures 5.6, 5.7 et 5.8. Les traces Abilene offrent des utilisations encore plus faibles confirmant ainsi que les réseaux dorsaux (*backbone*) sont peu chargés.
Les Figures 5, 6 et 7, en annexe 3 paragraphe 2.1, montrent de manière équivalente les variations du débit agrégé des trois traces (calculé sur des intervalles successifs de 100ms).

[2]http ://pma.nlanr.net/Traces/Traces/long/ipls/1 et http ://pma.nlanr.net/Special/ipls3.html

Dans les prochaines sections, nous examinons de plus près le nombre de flots en cours, le débit des flots et la distribution de la taille des paquets.

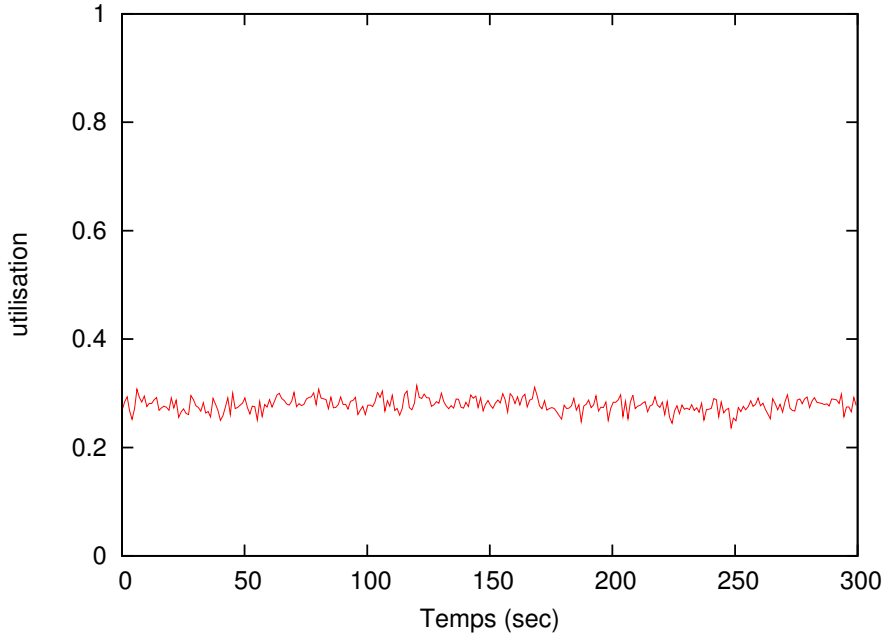


FIG. 5.6 – Variations de l'utilisation du lien ADSL en fonction du temps

5.3.2 Nombre de flots en cours

Le nombre moyen de flots en cours rapporté dans le tableau 5.1 a été évalué en considérant que les flots sont toujours en cours 20 secondes après l'observation de leur dernier paquet (*time out*). En moyenne, le nombre de flots en cours sans tenir compte du time out (i.e. un flot est considéré comme terminé dès l'observation de son dernier paquet) atteint 13000 flots sur le lien OC192 (*Ab-III*) contre 5900 sur le lien OC3 (ADSL) ; confirmant ainsi la croissance de ce nombre avec la capacité des liens. Les Figures 8, 9 et 10 spécifiées dans l'annexe 3, paragraphe 2.2 tracent l'évolution du nombre de flots en cours sur les liens considérés sur la durée d'observation sans tenir compte du *time out*. En considérant un *time out* de 20 secondes, le nombre de flots en cours est représenté par les figures 5.9, 5.10 et 5.11. Dans ce dernier cas, le nombre de flots en cours peut aller jusqu'à 162000 flots sur la trace *Ab-I*. Notons que le nombre de flots en cours sur la trace *Ab-I* est plus élevé que sur *Ab-III* lorsqu'on tient compte d'un *time out* de 20 secondes. Ceci est dû au nombre important de flots à un seul paquet dans cette trace et au fait qu'elle comporte plus de flots que *Ab-III*.

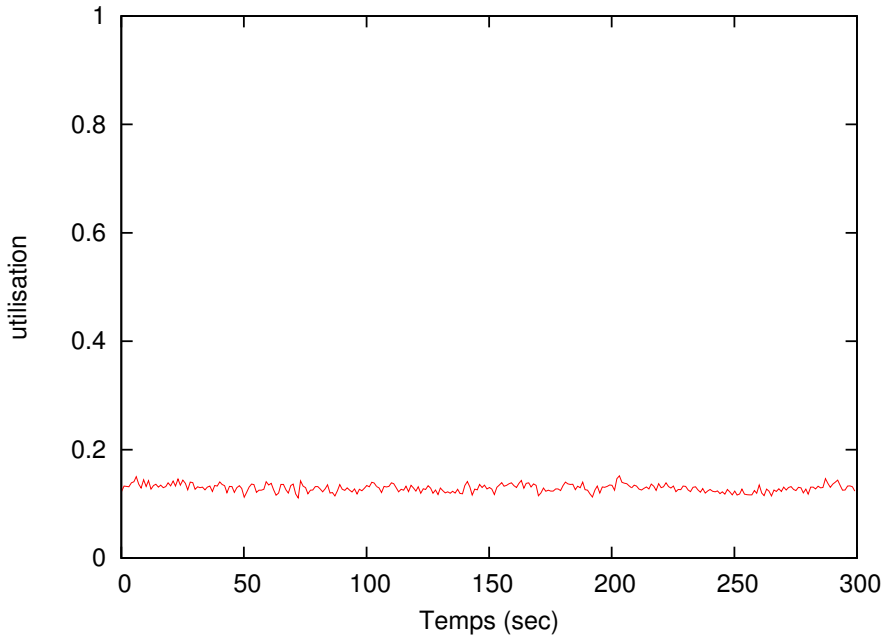


FIG. 5.7 – Variations de l'utilisation du lien *Ab-I* en fonction du temps

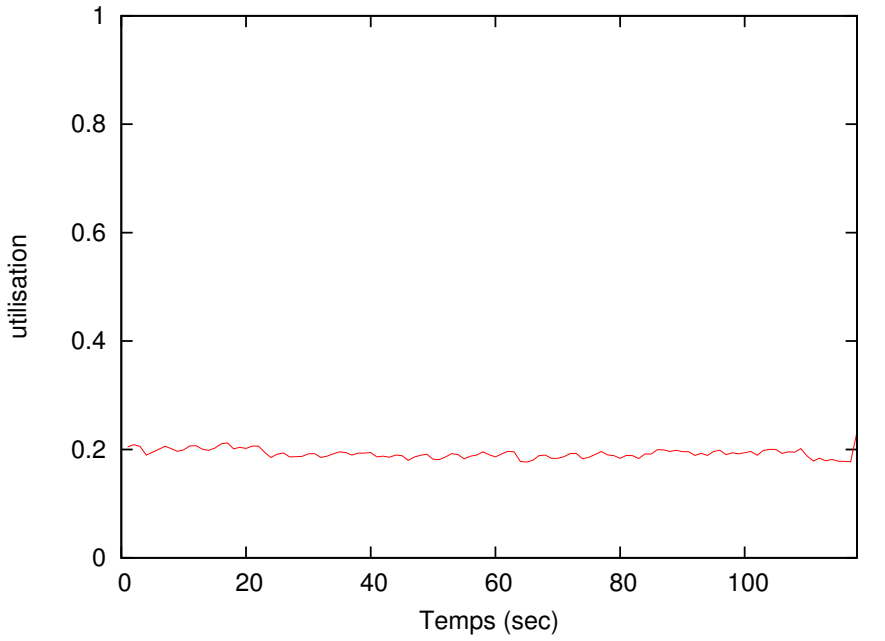


FIG. 5.8 – Variations de l'utilisation du lien *Ab-III* en fonction du temps

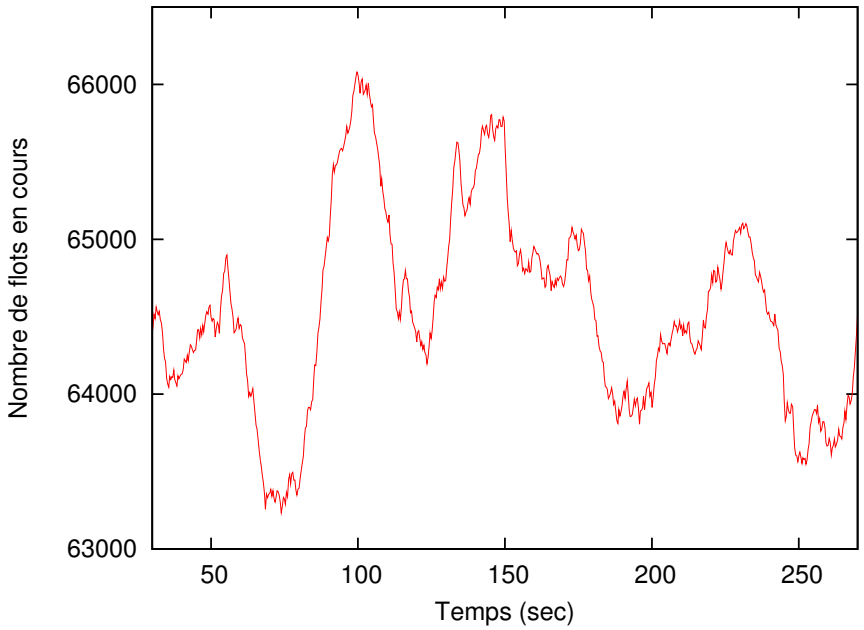


FIG. 5.9 – Evolution du nombre de flots en cours, TO = 20 sec, trace ADSL

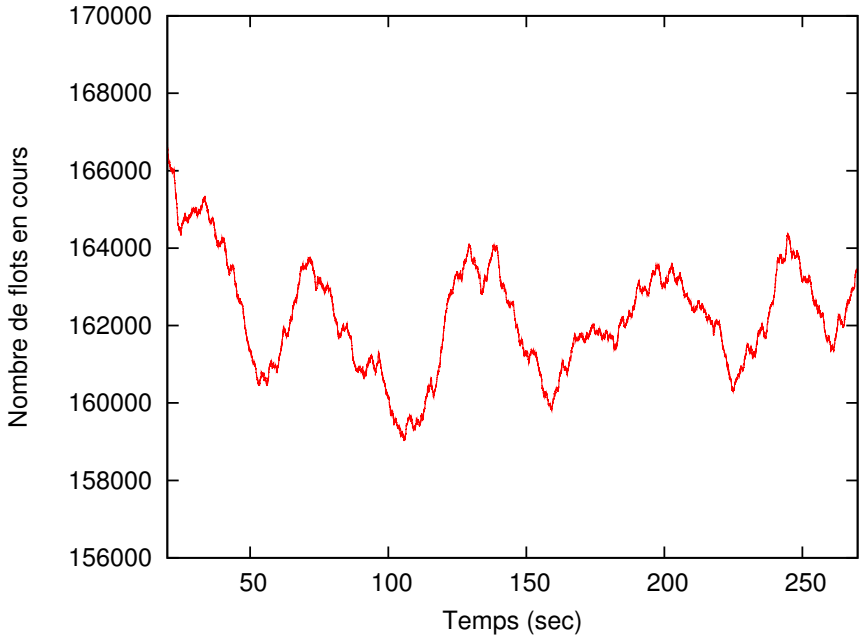


FIG. 5.10 – Evolution du nombre de flots en cours, TO = 20 sec, trace *Ab-I*

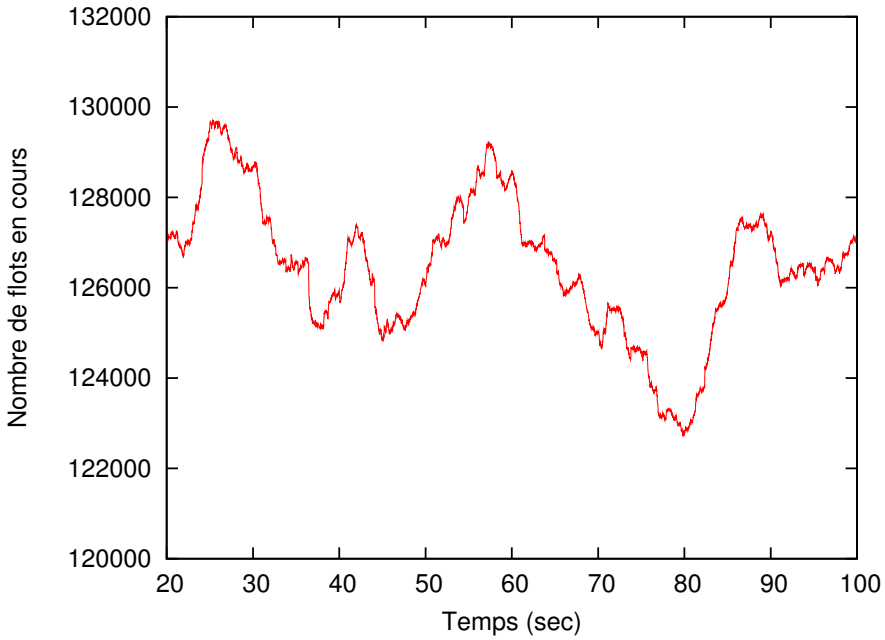


FIG. 5.11 – Evolution du nombre de flots en cours, TO = 20 sec, trace *Ab-III*

5.3.3 Débits des flots

Les caractéristiques des flots IP ont fait l'objet d'études relativement récentes [26, 99, 110] selon différents critères tels que leur taille, leur durée, leur *burstiness* et leur débit. Le débit des flots constitue un aspect primordial à prendre en compte dans le cadre de l'étude de l'extensibilité des schedulers basés sur du *fair queuing* comme on va pouvoir le constater.

Les Figures 5.12 et 5.13 indiquent les complémentaires des distributions des débits moyens et crêtes, respectivement, des flots évaluées sur les traces en question. Nous avons adopté la même convention que Zhang et. al. [110] pour évaluer le débit moyen des flots en ne considérant que les flots dont la durée est au moins égale à 100ms.
Les trois traces présentent des ordres de grandeurs différents du débit moyen des flots. En effet, le débit moyen des flots ADSL est limité à 1 Mb/s ; tandis que les flots de la trace *Ab-I* obtiennent un débit moyen maximum de 10 Mb/s même si la majorité des flots ont de faibles débits (inférieurs à 1 Mb/s), très variables d'un flot à un autre. Ces résultats rejoignent les observations de Zhang et. al. [110]. En revanche, les flots de la trace *Ab-III* réalisent des débits moyens très élevés (un maximum de 300 Mb/s). Nous avons pu vérifier que ceci résulte de l'utilisation de *jumbo* paquets d'une taille de 9000 octets par les flots en question.

Le débit crête des flots constitue une mesure alternative au débit moyen et peut être assimilé au débit intrinsèque des flots (le débit qu'ils pourraient atteindre si

le lien était de capacité infini). Nous considérons que le débit crête est le débit maximum atteint par le flot sur des périodes successives de 10ms afin de pouvoir capturer les différentes variations de débits. D'après la Figure 5.13, on observe que le débit crête des flots a un comportement semblable au débit moyen en fonction des traces avec naturellement des valeurs plus importantes. Ainsi, le débit crête maximum atteint par les flots de la trace *Ab-I* est 100 Mb/s, il est de 1.5 Gb/s pour la trace *Ab-III*; alors que pour la trace ADSL le débit crête reste 1 Mb/s correspondant à la limite imposée par la ligne d'accès.

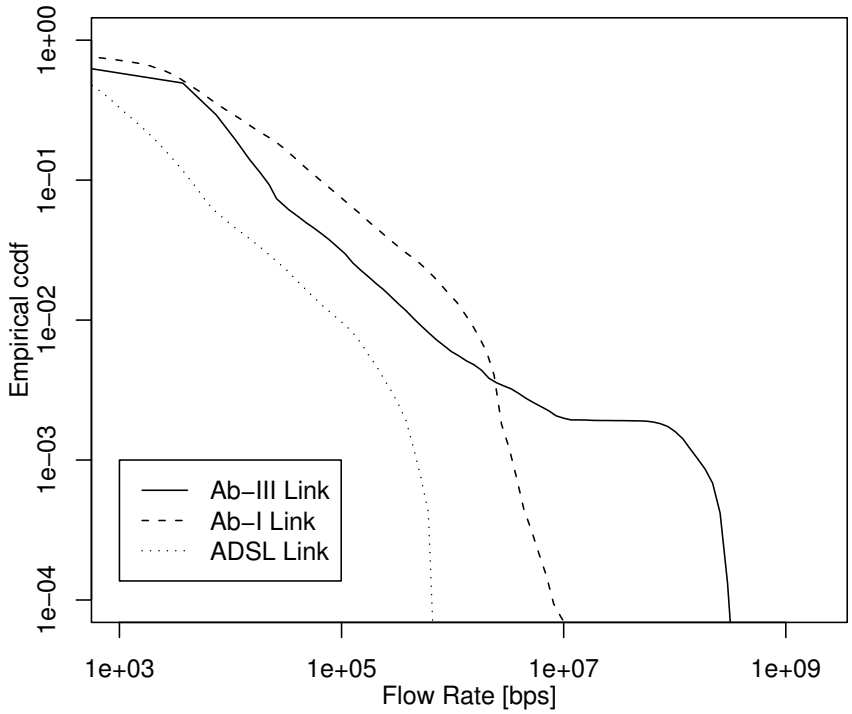


FIG. 5.12 – La distribution complémentaire des débits moyens des flots

5.3.4 Distribution de la taille des paquets

La distribution de la taille des paquets dans les trois traces se caractérise par deux principales valeurs relatives aux tailles des paquets ACK et MTU. Ces résultats sont dans la lignée des observations faites sur des traces de trafic issues du réseau de Sprint [3]. Le tableau 5.2 indique les principales tailles de paquets ainsi que leurs proportions pour les trois traces.
On peut remarquer que les tailles moyennes de paquets dans les traces *Ab-I* et *Ab-III* sont plus importantes que celle de la trace ADSL, ce qui se traduit par des

[3]http ://ipmon.sprint.com/packstat/packetoverview.php

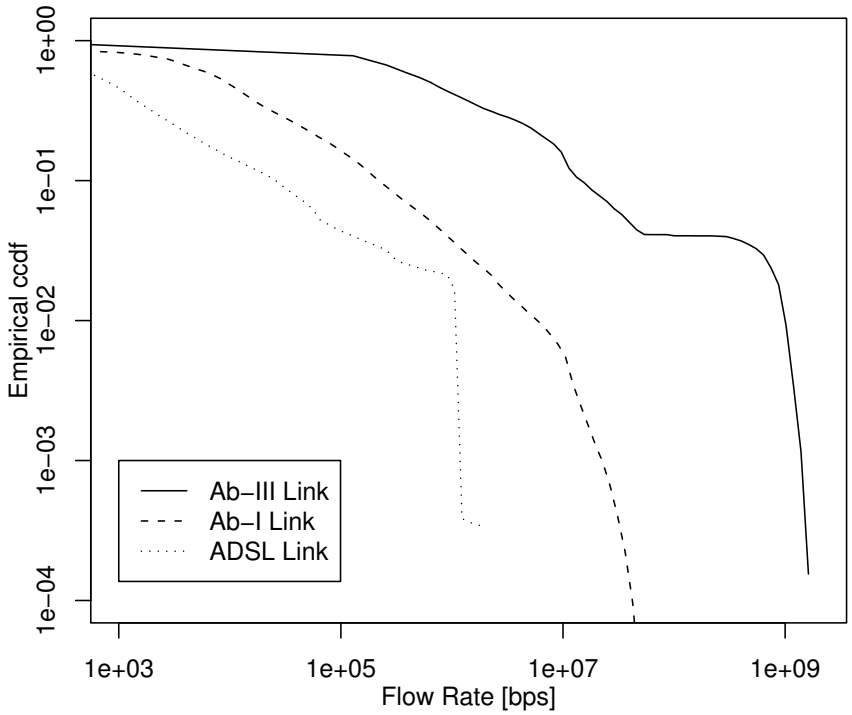


FIG. 5.13 – La distribution complémentaire des débits crêtes des flots

proportions de paquets MTU plus élevées. De façon générale, les flots de débits importants se distinguent par des tailles de paquets plus grandes.

5.4 Evaluation par simulation de l'extensibilité du *fair queueing*

Afin de confirmer la propriété d'extensibilité des algorithmes de *fair queueing*, nous avons conduit, dans un premier temps, des simulations *ns* qui utilisent les traces de trafic réelles décrites dans la section 5.3. Plus concrètement, les traces utilisées consistent en une suite de paquets d'une taille donnée arrivant à des instants précis et appartenant à divers flots. Les simulations ont permis de reproduire les traces en question en appliquant l'algorithme PFQ en tant que discipline de service du lien simulé. Le but de ces simulations étant d'observer la distribution de la taille de la liste des flots actifs AFL.

ADSL		Ab-I		Ab-III	
Taille (octets)	Prop (%)	Taille (octets)	Prop (%)	Taille (octets)	Prop (%)
40	29	40	29	40	17
48	6	52	8	52	5
1340	6	552	6	668	13
1460	5	1500	28	1420	5
1500	9	autres	29	1500	26
autres	45	-	-	9000	13
-	-	-	-	autres	21

TAB. 5.2 – Distribution de la taille des paquets des trois traces.

5.4.1 Charges originales

Comme mentionné au cours de la section 5.3, les charges originales des trois liens sont relativement faibles, entraînant par la même occasion un nombre de flots actifs très restreint. La figure 5.14 représente la distribution complémentaire de la fonction de répartition de la taille de la liste des flots actifs, i.e. la probabilité que le nombre de flots actifs soit plus grand qu'une certaine valeur.
Comme on peut le constater, au pire des cas, le nombre de flots actifs atteint 21 flots seulement. Cette valeur a été obtenue pour le lien ADSL, car il est le plus chargé en dépit du fait que le nombre de flots en cours y soit moins important que dans les deux autres liens. Ceci constitue une première indication concernant le découplage qui existe entre le nombre de flots en cours et le nombre de flot actifs, d'une part, et entre la capacité du lien et le nombre de flots actifs, d'autre part.

5.4.2 Charges modifiées : 0.6 et 0.9

Les charges originales n'étant pas suffisamment importantes pour causer une hausse du nombre de flots actifs, nous avons simulé les trois traces en réduisant les capacités des liens afin d'obtenir des charges de 0.6 et 0.9. Les différentes capacités simulées sont données par le tableau 5.3. La taille des files de paquets utilisée au cours des simulations garantissait l'acheminement de tous les paquets relatifs aux traces sans aucune perte, ce qui permet de ne pas altérer le comportement des traces telles que collectées sur les liens à capacité plus élevée. On rappelle que la charge est définie comme étant le rapport entre, d'une part, le taux d'arrivée des flots multiplié par leur taille moyenne et, d'autre part, la capacité du lien.

La figure 5.15 trace la distribution complémentaire de la fonction de répartition (cumulative) de la taille de l'AFL obtenue pour les trois traces avec les configurations de charge de 0.6 et 0.9.

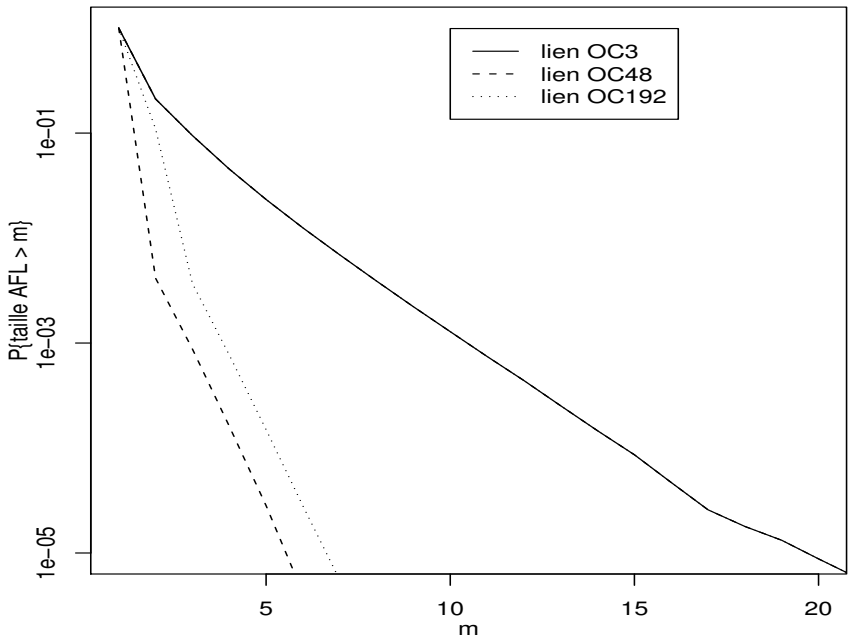


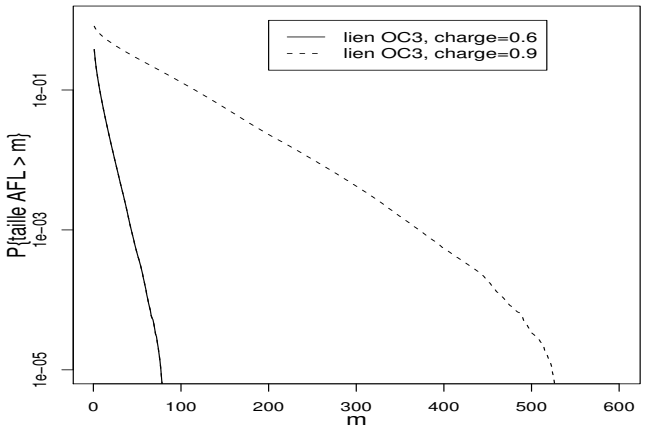
FIG. 5.14 – Distribution complémentaire de la taille de l'AFL, charges orginales

Charge	ADSL	Ab-I	Ab-III
0.6	72M	541M	3.16G
0.9	48M	361M	2.11G

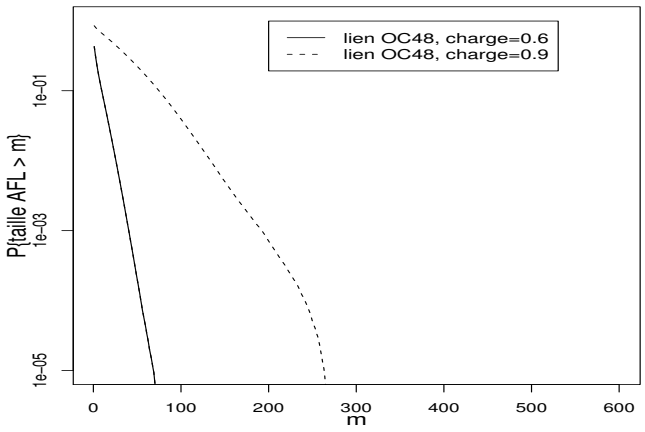
TAB. 5.3 – Capacités simulées (en b/s) pour les trois traces.

La principale observation que l'on peut faire est que le nombre de flots actifs reste relativement faible pour les trois traces et ce même sous une utilisation aussi élevée que 90%. En effet, ce nombre est de l'ordre de quelques centaines (approximativement 550 au pire des cas) ; il est donc nettement inférieur au nombre de flot en cours (voir tableau 5.1).
La deuxième observation est que la taille de l'AFL ne dépend pas de la capacité du lien puisque le nombre de flots actifs ne croît pas avec le débit du lien comme on pourrait le croire. En réalité, sur les traces simulées, le nombre de flots actifs le plus élevé a été atteint sur le lien ADSL qui est de moindre capacité. Ceci est dû au fait que la majorité écrasante des flots dans la trace ADSL appartiennent à la catégorie *non bottlenecked*, car leur débit d'accès est limité à 1 Mb/s uniquement. L'impact de la proportion des flots *bottlenecked* et *non bottlenecekd* (autrement dit, de leurs débits intrinsèques) au sein des traces sur la taille de l'AFL sera mis en évidence par le modèle analytique développé dans la prochaine section.

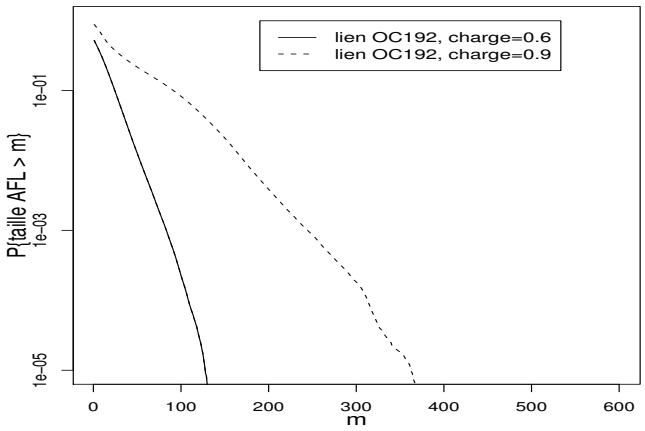
La figure 5.16 illustre les variations du *fair rate* (débit équitable) tel que calculé



(a) *ADSL*



(b) *Ab-I*



(c) *Ab-III*

FIG. 5.15 – Distribution complémentaire de la taille de l'AFL, simulations, charges 0.6 et 0.9

par l'algorithme PFQ en utilisant un lissage exponentiel de paramètre 0.9.
En ce qui concerne la trace ADSL, le *fair rate* lissé varie aux environs de 10 Mb/s sous la charge 0.9. En dépit du fait que ses variations instantanées peuvent être plus faibles, le *FR* est pratiquement tout le temps supérieur à 1 Mb/s qui est le débit crête des flots. Ceci confirme que la quasi-totalité des flots restent *non bottlenecked*.
Avec la trace *Ab-I*, lorsque la charge est de 0.6, le *fair rate* est supérieur à 300 Mb/s. Une faible proportion des flots (8%) est tout de même *bottlenecked* à cause des variations instantanées sur de courtes échelles de temps. Sous 90% de charge, le *fair rate* moyen chute à 55 Mb/s, la proportion des flots *bottlenecked* est donc naturellement plus élevée (27%).
Pour ce qui est de la trace *Ab-III*, le *fair rate* oscille au tour de 1.5 Gb/s pour une charge de 0.6. Il est inférieur à 400 Mb/s lorsque la charge est 0.9. Dans ce dernier cas, la proportion des flots *bottlenecked* atteint 72%.

A ce stade de notre analyse, on peut dores et déjà retenir que l'ordonnancement de type *fair queueing* est à la fois extensible et faisable. En effet, son extensibilité résulte du fait que le nombre de flots actifs est indépendant de la capacité ; et sa faisabilité est due au fait que ce même nombre est relativement faible.

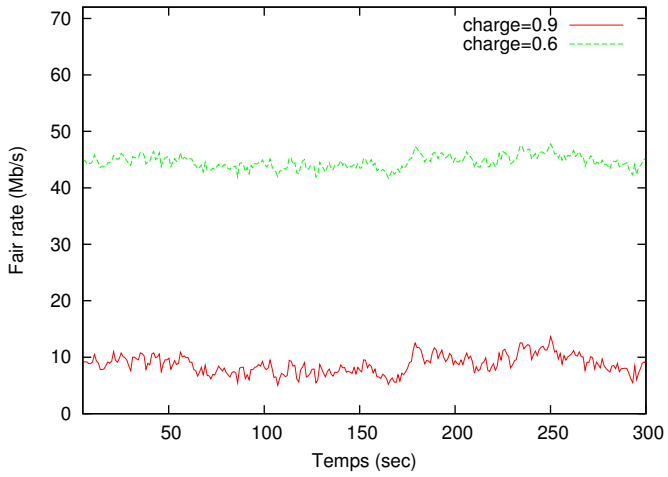
5.5 Modèle analytique multi-débit

Dans cette section, nous décrivons un modèle analytique qui permet d'expliquer les résultats de simulations présentés ci-dessus. Ce modèle tient compte des caractéristiques du trafic, en particulier des débits intrinsèques des flots [68].

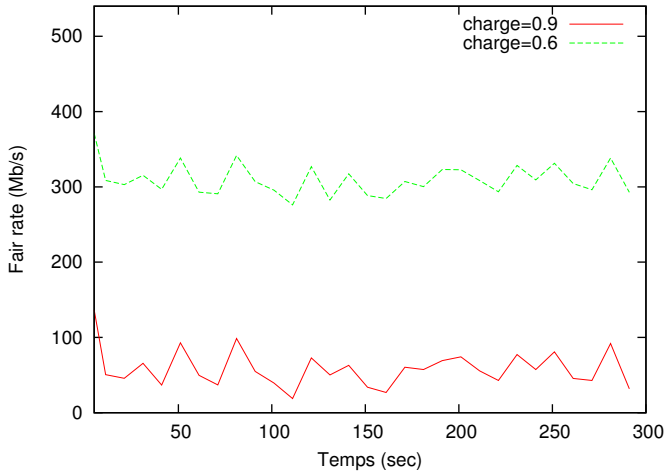
5.5.1 Modèle de trafic

On considère le problème de partage de bande passante sur un lien de capacité C. On suppose que le trafic est conforme au modèle Poisson de session introduit dans la section 2.3 et donc que les flots arrivent suivant un processus de Poisson. En outre, on considère que les flots ont un débit intrinsèque constant ayant une valeur c_i parmi l'ensemble $\{c_1, \ldots, c_M\}$ où $c_1 > c_2 > \ldots > c_M$. Les flots sont ainsi répertoriés en M classes, chaque classe étant caractérisée par un débit associé c'est-à-dire que les flots de débit intrinsèque c_i constituent la classe i. Par convention, on notera $c_0 = C$ et $c_{M+1} = 0$. Notons par $X_i(t)$ le nombre de flots en cours appartenant à la classe i à l'instant t.

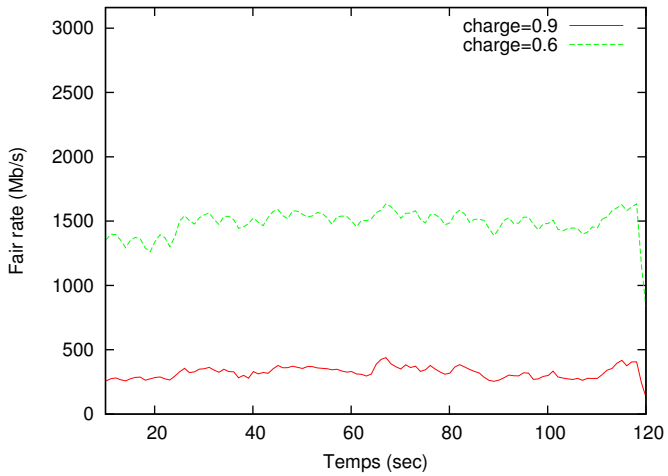
L'analyse que l'on va présenter est basée sur une séparation des échelles de temps entre le niveau flot et le niveau paquet. Autrement dit, on suppose que les X_i sont fixes et nous évaluons la distribution du nombre de flots actifs pour chaque état $x = \{x_1, \ldots, x_M\}$.
Par la suite, nous calculons la distribution stationnaire du nombre de flots en cours X, ce qui permettra de déduire la distribution inconditionnelle de la taille de l'AFL.



(a) *ADSL*



(b) *Ab-I*



(c) *Ab-III*

FIG. 5.16 – Variations du *fair rate* vs. temps, simulations, charges 0.6 et 0.9

Le modèle tient compte à la fois de l'échelle flot mais aussi de l'échelle paquet qui régissent le comportement de l'AFL.

5.5.2 Flots actifs et flots *bottlenecked*

Les flots dont le débit est supérieur au débit équitable (*fair rate*) sont sujets au partage max-min imposé par les algorithmes de *fair queueing* et réalisent, de fait, le même débit équitable θ. Lorsque les flots en cours ne sont plus en mesure de maintenir leur débit crête (intrinsèque) le partage équitable prend effet, autrement chaque flot de classe i obtient son débit c_i. Ainsi, si $\sum_{i=1}^{M} x_i c_i > C$ alors on a :

$$\theta = \frac{C - \sum_{i>J} x_i c_i}{\sum_{i \leq J} x_i} \tag{5.4}$$

où $J = J(x)$ est l'unique entier qui vérifie $c_J \geq \theta > c_{J+1}$ avec $1 \leq J \leq M$. En d'autres termes, J constitue la frontière entre les classes de flots *bottlenecked* et *non bottlenecked*.
De façon plus intuitive, le débit équitable est le débit réalisé par les flots *bottlenecked*. Il est égal au rapport entre la capacité résiduelle (non utilisée par les flots *non bottlenecked* dont les débits sont inférieurs au débit équitable) et le nombre de flots *bottlenecked* se partageant le lien.
Dans ce cas de figure, les flots dont le débit crête est supérieur ou égal à c_J réalisent le débit équitable θ tandis que les autres flots maintiennent leurs débits crêtes. Naturellement, si $\sum_{i=1}^{M} x_i c_i \leq C$ tous les flots réalisent leurs débit crêtes, on notera alors $J(x) = 0$. Bien entendu, l'indice J dépend de l'état du système représenté par le nombre de flots en cours dans chaque classe.

5.5.3 La notion de *busy cycle*

Après avoir introduit les principales notations du modèle, on revient au fonctionnement de l'algorithme PFQ pour comprendre la dynamique de l'évolution de la taille de la liste des flots actifs.

Tous les flots *bottlenecekd* sont insérés dans l'AFL. De plus, tous les flots *non bottlenecked* qui ont émis des paquets dans un passé récent, de sorte que leurs *finish tags* soient inférieurs au temps virtuel, sont également maintenus dans l'AFL.

On définit la notion de *busy cycle*, illustrée dans la figure 5.17, comme étant la période pendant laquelle le temps virtuel ne change pas et au moins un flot est *bottlenecked*. Pendant le service de paquets appartenant à des flots *bottlenecked* ayant la même estampille, des paquets de flots *non bottlenecked* peuvent arriver. Ils acquièrent alors le statut prioritaire avec une estampille égale au temps virtuel. Par conséquence, ils rallongent la durée du *busy cycle*.
Chaque paquet appartenant à un flot *non bottlenecked* qui arrive durant un *busy cycle* donne lieu à nouvelle entrée dans la liste des flots actifs. La taille de l'AFL à la fin du *busy cycle* est égale à la somme du nombre de flots *bottlenecked* et le nombre

de flots *non bottlenecked* nouvellement insérés.
Une fois que le temps virtuel change, plusieurs flots actifs (en particulier les flots *non bottlenecked*) sont susceptibles d'être supprimés de l'AFL. D'autre part, quand la file de paquets se vide (fin d'une *busy* période) la totalité de l'AFL est purgée, car tous les flots deviennent inactifs. On comprend alors pourquoi les algorithmes de *fair queueing* ne doivent gérer que les flots actifs et non pas les flots en cours sur le lien.

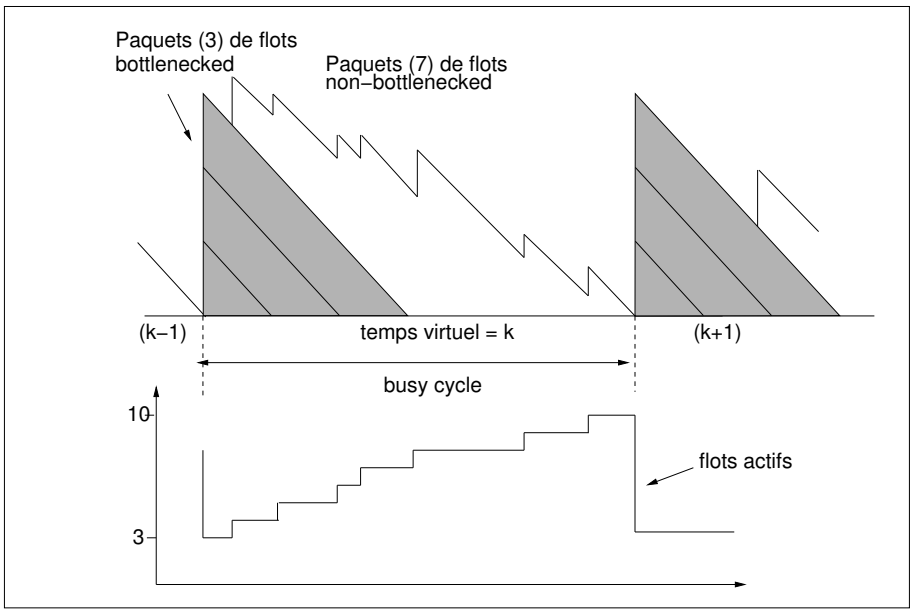


FIG. 5.17 – Illustration de la notion de *busy cycle.*

Le comportement de la liste des flots actifs, et donc sa distribution, dépendent non seulement de l'échelle flot (au travers des arrivées des flots et de leurs débits intrinsèques) mais aussi de l'échelle paquet. En effet, la durée de présence des flots *non bottlenecekd* au sein de l'AFL est intimement liée aux propriétés de *busy* périodes des files d'attente. Ainsi, la distribution de la taille des paquets joue un rôle important lors de l'évaluation de la distribution de la taille de l'AFL. On doit également faire des hypothèses concernant le processus d'arrivée des paquets appartenant aux flots *non bottlenecked.*

En premier lieu, on suppose que les paquets des flots *bottlenecked* ont une taille maximum égale à MTU. En effet, cette hypothèse garantit qu'à la suite d'un *busy cycle* tous les flots *non bottlenecked* soient supprimés de l'AFL, puisque le temps virtuel prend des valeurs multiples de MTU (donc le nouveau temps virtuel est forcément supérieur ou égal aux *finish tags* des flots *non bottlenecked.* Rappelons que, par définition, un flot *non bottlenecked* envoie au plus MTU octets par *busy cycle*). Ceci constitue une hypothèse pire cas dans la mesure où elle implique que la durée

des *busy cycle* soit plus longue puisque le temps virtuel change moins fréquemment. Cette considération simplificatrice n'en demeure pas moins raisonnable au regard de la distribution de la taille des paquets des flots de débits crêtes importants observée sur les traces de trafic.

En second lieu, afin de modéliser le processus d'arrivée des paquets des flots *non bottlenecked*, on suppose que chacun d'entre eux émet de façon indépendante son paquet durant un intervalle de temps $[t, t+dt]$ après le début d'un *busy cycle* avec une probabilité $\alpha dt + o(dt)$. Ce processus d'arrivée Poisson à source finie facilite l'analyse tout en tenant compte de la population finie du nombre de flots *non bottlenecked* donné par : $\sum_{i>J} x_i$.

En se basant sur les hypothèses ci-dessus, on peut évaluer la distribution de la taille de l'AFL. On sait que la taille de la liste des flots actifs atteint son maximum juste avant la fin d'un *busy cycle* (où certains flots accumulés sont effacés). Afin d'évaluer la distribution de ce nombre noté *MaxList*, on suppose que chaque flot *non bottlenecked* émet au plus un seul paquet par *busy cycle*.

5.5.4 Distribution conditionnelle de la taille de l'AFL

L'évolution de la taille de la liste des flots actifs telle que décrite dans la section précédente nous indique que l'estimation du nombre de flots *non bottlenecked* contribuant à *MaxList* peut être obtenue grâce à l'analyse de la *busy* période d'une file d'attente commençant par un premier service exceptionnel. Ce dernier correspond à la transmission des paquets des flots *bottlenecekd* de taille MTU ; le nombre de ces paquets étant donné par $\sum_{i \leq J} x_i$. Rappelons que l'ordre de service des paquets ayant la même estampille n'a aucun impact sur *MaxList*. Il est donc plus commode de supposer que l'on commence par servir les paquets des flots *bottlenecked* avant ceux des flots *non bottlenecked* au cours d'un *busy cycle* donné.

Il est connu que l'analyse d'une file d'attente requiert la connaissance, entre autres, du processus d'arrivée des clients ainsi que de la loi de service. Dans notre cas, la population des flots *non bottlenecked* est donnée par :
$N = \sum_{i>J} x_i$. Le processus d'arrivée des paquets appartenant aux flots *non bottlenecked* est alors obtenu en considérant que chaque flot (parmi N) émet au plus un paquet par *busy cycle* (voir section précédente).
La distribution du temps de service est celle de la taille des paquets des flots *non bottlenecked* que l'on note $F(s)$. On suppose que les tailles des paquets sont i.i.d (indépendante et identiquement distribuées) pour tous les flots *non bottlenecked* en ignorant sa dépendance vis-à-vis du nombre de flots par classe et la distribution associée (une certaine corrélation entre le débit des flots et la taille de leurs paquets existe). On considère que la taille des paquets est mesurée en unités de MTU et on note par σ la valeur moyenne.
Le taux d'arrivée moyen des paquets par flot *non bottlenecked* α est donné par $\alpha = R/(N\sigma)$ où $R = \sum_{i>J} x_i c_i$ est le débit cumulé des flots *non bottlenecked*. Ainsi,

la probabilité que k paquets arrivent pendant un intervalle de durée u après le début d'un *busy cycle* n'est rien d'autre qu'une loi binomiale :

$$\Lambda_N(k,u) = \binom{N}{k}(1-e^{-\alpha u})^k e^{-(N-k)\alpha u} \tag{5.5}$$

Ceci nous permet de déduire la distribution conditionnelle de *MaxList* indiquée dans la proposition suivante.

Proposition 1. *Soit $g(m;b,n,r)$ la distribution conditionnelle de MaxList étant donnés b flots bottlencked et n flots non bottlenecked contribuant à un débit cumulé r. En considérant que le processus d'arrivée des paquets des flots non bottlenecked suit une loi Λ et que la distribution de leur taille est F, on a :*

$$g(m;b,n,r) = \begin{cases} \frac{1}{m}\int \Lambda_n(m-1,u)dF^{[m]}(u), & b=0, m\geq 1, \\ \Lambda_n(0,b), & b>0, m=b, \\ b\int \Lambda_n(m-b-1,u)/u dF^{[m-b]}(u-b), & b>0, m>b, \end{cases} \tag{5.6}$$

où $F^{[m]}$ est le produit de convolution d'ordre m de F.

Démonstration. La formule est dérivée à partir des résultats classiques de la file M/G/1 en substituant la loi de Poisson associée par la distribution Λ_n. Cette adaptation est possible parce que le processus d'arrivées considéré a les mêmes propriétés que le processus de Poisson, ce qui permet d'appliquer le lemme combinatoire de Takàcs [103, page 231] et sa généralisation par Niu et Cooper [85].
Les trois expressions de la distribution $g(m;b,n,r)$ reflètent les trois cas possibles de combinaison entre la présence de flots *bottlenecked* et *non bottlenecked* ou pas pour initier un *busy cycle* :
La première formule, où aucun flot n'est *bottlenecked* (b=0) se ramène au résultat connu du nombre de clients servis lors d'une *busy* période d'une file M/G/1 [103, page 63].
Le deuxième cas exprime, tout simplement, la probabilité qu'aucun paquet émis par un flot *non bottlenecked* n'arrive durant le *busy cycle* ($m=b$).
Le troisième et dernier cas est dérivé à partir des résultats de [85] concernant le nombre de clients servis pendant une *busy* période commençant par un service exceptionnel de b paquets appartenant aux flots *bottlenecked*. □

Dans le but de dimensionner la liste des flots actifs, il serait plus judicieux de déterminer la distribution de la taille de l'AFL au moment où un nouveau flot doit être rajouté, puisque la saturation de l'AFL n'a d'impact que lorsqu'on ne peut pas insérer de nouveaux flot actifs (leurs paquets sont néanmoins transmis dans la file prioritaire avec PFQ).
Connaissant la distribution de *MaxList*, on peut en déduire la distribution de la taille de l'AFL *AddToList* lorsqu'un nouveau flot devrait être rajouté. Sous l'hypothèse de quasi stationnarité, la population des flots en cours x est fixe ; seuls les flots *non bottlenecked* sont rajoutés à la liste des flots actifs. La distribution conditionnelle de *AddToList* est exprimée dans la proposition suivante.

Proposition 2. *Soit $h(m;b,n,r)$ la distribution conditionnelle de* AddToList *étant donnés b flots bottlenecked et n flots non bottlnecked contribuant à un débit cumulé r. On a, pour $m \geq b \geq 0, n > 0$:*

$$h(m;b,n,r) = \frac{\sum_{k>m} g(k;b,n,r)}{\sum_{l>b}(l-b)g(l;b,n,r)} \tag{5.7}$$

Démonstration. La distribution $g(k;b,n,r)$ représente la proportion des *busy cycles* qui commencent avec une taille d'AFL égale à b et qui se terminent avec une taille de k suite à l'arrivée de $k-b$ paquets de flots *non bottlenecked*.
La probabilité qu'une arrivée arbitraire se produise pendant un tel *busy cycle* est donc $(k-b)g(k;b,n,r)/\sum_{l>b}(l-b)g(l;b,n,r)$.
La probabilité que l'arrivée en question trouve m flots au sein de l'AFL est $1/(k-b)$ pour $b \leq m < k$.
Enfin, l'expression de h est obtenue en multipliant les deux dernières probabilités et en sommant sur toutes les valeurs possibles de k. □

Après avoir effectué cette analyse au niveau paquet pour déduire l'expression de la distribution de la taille de l'AFL, comme on peut le constater, il reste à évaluer les distributions du nombre de flots *bottlenecked* et du débit cumulé des flots *non bottlenecked* afin de déduire la distribution de la taille de l'AFL inconditionnelle. Pour ce faire, il faut revenir au niveau flot afin d'évaluer la distribution du nombre de flots en cours.

5.5.5 Distribution du nombre de flots en cours

On s'intéresse maintenant à la distribution du nombre de flots en cours $X(t)$. Pour ce faire, nous considèrons un système fluide où les débits des flots sont réajustés instantanément suite aux arrivées et départs des flots.
Les algorithmes de *fair queueing* réalisent un partage de type max-min avec un débit équitable θ tel que défini dans l'équation (5.4). Le problème est que l'allocation max-min n'est pas tractable analytiquement sous des hypothèses de trafic réaliste (i.e. sans supposer des distributions de taille de flots exponentielles).
Afin de contourner cet inconvénient, nous considèrons plutôt une allocation de type *balanced fairness* qui permet d'aboutir à une formule simple de la distribution de X [18]. Il est clair que cela suppose que la distribution du nombre de flots en cours soit approximativement la même avec les deux types d'allocation. Nous vérifierons, partiellement, par la suite que c'est le cas.

L'allocation *balanced fair*

Tout comme le modèle PS, l'allocation *balanced fair* présente l'avantage d'être insensible aux caractéristiques détaillées du trafic (notamment à la distribution de la taille des flots) sous le modèle Poisson de sessions introduit dans la section 2.3.

Il est à noter que cette propriété d'insensibilité est un facteur clés dans l'optique de développer des règles d'ingénierie et de dimensionnement robustes sans avoir à se soucier des statistiques détaillées du trafic qui, du reste, peuvent changer au cours de temps. La formule d'Erlang B pour les réseaux téléphoniques en est la parfaire illustration.
L'expression de la distribution stationnaire de X ne dépend que des demandes de trafic par classe a_i où $a_i = \lambda_i \times S_i$ avec λ_i représentant le taux d'arrivée des flots de la classe i et S_i leur taille moyenne. La charge offerte est égale à a/C où $a = \sum_i^M a_i$ constitue la demande totale.
Conformément aux travaux de Bonald et Virtamo [20], nous adoptons les notations suivantes : $c^x = \prod_{i=1}^M c_i^{x_i}$, $a^x = \prod_{i=1}^M a_i^{x_i}$ et $x! = \prod_{i=1}^M x_i!$; e_i étant le vecteur de dimension M dont la composante i vaut 1 et tous les autres éléments valent 0.
La distribution stationnaire π_{BF} est alors donnée par :

$$\pi_{BF}(x) = \Phi(x)a^x/G \tag{5.8}$$

où G est la constante de normalisation qui peut être calculée en utilisant la formule récursive donnée dans [20]. La fonction de balance ϕ est obtenue récursivement par :

$$\Phi(x) = \begin{cases} 1/(x!c^x) & \text{si } xc^T \leq C, \\ \sum_{i=1}^M \Phi(x-e_i)/C & \text{si } xc^T > C. \end{cases} \tag{5.9}$$

Relations de récurrence

Soit $B_j = \sum_{i \leq j} X_i$, $N_j = \sum_{i>j} X_i$ et $R_j = \sum_{i>j} X_i c_i$ pour $0 \leq j \leq M$.
Il s'agit d'évaluer la distribution jointe deB_J, N_J et R_J à partir de la distribution de $\pi_{BF}(x)$ où J est l'indexe défini dans la section 5.5.2. Notons que J forme une partition de l'espace d'état x, on a donc, entre autres, $\theta = (C-R)/B$ et
$c_J \leq \theta < c_{J+1}$.

Soit $Q_J^{BNR}(b,n,r)$ la distribuion jointe de B_J, N_J et R_J. Comme elle fait intervenir trois variables, son évaluation directe n'est pas pratique pour des raisons de capacité mémoire énorme requise d'un point de vue programmation. Ainsi, il est préférable d'évaluer la distribution approximative suivante :

$$\begin{aligned} Q_J^{BNR}(b,n,r) &= \mathbb{P}\{B_J = b | N_J = n, R_J = r\} Q_J^{NR}(n,r) \\ &\approx Q_J^{BR}(b,r) Q_J^{NR}(n,r)/Q_J^R(r) \end{aligned} \tag{5.10}$$

Cette approximation est possible car N_J et R_J sont très corrélés puisque R représente le débit cumulé des flots *non bottlenecked*.
Pour évaluer les distributions jointes à deux variables, les relations de récurrence suivantes sont appliquées (leurs démonstrations sont données en annexe 4).

Proposition 3. *La distribution jointe de B_J et R_J pour $0 \leq J \leq M$, est donnée par $Q_J^{BR}(b,r) = q_J(b,r)$, où $q_J(b,r)$, pour $0 \leq j \leq M$, satisfait les relations de*

récurrence ci-dessous :

$$\begin{aligned} q_j(b,r) &= \sum_{i\le j}\frac{a_i}{C}[q_j(b-1,r)+\sum_{s=C-r-c_i+1}^{C-r} p_j(b-1,s,r)] \\ &+ \sum_{i>j}\frac{a_i}{C}[q_j(b,r-c_i)+\sum_{s=C-r+1}^{C-r+c_i} p_j(b,s,r-c_i)] \end{aligned} \tag{5.11}$$

et

$$\begin{aligned} p_j(b,s,r) &= \sum_{i\le j}\frac{a_i}{(s+r)\wedge C}p_j(b-1,s-c_i,r) \\ &+ \sum_{i>j}\frac{a_i}{(s+r)\wedge C}p_j(b,s,r-c_i). \end{aligned} \tag{5.12}$$

Notons que q_j et p_j sont égaux à 0 si un de leurs arguments est négatif, $q_j(0,0) = p_j(0,0,0) = \pi_{BF}(0)$ est déterminée à partir de la condition de normalisation.

Proposition 4. *La distribution jointe de N_J et R_J pour $1 \le J \le M$, est donnée par $Q_J^{NR}(n,r) = q_J(n,r)$, où $q_j(n,r)$, pour $1 \le j \le M$, satisfait la relation de récurrence suivante :*

$$\begin{aligned} (1-\sum_{i\le j}\frac{a_i}{C})q_j(n,r) &= \sum_{i\le j}\frac{a_i}{C}\sum_{s=C-r-c_i+1}^{C-r} p_j(s,n,r) \\ +\sum_{i>j}\frac{a_i}{C}[q_j(n-1,r-c_i)+&\sum_{s=C-r+1}^{C-r+c_i} p_j(s,n-1,r-c_i)] \end{aligned} \tag{5.13}$$

et

$$\begin{aligned} p_j(s,n,r) &= \sum_{i\le j}\frac{a_i}{(s+r)\wedge C}p_j(s-c_i,n,r) \\ &+ \sum_{i>j}\frac{a_i}{(s+r)\wedge C}p_j(s,n-1,r-c_i) \end{aligned} \tag{5.14}$$

Notons que q_j et p_j valent 0 si un de leurs arguments est négatif, $p_j(0,0,0) = \pi_{BF}(0)$ est déterminée à partir de la condition de normalisation.

Comparaison entre les allocations max-min et *balanced fair*

L'avantage que procure la simplicité de l'expression de la distribution de X dans le cadre d'une allocation *balanced fair* n'est exploitable que si cette même distribution n'est pas très éloignée de celle obtenue avec le partage max-min qui nous intéresse.

Un travail précédent avait déjà montré que les allocations max-min et *balanced fair* offrent des performances similaires à l'échelle flot [18].
Afin de vérifier cela dans notre contexte, nous avons conduit une comparaison entre les deux types d'allocations en effectuant des simulations fluides du partage max-min. Les comparaisons concernent les distributions marginales de B et R (pour tout J) obtenues par simulations avec celles prédites par les formules analytiques de *balanced fair*.

Nous avons simulé un partage max-min au sein d'un lien alimenté par des arrivées Poisson de flots selon deux scénarios. Dans le premier cas, la taille des flots suit une loi de Weibull de forme 0.3 (ce choix est motivé par le souci de générer une distribution à queue lente). Dans le second scénario, une distribution exponentielle pour la taille des flots a été choisie.
Lors des deux scénarios, la taille moyenne des flots est de 1 Mb. Les différents flots sont répartis en 8 classes, chacune étant caractérisée par un débit intrinsèque tel que indiqué dans le tableau 5.4. La classification des flots a été obtenue à partir de la trace *Ab-III*. Les taux d'arrivées des flots ont été calculés afin de produire les proportions de trafic en fonction de chaque classe.

Classe	Débit c_i (b/s)	Proportion a_i/a (%)
$\mathcal{C}_1$	1.5G	21
$\mathcal{C}_2$	1G	22
$\mathcal{C}_3$	800M	17
$\mathcal{C}_4$	500M	7
$\mathcal{C}_5$	100M	6
$\mathcal{C}_6$	50M	17
$\mathcal{C}_7$	10M	6
$\mathcal{C}_8$	1M	4

TAB. 5.4 – Classification des flots simulés, trace *Ab-III*.

Sous la charge 0.6, la majorité des flots sont *non bottlenecked*. Par conséquent, ils obtiennent leurs débits intrinsèques et l'impact des deux allocations est négligeable, puisque $J = 0$ avec une très grande probabilité. On assiste alors, naturellement, à un comportement identique entre les deux allocations. Il est plus intéressant de se pencher sur le cas où la charge vaut 0.9.
Les figures 5.18 et 5.19 représentent, respectivement, les distributions marginales de B et R. La comparaison est effectuée entre l'allocation max-min (avec des tailles de flots suivant des lois exponentielle et Weibull) et l'allocation *balanced*. Comme on peut le constater, l'impact de la distribution de la taille des flots n'est guère important dans le cadre de l'allocation max-min. Mais ce qui encore plus intéressant, c'est que les deux allocations aboutissent à des distributions marginales de B et R qui sont assez proches. Notons que les pics qui apparaissent dans la distribution de

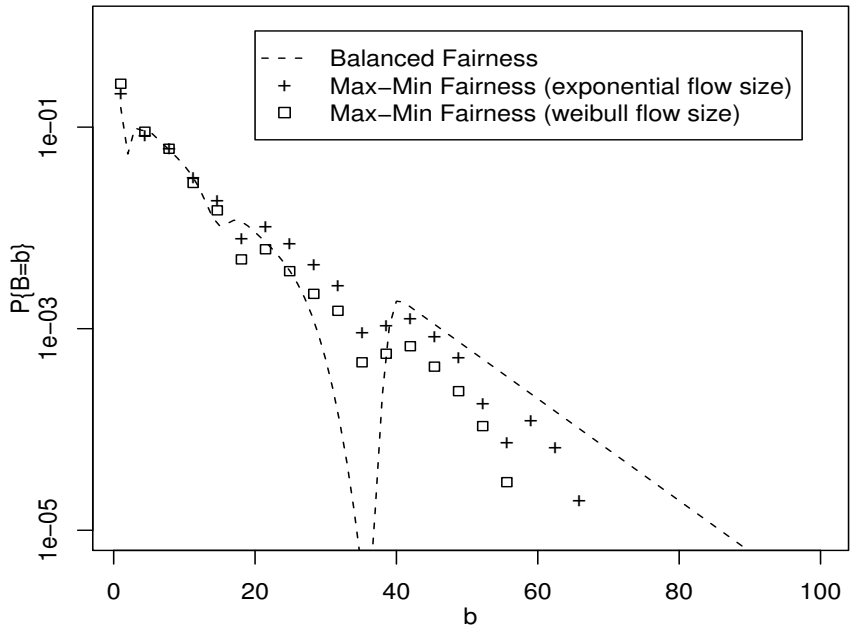


FIG. 5.18 – Comparaison des allocations max-min et *balanced*, distribution marginale de B, charge $= 0.9$, trace *Ab-III*.

B sont dus à la nature du modèle fluide où tous les flots d'une même classe ont un débit identique, ce qui implique que ces flots deviennent *bottlenecked* en même temps.

Nous avons également effectué des comparaisons entre les allocations max-min et *balanced fair* en utilisant les traces ADSL et *Ab-I*. Pour ce qui est de la trace ADSL, comme on a pu déjà le voir, pratiquement tous les flots sont *non bottlenecked*. Le partage est donc le même quelque soit le type d'allocation. En revanche, avec la trace *Ab-I* sous la charge 0.9, certains flots deviennent *bottlencekd*. Les simulations ont montré que les deux allocations offrent des résultats à peu près semblables comme dans le cas de la trace *Ab-III*.

5.5.6 Distribution de la taille de l'AFL

Une fois la distribution jointe $Q^{BNR}(b,n,r)$ calculée, il ne reste plus qu'à évaluer la distribution inconditionnelle de la taille de l'AFL. En combinant les résultats des sections 5.5.4 et 5.5.5. On a :

$$\mathbb{P}\{\text{taille AFL} = m\} = \begin{cases} \sum_{(b,n,r):J<M} h(m;b,n,r) Q_J^{BNR}(b,n,r) & J < M \\ Q_M^{BNR}(m,0,0) & J = M \end{cases}$$

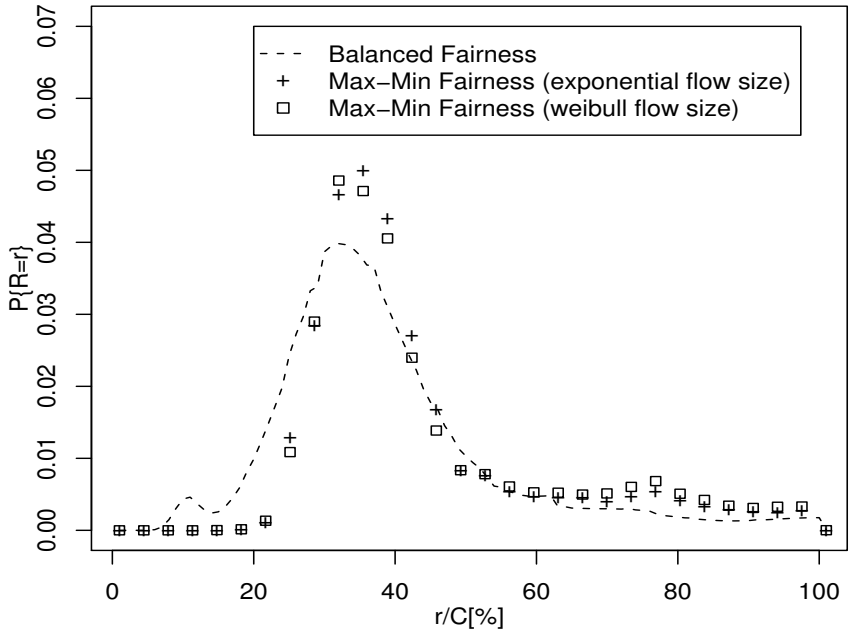


FIG. 5.19 – Comparaison des allocations max-min et *balanced*, distribution marginale de R, charge = 0.9, trace *Ab-III*.

En étant en mesure de calculer la distribution de la taille de l'AFL, nous avons utilisé les données relatives aux trois traces décrites dans la section 5.3 pour comparer les résultats analytiques avec ceux des simulations *ns*.
Pour ce faire, il fallait extraire les classes de flots à partir des traces de trafic. La classification des flots en fonction de leur débit crête ainsi que les proportions de demande de trafic par classe sont spécifiées dans le tableau 5.5 pour chacune des traces. Les valeurs en question ont été obtenues en choisissant un certain nombre de débits crêtes représentatifs couvrant l'ensemble de la distribution des débits des flots indiquée par la figure 5.13. A partir des traces, chaque flot est affecté à la classe dont le débit crête est le plus proche. Les proportions de demande de trafic sont ainsi déduites en tenant compte de la taille des flots appartenant aux différentes classes. Il est clair que cette méthode de classification constitue une simplification par rapport au trafic réel, dans la mesure où les flots n'ont pas un débit constant tout au long de leur durée de vie, et encore moins des valeurs restreintes. Néanmoins elle permet de partitionner les flots en fonction de leur débit crête, ce qui conditionne le partage effectué par l'algorithme de *fair queueing*.

Ainsi, le dernier paramètre qui reste à spécifier est la distribution de la taille des paquets des flots *non bottlenecked*. Une distribution bimodale a été appliquée. Ceci est dicté par les contraintes de l'évaluation numérique de la convolution de la proposition 1 qui est compliquée avec une distribution multimodale (plusieurs valeurs

	ADSL		Ab-I		Ab-III	
Classe	c [b/s]	a_i/a [%]	c [b/s]	a_i/a [%]	c [b/s]	a_i/a [%]
$\mathcal{C}_1$	1M	52	100M	24	1.5G	21
$\mathcal{C}_2$	512K	26	20M	10	1G	22
$\mathcal{C}_3$	128K	12	15M	9	800M	17
$\mathcal{C}_4$	50K	10	12M	14	500M	7
$\mathcal{C}_5$	-	-	10M	14	100M	6
$\mathcal{C}_6$	-	-	5M	11	50M	17
$\mathcal{C}_7$	-	-	1M	4	10M	6
$\mathcal{C}_8$	-	-	500K	12	1M	4
	$a = 40$Mb/s		$a = 345$Mb/s		$a = 1.9$Gbps	

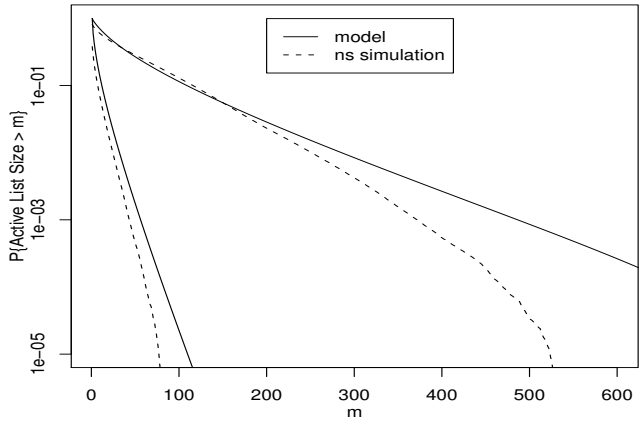
TAB. 5.5 – Classification des flots.

de taille de paquets).
Les valeurs choisies pour la distribution bimodale sont 40 et MTU octets avec les proportions relatives aux traces (50% pour chacune des deux valeurs dans le cas des traces ADSL et *Ab-I*, 30% et 70%, respectivement, pour la trace *Ab-III*). La valeur de MTU était de 1500 octets pour les traces ADSL et *Ab-I* et de 9000 octets pour la trace *Ab-III*.

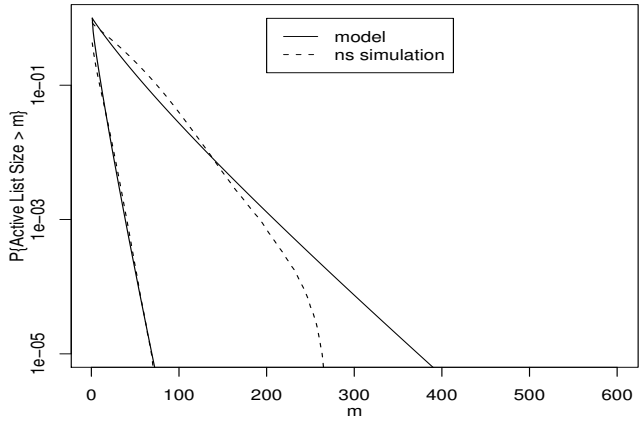
Les résultats obtenus pour la taille de la liste des flots actifs sont présentés dans la figure 5.20. La première impression qui en ressort est que le modèle analytique permet de prédire convenablement les résultats de simulations *ns*. Ces résultats sont discutés au cours de la prochaine section.

5.6 Analyse des résultats

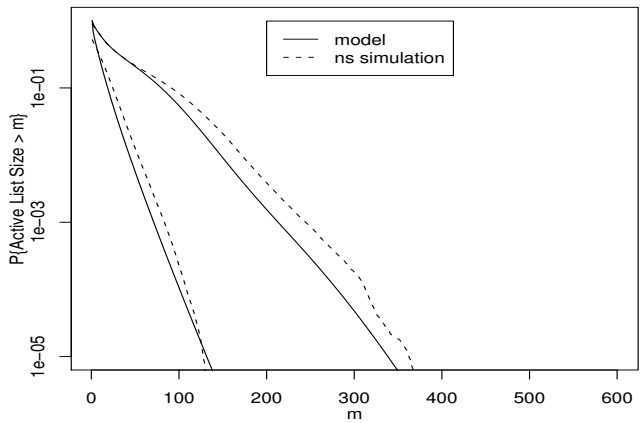
L'analyse des résultats et des paramètres du modèle analytique fait jaillir certains points cruciaux à prendre en compte en vue du dimensionnement de la liste des flots actifs pour des algorithmes de *fair queueing*.
En premier lieu, le nombre de flots en cours n'a pas d'incidence directe sur le nombre de flots actifs. Cette observation est valable dans le cadre d'une allocation *balanced*, mais elle peut l'être également pour une allocation max-min pour les raisons évoquées auparavant. Il s'avère donc que l'écart entre le nombre de flots en cours et celui des flots actifs est très important.
Les principales caractéristiques du trafic qui ont un impact direct sur la taille de l'AFL sont la charge totale offerte, la taille des paquets des flots *non bottlenecked* et la répartition des débits des flots (qui détermine les proportion des flots *bottlenecked* et *non bottleneked*). C'est pourquoi d'ailleurs, en dépit de son imprécision inhérente,



(a) *ADSL*



(b) *Ab-I*



(c) Ab-III

FIG. 5.20 – Distribution complémentaire de la taille de l'AFL, modèle analytique multi-débit vs. simulation, charges 0.6 et 0.9

la classification des flots dans le modèle fluide en un ensemble fini de débits crêtes produit des résultats satisfaisants et une bonne approximation des simulations réalisées au niveau paquet.
En résumé, le fait qu'il y ait des centaines de milliers de flots en cours sur des liens à grandes capacités ne devrait pas constituer un handicap pour la mise en place d'algorithmes de *fair queueing*, puisque le nombre de flots actifs lui se limite à quelques centaines seulement.

Un second aspect à considérer est que le nombre de flots *bottlenecked* est très faible et ce même sous des charges aussi élevées que 0.9 (conformément aux prédictions du modèle PS). La majorité des flots actifs correspond à des paquets émis sporadiquement par les flots *non bottlenecked*. Leur temps de présence dans la liste des flots actifs se résume à la durée d'un *busy cycle*. Plus la charge due aux flots *non bottlenecekd* est importante, plus la durée des *busy cycles* est rallongées et, par conséquent, plus grand est le nombre de flots actifs accumulés. Ceci explique la raison pour laquelle la trace ADSL a donné lieu au pire comportement de la taille de l'AFL. En revanche, la présence de flots à fort débit dans la trace *Ab-III* a eu pour effet de diminuer le nombre de flots actifs par rapport au lien OC3. Dans le cadre de *cross-protect* [73], où le *priority load* est limité à une valeur maximum et le *fair rate* est maintenu à un seuil minimum, le cas extrême pour la taille de l'AFL serait atteint lorsque la charge prioritaire est au maximum autorisée (si on suppose qu'elle reste inférieure à 1) et que le nombre maximum de flots *bottlenecked* compatible avec le débit équitable minimum est présent.

Enfin, comme mentionné précédemment, la taille de l'AFL dépend à la fois du niveau flots par le biais des débits des flots et du niveau paquets via la distribution de la taille des paquets des flots *non bottlenecked*. Intuitivement, plus la taille des paquets des flots *non bottlenecked* est faible, plus leur taux d'arrivée est grand (pour une même charge). L'augmentation du taux d'arrivée signifie l'augmentation du nombre d'entrées crées dans la liste des flots actifs.

5.7 Extension des résultats à PDRR

L'étude présentée concerne l'algorithme PFQ, toutefois ces résultats peuvent être étendus à d'autres algorithmes basés sur la notion d'estampille temporelle décrits dans la section 4.2.2. En effet, peu importe l'algorithme exact qui détermine le calcul de l'estampille affectée à chaque paquet, la philosophie de rajout et de suppression des flots actifs reste la même.
Cette remarque est également valable pour les algorithmes à base de *round robin* (dont certains ont été présentés dans la partie 4.2.3). D'ailleurs des simulation *ns* ont été conduites avec les mêmes traces de trafic en utilisant l'algorithme PDRR et ont donné lieu à des résultats pratiquement similaires comme le montre la figure 5.21.

Notons que le nombre de flots actifs est légèrement plus grand avec PDRR qu'avec PFQ pour la trace *Ab-III*. Ceci est dû au fait que la condition de suppression des flots avec PFQ est plus précise puisqu'elle tient compte du *finish tag* (qui lui dépend de la tailles des paquets) alors qu'avec PDRR les flots ne sont supprimés que si leur file est vide lorsqu'ils sont visités, d'autant plus que le quantum vaut 9000 octets pour *Ab-III* (ce qui rallonge le cycle).

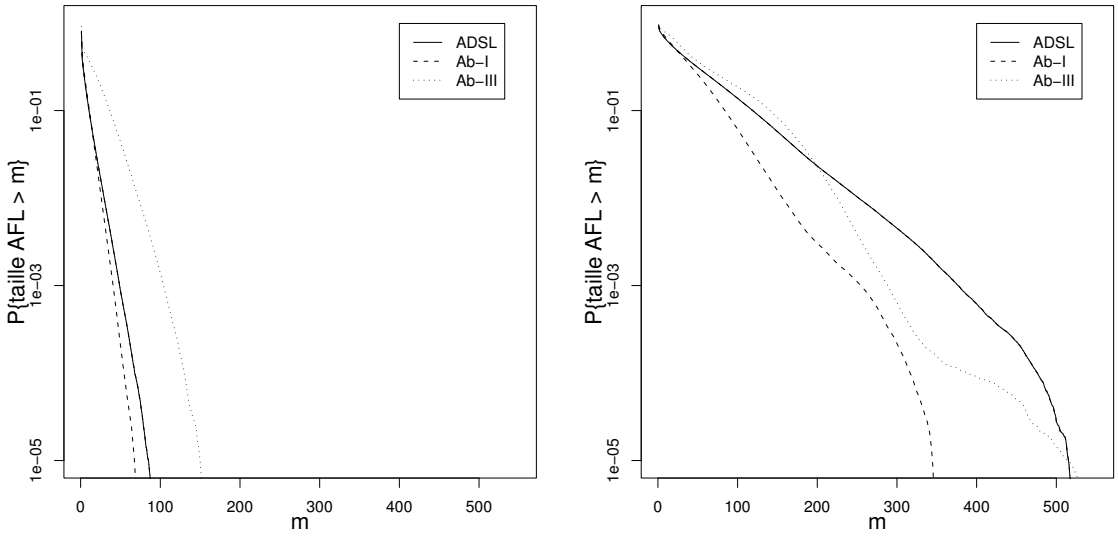


FIG. 5.21 – Distribution complémentaire de la taille de l'AFL, charges 0.6 (à gauche) et 0.9 (à droite), algorithme PDRR

La présente discussion pourrait également servir pour déterminer le nombre de flots à prendre en compte dans certains mécanismes d'équité approximative décrits dans la section 4.3 dans un cadre de trafic dynamique et donc plus réaliste.

5.8 Surcharge

On a pu voir que le nombre de flots actifs reste faible même sous une utilisation de 90%. D'autres simulations effectuées avec une charge de 0.95 (La figure 5.22 représente la distribution complémentaire de la taille de l'AFL pour les traces de trafic de la section 5.3 en utilisant l'algorithme PDRR) ont indiqué que le *fair queueing* reste extensible. Par contre, en situation de surcharge, où le trafic offert dépasse la capacité du lien, le nombre de flots en cours et le nombre de flots actifs croissent nettement. La taille de la liste des flots actifs étant limitée, le scheduler se comporterait comme une file FIFO une fois l'AFL saturée (les paquets sont insérés dans la file sans que leurs flots ne soient rajoutés à l'AFL). De ce fait, le débit des flots chute drastiquement et les performances se dégradent.

Il est clair qu'il faut envisager des mécanismes qui permettent de parer à ce genre de situation. Aucune discipline de service ni mécanisme d'AQM ne sont en mesure de le faire. La solution que l'on préconise est l'application d'un contrôle d'admission implicite qui rejette le surplus de trafic proactivement. Pour ce faire, la mesure du débit équitable fournie par un scheduler de type *fair queueing* constitue un critère d'admission adéquat, comme on a pu le voir dans le chapitre 2.

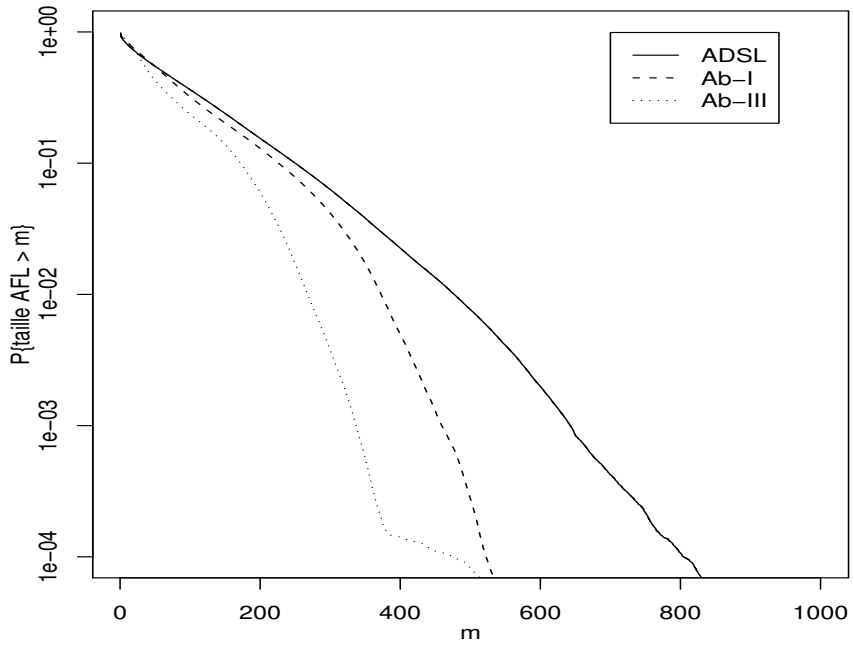


FIG. 5.22 – Distribution complémentaire de la taille de l'AFL, charge 0.95.

La taille de la liste des flots actifs pourrait être encore réduite en appliquant une insertion probabiliste des nouveaux flots, ce qui diminuerait le nombre d'ajouts dus aux flots *non bottlenecked*.
Une autre approche intéressante consisterait à identifier les flots *non bottlenecked* afin de se contenter d'insérer uniquement les flots *bottlenecked* dans l'AFL, puisqu'ils sont les seuls à être directement concernés par le partage équitable selon l'algorithme de FQ mis en place. Nous explorons cette possibilité dans le prochain chapitre.

Chapitre 6

Réduction du coût d'implémentation de l'architecture FAN 2^{nd}G

Au cours de ce dernier chapitre, nous proposons des améliorations permettant de réduire le coût de mise en œuvre de l'architecture *cross-protect* [67].

Plus précisément, ces améliorations concernent, d'une part, la gestion des flots actifs par les algorithmes de *fair queueing* précédemment décrits, et d'autre part, la gestion des flots en cours par le mécanisme du contrôle d'admission.

6.1 Réduction du coût de gestion des flots actifs

Lors du chapitre précédent nous avons vu que les algorithmes de *fair queueing* maintiennent une liste des flots actifs afin de pouvoir ordonnancer leurs paquets. On s'est également rendu compte, qu'en réalité seule la connaissance des flots *bottlenecked* est indispensable. En d'autres termes, il n'est pas nécessaire de maintenir d'états pour les flots *non bottlenecked* au sein de la population des flots actifs. Rappelons que les flots *non bottlenecked* émettent des paquets espacés à un débit inférieur au débit équitable. Les MTU octets qu'ils envoient, au plus, au cours du même cycle étant traités en priorité, aucune information concernant ces flots n'a besoin d'être retenue (du moins avec PFQ et PDRR).

Sachant que les flots actifs sont majoritairement *non bottlenecked*, il est possible de réduire la taille de la liste AFL, en y insérant uniquement les flots *bottlenecked*. Pour accomplir cela, il faut être en mesure de les identifier. En effet, les flots *non bottlenecked* sont momentanément inclus dans l'AFL pour permettre la détection des nouveaux flots *bottlenecked*, ce qui a pour effet d'augmenter considérablement la capacité mémoire nécessaire à cette structure par rapport à celle qui est strictement requise.

De plus, la taille de la liste des flots actifs dépend des caractéristiques du trafic, notamment du rapport entre le débit du lien et le débit intrinsèque des flots (le débit qu'ils pourraient atteindre si le lien était de capacité infinie et compte tenu des autres limitations de débit sur leur chemin). En identifiant autrement les flots

bottlenecked, la capacité de l'AFL peut être limitée à une centaine de flots. Cette capacité est largement indépendante des caractéristiques du trafic et constitue donc un dimensionnement robuste.

Dans le prochain paragraphe, nous décrivons la méthode d'identification des flots *non bottlenecked*. Par la suite, nous évaluons, par simulation, le bénéfice apporté par ce procédé qui a fait l'objet d'un dépôt de brevet [66].

6.1.1 Identification des flots *non bottlenecked*

On cherche à détecter les flots *non bottlenecked* sans avoir à les rajouter à l'AFL. Pour y parvenir, nous utilisons une structure de donnée supplémentaire nommée *NewFlows* qui consiste en un tableau de M éléments de b bits. A chaque flot f correspond un élément unique i de ce tableau déterminé par une fonction de hachage (appliquée à l'identifiant du flot). Chaque élément représente un compteur, $Bytes.i$, du nombre d'octets émis par le flot associé dans la partie prioritaire de la file.

La mise à jour des valeurs de $Bytes.i$ est simple à effectuer (incrémentation par la taille des paquets du flot f), les instants de remise à 0 (RAZ) qu'on décrira plus loin déterminent l'identification des flots *non bottlenecked*.

Dès que la valeur de $Bytes.i$ dépasse un certain seuil appelé $MaxBytes_i$, le flot est considéré comme *bottlenecked*; il est donc rajouté à la liste des flots actifs. En revanche, tant que la valeur de $Bytes.i$ reste inférieure au seuil en question, les paquets continuent à être insérés dans la file prioritaire et le flot f n'est pas rajouté à l'AFL.
Le seuil $MaxBytes_i$ peut être égal à MTU (i.e., identique pour tous les flots) ou bien au quantum du flot (dans le cadre de PDRR).

Les instants de RAZ sont choisis de façon à garantir que les flots qui émettent plus de MTU octets par cycle soient *bottlenecked*, il s'agit des flots dont le débit est supérieur au débit équitable. La durée du cycle est directement déduite à partir de la mesure effectuée du *fair rate*. Soit $FR(t)$ la mesure du *fair rate* à l'instant t, *NewFlows* est remis à 0 aux instants $t_n, n = 1, 2, \ldots$ avec $t_{n+1} = t_n + 8 \times \mathrm{MTU}/\mathrm{FR}(t_n)$. On désignera par intervalle de référence les époques entres deux RAZ successives.

Comme mentionné ci-dessus, le compteur $Bytes.i$ est codé sur b bits. Bien entendu, plus la valeur de b est grande plus le compteur est précis mais moins on obtient de gain mémoire. Afin d'optimiser la précision du compteur, nous adaptons la valeur de l'unité à la limite $MaxBytes_i$ du flot concerné. Plus spécifiquement, le compteur serait incrémenté suite à l'arrivée d'un paquet de taille L de la manière suivante :
$Bytes.i \mathrel{+}= \lceil (2^b - 1)L/MaxBytes_i \rceil$ où $\lceil x \rceil$ désigne l'entier immédiatement supérieur ou égal à x. Ainsi, lorsque le compteur déborde, le flot devient *bottlenecked* et il est inséré dans l'AFL. Par construction, la liste des flots actifs ne contient que les flots *bottlenecked*.

Le cas particulier $b = 1$ mérite une attention particulière. En effet, tout flot est considéré comme *bottlenecked* à partir du moment où il émet au moins deux pa-

quets dans le même intervalle de référence indépendamment de leurs tailles. Malgré l'imprécision qui en résulte, ce choix est attractif car il minimise la taille de *New-Flows*. Les simulations que l'on présentera dans la section 6.1.5 révèlent que ce choix conduit à des résultats satisfaisants.

La précision de ce mécanisme d'identification des flots *non bottlenecked* est discutée dans le paragraphe 6.1.4.

Les algorithmes PFQ et PDRR sont modifiés afin de prendre en compte le mécanisme d'identification des flots *non bottlenecked* que nous venons de décrire. Nous présentons les pseudo-codes qui en découlent dans les deux prochains paragraphes.

6.1.2 pseudo-code modifié de PFQ

Le pseudo-code modifié de PFQ (avec $b = 1$) faisant suite à l'arrivée d'un paquet appartenant à un flot f est indiqué ci-dessous. Les instructions en italique sont celles qui différent par rappport au pseudo-code classique de PFQ présenté dans 2.2.2. Les instructions relatives à la fin de traitement d'un paquet restent inchangées, elles ne sont donc pas spécifiées ici.

```
1. si (PIFO saturée) alors rejeter le paquet en tête de la file la plus longue
2. si (f ∈ AFL)
3.     f.backlog += L
4.     insérer {paquet,f.finishTag} dans PIFO
5. sinon
6.     calculer image i du flot f dans NewFlows
7.     si (Bytes.i = 0)
8.         Bytes.i = 1
9.         insérer {paquet,temps virtuel} dans PIFO derrière P et mettre à jour P
10.        priority bytes += L
11.    sinon
12.        insérer {paquet,temps virtuel} dans PIFO
13.        si (AFL n'est pas saturée)
14.            rajouter flot f
15.            f.finishTag = temps virtuel +L
16.            f.backlog = L
```

Si le flot est déjà actif, le paquet est inséré dans la file FIFO du flot. Par contre, s'il n'est pas encore présent dans l'AFL, c'est *Bytes.i* qui détermine où il sera inséré. On constate que lorsque le flot f bascule en *bottlenecked*, son paquet est inséré dans la PIFO avec une estampille égale au temps virtuel (car on ignore la taille de son premier paquet, cette petite imprécision n'est toutefois pas préjudiciable pour les autres flots) mais pas dans la partie prioritaire.

6.1.3 pseudo-code modifié de PDRR

De la même manière que pour PFQ, le pseudo-code modifié de PDRR [1] exécuté lors de l'arrivée d'un paquet p de taille L comporte des instructions en italique pour

[1] On suppose que la liste AFL n'est pas saturée

se démarquer du pseudo-code présenté en 2.5.1.

```
1. si (buffer saturé) alors
2.       Rejeter un paquet
3. f = Exraire_flot(p)
4. si (f ∈ AFL)
5.       Insérer(Queue_i,p)
6.       f.backlog +=L
7. sinon
8.       calculer image i du flot f dans NewFlows
9.       si (Bytes.i = 0)
10.          Bytes.i = 1
11.          Insérer(PQ,p)
12.      sinon
13.          rajouter f à AFL
14.          Insérer(Queue_i,p)
15.          f.backlog +=L
16.          f.DC = 0
```

L'extraction de paquets reste inchangée. La file PQ est visitée en priorité, quand elle est vide, les autres files envoient leurs paquets en fonction des quanta et déficits.

Une réalisation typique de la liste des flots actifs avec PDRR dans ce cas est donnée par la figure 6.1. Elle comporte 6 champs occupant 16 octets de mémoire par entrée (si on considère que les pointeurs et compteurs sont stockés sur 2 octets et que l'identifiant est sur 4 octets). Le champ *Next* indique le prochain flot dans le cycle DRR, DC le compteur de déficit et *backlog* la taille en octets de la file associée au flot. La liste de paquets consiste en deux pointeurs, l'un sur la tête et l'autre sur la fin de la file associée au flot. Le champ *ByteCount* qui servait à indiquer le nombe d'octets insérés en priorité par le flot n'est plus nécessaire.

identificateur de flot	backlog	Quantum	DC	Next	liste chainee de paquets
f	f.backlog	f.Quantum	f.DC	f.Next	f.FIFO

FIG. 6.1 – Structure de l'AFL avec PDRR modifié

6.1.4 Précision du mécanisme

La simplification du mécanisme d'identification des flots *bottlenecked* induit un certain degré d'imprécision. Des erreurs de type faux positif et faux négatif peuvent

se produire. Dans le premier cas, un flot peut être considéré à tort comme *bottlenecked* et dans le second cas, un flot réellement *bottlenecked* n'est pas détecté en tant que tel.

Les erreurs de type faux positif surgissent dans l'un des deux cas suivants :

- La granularité du compteur représentée par le nombre de bits b peut conduire à une surestimation du nombre d'octets envoyés en priorité par un flot donné. Par exemple, quand b vaut 1, dès qu'un flot émet deux paquets dans le même intervalle de référence, il est considéré comme *bottlenecked* alors que la somme des tailles des deux paquets pourrait être inférieure à *MaxBytes*.
- L'utilisation d'une fonction de hachage pour le calcul des images des flots peut conduire à une collision, où les images de deux, voire plusieurs, flots distincts coïncident au sein de *NewFlows*. Un des flots pourrait alors causer le débordement du compteur associé, sans qu'il ait un débit supérieur au débit équitable.

La probabilité du premier type d'erreur dépend des caractéristiques du trafic et notamment des valeurs de *MaxBytes*. Au cours de nos simulations (voir section 6.1.5), nous avons constaté que $b = 1$ est presque aussi efficace que le choix de $b = 8$ lorsque *MaxBytes* vaut MTU pour tous les flots. Il est clair qu'une valeur de b plus importante que 1 serait souhaitable si un partage pondéré est envisagé, où certains flots disposent d'un *MaxBytes* élevé.

La probabilité du deuxième type d'erreurs peut êtte contrôlée via le choix du nombre de compteurs dans la structure *NewFlows* : M.
En effet, si on considère une réalisation en *bitmap*, c'est-à-dire que $b = 1$, et que n paquets provenant de différents flots arrivent pendant un intervalle de référence ; alors en supposant que x faux positifs se produisent, on déduit que $n - x$ images sont occupées. On sait alors que la probabilité que cet événement se produise peut être calculée à partir des résultats de [40, page 102]. Soit $p_k(M, n)$ la probabilité que k images soient vides (non occupées), on a alors :

$$p_k(M,n) = \binom{M}{k} \sum_{\nu=0}^{M-k} (-1)^\nu \binom{M-k}{\nu} \left(1 - \frac{k+\nu}{M}\right)^n$$

avec $k = M - n + x$ dans notre cas. L'évaluation directe de cette formule pour des valeurs de n importantes n'est pas pratique. Une approximation peut être obtenue en calculant la limite lorsque n tend vers l'infini. Ainsi, le nombre de faux positifs suit approximativement une loi de Poisson d'une moyenne $n^2/2M$.
Notons tout de même que seules les collisions qui conduisent à un débordement du compteur peuvent être nuisibles. En effet, si l'image de deux flots coïncide sur des intervalles de référence distincts, ou sans que le compteur ne dépasse *MaxBytes*, cela est sans effet sur les flots en question.

Les valeurs que n peut prendre seront abordées au cours de la partie simulation où l'utilisation de trace de trafic réel suggère qu'une taille relativement faible d'un

bitmap de quelques milliers de bits est suffisante pour limiter la probabilité de faux positifs à approximativement 1%.

Les conséquences d'un faux positif sur les performances des flots ne sont pas critiques. En fait, leur effet ne devrait pas perdurer au delà de la durée d'un intervalle de référence, où un seul paquet du flot reconnu comme étant *bottlenecked* par erreur ne bénéficierait pas du traitement prioritaire. Si le flot en question a bien un débit inférieur au débit équitable, son compteur devrait être réinitialisé lors des prochains cycles et, par conséquent, retrouverait son statut de *non bottlenecked*. L'augmentation des délais qui en résulte est d'autant moins importante sur des liens à grande capacité.

Les erreurs de type faux négatif peuvent se produire pour les flots dont le débit est légèrement plus important que le débit équitable. La figure 6.2 illustre ce concept, où un flot ayant un débit légèrement supérieur au débit équitable n'est détecté en tant que *bottlenecked* que lors du troisième intervalle de référence, puisqu'il y envoie deux paquets alors que les autres paquets étaient mal placés par rapport aux instants de RAZ. Néanmoins, l'insertion des premiers paquets dans la partie prioritaire de la file ne nuit guère à l'efficacité de l'ordonnancement tant que le *priority load* reste contrôlé.
Il est clair que ce type d'erreur n'affecte que les flots dont le débit est proche du débit équitable, et que de toute façon ses flots oscillent entre les catégories *bottlenecked* et *non bottlenecked* suite aux fluctuations du *fair rate* indépendamment de l'utilisation de la structure *NewFlows*.

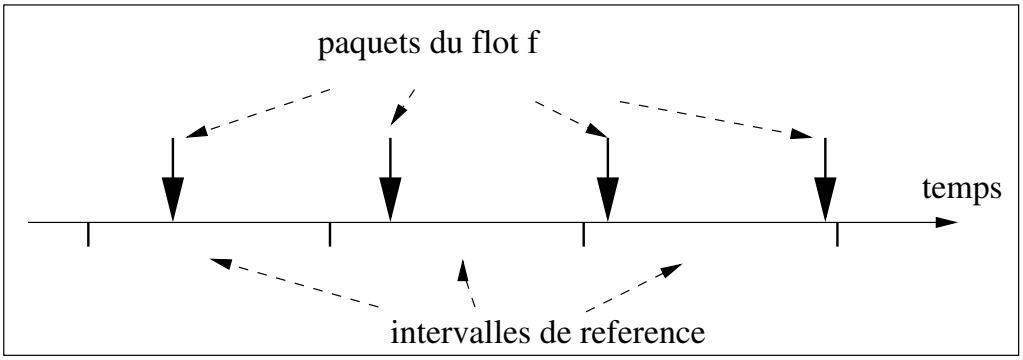


FIG. 6.2 – Détection tardive d'un flot *bottlenecked* ($b = 1$)

Dans l'optique de réduire davantage la capacité mémoire requise, la structure de *NewFlows* peut être plus complexe qu'un simple tableau. Les techniques du filtre de Bloom [15] ou filtres multi-étages [39] peuvent être employées à cet effet. Naturellement, ceci exigerait d'appliquer plusieurs fonctions de hachage.

6.1.5 Evaluation par simulation

Nous avons conduit des simulations *ns* en utilisant deux traces de trafic réelles : la trace ADSL et la trace *Ab-I* déjà décrites dans la section 5.3 du chapitre précédent. Le but de ces simulations est de vérifier, en premier lieu, l'apport des modifications des algorithmes PFQ et PDRR dans la réduction de la taille de la liste des flots actifs. Dans un second temps, il s'agit de confirmer que ces modifications n'entachent pas le fonctionnement normal.
On rappelle que la trace ADSL se caractérise par la prédominance des flots *non bottlenecked* à cause des limitations de leur débit d'accès, elle devrait mettre en évidence le gain réalisable par notre méthode d'identification des flots *non bottlenecked*.

Afin d'avoir une taille de l'AFL potentiellement importante, la charge simulée est 0.9. La valeur choisie pour *MaxBytes* est MTU pour tous les flots et le nombre de compteurs est $M = 16000$. Deux solutions ont été testées, l'une avec $b = 1$ et l'autre avec $b = 8$.

La figure 6.3 trace la distribution complémentaire de la taille de l'AFL avec les trois configurations : $b = 1$, $b = 8$ et la version originale de PDRR où les flots actifs sont rajoutés systématiquement à l'AFL (i.e. sans utilisation de NewFlows).
On remarque une baisse drastique de la taille de l'AFL dans le cadre de la trace ADSL. En effet, dans ce cas le nombre maximum de flots actifs insérés dans l'AFL est 4 (avec les solutions $b = 1$ et $b = 8$) alors qu'il avoisinait les 500 flots avec l'algorithme PDRR classique. Ce résultat confirme l'efficacité du mécanisme dans la détection des flots *bottlenecekd* minoritaires dans la trace ADSL.
Le gain est moins éclatant avec la trace *Ab-I*, il demeure néanmoins appréciable puisqu'il représente approximativement un facteur de 3 . Ceci est dû à la plus grande proportion de flots *bottlenecked* présents dans la trace *Ab-I*.
Notons que la solution *bitmap* ($b = 1$) est presque aussi bonne que $b = 8$ (compteurs sur un octet) légitimant son intérêt en vue d'un gain mémoire plus important.

La probabilité de faux positifs observée au cours des simulations est $3\ 10^{-3}$ pour les deux traces. Elle est en ligne avec celle prédite par la formule analytique de la section 6.1.4 : $n/2M$ $(= (n^2/2M)/n)$, où n représente le nombre de flots considérés comme *non bottlenecked* (ou de façon équivalente le nombre de paquets prioritaires) qui arrivent par intervalle de référence. La valeur moyenne de n est 13.8 pour la trace *Ab-I*, elle vaut 15 pour la trace ADSL. Les variations de n (échantillonnées) sont données par la figure 6.4, comme on peut le constater des pics importants peuvent être atteints.

La valeur de n dépend du taux d'arrivée des paquets des flots *non bottlenecked* et des durées des intervalles de référence. En se basant sur les résultats présentés dans le paragraphe 5.5 du chapitre précédent, la valeur moyenne minimum du *fair rate* sous une utilisation de 90% devrait être supérieure à $0.1C$, ce qui induit un intervalle de référence d'une durée maximum de $8\times$ MTU$/0.1C$. Si on suppose que la taille moyenne des paquets des flots *non bottlenecked* est γMTU ($\gamma = 0.3$, par exemple) alors le taux d'arrivée de ces paquets est inférieur à $0.9C/(8\gamma\text{MTU})$ (puisque la

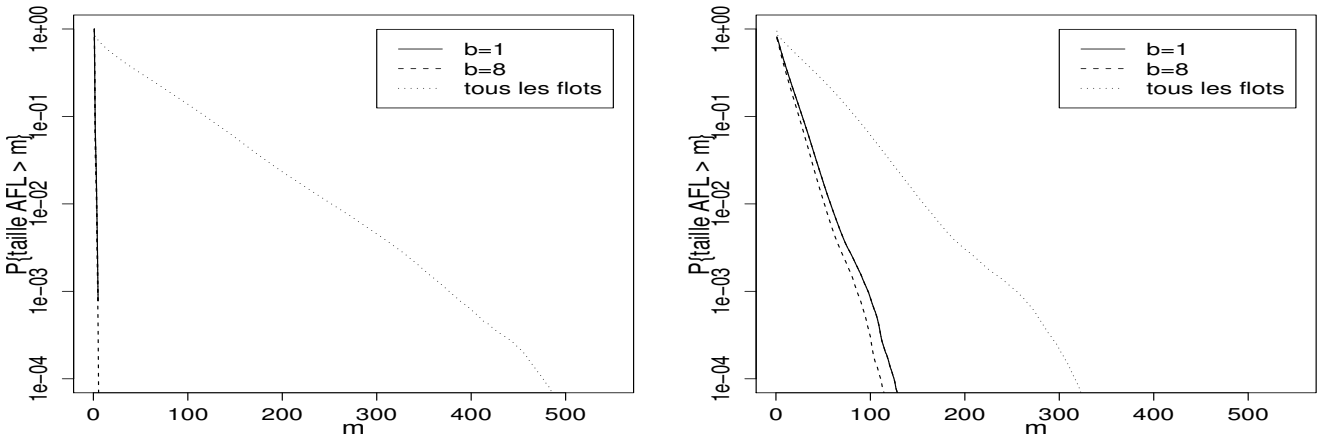


FIG. 6.3 – Distribution complémentaire de la taille de l'AFL, traces ADSL (gauche) et *Ab-I* (droite), PDRR modifié

charge totale est 0.9, la charge due aux flots *non bottlenecked* ne peut être que plus faible). Ceci nous donne $n = 9/\gamma$.

Signalons que la pénalité résultante d'un faux positif pour un flot *non bottlenecked* est minime dans le cas de la trace *Ab-I*, puisque le délai moyen des paquets est de 0.82ms seulement (le temps de transmission d'un paquet MTU est 33 μs).

Des simulations avec $M = 1000$ ont également été conduites, elles ont donné lieu à une probabilité de faux positif de 6 10^{-3} et un gain important dans la taille de l'AFL, en particulier pour la trace ADSL. Ce choix s'avère donc satisfaisant.

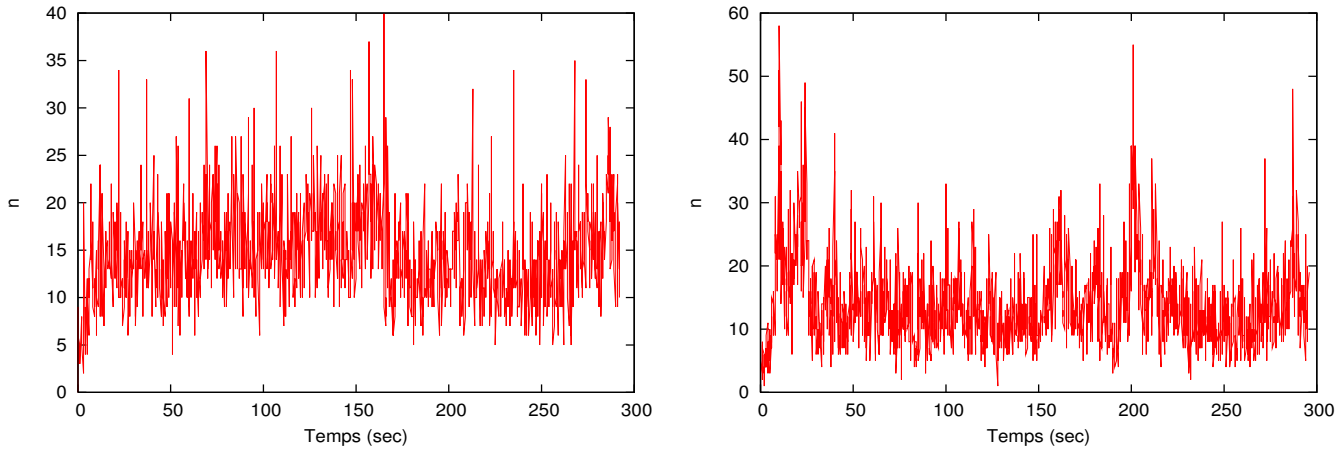


FIG. 6.4 – Nombre de flots *non bottlenecked* arrivant par intervalle de référence, traces ADSL (gauche) et *Ab-I* (droite), PDRR modifié

Maintenant qu'on a pu voir que la version modifiée de PDRR permet de réduire la taille de la liste des flots actifs conformément aux attentes, il reste à vérifier que cela n'a aucune incidence sur le fonctionnement habituel de l'algorithme.

La figure 6.5 indique que l'évolution du *fair rate* normalisé reste inchangée avec la version modifiée de PDRR, puisque les deux courbes sont confondues pour les deux traces.

La figure 6.6 trace l'évolution du *priority load*. On retrouve les mêmes variations pour la trace ADSL, puisque la quasi totalité des paquets sont insérés en priorité. Pour ce qui est de la trace *Ab-I*, on constate que la valeur du PL est légèrement plus importante avec l'algorithme PDRR modifié, car les flots *non bottlenecked* sont détectés en fonction du *fair rate* moyen (lissé) et non pas instantané comme c'est le cas dans PDRR. Néanmoins, ceci n'a pas d'impact sur le délai moyen des paquets traités en priorité qui est très faible : 0.05ms.
On en déduit que le comportement de l'algorithme ne change pas, ce qui fait que l'apport de la structure *NewFlows* ne peut être que bénéfique. Ainsi, la nette réduction de la taille de l'AFL se fait de manière transparente par rapport au fonctionnement de l'algorithme.

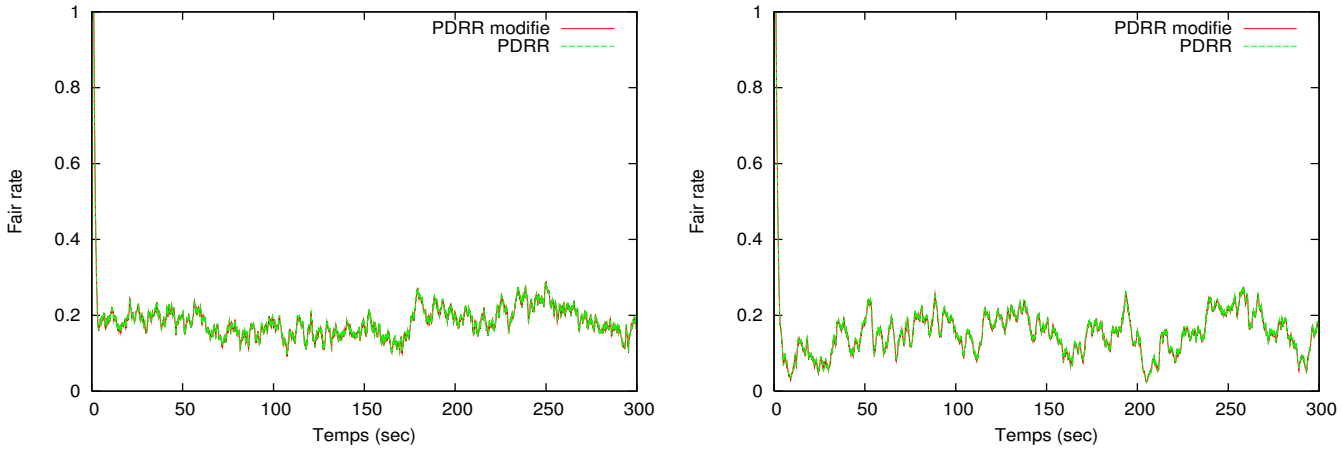


FIG. 6.5 – Evolution du FR avec PDRR et PDRR modifié, traces ADSL (droite) et *Ab-I* (gauche)

La version modifiée de PFQ permet d'avoir une réduction de la taille de l'AFL tout aussi importante que celle obtenue avec PDRR, comme illustré par la figure 6.7. Les autres résultats étant également à peu près similaires à ceux de PDRR, nous omettrons de les présenter. On en conclut que la structure *NewFlows* est également efficace avec l'algorithme PFQ

Ainsi, en dimensionnant la taille de la liste des flots actifs pour abriter 100 flots au maximum, sans avoir à tenir compte des caractéristiques du trafic, une faible probabilité de saturation est obtenue (10^{-3} avec la trace *Ab-I*).

Des résultats pratiquement identiques à ceux de *Ab-I*, qu'on ne présentera pas par souci de concision, sont obtenus en simulant la trace *Ab-III*.

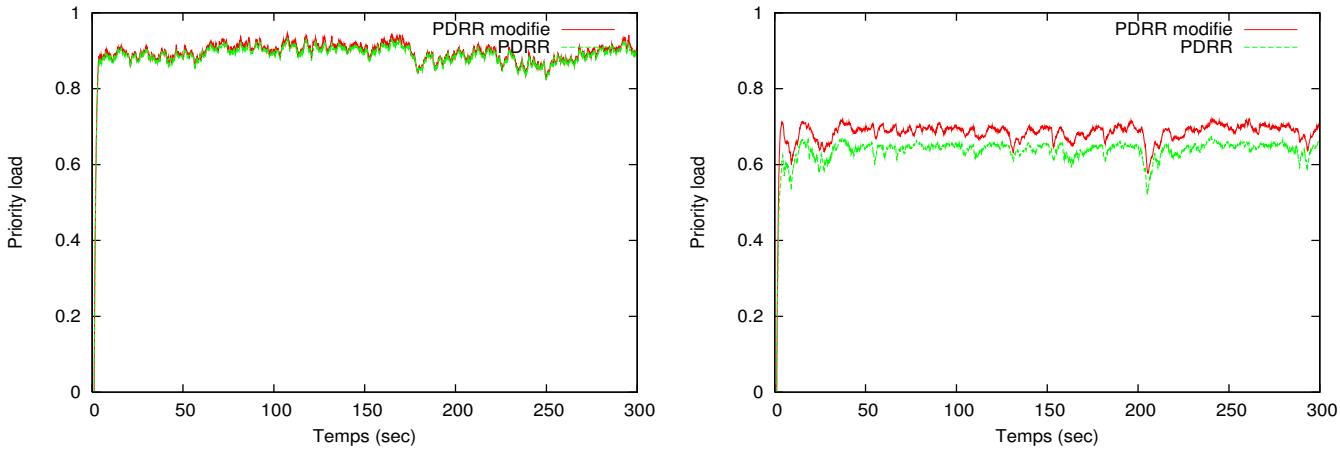


FIG. 6.6 – Evolution du PL avec PDRR et PDRR modifié, traces ADSL (gauche) et *Ab-I* (droite)

6.2 Réduction du coût de gestion des flots en cours

Comme mentionné au cours du paragraphe 2.1.2 du chapitre 2, l'identification des nouveaux flots se fait grâce à la liste des flots en cours PFL. En effet, l'identification basée sur les paquets SYN et SYNACK comme proposée dans [75, 83] se limite aux flots TCP.
Nous avons également vu que le nombre de flots en cours sur des liens à grande capacité est extrêmement élevé (cf. section 5.3.2). Il est donc primordial de choisir une structure adéquate et efficace pour implémenter la liste des flots en cours.

La gestion de la PFL se fait en *soft state*, i.e. les nouveaux flots sont insérés suite à l'arrivée de leur premier paquet, rafrachis à chaque arrivée d'un de leurs paquets et ils sont supprimés s'ils n'envoient aucun paquet pendant une durée égale au *time out*. Les informations minimales alors associées à un flot sont son identifiant et l'instant de passage de son dernier paquet.

Le champ d'utilisation de la PFL ne se réduit pas uniquement aux besoins du contrôle d'admission implicite appliqué au sein d'une architecture FAN. Elle peut servir pour la détection d'attaques ou du routage par politiques [108] auxquels cas, d'autres informations doivent être maintenues outre les informations minimales.
La principale difficulté que pose la gestion de cette structure est l'opération de purge des flots expirés, i.e. leur détection et suppression. En effet, une approche naïve consisterait à parcourir systématiquement toutes les entrées de la liste à la recherche de flots expirés, ce qui devrait être réalisé par un processus de type *garbage collector* en tâche de fond. Une solution plus efficace consiste à faire un balayage sélectif, mais nécessite de classifier les flots en fonction de leur date d'expiration prévue. Ainsi, seuls les flots susceptibles d'avoir expirés sont visités à un instant donné.

Dans le souci d'être complet, nous décrivons dans la prochaine section deux méthodes de gestion des flots en cours, proposées par d'autres membres de l'équipe,

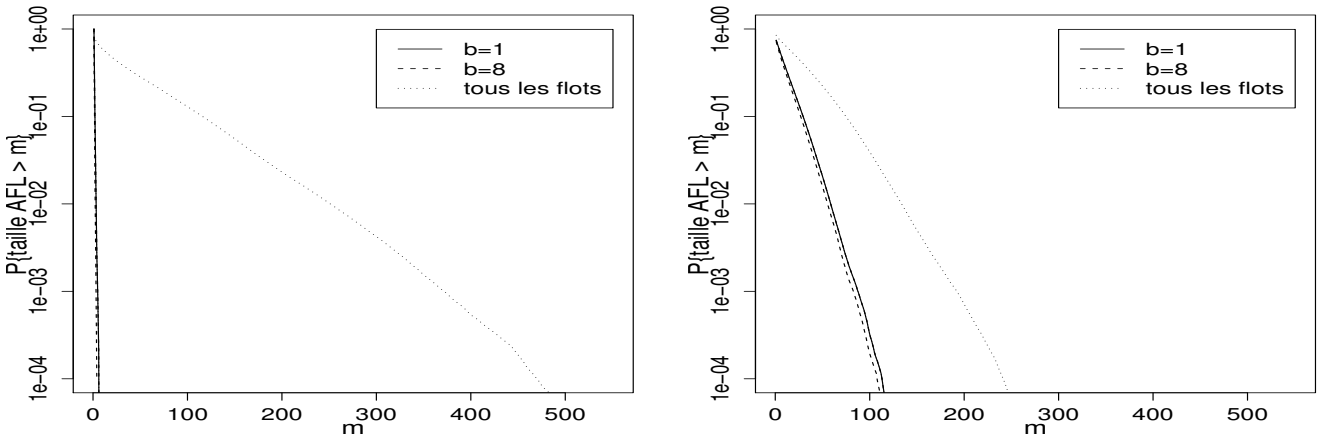


FIG. 6.7 – Distribution complémentaire de la taille de l'AFL, traces ADSL (gauche) et *Ab-I* (droite), PFQ modifié

qui simplifient considérablement l'opération de purge. Ces solutions reposent sur deux structures différentes : *bitmap* et tableau.

Dans le dernier paragraphe de ce chapitre, nous présentons une technique évoquée au cours du chapitre 2 qui permet de réduire le nombre de flots rajoutés à la liste des flots en cours.

6.2.1 PFL à adressage direct

Dans l'implémentation existante, la recherche d'une entrée particulière dans une structure table de flots mixte (tableau plus liste chaînée) repose sur une combinaison d'adressage direct et d'opération de recherche. Une fonction de hachage sur une partie de l'identifiant du flot permet de localiser un espace mémoire, une recherche ciblée (du flot en question) est ensuite effectuée parmi les flots présents dans cette portion (par exemple, une recherche séquentielle dans le cas d'une liste chaînée) [108].

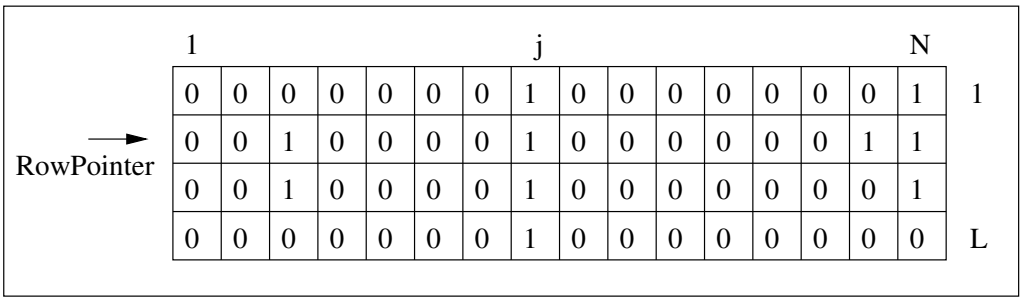
Les nouvelles méthodes proposées adoptent un modèle d'adressage purement direct. La fonction de hachage appliquée sur l'identifiant du flot détermine le registre qui lui est associé. Le registre en question permet de savoir si le flot est en cours ou pas.
Deux réalisations possibles sont décrites, l'une est basée sur un *bitmap* et l'autre sur un tableau. Elles permettent de réduire le coût de l'opération de purge, celle-ci s'effectuant de manière globale plutôt que flot par flot.

Bitmap

Dans ce cas, le statut du flot est codé sur un seul bit. La valeur 1 indique que le flot est en cours, la valeur 0 indique qu'il a expiré. Afin de tenir compte de la notion

temporelle, une matrice à deux dimensions est utilisée. Le nombre de colonnes N est égal aux différentes valeurs possibles retournées par la fonction de hachage. Le nombre de lignes L est un paramètre du système à déterminer. Le bitmap représenté dans la figure 6.8 est mis à jour de la façon suivante :

1. A chaque arrivée d'un nouveau paquet, tous les bits de la colonne j, correspondant à l'adresse du flot retournée par la fonction de hachage, sont mis à 1.
2. Les bits de la ligne i modulo(L) sont mis à 0 aux instants $i\tau$.



	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	1
→	0	0	1	0	0	0	0	1	0	0	0	0	0	0	1	1
	0	0	1	0	0	0	0	1	0	0	0	0	0	0	0	1
	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	L

Tous les bits de la colonne j sont mis à 1 indiquant l'arrivée d'un paquet dont le flot est associé à j depuis la dernière remise à 0 d'une ligne. La variable RowPointer indique la prochaine ligne à réinitialiser.

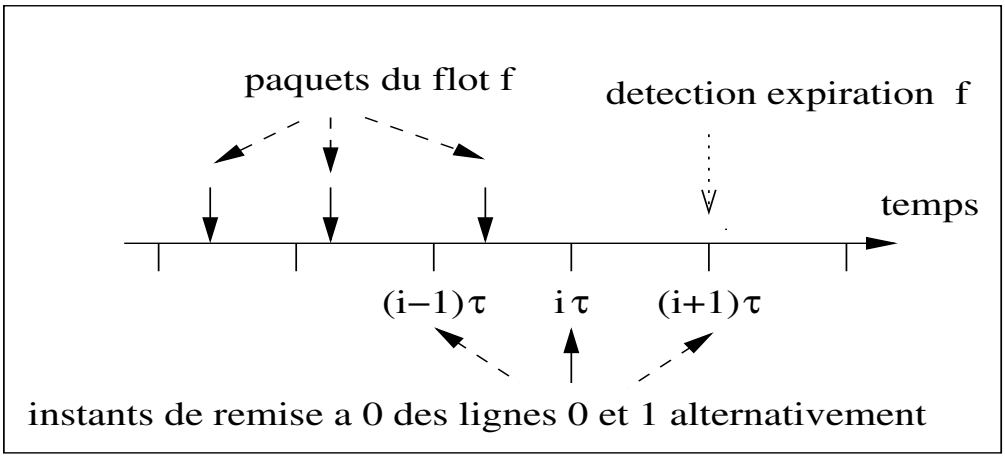
FIG. 6.8 – Représentation d'un *bitmap*

Si tous les bits de la colonne j sont à 0, le (ou les) flot(s) correspondant(s) est (sont) considéré(s) expiré(s).

Le choix de τ et L détermine la valeur et la précision du *time out* utilisé pour la détection de l'expiration des flots. Un flot expire T secondes après l'arrivée de son dernier paquet tel que :
$(L-1)\tau < T \leq L\tau$. On peut alors choisir $(L-1)\tau = time\ out$.
L'imprécision due à la granularité de L induit le maintien des flots avec une durée moyenne $\tau/2$ supplémentaire par rapport à celle qui est strictement requise comme indiqué dans la figure 6.9.

Les erreurs de type faux positif se produisent lorsqu'un nouveau flot est accepté en situation de congestion parce que son identifiant coïncide avec celui d'un autre flot en cours. En supposant que la fonction de hachage est uniforme, le taux de faux positifs est donné par le rapport entre la moyenne du nombre de flots en cours et N. Il peut donc être contrôlé par le choix de N.

En réalité, un taux de faux positifs aussi élevé que 10% peut être toléré car le contrôle d'admission basé sur les mesures se distingue par des facultés d'autocorrection qui permettent de remédier à des admissions erronées. Ce qui compte c'est qu'il n'y ait pas d'erreurs de type faux négatif, où un flot déjà en cours est bloqué. Comme on a pu le voir, par construction, le *bitmap* est en mesure de reconnaître



Dans cet exemple, $L = 2$ et donc τ = time out. Le dernier paquet du flot f arrive durant l'intervalle $(i-1)\tau$, l'expiration du flot n'est détectée qu'à l'instant de remise à 0 $(i+1)\tau$. L'imprécision dans la détection de l'expiration du flot dépend de l'instant d'arrivée du dernier paquet pendant l'intervalle de remise à 0.

FIG. 6.9 – Détection tardive de l'expiration d'un flot

tous les flots en cours puisque tout flot associé à une colonne dont toutes les valeurs sont nulles est forcément nouveau.

Le choix de $L = 2$ permet d'optimiser la capacité mémoire requise pour le *bitmap*. En effet, en appliquant la loi de Little et pour un taux de faux positifs x donné, N doit être proportionnel à la durée de séjour moyenne d'un flot :

$$Nx = \lambda(d + TO + \tau/2)$$

où λ est le taux d'arrivée des flots, d est la durée moyenne des flots et TO le *time out*. En remplaçant τ par $TO/(L-1)$ et sachant que $L \geq 2$, on déduit, en dérivant l'expression obtenue de NL, que la taille totale du *bitmap* est minimisée lorsque $L = 2$, car la fonction associée est croissante sur l'intervalle $[2, +\infty[$.

Une implémentation *hardware* du bitmap utilisant des basculs bi-stables est présentées dans [67].

Tableau

La seconde réalisation possible (adaptée à une implémentation *software*) consiste à utiliser un tableau de N éléments de b bits. L'élément j contient l'instant d'observation du dernier paquet du flot qui lui est associé par le biais d'une fonction de hachage. Un flot n'est plus considéré en cours si la différence entre le temps d'horloge courant et la valeur stockée dans j est supérieure à M. L'horloge est un compteur codé en b bits qui est incrémenté chaque σ secondes.

Le choix de M et σ détermine la valeur et la précision du *time out*. Un flot expire après un temps T tel que $(M-1)\sigma < T \leq M\sigma$. Par conséquent, on peut choisir

$(M-1)\sigma = time\ out$.

Outre les erreurs de type faux positif suite aux collisions possibles entre les flots, un flot pourrait être considéré en cours à tort. Ceci se produit lorsque le temps écoulé depuis l'émission de son dernier paquet est compris entre 2^bK et $2^bK + M$ pour un entier K quelconque (i.e. quand le compteur reboucle) [2]. En supposant que les valeurs de l'horloge et du tableau sont indépendantes, la probabilité de faux positifs est $M/2^b$.

Dimensionnement de la PFL

Il est nécessaire de dimensionner la liste des flots en cours pour limiter la probabilité de saturation. Une structure de la PFL à base d'une table (ou liste) est saturée lorsqu'il n'y a plus d'espace mémoire permettant de rajouter un nouveau flot.
La saturation d'une structure à adressage direct est moins bien définie. Lorsque le taux de faux positifs dépasse une valeur maximum jugée tolérable, la structure peut être considérée saturée. Néanmoins, par rapport à une table, une structure à adressage direct présente l'avantage d'avoir des entrées d'une taille beaucoup plus faible (2 bits seulement dans la réalisation proposée dans la section 6.2.1).

Le nombre moyen de flots en cours est égal au produit du taux d'arrivée des flots fois leur temps de séjour moyen. Ce dernier est représenté par la somme de la durée moyenne d'un flot et du *time out*. Une valeur du *time out* de 2 secondes peut être utilisée pour les besoins du contrôle d'admission. Ainsi, le nombre de flots à maintenir dans la PFL dépend du type de trafic et des caractéristiques des flots en terme de durée. A titre indicatif, la durée moyenne des flots de la trace *Ab-I* est 856ms et leur taille moyenne vaut 5205 octets. En extrapolant ces chiffres pour un lien OC192 (10Gb/s) chargé à 90%, on aurait un taux d'arrivée de (9G/41640). En considérant une durée moyenne de 2.856 secondes, on obtiendrait alors 62000 flots, environ. La variance du nombre de flots devrait également être prise en compte. La structure à adressage direct devrait avoir une taille dix fois plus grande que le nombre de flots en cours pour assurer un taux de faux positif de 10%.

6.2.2 Insertion probabiliste dans la PFL

Afin de diminuer le nombre d'insertions au sein de la PFL, les flots peuvent être rajoutés avec une probabilité p et non pas systématiquement à la suite de la détection de leur premier paquet. Ainsi, une fraction de (1 - p) des flots à un seul paquet ne sont pas rajoutés à la liste. Les longs flots sont, quant à eux, rajoutés après l'émission de $p/(1-p)$ paquets en moyenne. En effet, on a :
Pr[flot inséré après émission de k paquets] $= p(1-p)^{k-1}$.

[2] Il existe des parades simples pour éviter ce genre d'erreurs, par exemple en rajoutant un bit qui indiquerait que le temps stocké n'est plus valable dès qu'un flot correspondant à l'entrée en question est rejeté

Etant donnée la distribution à queue lente de la taille des flots [110], un choix de $p = 0.1$, par exemple, permettrait de réduire considérablement le nombre d'insertions effectuées dans la PFL. En utilisant les données des traces *ADSL* et *Ab-I*, présentées dans la section 5.3, on aboutit à 15% des flots seulement qui sont rajoutés à la liste des flots protégés lorsque $p = 0.1$.
D'un autre côté, comme seuls les flots longs sont rajoutés à la PFL, la diminution du nombre d'insertions ne se traduit pas par une réduction proportionnelle dans la taille nécessaire pour la PFL, ce qui est quelque peu contre intuitif. Ceci s'explique par la loi de Little puisque la diminution du taux d'arrivée est relativement compensée par l'augmentation du temps de séjour moyen.
Néanmoins, une insertion probabiliste avec $p = 0.1$ et en considérant un *time out* de 2 secondes, induit une réduction de 70% du nombre de flots en cours moyen insérés dans la PFL sur la trace *Ab-I*, et une réduction de 63% avec la trace *ADSL*. Le gain résultant par rapport à la taille de la PFL croît avec la valeur du *time out*.

Le tableau 6.1 indique les tailles moyennes et maximales de la liste des flots protégés en fonction du *time out* (0, 2 et 20 secondes), ainsi que le gain obtenu G lorsqu'une insertion probabiliste est effectuée avec $p = 0.1$ sur les traces en question. Le gain G représente la différence de l'occupation (entre les tailles relatives à l'insertion systématique et celle à l'insertion probabiliste) divisée par la taille obtenue lorsque tous les flots sont insérés. Les résultats indiquent qu'un gain significatif peut être atteint, d'où l'intérêt d'une telle approche.
Cependant, le gain apporté doit être confronté avec la fraction des flots qui sont interromps (bloquées) après l'émission de quelques paquets, parce qu'ils n'acquièrent pas tout de suite le statut de protection que leur confère l'insertion dans la PFL.
La simulation de la trace *ADSL* avec une charge offerte de 1.08 avec contrôle d'admission en appliquant une insertion probabiliste, où $p = 0.1$, en utilisant PDRR a conduit à l'interruption de 1%, seulement, des flots. Le taux de blocage total était 5% en appliquant des seuils d'admission de $0.01C$ pour le FR et 0.9 pour le PL.

TO (sec)	*Ab-I*			*ADSL*		
	moy	max	G	moy	max	G
0	4455	4638	0.37	4244	4434	0.46
2	6764	7072	0.70	5072	5300	0.63
20	27547	28109	0.83	12499	13027	0.81

TAB. 6.1 – Tailles moyenne et maximale de la PFL, insertion probabiliste, $p = 0.1$

Conclusion

On conclut ce mémoire en résumant les principaux résultats obtenus et en soulignant les perspectives de recherche qui en découlent.

Contributions

Cette thèse traite du problème de la qualité de service dans les réseaux IP. Plus précisément, nous avons évalué une nouvelle architecture de QoS orientée flot, qui se veut simple et efficace, appelée *cross-protect*.

Dans un premier temps, nous avons mis l'accent sur l'inadéquation des mécanismes de QoS existants, ce qui a motivé la recherche d'une solution alternative. Une étude de la performance du trafic élastique, comparant le contrôle d'admission sélectif et le modèle de service Diffserv, a indiqué que ce dernier permet de protéger le trafic *premium* en cas de surcharge mais au prix d'un gaspillage de ressources dû à l'interruption de certains flots *best effort* qui souffrent d'une dégradation de débit inacceptable ; causant ainsi une performance non seulement mauvaise mais aussi imprédictible. Le contrôle d'admission sélectif opère une différenciation en terme d'accessibilité au réseau en cas de surcharge (les flots *premium* sont acceptés en priorité par rapport aux flots *best effort*) tout en garantissant, indistinctement, un débit satisfaisant pour tous les flots admis.
Ainsi, cette étude comparative a mis en évidence la nature différente de la protection du trafic apportée par les deux mécanismes, justifiant par la même occasion l'emploi d'un contrôle d'admission implicite au niveau flot.

Par la suite, nous avons présenté l'architecture *cross-protect* qui propose un concept innovant, celui de la différenciation implicite, apportant de la QoS aux flots IP. Cette architecture combine l'utilisation du contrôle d'admission implicite et du *fair queueing* par flot, à mettre en œuvre dans les routeurs IP. Il s'agit d'offrir une QoS sans se baser sur la notion de classes de service. La différenciation se faisant en fonction du débit des flots.

Ce concept a été validé par simulation. Nous avons vérifié que *cross-protect* permet d'effectuer une différenciation de service implicite en faveur des flots dont le débit est inférieur au débit équitable. Il en résulte un débit adéquat pour les flots

élastiques et des délais ainsi que des pertes négligeables pour les paquets des flots *streaming*.

L'ordonnancement des paquets repose sur un algorithme de type *fair queueing*. Deux réalisations possibles ont été proposées et décrites : PFQ et PDRR. Le premier algorithme appelé PFQ est basé sur la notion d'estampille temporelle, et le second en l'occurrence PDRR est de type *round robin*. Ils offrent des performances très proches dans des scénarios de trafic dynamique, où les flots arrivent et quittent le système conformément à ce qui se passe dans la réalité. Ils garantissent un partage équitable de la bande passante pour les flots élastiques, les paquets des flots *streaming* sont, quant à eux, traités en priorité sans nécessité de marquage explicite. L'application du contrôle d'admission grâce aux indicateurs de congestion fournis (*fair rate* et *priority load*) par les algorithmes PFQ ou PDRR permet de préserver une bonne performance, quelle que soit la charge offerte.

Contrairement aux réseaux d'intégration de service classiques, où un marquage explicit est effectué afin de traiter les paquets des flots temps-réel en priorité, dans le cadre de *cross-rotect*, la charge induite par le trafic prioritaire sur certains liens peut être importante. L'application du contrôle d'admission est alors critique. En effet, la qualité de service des flots *streaming* se traduit par de faibles délais par paquet. Pour ce faire, la charge de la partie prioritaire de la file gérée avec PFQ ou PDRR doit être inférieure à un certain seuil dans l'optique de réaliser un multiplexage sans buffer (au niveau fluide). Ce type de multiplexage présente l'avantage d'être indépendant des caractéristiques détaillées du trafic. Un contrôle d'admission basé sur les mesures est utilisé à cet effet.

Un algorithmes MBAC simple, qui suppose que le nombre de rafales suit une loi de Poisson, a été proposé et testé par le biais de simulations *ns* avec une variété de modèles de trafic (synthétique, vidéo et réel). Cet algorithme est adapté au contexte de *cross-protect*, dans la mesure où le seuil d'admission est déterminé à partir de la seule connaissance d'un débit crête maximum pour tous les flots. Ainsi, les nouveaux flots sont bloqués dès que la mesure de la charge dépasse le seuil en question.
Nous avons également effectué des comparaisons avec un algorithme basé sur une approximation Gaussienne du débit agrégé des flots, qui présente l'avantage de mieux s'adapter à un trafic hétérogène. On a pu voir qu'un taux d'utilisation important du lien peut être atteint tout en maintenant de faibles délais, notamment lorsque le rapport entre la capacité du lien et le débit crête maximum considéré est grand.

Le second volet d'évaluation de l'architecture *cross-protect* concernait l'extensibilité des algorithmes de *fair queueing*. En considérant la nature statistique et dynamique du trafic IP, on a pu voir que l'ordonnancement de type *fair queueing* est non seulement extensible mais aussi faisable. Son extensibilité dérive du fait que le nombre de flots actifs (ceux qui doivent être gérés par l'ordonnancement) est indépendant de la capacité du lien. Ce nombre est limité, avec une très grande probabilité, à quelques centaines de flots seulement, ce qui entraîne la faisabilité

du *fair queueing* tant que la charge du lien n'est pas trop proche de 1 (même une utilisation aussi élevée que 95% ne remet pas en cause ces conclusions). Le rôle du contrôle d'admission étant justement de maintenir une charge totale inférieure à 1.

Une évaluation de la distribution de la taille de la liste des flots actifs a été conduite par des simulations *ns* utilisant des traces de trafic réelles relativement récentes ainsi que par un modèle analytique explicatif. Les principaux enseignements que l'on a pu tirer sont que l'extensibilité des algorithmes de *fair queueing* dépend du nombre de flots actifs et non pas du nombre de flots en cours. Le nombre de flots actifs, quant à lui, dépend de la charge totale offerte, de la proportion des flots *bottlenecked* et *non bottlenecked* obtenue en fonction des débits des flots ainsi que de la distribution de la taille des paquets appartenant aux flots *non bottlenecked*. Par conséquent, le débit intrinsèque des flots (défini comme étant le débit que les flots pourraient atteindre en l'absence de congestion sur le lien) constitue un facteur important lors de l'évaluation du nombre de flots actifs.

Nous avons également vu que sur les liens *backbone*, très peu de flots sont *bottlenecked*, car ils sont contraints par leur débit d'accès dans la plupart des cas. Ce résultat peut entraîner des implications importantes dans l'évaluation de certains mécanismes d'AQM et pour le dimensionnement des buffers des routeurs, où généralement un nombre fixe de connexions est considéré.

Cette étude a également été l'occasion de mettre en exergue certaines caractéristiques du trafic IP sur les traces réelles considérées. Les principales tendances des études passées en termes de distribution de la taille des flots, distribution des débits des flots et distribution de la taille des paquets ont été confirmées notamment pour les traces ADSL et *Ab-I*; la trace *Ab-III* exhibant un comportement particulier qui se matérialise par des débits par flots très élevés.

L'architecture *cross-protect* offre plusieurs avantages. Outre le maintien de l'interface réseau/utilisateur actuelle, son introduction pourrait être faite de façon incrémentale (routeur par routeur ou AS par AS) puisqu'elle ne nécessite pas de standards particuliers (mise à part éventuellement pour l'identification des flots).
De plus, à partir du moment où tous les flots admis ont une garantie de QoS, une tarification basée sur le volume est envisageable dans la mesure où chaque octet envoyé est redevable.

Nous espérons que ces résultats inciteront les constructeurs de routeurs à mettre en œuvre le mécanisme *cross-protect* avec des ordonnancements de type *fair queueing* afin de bénéficier des avantages qu'ils offrent. En particulier, ceci permettrait d'utiliser d'autres protocoles de transport plus efficaces que TCP pour effectuer des transferts de données sur des liens à grande capacité, puisque l'équité du partage serait assurée par le scheduler.
Dans cette optique d'industrialisation, un perfectionnement de l'architecture FAN a été proposé dans le but de réduire davantage la taille de la liste des flots actifs. Ceci est possible en identifiant les flots *non bottlencked* grâce à l'emploi d'une structure

de donnée supplémentaire, qui nécessite une faible capacité mémoire. Ainsi, seuls les flots *bottlenecked* sont insérés dans l'AFL. L'efficacité de cette approche a été validée par simulation.

Perspectives

Le premier axe de recherches futures concerne la robustesse des algorithmes MBAC face à des situations de surcharge subite. Ainsi, les algorithmes présentés dans le chapitre 3 peuvent être améliorés notamment pour éviter d'accepter trop de flots suite à une panne, où le trafic est redirigé vers un autre lien. En effet, l'admission de nouveaux flots ne se répercutent pas immédiatement dans la mesure de la charge. Les pistes de recherche envisageables concernent le rajout d'un débit représentatif à la mesure de la charge à chaque nouveau flot accepté, comme spécifié dans l'algorithme de Grossglauser et Tse. L'addition systématique du débit crête peut conduire à une sous-utilisation du lien, notamment lorsque la majorité des flots sont courts, ils n'atteignent donc pas le débit crête considéré. D'autres solutions sont donc à explorer, où il serait peut être préférable de tenir compte de la quantité de la bande passante disponible pour déterminer le débit représentatif à considérer.

Un autre aspect qui mérite sûrement d'être creusé est l'adaptation de *cross-protect* en vue d'une éventuelle mise en place au sein des réseaux d'accès. Cette extension présente quelques difficultés à surmonter. En effet, l'étude effectuée dans cette thèse considère un réseau cœur où le partage de la bande passante est parfaitement équitable. Sur les réseaux d'accès, où les liens sont de capacités plus faibles, un partage pondéré pourrait être envisagé, notamment pour des raisons de tarification. Le choix du scheduler approprié reste à déterminer.

La présente étude s'applique à un réseau purement IP. La co-existence de *cross-protect* avec le mécanisme MPLS (*Multi Protocol Label Switching*) est cruciale pour à la mise en œuvre d'une architecture FAN à grande échelle, car de nombreux opérateurs envisagent d'utiliser MPLS dans leur réseau *backbone*, notamment dans le cadre de service VPN, entre autres.

L'application d'un routage adaptatif en cas de blocage de flots sur une route donnée pourrait également faire l'objet d'une étude approfondie. Ceci permettrait d'effectuer un partage de charge (*load balancing*) dynamique en fonction de l'état de congestion des différents liens, en exploitant les capacités d'équilibrage de charge dans les routeurs et en tenant compte éventuellement du champ *flow label* de IPv6. Ainsi, le blocage de flots ne constituerait plus une source de frustration pour les utilisateurs, mais plutôt une opportunité supplémentaire d'accéder au réseau dans de bonnes conditions.

Enfin, l'industrialisation de *cross-protect* sur des routeurs commerciaux nécessiterait la prise en compte de certains aspects, éventuellement de type *hardware*, qui dépassent le cadre de nos compétences.

Annexe 1
Expérimentations

Dans cet annexe nous rapportons des résultats d'expérimentations, conduites par d'autres membres de l'équipe, qui confirment les observations dérivées des simulations présentées dans les chapitres 1 et 2.

1 Expérimentation de Diffserv

Afin de compléter et valider les résultats de simulations présentés dans le chapitre 1, paragraphe 1.2.1, des tests sur une plate-forme représentée dans la figure 1, constituant une réplique à échelle réduite d'un réseau dorsal IP, ont été conduits. La plate-forme est composée de routeurs, de *switchs*, de sources de trafic et d'une machine puits. Les liens connectant les routeurs ont une capacité de 155 Mb/s. Les deux PCs Linux servant de source de trafic utilisent un logiciel de génération de flots (*Genflot*) adapté pour des scénarios de trafic dynamiques [8]. Ces expérimentations ont été effectuées par N. Benameur et S. Oueslati.
Les flots TCP arrivent selon un processus de Poisson, leur taille suit une loi exponentielle de moyenne 10 Mo. Cette valeur est élevée en raison de contraintes techniques concernant le logiciel de génération de flots à l'époque où ces tests ont été réalisés. D'ailleurs, un deuxième générateur de trafic commercial (*smartbit*) est utilisé pour porter le lien à saturation (génération d'un flot UDP permanent à 75 Mb/s). Au cours de ces expérimentations, les proportions des flots PR (*premium*) et BE (*best effort*) étaient de 45% et 55% respectivement ; ces deux classes étaient gérées avec le mécanisme WFQ. Le scénario de trafic n'est donc pas tout à fait conforme à celui simulé. Les poids associés à WFQ étaient respectivement de 70% et 30% pour les classes PR et BE. La proportion offerte du trafic BE est supérieure au poids WFQ associé.

La figure 2 à gauche confirme que le nombre de flots BE croît constamment comme observé avec les simulations. La figure à droite trace le débit réalisé par les flots. Comme prévu, les flots PR obtiennent des débits élevés (au tour de 10 Mb/s) alors que les flots BE réalisent des débits très faibles qui tendent vers 0.

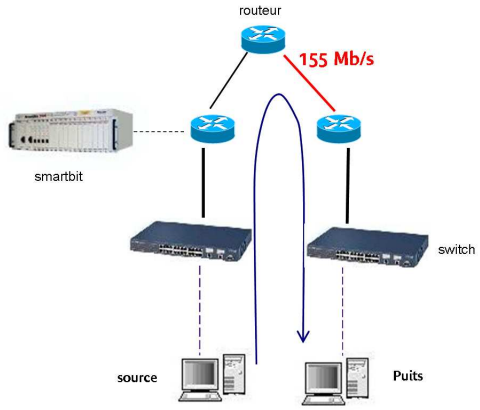


FIG. 1 – Plate-forme d'expérimentation de Diffserv

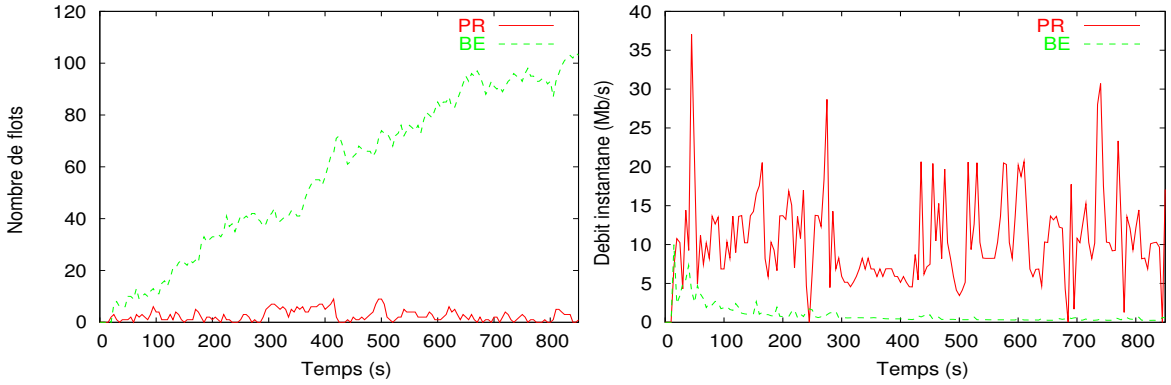


FIG. 2 – Evolution du nombre (gauche) et des débits (droite) des flots PR et BE avec le scénario WFQ de Diffserv, expérimentation, $\rho = 1.2$

2 Expérimentation du contrôle d'admission sélectif

Afin d'évaluer la faisabilité du contrôle d'admission, un prototype de tests a été développé. Il s'agit d'un programme qui contrôle une carte matérielle mettant en œuvre une mémoire Trie à grande vitesse [1] qui réalise la liste des flots en cours. Bien que pouvant fonctionner sur un lien Gigabit Ethernet, les expérimentations ont été conduites sur un lien à 100 Mb/s à cause essentiellement des limitations du générateur de trafic.

La charge offerte est $\rho = 1.2$ dont 30% provient des flots PR. Le trafic est généré grâce à l'outil *Genflot* en utilisant deux machines puits, une pour chaque classe. Le prototype reconnaît les flots *premium* en fonction de leur adresse destination.
Un contrôle d'admission sélectif est employé avec des seuils de 1% et 4% de la capacité du lien pour les flots PR et BE, respectivement. La taille des flots suit une loi Pareto de moyenne 7.5 Mo et un facteur de forme de 1.5.

Encore une fois, le blocage de 31% des flots ne concerne que la classe BE ; ce qui

permet d'assurer une bonne performance aux deux classes tout en privilégiant les flots PR en terme d'accessibilité.
La figure 3 de gauche trace l'évolution du nombre de flots PR et BE en fonction du temps écoulé. Le nombre moyen est proportionnel à la charge induite par chaque classe en tenant compte du taux de blocage. Comme prévu, ce nombre reste borné, il est limité à 50 dans ces expérimentations.

La figure 3 de droite représente le débit instantané moyenné sur des périodes de durée 5 secondes pour les deux classes de trafic. Les valeurs des débits varient entre 2 et 5 Mb/s. Le débit moyen réalisé par les flots PR est 3.6 Mb/s (proche des 4%), il est de 2.7 Mb/s pour les flots BE. Cette légère différence semble être due (en partie) au fait que la machine puits dédiée au trafic BE gère plus de flots, ce qui augmente ses temps de traitement.

Qualitativement, ces résultats sont conformes à ceux obtenus avec les simulations *ns*, présentés dans la section 1.2.2.

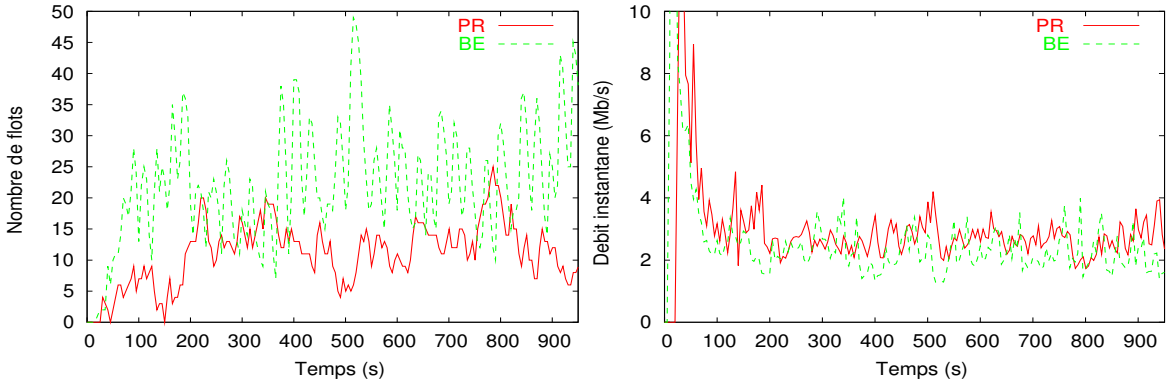


FIG. 3 – Evolution du nombre (gauche) et des débits instantanés (droite) des flots PR et BE avec contrôl d'admission, expérimentation, $\rho = 1.2$

3 Expérimentation de *cross-protect*

Afin de valider la faisabilité de *cross-protect* en pratique, une implémentation logicielle a été développée et intégrée au noyau Linux par la suite, par un autre membre de l'équipe. Cette plate-forme a également pour but d'identifier certains problèmes qui pourraient se présenter en cas de mise en œuvre sur des routeurs commerciaux. Elle permet, en outre, d'effectuer une comparaison avec les résultats obtenus par simulation présentés dans la section 2.4.1.

La figure 4 illustre la plate-forme développée. L'expérimentation utilise trois PCs répartis sur deux VLANs. L'un des PCs fait office de générateur de trafic sur le premier VLAN, un autre opére comme puits de trafic sur le second sous-réseau et le

dernier, en coupure entre les deux VLANs, met en œuvre *cross-protect*. Le *smartbit* est un générateur de trafic qui peut être utilisé conjointement avec les PCs source.

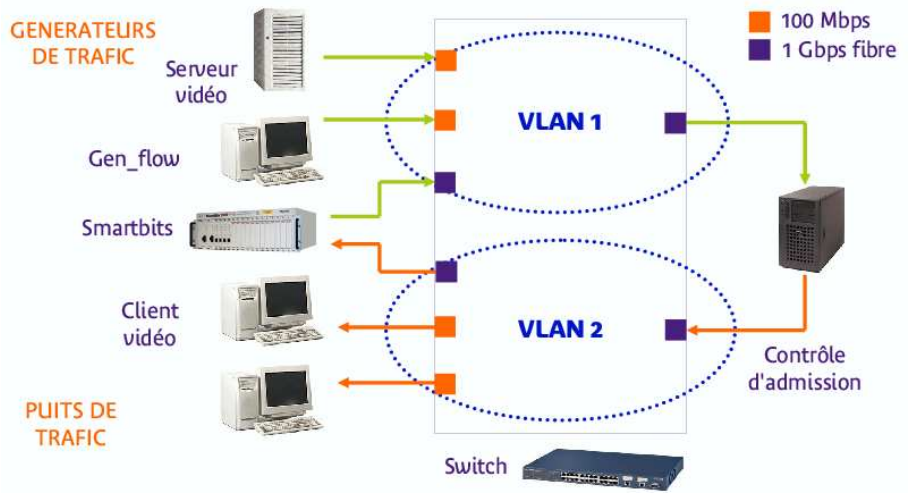


FIG. 4 – Plate-forme de tests

3.1 Génération de trafic

L'application *Genflot* est utilisée pour générer du trafic élastique artificiel selon un scénario dynamique. A l'instar des simulations de la section 2.4.1, le trafic offert se compose de 80% de flots TCP et 20% de flots UDP. La durée de chaque expérience est de 20 minutes.
L'arrivée des flots suit un processus de Poisson dont l'intensité permet d'obtenir des charges de 0.9 ou 1.1. La taille des paquets de tous les flots étant de MTU octets.
La distribution de la taille des flots TCP est exponentielle avec une moyenne de 100 Ko.
Les flots UDP sont de type *on/off* et leur durée suit une loi exponentielle de moyenne 1 minute. La durée des périodes *on* et *off* est de 500ms chacune. Quant au débit d'émission dans l'état *on*, il est de 64Kb/s. La génération de ces flots se fait grâce à l'utilitaire *mgen* [3].

On constate qu'une des différences par rapport au profil de trafic simulé avec *ns* est liée, pour des raisons pratiques, à la distribution de la taille des flots TCP ainsi que sa moyenne. Notons également que les durées des périodes *on* et *off* pour les flots UDP sont déterministes et non pas exponentielles.

Les paquets générés arrivent sur l'interface réseau de la machine réalisant *cross-protect*, ils sont recopiés vers la carte de sortie et transmis suivant l'ordre imposé par PFQ. On ne spécifie pas les différentes étapes intermédiaires ainsi que les problèmes techniques rencontrés, notamment ceux liés à la gestion des buffers du noyau et

[3] http ://mgen.pf.itd.nrl.navy.mil/

l'émulation du temps de transmission. Pour de plus amples détails, le lecteur pourra se référer à [3]. La capacité du lien contrôlé est 100 Mb/s, qui peut être bridée à 10 Mb/s.

3.2 Résultats

Les résultats présentés concernent 4 scénarios d'expérimentations effectués avec la version qui n'était pas encore integrée au noyau. Le premier est relatif à une charge normale, le second à une surcharge avec contrôle d'admission, le troisième se déroule sur un lien à 100 Mb/s au lieu de 10 Mb/s et enfin le dernier consiste à utiliser du trafic vidéo en plus du trafic artificiel.

Analyse en régime de charge normale

Dans ce scénario, la demande de trafic est 9 Mb/s ($\rho = 0.9$). La figure 5 trace l'évolution de divers paramètres liés au fonctionnement de PFQ.
Le *fair rate* varie entre 10% et 60% de la capacité du lien indiquant un bon débit pour les flots TCP.
La valeur du *priority load* est légèrement supérieure à 20%, ce qui se matérialise par de faibles délais pour les paquets des flots UDP, qui sont tous insérés en priorité. Le délai moyen obtenu est plus grand que celui relatif aux simulations, car la taille des paquets UDP est plus importante dans les expérimentations. La valeur du PL est plus petite que celle observée en simulation, ceci est dû au fait qu'une plus faible proportion de paquets TCP est insérée en priorité. En effet, le nombre de paquets TCP insérés en priorité à un instant donné ne dépasse quasiment jamais 3. Cette différence provient de la taille moyenne des flots TCP qui est de 100 Ko au lieu de 25 Ko en simulation, du coup les flots parviennent à sortir plus souvent de la phase de *slow start*. De plus, avec la distribution Pareto tronquée (utilisée en simulation), la majorité des flots sont de faible taille, alors que dans les expérimentations, la distribution exponentielle implique une répartition plus équilibrée.
Conformément à nos attentes, les flots actifs constituent une sous population des flots en cours dont le nombre reste inférieur à 100 flots. La taille de l'AFL est encore plus faible, elle est limitée à 30 flots. Ces valeurs sont légèrement inférieures à celles des simulations parce que les flots TCP quittent le système plus rapidement en expérimentation, vu que la proportion des éléphants y est plus importante et que le FR est plus élevé.

Analyse en régime de surcharge avec contrôle d'admission

Dans cette configuration, la demande de trafic est de 11 Mb/s ($\rho = 1.1$). Le contrôle d'admission est activé dès que la valeur mesurée du *fair rate* descend en dessous de 100 Kb/s et les nouvelles connexions sont alors refusées.
La figure 6 illustre l'évolution du FR, qui oscille au tour du seuil d'admission, et celle du PL qui varie entre 20% et 30%. L'utilisation du lien est pratiquement à

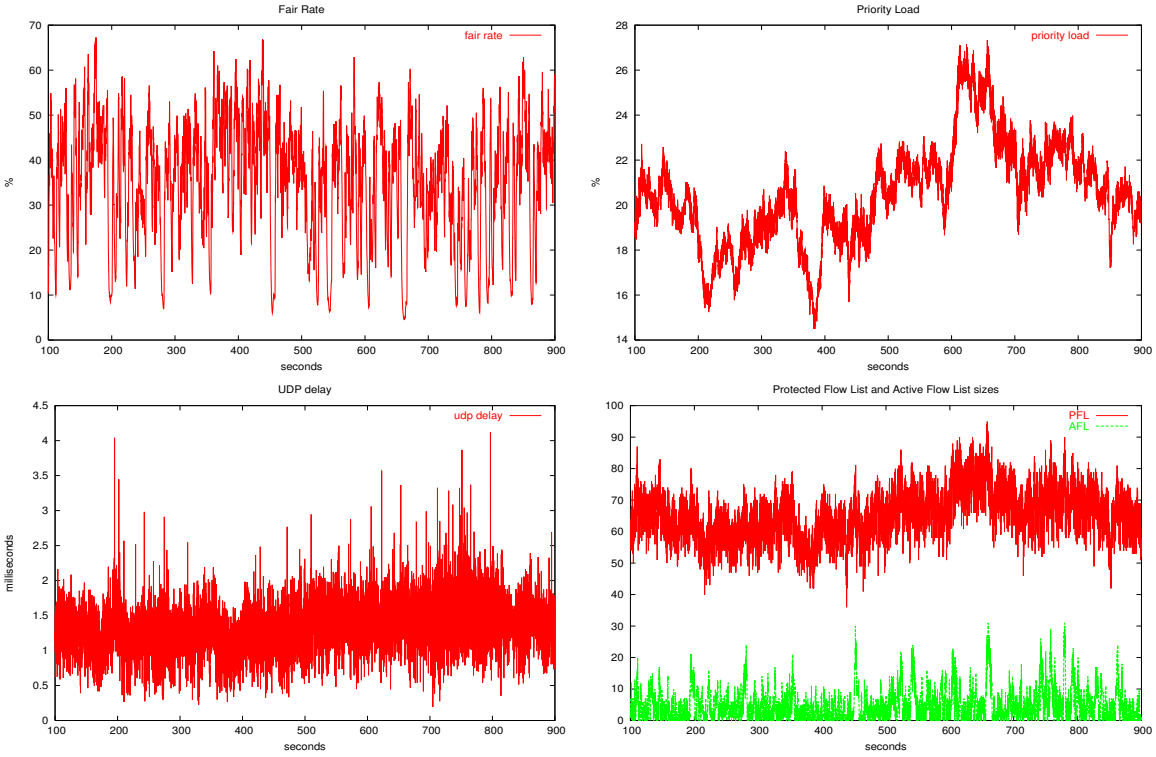


FIG. 5 – Variations du FR (en haut à gauche), PL (en haut à droite), délais des paquets UDP (en bas à gauche) et tailles PFL et AFL (en bas à droite) en focntion du temps, $\rho = 0.9$

100%. Le délai moyen des paquets UDP est de l'ordre de 1.5ms. Le nombre de flots en cours et celui des flots actifs restent bornés.

Ces résultats sont en ligne avec ceux issus des simulations en surcharge. La principale information étant que *cross-protect* est en mesure de fournir une bonne performance en situation de surcharge grâce au contrôle d'admission.

Analyse du cas d'un lien à 100 Mb/s

Dans cette partie, nous nous intéressons au fonctionnement de *cross-protect* sur un lien à 100 Mb/s en surcharge ($\rho = 1.1$) avec l'application du contrôle d'admission. La taille moyenne des flots est 1 Mo, essentiellement à cause de limitations liées à la génération du trafic.
La figure 7 trace le nombre de flots actifs et le nombre de flots en cours. Encore une fois, les résultats de simulation sont confirmés. On observe une nette augmentation du nombre de flots en cours, qui dépasse 700 flots, alors que le nombre de flots actifs reste inférieur à 200 flots. L'augmentation du nombre de flots actifs (par rapport au cas où $C = 10$ Mb/s) s'explique par le changement de la proportion des flots *non bottlenecked*.

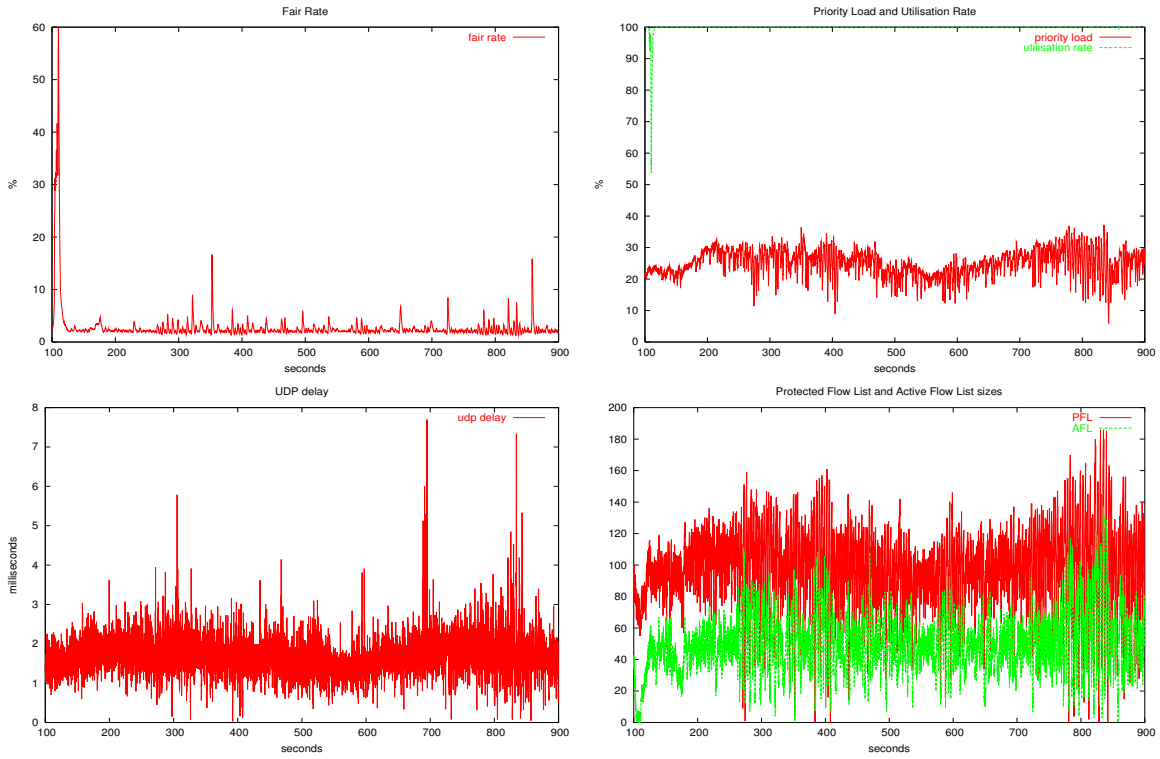


FIG. 6 – Variations du *FR* (en haut à gauche), *PL* (en haut à droite), délais des paquets UDP (en bas à gauche) et tailles PFL et AFL (en bas à droite) vs. temps, $\rho = 1.1$

Analyse du scénario avec trafic vidéo

Afin de mettre en évidence l'apport de *cross-protect* de façon plus visuelle et qualitative, une analyse de la performance d'une application de vidéo *streaming* a été conduite.
Outre le trafic artificiel décrit dans la section 3.1 de cet annexe, une vidéo codée en MPEG avec *vlc*[4] a été rajoutée, ses variations de débit sont illustrées par la figure 8.
Sous une charge de 0.9 (sur un lien à 10 Mb/s), avec une simple file FIFO (dont le buffer est limité à 300 paquets), on observe une nette dégradation de la qualité de la vidéo à cause des pertes de paquets qu'elle subit.
En appliquant *cross-protect*, la qualité de l'image de la vidéo est bien meilleure grâce à l'algorithme PFQ qui rejette des paquets appartenant aux flots dont le *backlog* est le plus important. C'est donc les connexions TCP qui sont visées par ces pertes.
Ainsi, même en charge normale, la différenciation implicite effectuée par *cross-protect* peut être bénéfique à des applications temps-réel.

[4]http ://www.videolan.org/vlc/

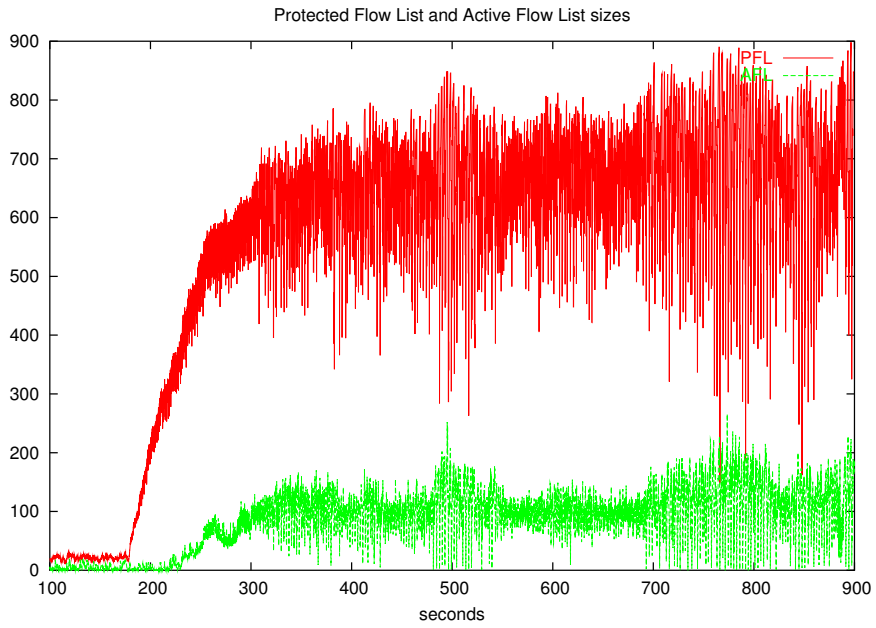


FIG. 7 – Variations de la taille de la PFL et AFL vs. temps, $C = 100$ Mb/s, $\rho = 1.1$

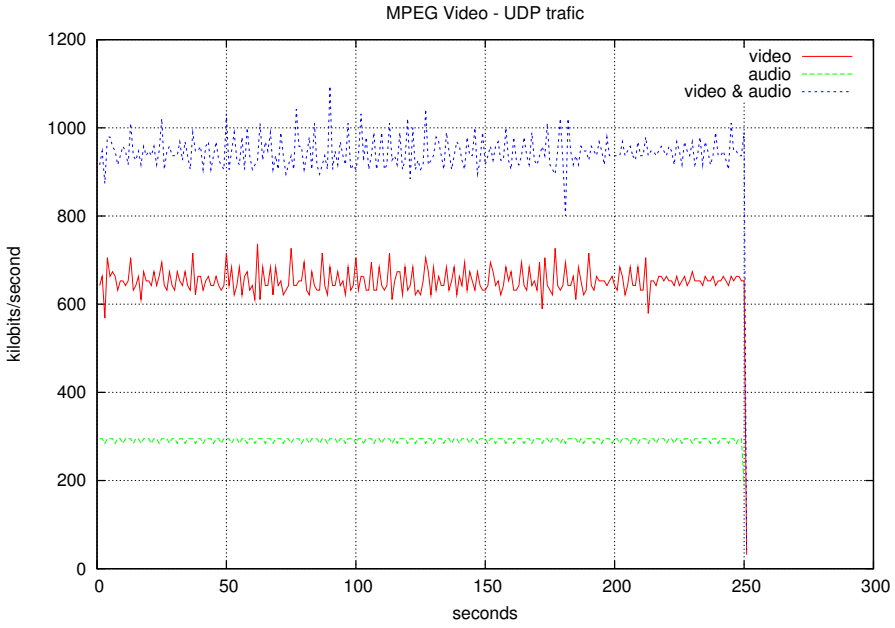


FIG. 8 – Variations du débit de la vidéo utilisée vs. temps

Annexe 2
Exemples de fonctionnement de PFQ et PDRR

Cet annexe met l'accent sur le fonctionnement des algorithmes PFQ et PDRR proposés, respectivement, dans les paragraphes 2.2 et 2.5. La figure 1 illustre un exemple de fonctionnement de PFQ étalé sur 8 étapes :

- Initialement, on suppose que la liste des flots actifs (AFL) contient deux flots, notés f1 et f2, dont les valeurs de *finishTag* sont VT+40 et VT+1500 respectivement, où VT représente le temps virtuel. La file PIFO contient un seul paquet appartenant au flot f2 et dont l'estampille temporelle vaut VT. Le pointeur P indique la partie prioritaire de la file.
- A l'arrivée d'un nouveau paquet appartenant au flot f2, de taille 1000 octets, il est inséré dans la PIFO avec une estampille égale au *finishTag*, lequel est mis à jour, puisque le flot f2 est déjà actif.
- Lorsque le premier paquet de f2 termine son service, le temps virtuel évolue. Par conséquent, le flot f1 est supprimé de l'AFL, car son *finishTag* (VT+40) devient inférieur au nouveau temps virtuel (VT+1500). Le second paquet de f2 commence son service.
- Un troisième paquet du flot f2 arrive, de taille 1500, il est rajouté à la PIFO et le *finishTag* de f2 est incrémenté par la taille du paquet en question.
- Un paquet provenant d'un flot, noté f3, arrive. Comme il s'agit d'un nouveau flot actif, son paquet est inséré dans la partie prioritaire de la file et le pointeur P est mis à jour. Le flot f3 est rajouté à l'AFL avec un *finsihTag* égal à la somme du temps virtuel courant (VT+1500) et la taille du paquet arrivant (500).
- Le paquet d'estampille VT+1500, du flot f2, achève son service. Le nouveau paquet à traiter est celui appartenant à f3. Le temps virtuel reste inchangé.
- A l'issue du service du paquet du flot f3, ce dernier est supprimé de l'AFL, puisque le nouveau temps virtuel est supérieur à son *finishTag*. Le paquet d'estampille VT+2500 de f2 entame son service.
- Lors de cette dernière étape, la file de paquet se vide. Ainsi, la liste AFL est purgée en effaçant le seul flot qui était actif, en l'occurrence f2.

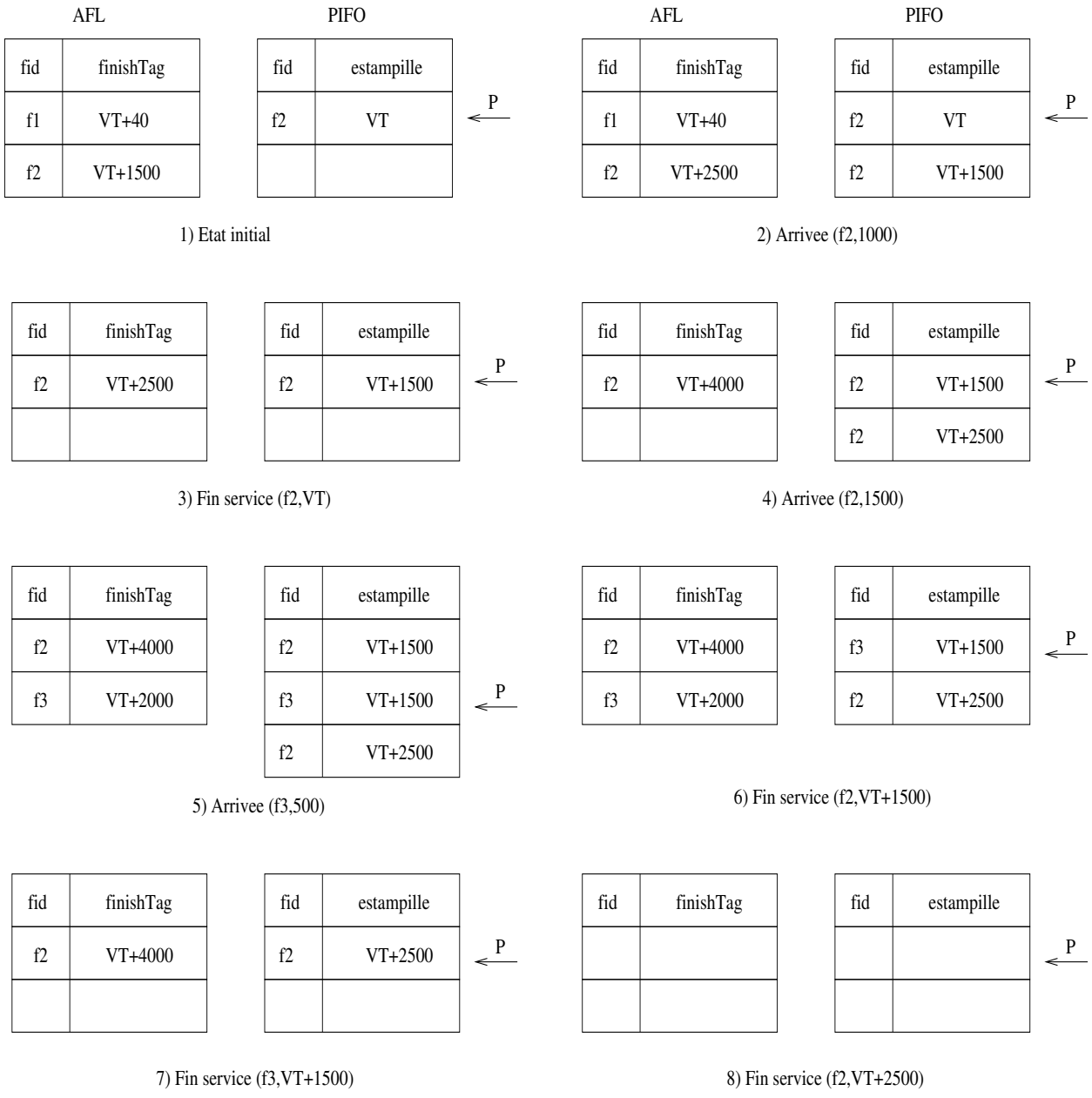


FIG. 1 – Exemple de fonctionnement de PFQ.

La figure 2 déroule un exemple de fonctionnement des algorithmes DRR (à gauche) et PDRR (à droite). On suppose qu'au début deux flots sont actifs et dont les files sont notées Queue_1 et Queue_2. La flèche indique la tête de la liste des flots actifs dans le cycle (i.e, le flot en cours de service ou bien le prochain flot à servir si la file PQ n'est pas vide dans le cas de PDRR). La valeur du quantum considérée dans cet exemple est la même pour tous les flots, elle est égale à 1500. Chaque rectangle représente un paquet avec une taille associée, le paquet en cours de service est hachuré dans les différents cas.

Dans ce qui suit nous décrivons le déroulement des différentes étapes de cette étude de cas :

- Initialement, le flot qui est en tête de liste est le flot 1, son déficit est augmenté par le quantum et son premier paquet (dont la taille est 500) commence son service ; les états de DRR et de PDRR sont identiques à cette étape.
- Un paquet appartenant à un nouveau flot arrive pendant le service du paquet précédent. Une file qui lui est associée est crée (Queue_3) dans les deux cas. Cependant, avec PDRR son paquet (de taille 750) est inséré dans la file prioritaire (PQ) alors qu'avec DRR il est inséré dans la file FIFO associée au flot, à savoir Queue_3. Le traitement du flot 1 est poursuivi dans les deux cas dans la mesure où la taille de son deuxième paquet est inférieure à son nouveau déficit ($500 < 1000$).
- Dans la troisième étape, le traitement du flot 1 est achevé puisque son déficit devient inférieur à la taille de son prochain paquet ($500 < 1000$) ; il est donc replacé en fin de liste active et la nouvelle tête de liste est le flot 2 dans les deux cas (DRR et PDRR). Selon l'algorithme DRR, le nouveau flot à traiter est par conséquent le flot 2, son déficit est incrémenté par le quantum et son premier paquet entame son service. Par contre, avec l'algorithme PDRR, c'est la file prioritaire (PQ) qui est servie puisqu'elle contient au moins un paquet, ce paquet appartient au flot 3 dont le déficit est décrémenté.
- Au cours de la 4^{ime} étape, avec DRR le nouveau flot à servir est le flot 3 tandis qu'avec PDRR il s'agit du flot 2, la file PQ étant devenue vide.
- La dernière étape constitue le début d'un nouveau cycle où le flot 1 est à nouveau en service. Notons que la file Queue_3 est supprimée avec chacun des deux scénarios car elle était devenue vide (lors de sa visite dans PDRR et à l'issue du service de son unique paquet dans DRR).

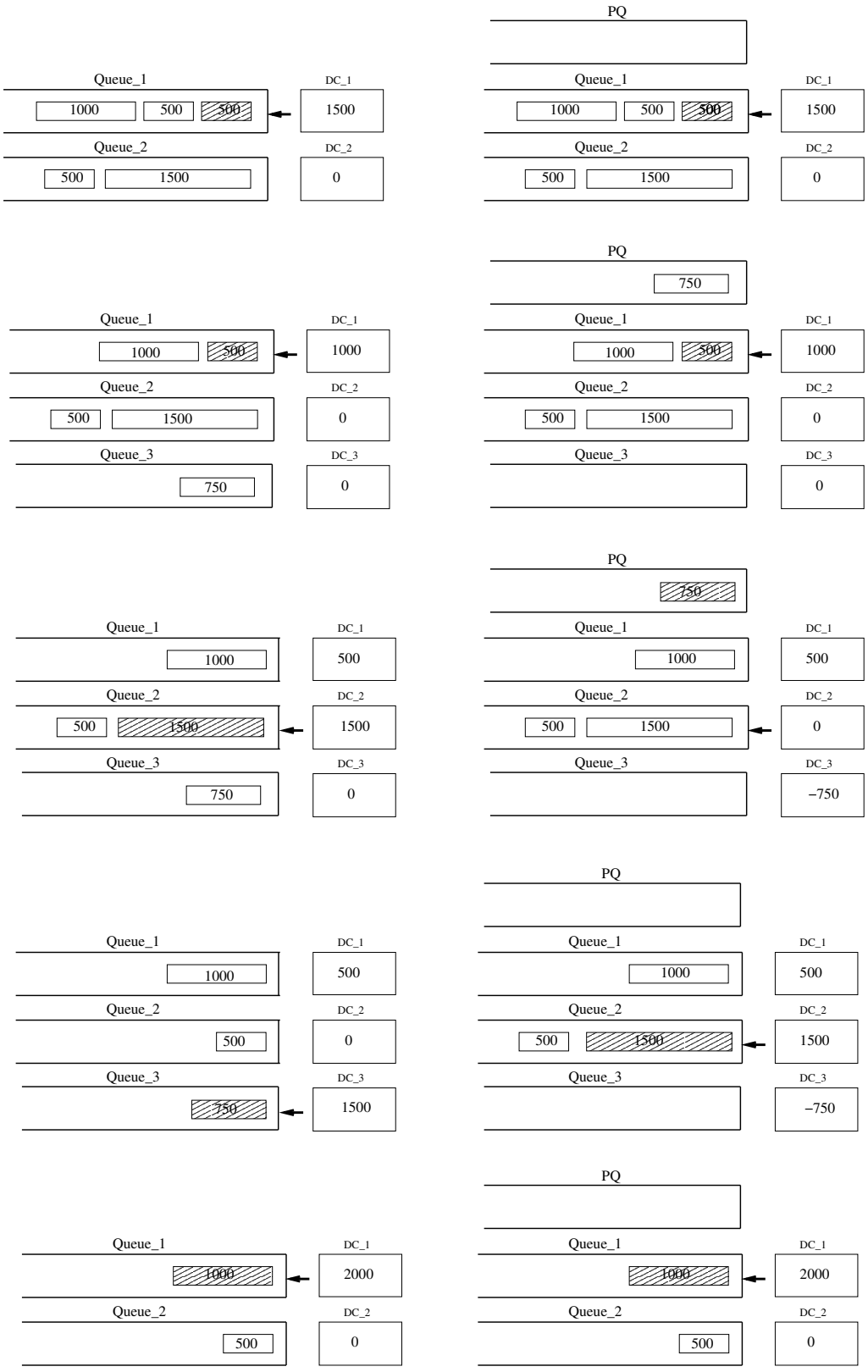


FIG. 2 – Exemple de fonctionnement de DRR (à droite) et PDRR (à gauche).

Annexe 3
Traces de trafic et traces vidéos

Dans cet annexe nous donnons des détails supplémentaires concernant les traces vidéos et réelles utilisées lors des simulations. En particulier, on décrit les variations de débit des vidéos et les variations du nombre de flots en cours ainsi que du débit cumulé des traces réelles.

1 Traces vidéos

Les variations de débits des 4 traces vidéos utilisées pour les simulations du paragraphe 3.5.2 sont données par les courbes des figures 1, 2, 3 et 4.

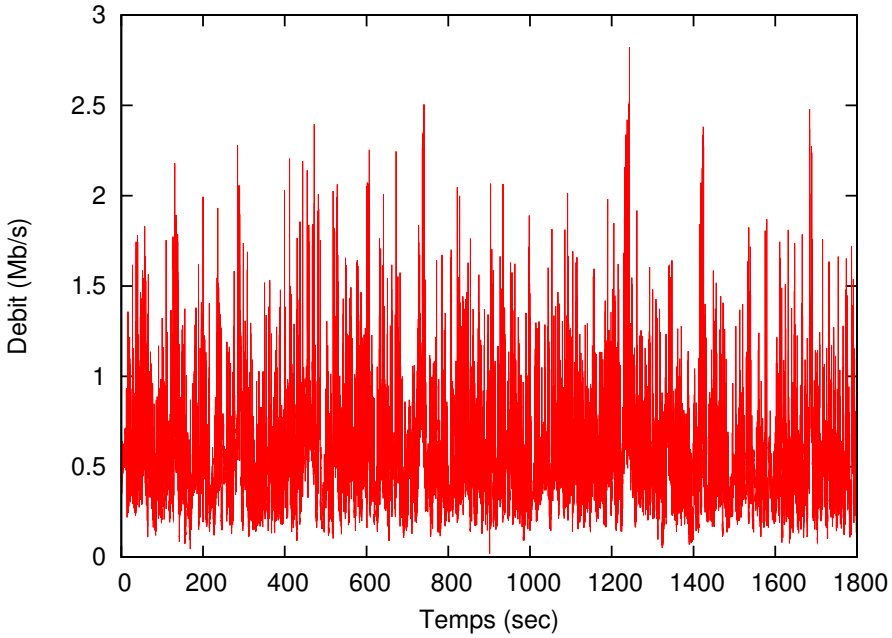


FIG. 1 – Variations de débit de la vidéo F1

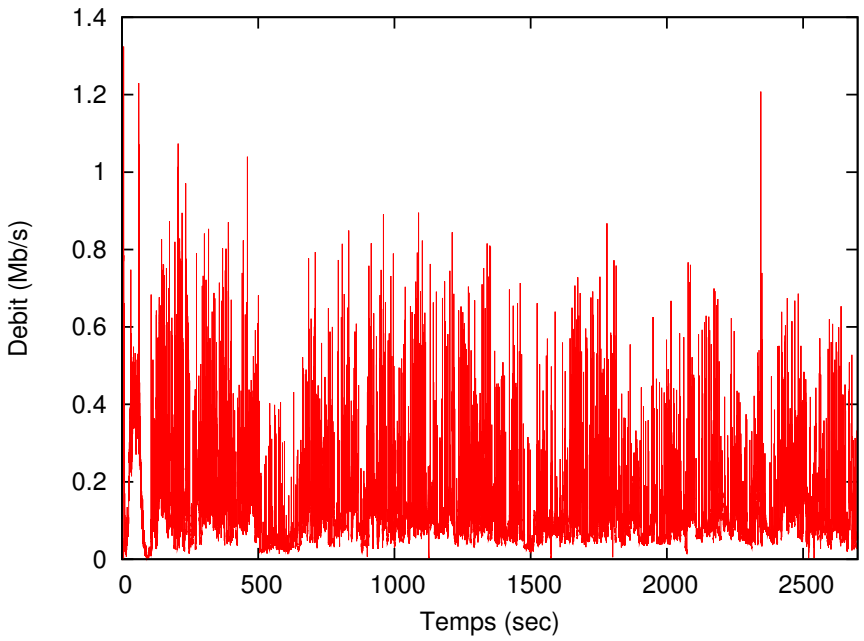


FIG. 2 – Variations de débit de la vidéo *Star Wars*

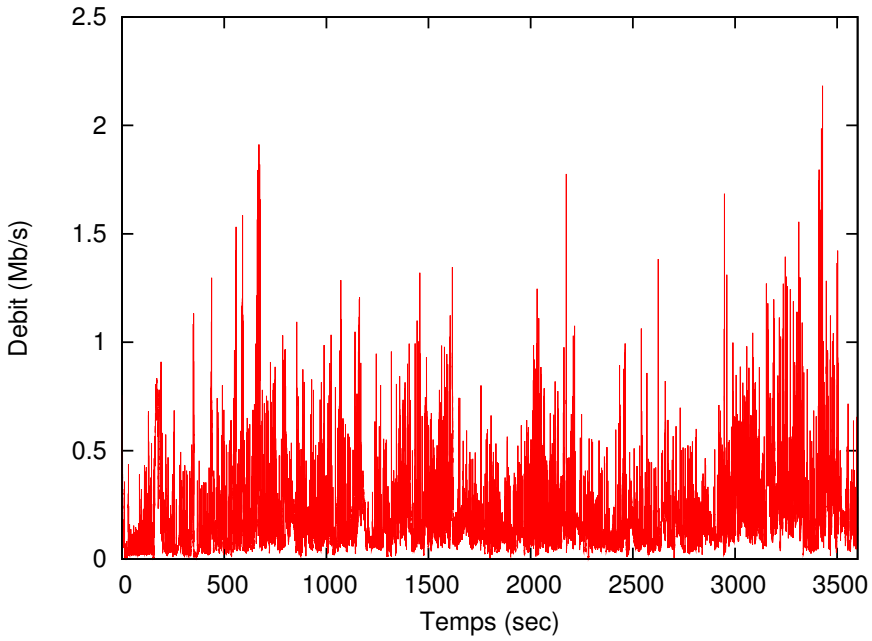


FIG. 3 – Variations de débit de la vidéo *Star Trek*

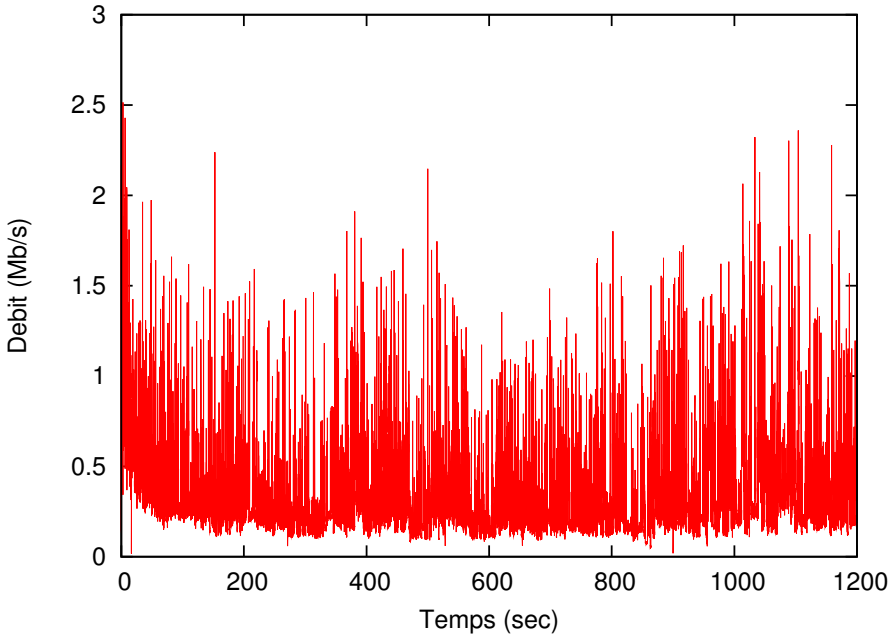


FIG. 4 – Variations de débit de la vidéo *South Park*

2 Traces réelles

Dans cette section nous traçons les courbes représentant les variations du débit cumulé et du nombre de flots en cours (sans tenir compte du *time out*) relatives aux traces de trafic décrites dans le paragraphe 5.3.

2.1 Variations du débit cumulé

Les figures 5, 6 et 7 représentent les variations du débit cumulé des trois traces de trafic réelles. Ses variations reflétent le taux d'utilisation des différents liens indiqué par les figures 5.6, 5.7 et 5.8.

2.2 Variations du nombre de flots en cours

Les figures 8, 9 et 10 tracent le nombre de flots en cours sur les trois liens en considérant que les flots sont terminés dès l'observation de leurs derniers paquets.

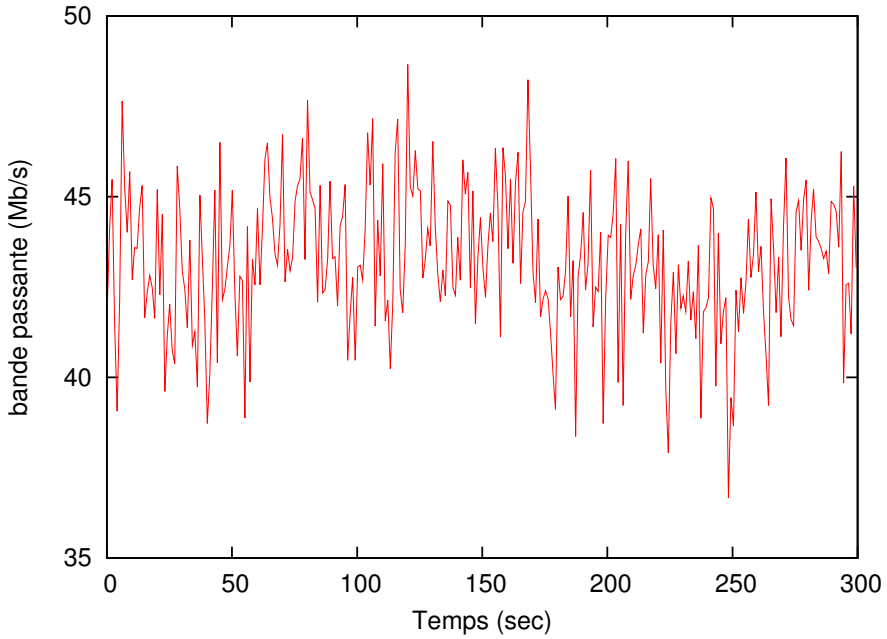


FIG. 5 – Variations du débit agrégé en fonction du temps, trace ADSL

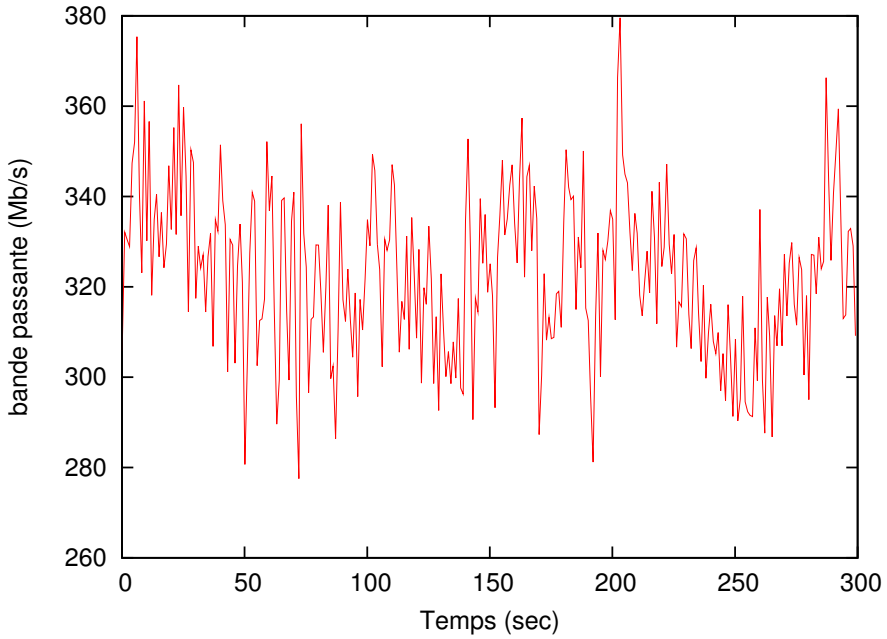


FIG. 6 – Variations du débit agrégé en fonction du temps, trace *Ab-I*

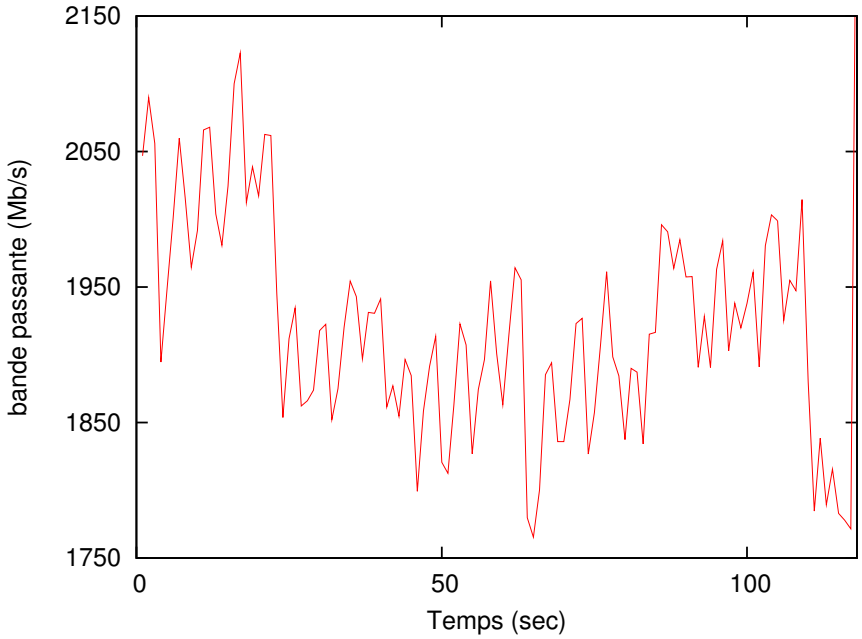


FIG. 7 – Variations du débit agrégé en fonction du temps, trace *Ab-III*

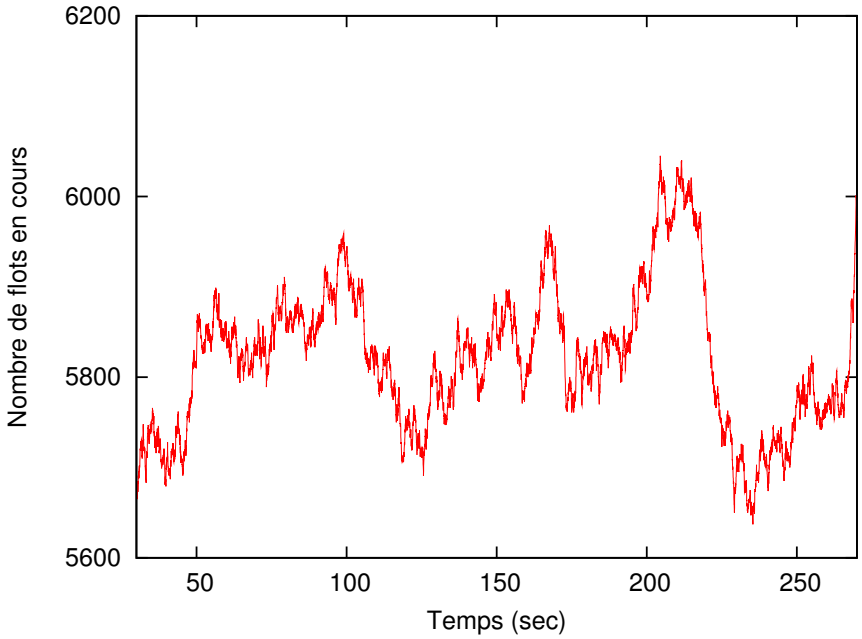


FIG. 8 – Evolution du nombre de flots en cours, TO = 0 sec, trace ADSL

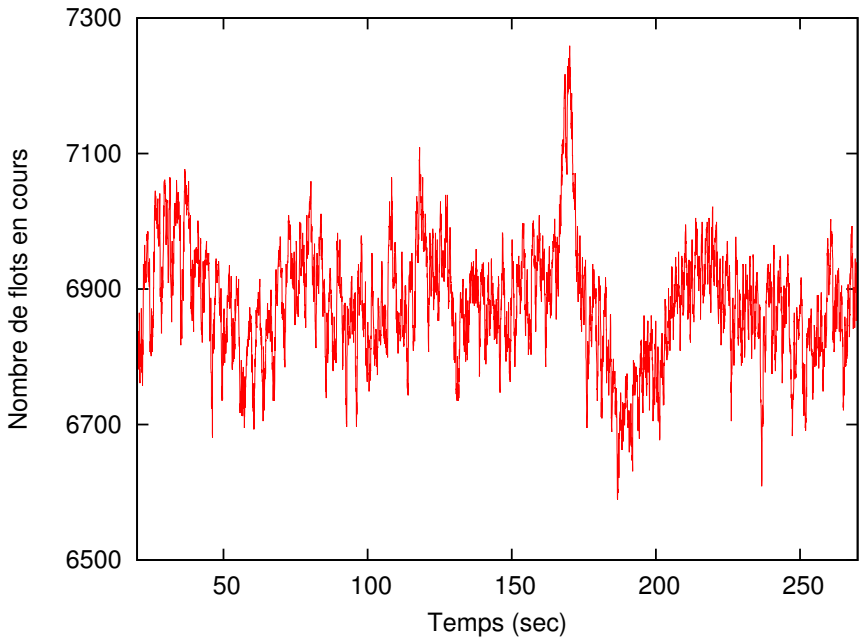


FIG. 9 – Evolution du nombre de flots en cours, TO = 0 sec, trace *Ab-I*

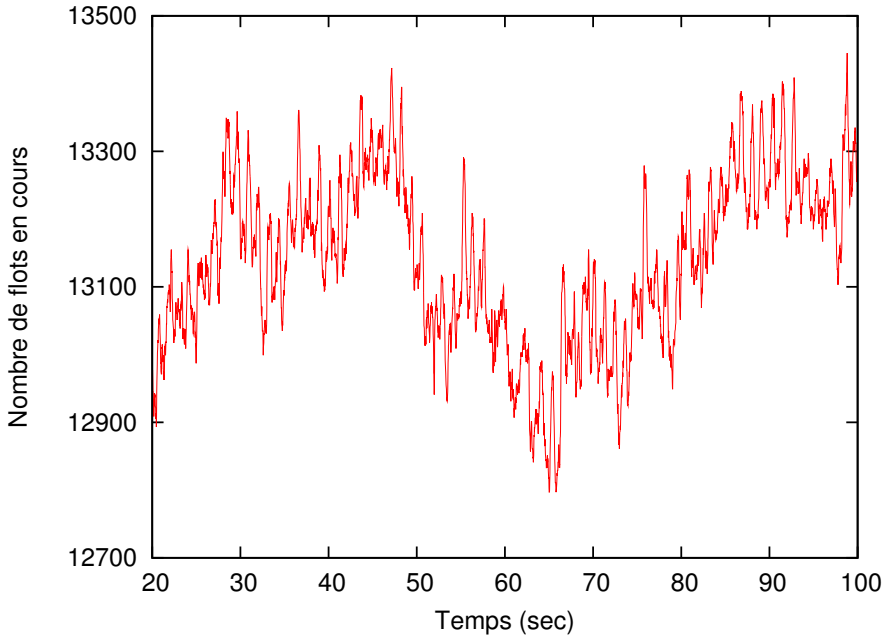


FIG. 10 – Evolution du nombre de flots en cours, TO = 0 sec, trace *Ab-III*

Annexe 4
Formules de récurrence

Cet annexe contient les démonstrations des formules de récurrence spécifiées dans les propositions 3 et 4 du paragraphe 5.5.5, qui ont servi à évaluer la distribution de la taille de la liste des flots actifs relative à l'étude de l'extensibilité des algorithmes de *fair queueing*.

1 Preuve de la proposition 3

Soit $S_j = \sum_{i \leq j} x_i c_i$ et $\mathcal{S}_j(b,s,r) = \{x : B_j = b, S_j = s, R_j = r\}$. On considère les probabilités $p_j(b,s,r) = \mathbb{P}\{B_j = b, S_j = s, R_j = r\}$ pour $j > 0$:

$$\begin{aligned} p_j(b,s,r) &= \sum_{x \in \mathcal{S}_j(b,s,r)} \pi(x) \\ &= \sum_{x \in S_j(b,s,r)} \sum_{i=1}^{M} \frac{a_i}{(s+r) \wedge C} \pi(x - e_i) \\ &= \sum_{i \leq j} \frac{a_i}{(s+r) \wedge C} \sum_{x \in S_j(b-1, s-c_i, r)} \pi(x) + \sum_{i > j} \frac{a_i}{(s+r) \wedge C} \sum_{x \in S_j(b,s,r-c_i)} \pi(x), \end{aligned}$$

ce qui correspond à l'équation (5.12).
La deuxième égalité provient des propriétés de (5.8) et (5.9), (voir [20]).
Si $b = 0$ alors $q(0,r)$, pour $r = 0, \ldots, C$, est donnée par la formule de récurrence Kaufman-Roberts ([20]).
Lorsque $b > 0$ on a : $S_J + R_J > C$. On obtient donc $q_j(b,r) = \sum_{s > C-r} p_j(b,s,r)$ pour $j > 0$ où

$$\begin{aligned} q_j(b,r) &= \sum_{s > C-r} p_j(b,s,r) = \\ &= \sum_{i \leq j} \frac{a_i}{C} \sum_{s > C-r} p_j(b-1, s-c_i, r) + \sum_{i > j} \frac{a_i}{C} \sum_{s > C-r} p_j(b,s,r-c_i), \\ &= \sum_{i \leq j} \frac{a_i}{C} \sum_{s > C-r-c_i} p_j(b-1,s,r) + \sum_{i > j} \frac{a_i}{C} \sum_{s > C-r} p_j(b,s,r-c_i), \end{aligned}$$

Les deux dernières sommes peuvent être écrites comme dans (5.11). $Q_J^{BR}(b,r)$ est obtenue en limitant $q_j(b,r)$ à la partition de l'espace d'état induite par J.

2 Preuve de la proposition 4

Soit $S_j = \sum_{i \leq j} x_i c_i$ et $\mathcal{S}_j(s,n,r) = \{x : S_j = s, N_j = n, R_j = r\}$. On considère les probabilités $p_j(s,n,r) = \mathbb{P}\{S_j = s, N_j = n, R_j = r\}$ pour $j > 0$:

$$\begin{aligned} p_j(s,n,r) &= \sum_{x \in \mathcal{S}_j(s,n,r)} \pi(x) \\ &= \sum_{x \in S_j(s,n,r)} \sum_{i=1}^{M} \frac{a_i}{(s+r) \wedge C} \pi(x - e_i) \\ &= \sum_{i \leq j} \frac{a_i}{(s+r) \wedge C} \sum_{x \in S_j(s-c_i,n,r)} \pi(x) + \sum_{i > j} \frac{a_i}{(s+r) \wedge C} \sum_{x \in S_j(s,n-1,r-c_i)} \pi(x), \end{aligned}$$

ce qui correspond à l'équation (5.14).
La deuxième égalité provient des propriétés de (5.8) et (5.9) (voir [20]) de la même manière que la preuve précédente.
Soit $q_j(n,r) = \sum_{s > C-r} p_j(s,n,r)$ où

$$\begin{aligned} q_j(n,r) &= \sum_{s > C-r} p_j(s,n,r) = \\ &= \sum_{i \leq j} \frac{a_i}{C} \sum_{s > C-r} p_j(s - c_i, n, r) + \sum_{i > j} \frac{a_i}{C} \sum_{s > C-r} p_j(s, n-1, r - c_i), \\ &= \sum_{i \leq j} \frac{a_i}{C} \sum_{s > C-r-c_i} p_j(s,n,r) + \sum_{i > j} \frac{a_i}{C} \sum_{s > C-r} p_j(s, n-1, r - c_i), \end{aligned}$$

Les deux dernières sommes peuvent être écrites comme dans (5.13) où le premier terme ne dépend pas de i.
Si $J = 0$ on obtient à nouveau la formule de récurrence Kaufman-Roberts.
$Q_J^{NR}(n,r)$ est obtenue en limitant $q_j(n,r)$ à la partition de l'espace d'état induite par J.

3 La formule de récurrence Kaufman-Roberts

$\mathbb{P}\{X_1 c_1 + \cdots + X_M c_M = n\} = q(n)$ pour $n \leq C$ satisfait la formule de récurrence suivante :

$$q(n) = \sum_{i=1}^{M} \frac{a_i}{n} q(n - c_i)$$

$q(n) = 0$ si $n < 0$.

Démonstration. La distribution jointe définie dans (5.8) et (5.9), lorsque $xc^T \leq C$, satisfait la relation $x_i c_i \pi(x) = a_i \pi(x - e_i)$.
D'où, en sommant sur i on a : $n\pi(x) = \sum_{i=1}^{M} a_i \pi(x - e_i)$.
Enfin, en sommant sur tout x tel que $xc^T = n$ on obtient la formule de récurrénce. □

Annexe 5
Expressions des distributions manipulées

Cet annexe fournit les expressions des différentes distributions mentionnées dans le document et qui ont été utilisées en simulations, ainsi que l'expression de la fonction de répartition du temps de séjour dans une file M/M/1.

1 Loi exponentielle

La distribution exponentielle de paramètre λ est défnie comme suit :

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0$$

2 Loi Pareto

Soit σ une variable aléatoire qui suit une loi de Pareto. La distribution Pareto de forme α et de paramètre b est donnée par :

$$f(\sigma) = \frac{\alpha}{b}\left(\frac{b}{\sigma}\right)^{\alpha+1}, \quad \sigma > b$$

La fonction de répartition a l'expression suivante :

$$Pr[\sigma < x] = 1 - \left(\frac{b}{x}\right)^{\alpha}, \quad x > b$$

Pour $\alpha > 1$, l'espérence est :

$$E[\sigma] = \frac{\alpha b}{\alpha - 1}$$

Pour $\alpha > 2$, la variance est :

$$V[\sigma] = \frac{\alpha b^2}{\alpha - 2} - \frac{\alpha b}{(\alpha - 1)^2}$$

3 Loi Pareto tronquée

Pour des raisons pratiques, la distribution de la taille des flots élastiques qui a été employée pour du trafic synthétique est une Pareto tronquée. Une loi Pareto tronquée se caractérise également par une valeur maximum qu'elle ne peut dépasser, notée M. Dans le programme de simulation, tant que la valeur obtenue est plus grande que M, un autre tirage aléatoire est effectué.

Soit $\tilde{\sigma}$ une variable aléatoire qui suit une loi Pareto tronquée à M, de paramètres α et b. On a :

$$\forall x \in [b, M],\ Pr[\tilde{\sigma} < x] = Pr[\sigma < x / \sigma < M]$$

d'où :

$$Pr[\tilde{\sigma} < x] = Pr[\sigma < x] / Pr[\sigma < M]$$

on obtient :

$$Pr[\tilde{\sigma} < x] = \frac{1 - (b/x)^{\alpha}}{1 - (b/M)^{\alpha}}$$

L'espérence est alors (pour $\alpha > 1$) :

$$E[\tilde{\sigma}] = E[\sigma] \frac{1 - (b/M)^{\alpha - 1}}{1 - (b/M)^{\alpha}}$$

4 Loi Weibull

Soit σ une variable aléatoire qui suit une loi de Weibull de paramètres $\alpha > 0$ et $\lambda > 0$. Sa densité est donnée par :

$$f(\sigma) = \alpha \lambda e^{-\lambda \sigma^{\alpha}} \sigma^{\alpha - 1}$$

5 Distribution complémentaire du temps de séjour dans une file M/M/1

Une file M/M/1 modélise des arrivées de clients Poisson et un temps de service exponentiel avec un seul serveur FIFO et un buffer infini. Soit S le temps de séjour, $1/\mu$ la durée moyenne de service et ρ la charge, on a alors :

$$Pr[S > t] = e^{-\mu(1-\rho)t}, \quad t \geq 0$$

Les courbes relatives à la file M/M/1 de la figure 2.12 ont été tracées à partir de l'équation ci-dessus.

Table des figures

1 Demande de trafic journalière et hebdomadaire sur un lien OC192 . . 5
2 Variation du débit d'une vidéo MPEG 6
3 Structure d'une session en flots et paquets 7
4 Modélisation des arrivées des flots 8

1.1 Réseau muni d'un contrôle d'admission au niveau d'un goulot d'étranglement 16
1.2 Topologie simulée 19
1.3 Diagrammes en rectangle sans Diffserv, ρ=0.9 (gauche) et ρ=1.2 (droite) 21
1.4 Diagrammes en rectangle, scénario PQ, ρ=1.2, flots PR (gauche) et flots BE (droite) 21
1.5 Nombre de flots des classes PR et BE vs. temps, simulations Diffserv 22
1.6 Evolution du nombre de flots PR et BE avec le scénario PQ de Diffserv et le contrôle d'admission, simulation avec impatience, ρ=1.2 24

2.1 Un routeur *cross-protect* 28
2.2 Opérations exécutées à l'arrivée d'un paquet 31
2.3 Partage max-min via un scheduler *fair queueing* (FQ) 32
2.4 Structure de la liste des flots actifs avec l'algorithme PFQ 33
2.5 Structure de l'échéancier avec l'algorithme PFQ 38
2.6 Topologie simulée 42
2.7 Résultats des simulations avec un lien à 10 Mb/s sous a) charge normale , b) surcharge sans contrôle d'admission, c) surcharge avec contrôle d'admission. Courbes relatives à 1) *PL*, 2) *FR* normalisé, 3) tailles PFL (rouge/foncé) et AFL (vert/clair), 4) tailles file prioritaire (rouge/foncé) et file totale (vert/clair). 44
2.8 Nombre de flots en cours (rouge/foncé) et actifs (vert/clair) sur un lien à 100 Mb/s en fonction du temps, charge 0.9 46
2.9 Distribution complémentaire des délais des paquets des flots *streaming* 48
2.10 Comparaison entre PDRR et PFQ. Courbes relatives à *PL* (en haut à gauche), *FR* normalisé (en haut à droite), tailles AFL (en bas à gauche), tailles file totale (en bas à droite), trafic synthétique. 53
2.11 Distribution complémentaire des délais des paquets et évolution du *fair rate*, trace Abilene plus flots CBR. 55

2.12 Comparaison des distributions des délais des packets du flot CBR avec PDRR et DRR, trafic synthétique (gauche) et trace Abilene (droite) . 56
2.13 Variations de la taille de l'AFL (gauche) et *priority load* (droite) vs. temps, trafic synthétique en surcharge 57

3.1 Probabilité de débordement (gauche) et délais (droite) vs. utilisation, trafic synthétique 73
3.2 Taux de blocage en fonction de la charge offerte 74
3.3 Délais vs. utilisation, seuil d'admission = 0.91, débit crête = 32 Kb/s 75
3.4 Distribution du nombre de paquets par intervalle avec du trafic *on/off* (gauche) et la trace ADSL (droite) 76

5.1 Distribution géométrique du nombre de flots, $\rho = 0.9$ 97
5.2 Nombre de flots moyen avec une loi géométrique en fonction de la charge 97
5.3 Distribution complémentaire de la taille de la l'AFL, charges : 0.16, 0.41, 0.84, 0.93. 99
5.4 Quantiles 99.9% des distributions du nombre de flots vs. charge : $m = 1, 100, 1000, 10000$. 101
5.5 Distribution complémentaire de la taille de l'AFL, simulation et analyse. 102
5.6 Variations de l'utilisation du lien ADSL en fonction du temps 104
5.7 Variations de l'utilisation du lien *Ab-I* en fonction du temps 105
5.8 Variations de l'utilisation du lien *Ab-III* en fonction du temps 105
5.9 Evolution du nombre de flots en cours, TO = 20 sec, trace ADSL . . 106
5.10 Evolution du nombre de flots en cours, TO = 20 sec, trace *Ab-I* . . . 106
5.11 Evolution du nombre de flots en cours, TO = 20 sec, trace *Ab-III* . . 107
5.12 La distribution complémentaire des débits moyens des flots 108
5.13 La distribution complémentaire des débits crêtes des flots 109
5.14 Distribution complémentaire de la taille de l'AFL, charges orginales . 111
5.15 Distribution complémentaire de la taille de l'AFL, simulations, charges 0.6 et 0.9 112
5.16 Variations du *fair rate* vs. temps, simulations, charges 0.6 et 0.9 . . . 114
5.17 Illustration de la notion de *busy cycle.* 116
5.18 Comparaison des allocations max-min et *balanced*, distribution marginale de B, charge = 0.9, trace *Ab-III*. 123
5.19 Comparaison des allocations max-min et *balanced*, distribution marginale de R, charge = 0.9, trace *Ab-III*. 124
5.20 Distribution complémentaire de la taille de l'AFL, modèle analytique multi-débit vs. simulation, charges 0.6 et 0.9 126
5.21 Distribution complémentaire de la taille de l'AFL, charges 0.6 (à gauche) et 0.9 (à droite), algorithme PDRR 128
5.22 Distribution complémentaire de la taille de l'AFL, charge 0.95. 129

6.1 Structure de l'AFL avec PDRR modifié 134

6.2 Détection tardive d'un flot *bottlenecked* ($b = 1$) 136
6.3 Distribution complémentaire de la taille de l'AFL, traces ADSL (gauche) et *Ab-I* (droite), PDRR modifié 138
6.4 Nombre de flots *non bottlenecked* arrivant par intervalle de référence, traces ADSL (gauche) et *Ab-I* (droite), PDRR modifié 138
6.5 Evolution du FR avec PDRR et PDRR modifié, traces ADSL (droite) et *Ab-I* (gauche) 139
6.6 Evolution du PL avec PDRR et PDRR modifié, traces ADSL (gauche) et *Ab-I* (droite) 140
6.7 Distribution complémentaire de la taille de l'AFL, traces ADSL (gauche) et *Ab-I* (droite), PFQ modifié 141
6.8 Représentation d'un *bitmap* 142
6.9 Détection tardive de l'expiration d'un flot 143
1 Plate-forme d'expérimentation de Diffserv 152
2 Evolution du nombre (gauche) et des débits (droite) des flots PR et BE avec le scénario WFQ de Diffserv, expérimentation, $\rho = 1.2$ 152
3 Evolution du nombre (gauche) et des débits instantanés (droite) des flots PR et BE avec contrôl d'admission, expérimentation, $\rho = 1.2$ . . 153
4 Plate-forme de tests 154
5 Variations du *FR* (en haut à gauche), *PL* (en haut à droite), délais des paquets UDP (en bas à gauche) et tailles PFL et AFL (en bas à droite) en focntion du temps, $\rho = 0.9$ 156
6 Variations du *FR* (en haut à gauche), *PL* (en haut à droite), délais des paquets UDP (en bas à gauche) et tailles PFL et AFL (en bas à droite) vs. temps, $\rho = 1.1$ 157
7 Variations de la taille de la PFL et AFL vs. temps, $C = 100$ Mb/s, $\rho = 1.1$ 158
8 Variations du débit de la vidéo utilisée vs. temps 158
1 Exemple de fonctionnement de PFQ. 160
2 Exemple de fonctionnement de DRR (à droite) et PDRR (à gauche). 162
1 Variations de débit de la vidéo F1 163
2 Variations de débit de la vidéo *Star Wars* 164
3 Variations de débit de la vidéo *Star Trek* 164
4 Variations de débit de la vidéo *South Park* 165
5 Variations du débit agrégé en fonction du temps, trace ADSL 166
6 Variations du débit agrégé en fonction du temps, trace *Ab-I* 166
7 Variations du débit agrégé en fonction du temps, trace *Ab-III* 167
8 Evolution du nombre de flots en cours, TO = 0 sec, trace ADSL . . . 167
9 Evolution du nombre de flots en cours, TO = 0 sec, trace *Ab-I* 168
10 Evolution du nombre de flots en cours, TO = 0 sec, trace *Ab-III* . . . 168

Liste des tableaux

1.1 Débit des flots élastiques, simulations Diffserv, $\rho = 0.7$ 21
1.2 Comparaison entre le scénario PQ de Diffserv et le contrôle d'admission (AC), simulation avec impatience, ρ=1.2 23

3.1 Seuils d'admission en fonction de C/p et ϵ 71
3.2 Simulation flots *on/off* homogènes, $\rho = 0.9$ 77
3.3 Simulation flots *on/off* hétérogènes, $\rho = 0.9$ 78
3.4 Vidéos simulées 78
3.5 Simulation trafic vidéo type1, $\rho = 1$ 80
3.6 Simulation trafic vidéo type2, $\rho = 1$ 80
3.7 Simulation trafic réel, $\rho = 1$ 80

5.1 Statistiques des traces de trafic 103
5.2 Distribution de la taille des paquets des trois traces. 110
5.3 Capacités simulées (en b/s) pour les trois traces. 111
5.4 Classification des flots simulés, trace *Ab-III*. 122
5.5 Classification des flots. 125

6.1 Tailles moyenne et maximale de la PFL, insertion probabiliste, $p = 0.1145$

Glossaire

AF : Assured Forwarding, 4
AFD : Approximate Fair Droping, 88
AFL : Active Flow List, 31
AQM : Active Queue Management, 18
ARED : Adaptive RED, 88
ATM : Asynchronous Transfer Mode, 59

CAM : Content Adressable Memory, 36
CBQ : Class Based Queueing, 3
CBR : Constant Bit Rate, 6
CCDF : Complementary Cummulative Distribution Function, 45
CL : Controlled load, 2
CSFQ : Core-Stateless Fair Queueing, 89

DoS : Denial of Service, 38
DRED : Dynamic RED, 88
DRR : Deficit Round Robin, 28
DSCP : DiffServ Code Point, 3

EF : Expedited Forwarding, 3
ERR : Elastic Round Robin, 85

FAN : Flow-Aware Networking, 2
FIFO : First In First Out, 31
FR : Fair Rate, 35
FRED : Flow Random Early Drop, 88

GPS : Generalized Processor Sharing, 82
GRED : Gentle RED, 87
GS : Guaranteed Service, 2

IETF : Internet Engineering Task Force, 3
IP : Internet Protocol, 1

LAN : Local Area Network, 6
LIFO : Last In First Out, 56

MBAC : Measurement-Based Admission Control, 10
MTU : Maximum Transfer Unit, 32

PDRR : Priority Deficit Round Robin, 28
PERR : Prioritized Elastic Round Robin, 85
PFL : Protected Flow List, 14
PFQ : Priority Fair Queueing, 26
PHB : Per Hop Behaviour, 3
PIFO : Push In First Out, 30
PL : Priority Load, 35
PQ : Priority Queueing, 18
PS : Processor Sharing, 82

QoS : Quality of Service, 1

RED : Random Early Detection, 87
RED-PD : RED with Preferential Dropping, 88
RSVP : Resource ReSerVation Protocol, 2
RTT : Round Trip Time, 39

SCFQ : Self Clocked Fair Queueing, 28
SFQ : Start-time Fair Queueing, 28
SRED : Stabilized RED, 89
SRR : Stratified Round Robin, 86

TCP : Transmission Control Protocol, 8

UDP : User Datagram Protocol, 8

WF^2Q : Worst-Case Fair Weighted Fair Queueing, 83
WFQ : Weighted Fair Queueing, 4
WRED : Weighted Random Early Detection, 4
WRR : Weighted Round Robin, 3

Bibliographie

[1] M. Accarion, C.Boscher, C.Duret, and J.Lattmann. Extensive Packet Header Lookup at Gb/s Speed for an Application to IP/ATM Multimedia Switch Router. In Proceedings of World Telecommunications Congress - International Switching Symposium, May 2000, Birmingham, UK.

[2] Y. Afek, Y. Mansour, Z. Ostfeld. Phantom : A Simple and Effective Flow Control Scheme. In ACM SIGCOMM, 1996.

[3] J. Augé. Evaluation et enrichissement d'un prototype de contrôle d'admission pour les flots IP sur plate-forme Linux. Mémoire d'ingénieur IIE, 2004.

[4] J. Aweya, M. Ouellette, D. Montuno. A Control Theoritic Approach to Active Queue Management. Computer Networks, vol. 36, no. 2-3, 2001.

[5] N. Benameur, S. Ben Fredj, S. Oueslati-Boulahia, J. Roberts. Quality of service and flow-aware admission control in the Internet. In Computer Networks, Vol 40, pages 57-71, 2002. (http ://perso.rd.francetelecom.fr/roberts/Pub/BBOR02.pdf).

[6] N. Benameur, A. Kortebi, S. Oueslati, J. Roberts. Selective protection in overload : differentiated services or per flow admission control ? Networks 2004, Vienna, Austria, June 2004.

[7] N. Benameur, S. Ben Fredj, F. Delcoigne, S. Oueslati-Boulahia and J.W. Roberts. Integrated Admission Control for Streaming and Elastic Traffic. QofIS 2001 organised by COST 263, Coimbra, Portugal, September 2001.

[8] N. Benameur S. Oueslati, J. Roberts. DE/DAC/OAT/12.03 Rapport sur l'expérimentation du contrôle d'admission orienté flot.

[9] S. Ben Fredj, T. Bonald, A. Proutière, G. Régnié and J.W. Roberts. Statistical bandwidth sharing : A study of congestion at flow level. In Proceedings. of ACM SIGCOMM 2001.

[10] S. Ben Fredj, S. Oueslati-Boulahia and J.W. Roberts. Measurement-based Admission Control for Elastic Traffic. Teletraffic Engineering in the Internet Era, ITC 17, Elsevier, December 2001. (http ://perso.rd.francetelecom.fr/roberts/Pub/BOR01.pdf).

[11] J. C. R. Bennett, H. Zhang. WF^2Q : Worst-Case Fair Weighted Fair Queueing. In Proceedings of IEEE INFOCOM, San Francisco, CA, 1996.

[12] J. C. R. Bennett, H. Zhang. Hierarchical Packet Fair Queueing Algorithms. In ACM SIGCOMM 1996.

[13] D. Bertsekas, R. Gallager. Data Networks. Prentice-Hall, 19887.

[14] S. Blake et al. An Architecture for Differentiated Services. RFC 2475, 1998.

[15] B. Bloom. Space/time tradeoffs in hash coding with allowable errors, Communication ACM, vol. 13, no. 7, pp. 422-426, July 1970.

[16] T. Bonald, S. Oueslati, J. Roberts. IP traffic and QoS control : Towards a flow-aware architecture. In Proceedings of World Telecom. Conf., Paris 2002. (http ://perso.rd.francetelecom.fr/roberts/Pub/BOR02a.pdf)

[17] T. Bonald, A. Proutière. On performance bounds for balanced fairness. Performance Evaluation, 2003.

[18] T. Bonald, A. Proutiére. Insensitivity in Processor Sharing Networks. In Proceedings of IFIP Performance 2002.

[19] T. Bonald, A. Proutière,, J.W. Roberts. Statistical Performance Guarantees for Streaming Flows using Expedited Forwarding. In IEEE INFOCOM, pages 1104–1112, 2001.

[20] T. Bonald, J. Virtamo, A recursive formula for multirate systems with elastic traffic. IEEE Communications Letters, 2005

[21] T. Bonald, J.W. Roberts. Congestion at flow level and the impact of user behaviour. Computer Networks 42 (2003) 521-536.

[22] B. Braden et al. Resource ReSerVation Protocol (RSVP)-Version 1 Functional Specification. RFC 2205, September 1997.

[23] B. Braden, D. Clark, J. Crowcroft, B. Davie, S. Deering, D. Estrin, S. Floyd, V. Jacobson, G. Minshall, C. Partridge, L. Peterson, K. Ramakrikshnan, S. Shenker, J. Wroclawski, L. Zhang. Recommendations on Queue Management and Congestion Avoidance in the Internet. IETF RFC 2309, 1998.

[24] L. Breslau, S. Jamin, S. Shenker. Comments on the Performance of Measurement-Based Admission Control Algorithms. In IEEE INFOCOM2000, Tel Aviv, Israel, March 2000.

[25] F. Brichet, A. Simonian. Measurement-based CAC for video applications using SBR service. Proceedings of the PMCCN Conference, IFIP WG 6.3 and 7.2, Tsukuba (Japan), November 1997, pp 285-304. Technilcal Document COST 257 TD (97)33.

[26] N. Brownlee, KC Claffy. Understanding Internet Traffic Streams : Dragonflies and Tortoises, IEEE Communications Magazine, Vol. 40, No. 10, Oct. 2002, Pages : 110-117.

[27] Z. Cao, Z. Wang, E.W. Zegura. Rainbow Fair Queueing : Fair Bandwidth Sharing Without per-Flow stats, In Proceedings IEEE/INFOCOM 2000.

[28] C. Casetti, J. Kurose. An Adaptive Algorithm for Measurement-based Admission Control in Integrated Services Packet Networks. Technical Report TR 96-76, workshop on high speed networs, October 1996.

[29] S. Cheung, C. Pencea. BSFQ : Bin Sort Fair Queueing. In IEEE INFOCOM 2002.

[30] M. Christiansen, K. Jeffay, D. Ott, F. D. Smith. Tuning RED for Web Traffic. In ACM SIGCOMM 2000.

[31] G. Chuanxiong. SRR : an O (1) Time Complexity Packet Scheduler for Flows in Multi-Service Packet Networks. In ACM SIGCOMM 2001.

[32] D.D. Clark, S. Shenker, L. Zhang. Supporting rteal-time applications in an integrated services packet network : Architecture and mechanism. SIGCOMM Sumposium on Communications Architectures and Protocols, pages 14-26, 1992.

[33] S. Crosby, I. Leslie, B. McGurk, J.T. Lewis, R. Russell, F. Toomey. Statistical Properties of a Near Optimal Measurement-based CAC Algorithm. In Proceedings IEEE ATM, June 1997.

[34] M. Crovella, A. Bestavros. Self-Similarity in World Wide Web Traffic Evidence and Possible Causes. IEEE/ACM Transactions on Networking, 5(6) :835-846, 1997.

[35] S. Deering, R. Hinden. RFC 2460 : Internet Protocol, Version 6 (IPv6) - Specification. 1998

[36] A. Demers, S. Keshav, S. Shenker. Design and Analysis of a Fair Queuing Algorithm. In Proceedings of ACM SIGCOMM, Austin, 1989.

[37] N.Duffield. Asymptotic Sampling Properties of Effective Bandwidth Estimation for Admission Control. Proceeding Infocom, pp 1532-1538. 1999.

[38] Z. Dziong, M. Juda, L. Mason. A Framework for Bandwidth Management in ATM Networks - Aggregate Equivalent Bandwidth Estimation Approach. IEEE/ACM Transactions on Networking 5, 1, February 1997.

[39] C. Estan, G. Varghese. New directions in traffic measurement and accounting. Proceedings of ACM Sigcomm 2002.

[40] W. Feller. Introduction to probability theory and its applications : Vol I. Wiley International, 1968.

[41] W. Feng, D. Dilip, Z. Kandlur, D. Saha, Z. Kang, G. Shin. Blue : A New Class of Active Queue Management. Technical Report, University of Michigan, 1999.

[42] W. Feng, D. Dilip, Z. Kandlur, D. Saha, Z. Kang, G. Shin. A Self Configuring RED Gateway. In Proceedings of IEEE INFOCOM, 1999.

[43] N. Figueira, J. Pasquale. Leave-in-time : A New Service Discipline for Real-time Communications in a Packet-Switching Network. In ACM SIGCOMM 1995.

[44] S. Floyd. Comments on Measurement-based Admissions Control for Controlled-Load Services. Lawrence Berckeley Laboratory, Tech. Rep., July 1996.

[45] S. Floyd, V. Jacobson. Random Early Detection Gatways for Congestion Avoidance. IEEE/ACM Transaction on Networking, 1(4) :397-413, 1993.

[46] S. Floyd, V. Paxson. Difficulties in Simulating the Internet. IEEE/ACM Transactions on Networking, Vol. 9, No. 4, Aug. 2001, Pages : 392-403.

[47] R.J. Gibbens, F.P. Kelly, P.B. Key. A Decision-Theoretic Approach to Call Admission Control in ATM Networks. In IEEE Journal on Selected Areas in Communications, Vol 13, No 6, pages 1101-1114, August 1995.

[48] R.J. Gibbens, F.P. Kelly. Measurement-based connection admission control. In 15th International Teletraffic Congress Proceedings, June 1997.

[49] S. J. Golestani. A Self-Clocked Fair Queuing Scheme for Broadband applications. In Proceedings of IEEE INFOCOM Toronto, 1994.

[50] P. Goyal, S. S. Lam, H. M. Vin. Determining End-to-End Delay Bounds in Heterogeneous Networks. In ACM/Springer-Verlag Multimedia Systems Journal, 1996.

[51] P. Goyal, H. Vin, H. Cheng. Start-time Fair Queuing : A Scheduling Algorithm for Integrated Services Packet Switching Networks. IEEE/ACM Transactions on Networking, Vol. 5, No. 5, Oct 1997, pages : 690-704.

[52] M. Grossglauser, D. Tse. A Framework for Robust Measurement-Based Admission Control. IEEE/ACM transactions on Networking, 5 (1), February 1997.

[53] M. Grossglauser, D. Tse. A Time-Scale Decomposition Approach to Measurement-Based Admission Control. submitted to IEEE/ACM Transactions on Networking, 2003.

[54] E. L. Hahne. Round-Robin Scheduling for Max-Min Fairness in Data Networks. IEEE Journal on Selected Areas in Communications 9(7) : 1024-1039, 1991.

[55] J. Heinanen, F. Bake, W. Weiss, J. Wroclawski. Assured Forwarding PHB Group. RFC 2597, June 1999.

[56] J.M. Hyman, A.A. Lazar,, G. Pacifici. A Separation Principle Between Scheduling and Admission Control for Broadband Switching. IEEE Journal on Selected Areas in Communications, 11(4) :605-616, May 1993.

[57] V. Jacobson, C. Nichols. An Expedited Forwarding PHB. RFC 2598, June 1999.

[58] S. Jamin, P.B. Danzig, S.J. Shenker,, L.Zhang. A Measurement-based Admission Control Algorithm for Integrated Services Packet Switched Networks (Extended Version). IEEE/ACM transactions on networking, February 1997.

[59] S. Kanhere, H. Sethu. Fair, Efficient and Low-latency Packet Scheduling Using Nested Deficit Round Robin. In Proceedings of IEEE HPSR, Dallas, 2001.

[60] S. Kanhere, H. Sethu, A. Parekh. Fair and Efficient Packet Scheduling Using Elastic Round Robin. IEEE Trtansactions on Parallele and Distributed Systems 13 (3), pages : 324-326, 2002.

[61] S. Kanhere, H. Sethu. Prioritized Elastic Round Robin : An Efficient and Low-Latency Packet Scheduler with Improved Fairness. Technical Report DU-CS-03-03, Department of Computer Science Drexel University, Philadelphia, 2003.

[62] F. Kelly. Reversibility and Stochastic Networks. J. Wiley and Sons, 1979.

[63] S. Keshav. A control-theoritic approach to flow control. In ACM Sigcomm 1991.

[64] P.S. Khedkar, S. Keshav. Fuzzy prediction of time-series. IEEE conference on fuzzy systems-92, 1992.

[65] R. Kompella, G. Varghese. Reduced State Fair Queueing for Edge and Core Routers. In Proceedings of the 14th ACM International Workshop on Network and Operating System Support for Digital Audio and Video (NOSSDAV), Kinsale, County Cork, Ireland, June 2004.

[66] A. Kortebi, L. Muscariello, J. Roberts. Procédé d'ordonnancement de paquets appartenant à des flots et équipement associé,n° FTR&D 2236, 2005.

[67] A. Kortebi, L. Muscariello, S. Oueslati, J. Roberts. Minimizing the Overhead in Implementing Flow Aware Networking. In ACM ANCS, Princeton, 2005.

[68] A. Kortebi, L. Muscariello, S. Oueslati, J. Roberts, Evaluating the Number of Active Flows in a Scheduler Realizing Statistical fair Bandwidth Sharing. In ACM SIGMETRICS, Banff, 2005.

[69] A. Kortebi, L. Muscariello, S. Oueslati, J. Roberts. On the Scalability of Fair Queueing. In Proceedings of ACM HotNets III 2004.

[70] A. Kortebi, S. Oueslati, J. Roberts. Implicit service differentiation using deficit round robin. ITC19, Beijing, China, 2005.

[71] A. Kortebi, S. Oueslati, J. Roberts. Procédé et dispositif d'ordonnancement de paquets pour leur routage dans un réseau avec détermination implicite des paquets à traiter en priorité, Demande de brevet INPI, France 0412063, 2005.

[72] A. Kortebi, S. Oueslati, J. Roberts. MBAC algorithm for streaming flows in Cross-protect. EuroNGI Workshop, Lund, Sweden, June 2004.

[73] A. Kortebi, S. Oueslati, J. Roberts. Cross-protect : Implicit Service Differentiation and Admission Control. In Proceedings of IEEE HPSR 2004.

[74] L. Kleinrock. Queueing System, Volume 2 : Computer Applications. Wiley 1976.

[75] A. Kumar, M. Hegde, S. Anand, B. Bindu, D. Thitumurthy and A. Kherani. Non-intrusive TCP connection admission control for bandwidth management of an Internet access link. In IEEE Communication Magazine Vol 38, No 5, pages 160-167, 2000.

[76] W.E. Leland, M.S. Taqqu, W. Willinger, D.V. Wilson. On the Self-similar Nature of Ethernet Traffic. IEEE/ACM Transactions on Networking, 2(1) :1–15, 1994.

[77] D. Lin, R. Morris. Dynamics of Random Early Detection, In Proceedings of ACM SIGCOMM 1997.

[78] M. MacGregor, W. Shi. Deficits for Bursty Latency-critical Flows : DRR++, In Proceedings IEEE ICON, Singapour, 2000.

[79] P. E. McKenney, Stochastic Fairness Queueing. In Proceedings of IEEE INFOCOM 1990.

[80] R. Mahajan, S. Floyd, D. Weatherall. Controling High-Bandwidth Flows at the Congested Router. Proceedings of ICNP 2001.

[81] L. Massoulié, J. Roberts. Arguments in favour of admission control for TCP flows. Proceedings of ITC 16, Edinburgh, June 1999.

[82] M. May, C. Diot, B. Lyes, J. Bolot. Influence of Active Queue Management Parameters on Aggregate Traffic Performance. Technical Report, RR-3995, http ://www.inria.fr/rrrt/rr-3995.html, 2000.

[83] R. Mortier, I. Pratt, C. Clark, S. Crosby. Implicit Admission Control. In IEEE Journal on Selected Areas in Communications, December 2000.

[84] J. Nagle. On Packet Switches with Infinite Storage. RFC 970, IETF, 1985.

[85] S. Niu, R. Cooper. Duality and Other Results for M/G/1 and GI/M/1 Queues, via a New Ballot Theorem. Mathematics of Operations Research, Vol. 14, No. 2, 1989, Pages : 281-293.

[86] T. J. Ott, T. V. Lakshman, L. H. Wong. SRED : Stabilized RED, Proceedings IEEE INFOCOM, 1999.

[87] R. Pan, L. Breslau, B. Prabhakar, S. Shenker. A Flow Table-Based Design to Approximate Fairness. In Hot Interconnects, Palo Alto, California, August 2002.

[88] R. Pan, B. Prabhakar, K. Psounis, CHOKe. A Stateless Active Queue Management Scheme for Approximating Fair Bandwidth Allocation. In Proceedings of IEEE INFOCOM 2002.

[89] A. K. Parekh, R.G. Gallager. A Generalized Processor Sharing Approach to Flow Control, the Single Node Case. In Proceedings of IEEE INFOCOM, Florence, 1992.

[90] J. Qiu, E.W. Knightly. Measurement-Based Admission Control with Aggregate Traffic Envelopes. IEEE/ACM transactions on networking, vol. 9, NO. 2, April 2001.

[91] S. Ramabhadran, J. Pasquale. Stratified Round Robin : A Low Complexity Packet Scheduler with Bandwidth Fairness and Bounded Delay. In Proceedings of ACM SIGCOMM 2003.

[92] A.R. Reibman, A.W. Berger. Traffic descriptors for VBR video teleconferencing over ATM networks. IEEE/ACM Transactions on Networking, Vol 3, No 3, 329-339, 1995.

[93] M. Reisslein. Measurement-Based Admission Control : A Large Deviations Approach for Bufferless Multiplexers. IEEE 0-7695-0722-0/00, 2000.

[94] M. Reisslein. Providing QoS with Measuremnt-based Admission Control. http ://www.fokus.de/usr/reisslein.

[95] J. Roberts. Internet Traffic, QoS and Pricing. In Proceedings of the IEEE,Vol 92, No 9, pp 1389-1399, Sept. 2004.

[96] J. Roberts. Self-Similar Network Traffic and Performance Evaluation, chapter Engineering for Quality of Service, pages 401-420. Wiley-Interscience, 2000.

[97] J. Roberts, U. Mocci, J. Virtamo. Broadband Network Teletraffic(Final Report of COST 242). LNCS 1155 Springer Verlag, 1996.

[98] J. Roberts, S. Oueslati Boulahia. Quality of Service by Flow Aware Networking. In Philosophical Transactions of The Royal Society of London, series A vol. 358, number 1773, August 2000.

[99] S. Sarvotham, R. Riedi, R. Baraniuk. Connection-Level Analysis and Modeling of Network Traffic, In Proceedings of ACM Internet Measurement Workshop 2001.

[100] M. Shreedhar, G. Varghese. Efficient Fair Queuing Using Deficit Round Robin. IEEE/ACM Transactions on Networking, Vol.4, No.3, June 1996.

[101] I. Stoica, S. Shenker, H. Zhang. Core-Stateless Fair Queueing : A scalable Architecture to Approximate Fair Bandwidth Allocations in High-Speed Networks. IEEE/ACM Transactions on Networking, Vol. 11, No. 1, Feb. 2003, Pages : 33 - 46.

[102] B. Suter, T.V. Lakshman, D. Stiliadis, A.K. Choudhury. Buffer Management Schemes for Supporting TCP in Gigabit Routers with Per-Flow Queueing. IEEE Journal in Selected Areas in Communications, August 1999.

[103] L. Takàcs, Introduction to the Theory of Queues. Oxford University Press, 1962, New York.

[104] S. Tsao, Y. Lin. Pre-order Deficit Round Robin : A New Scheduling Algorithm for Packet Switched Networks. Computer Networks 35 (2-3), pages : 287-305, 2001.

[105] H. van den Berg, M. Mandjes. Measurement-based admission controle : algorithms, evaluation, and research perspectives. COST 257,1999.

[106] J. Virtamo, Idle and busy period distributions of an infinite capacity N*D/D/1 queue. Proceedings of ITC 14, Elsevier, 1994.

[107] P. White, J. Crowcroft. Integrated Services in the Internet : State of the Art. In IEEE, volume 85, 1997.

[108] J. Xu, M. Singhal. Cost-Effective Flow Table Designs for High-Speed Routers : Architecture and Performance Evaluation. IEEE Transactions on Computers, Vol. 51, No. 9, September 2002.

[109] L. Zhang. Virtual Clock : A New Traffic Control Scheme for Packet Switching Networks. In ACM SIGCOMM 1990.

[110] Y. Zhang, L. Breslau, V. Paxson, S. Shenker. On the Characteristics and Origins of Internet Flow Rates. In Proceedings of ACM SIGCOMM 2002.

[111] http ://www.caspiannetworks.com/home.asp

[112] Caspian Networks. Flow-Based Routing : Rationale and Benefits. White paper, 2003 http ://www.caspiannetworks.com/documents/Apeiro Flow State.pdf

[113] http ://www.cisco.com/wrap/public/63/mdrr_wred_overview.html

[114] http ://www-tkn.ee.tu-berlin.de/research/trace/trace.html

[115] http ://www-tkn.ee.tu-berlin.de/research/trace/pics/FrameTrace/mp4/index94b6.html